LLD Handbook — Vol 3: Interview Approach and Case Studies I

OOP · UML · SOLID · Clean Code · Refactoring · Design Patterns · Concurrency · 45+ case studies in Java

4 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — The LLD Interview & How to Approach It**
   - 1.1 What this round is, and what this handbook covers
   - 1.2 How the interview runs
   - 1.3 A complete answer in five minutes: "Design a music playlist"
   - 1.4 The step-by-step answer framework
   - 1.5 From one line to a list of requirements
   - 1.6 Finding the classes
   - 1.7 Connecting the classes (preview)
   - 1.8 Keeping the design honest with SOLID
   - 1.9 The design patterns that come up in this round
   - 1.10 Code, edge cases and concurrency: how to write interview code
   - 1.11 Worked mini-example, end to end: a small parking lot
   - 1.12 Communication tips
   - 1.13 How to use this handbook, and a study plan
   - 1.14 Common mistakes
   - In the interview
   - Self-check
   - TL;DR
2. **Chapter 14 — Case Studies: Everyday Objects**
   - 14.1 Design a Parking Lot
   - 14.2 Design a Vending Machine
   - 14.3 Design a Coffee Vending Machine
   - 14.4 Design an ATM
   - 14.5 Design an Elevator System
   - 14.6 Design a Traffic Signal Control System
   - 14.7 Design a Shipping Locker System (Amazon Locker)
   - Chapter summary
3. **Chapter 15 — Case Studies: Booking & Inventory (Part 1)**
   - Why booking problems are interview favourites
   - 15.1 Design a Library Management System
   - 15.2 Design a Movie Ticket Booking System (BookMyShow)
   - 15.3 Design a Concert Ticket Booking System
   - 15.4 Design a Hotel Management System
   - Chapter summary
4. **Chapter 16 — Case Studies: Booking & Inventory (Part 2)**
   - Design a Car Rental System
   - Design a Restaurant Management System
   - Design an Airline Management System
   - Design a Course Registration System
   - Design a Meeting Scheduler
   - Chapter summary

Chapter 1 — The LLD Interview & How to Approach It

What you will learn in this chapter

- What the low-level design (LLD) round is, its names at different companies, and how it differs from system design; the **object-oriented design (OOD) round** vs the **machine-coding round**.
- A complete answer before any theory (a music playlist in Java).
- A repeatable framework, with a time budget, that turns one sentence into working code: requirements, classes, a SOLID check (five rules of thumb that keep classes easy to change), and the six patterns that come up most.
- What interviewers score, how to talk while you design, the costliest mistakes, a study plan and a map of the handbook (Chapters 1–22).
- A second worked example (a small parking lot) run through every step.

1.1 What this round is, and what this handbook covers

What it is

The **low-level design** round gives you **a single application** to design while an interviewer watches, usually in 45–60 minutes (a machine-coding round stretches this to 90 minutes or 2 hours). Not the servers underneath: **the set of classes inside the program** — a parking lot, a vending machine, a chess game, Splitwise (an app that splits shared expenses between friends), an elevator.

Term	Meaning
Class	A named type in code holding data (fields) and the methods (functions that belong to the class) that work on it.
Object	One concrete instance of a class, with real values in its fields.
Responsibility	The one job a class does, statable in a single sentence.
Relationship	How two classes are connected: one uses, holds, owns, or extends the other.
UML	Unified Modeling Language: the standard symbols for drawing classes and their relationships.
Design pattern	A named, reusable answer to a recurring design problem (e.g. Strategy, Observer).

Analogy: system design is the **city plan** (roads, power, water); low-level design is the **floor plan of one house** (which rooms, what each is for, which doors connect them). Engineers who write classes daily still struggle here because daily work seldom makes you **explain aloud** why each class exists and **justify** how they connect.

What you should be able to do after this handbook

- Turn one sentence ("Design a parking lot") into a bounded requirement list with an explicit out-of-scope list.
- Pick the classes a problem needs, with **exactly one job** apiece, and relate and draw each pair correctly.
- Code the Java methods behind the core flows until they run.
- Say which design pattern you used **and which you rejected**, and why.

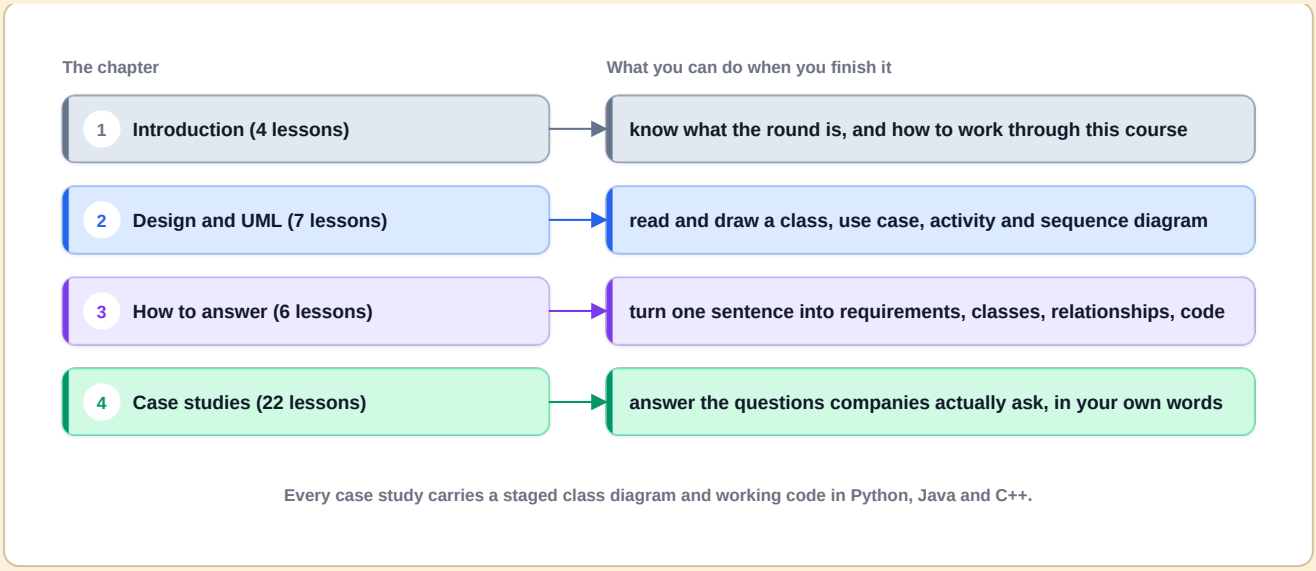


Figure — The four parts of the course material this handbook absorbs: introduction, design and UML notation, the answering method, and the case studies (course diagram)

The handbook follows that arc (understand the round, the notation, the **method**, then case studies) and adds full chapters on OOP, SOLID, design principles, clean code, refactoring (improving code structure without changing behaviour), all 23 GoF patterns (the catalogue from the 1994 "Gang of Four" book) and concurrency (several pieces of work running at once and sharing data), so you need no second resource.

Every case study uses one template: the question and chosen requirements (with out of scope); actors and use cases (often a use case diagram: who uses the system and for what); the class diagram built in stages; diagrams for key flows (activity diagrams show the steps of a process, sequence diagrams the messages objects send each other in time order); working Java with a driver `Main` (a class whose `main` method runs the example) and its real output; patterns named **only where really used**; and "Extensions & follow-up questions" listing the simplifications and what to say when the interviewer notices them.

Who this is for, and what you need

For anyone facing this round, writing OO code without ever defending a class diagram aloud, or prepared only for system design. The method also splits new features into classes at work.

- **Some object-oriented language**, enough that a class with a constructor (the special method that sets up a new object) and a few methods reads easily. All examples are **Java 21** (a recent long-term-support release).
- **No UML** (Chapter 3 teaches it from zero).
- **No system design.** That round asks how a product survives millions of users (servers, databases, caches, queues; see System Design Fundamentals). This round stays **inside one process** (one running program). Practising one gives little for the other.
- Data structures only at basic level (a separate round): a **list** keeps items in order, a **map** looks a value up by key, a **set** holds no duplicates, a **queue** hands items out first-in first-out, a **heap** gives the smallest or largest item quickly.

It will not teach a language from scratch or give scripts to memorise: a memorised answer is useless the moment the question is new. **The reusable part is the method.**

> **TL;DR**
>
> The LLD round = design the classes inside one application, out loud, then code the main flows. Prerequisite: one OO language. The method beats memorised answers.

1.2 How the interview runs

What it is

The interviewer says one sentence, "Design a parking lot", and stops. Nothing tells you how many classes, whether to code, or how long you have. **That silence is the hard part.**

Name	Where you hear it	What is different
Object-oriented design (OOD) interview	US big tech, many product companies	Whiteboard/doc: requirements, classes, diagram, code for main flows.
Low-level design (LLD)	Indian product companies, general usage	Same as OOD; the umbrella term.
Machine-coding round	Commonly reported at Indian product companies such as Flipkart, Swiggy and PhonePe	**Working, runnable code is mandatory**, usually in a 90-minute to 2-hour window.

LLD vs HLD (system design) — same product, different zoom:

	Low-level design (this handbook)	High-level / system design
Scope	Inside one application or service	Many services and machines
Vocabulary	Classes, interfaces, fields, methods, relationships, patterns	Servers, load balancers (spread requests across servers), databases, caches, queues, sharding (splitting one database across machines)
Output	Class diagram + code	Architecture diagram + capacity numbers
Typical question	"Design the classes for a parking lot"	"Design a parking-lot booking service for 10 million users"

OOD round vs machine-coding round

	OOD / LLD discussion round	Machine-coding round
Duration	45–60 minutes	Usually about 90 minutes of coding (often 2 hours including evaluation)
Where	Whiteboard, shared doc, or IDE (code editor with build and run tools)	Your own IDE, on your laptop or a remote machine
What you produce	Requirements, use cases, class diagram, code for 2–3 main flows	A **complete, running program** implementing the minimum required features
Storage	Not discussed much	**In-memory** (maps and lists); no real database unless asked
Interaction	Continuous conversation	Problem statement up front, mostly silent coding, then a **demo and code walkthrough**
What is judged	Bounding the problem, responsibilities, relationships, trade-offs, patterns	Working code, **data models**, **service interfaces**, **extensibility**, readability, edge cases, sometimes tests and thread safety
Common failure	Talking without converging, or coding too early	Not finishing, or one giant class that cannot be extended

Machine coding gives a **minimum feature set**; you choose data structures, design **service interfaces** a client (e.g. a command-line driver) calls, and present **working code** with reasoning and patterns. Common for SDE-1/SDE-2 hiring (Software Development Engineer levels: entry, mid).

Terms: **data models** = classes holding the problem's data (`Ticket`, `Spot`); **service interface** = the public methods a caller uses (`park`, `exit`); **extensibility** = adding a feature without rewriting; **edge cases** = unusual boundary inputs (empty, full, negative, duplicate); **thread safety** = staying correct when several **threads** (independent lines of execution inside one program) use the code at once.

How it works: the five core steps

1. **Clarify the requirements** — one sentence → a short written list of what the system must do.
2. **Find the actors and use cases** — who uses it (actors) and what each does (use cases).
3. **Find the classes** — one responsibility each.
4. **Connect the classes** — how each pair relates; a class diagram.
5. **Write the code** — the classes plus the 2–3 methods of the main flows.

Order matters more than speed: skip step 1 and step 3 invents classes no requirement asked for. (Section 1.4 expands these into eleven finer steps.)

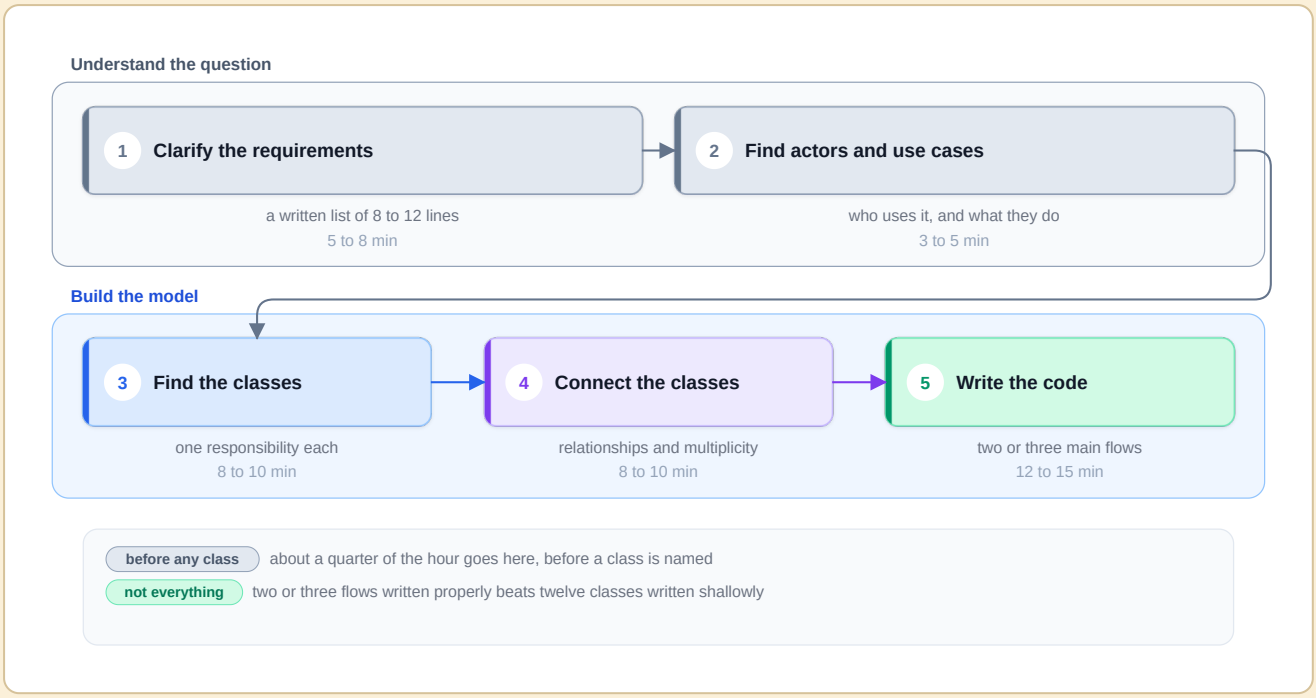


Figure — The five steps split into "understand the question" and "build the model", with a time budget for each (course diagram)

Up to **a quarter of the hour goes before any class is named**; the largest block, code, covers only two or three flows. **Nobody completes the whole product, and you are not expected to**: the design is judged on the part you built properly.

Time-boxing

45-minute OOD round:

Step	Time	What exists at the end
Clarify requirements	5–8 min	8–12 requirements + out-of-scope list
Actors and use cases	3–5 min	2–4 actors, 6–10 use cases
Find the classes	8–10 min	Class list, one responsibility each
Connect the classes	8–10 min	Class diagram with relationships and multiplicity (how many of each)
Write the code	12–15 min	2–3 methods implementing the main flows

90-minute machine-coding round:

Phase	Time	Output
Read, list requirements and assumptions	5–10 min	Comment block at the top of `Main.java`
Model: entities, enums, service interfaces	10–15 min	Empty classes/interfaces that **compile** (the compiler accepts them)
Core flow end to end	30–40 min	One **happy path** (main flow, valid input) running from a driver `main`
Remaining features, validation, edge cases	15–20 min	All "must have" features working
Polish: names, exceptions, one extension point, demo script	5–10 min	Clean demo output
Buffer	5 min	Never zero

(**Enums** are types with a fixed list of named values; Section 1.6.) Rule of thumb: **get something running by the halfway mark** — a running partial solution beats a beautiful non-compiling one.

What the interviewer is actually judging

1. **Did you bound the problem?** A clear bounded set beats a broad unbounded design.
2. **Does each class have one job statable in one sentence?** If not, it is normally two classes glued together.
3. **Are the relationships right?** Holding, owning and extending are different choices.
4. **Does the code match the diagram?** A method with no place in the diagram means one of them is wrong.
5. **Can you explain a trade-off?** Naming the road taken and the road not taken scores higher.

Machine coding adds: **runs**, **modular** (new vehicle type / payment / rule = new class), **readable**, **handles bad input**, and (senior) **thread-safe** shared state. Not graded: perfect UML or every diagram type — **the model is graded; notation is presentation.**

Sizing: a parking lot has ~12 requirements → ~12 classes, ~15 relationships. Three classes implement the main flows (issue a ticket, hold a vehicle, price the stay): write those properly, name the other nine in a sentence each. The costliest mistake is writing all twelve at equal depth so no flow works end to end.

> **TL;DR**
>
> OOD, LLD and machine coding are one family; machine coding needs running code in ~90 minutes. Five steps in order, a quarter of the time before naming a class, two or three flows built properly.

1.3 A complete answer in five minutes: "Design a music playlist"

A complete answer first, theory later. It is small on purpose; later case studies are ~10× the size but follow **the identical outline**. (Diagram notation: Chapter 3.)

Step 1: requirements

"Design a music playlist." Three clarifying questions give three requirements:

- Songs sit in a playlist and **their sequence is significant**.
- Someone can ask for **the following song**; the player tracks its current position.
- A user can turn **shuffle** on, changing the play order.

Out of scope, agreed out loud: user accounts, downloads, storage, the audio itself. The "not building" list earns as much credit as the "building" list.

Step 2: actors and use cases

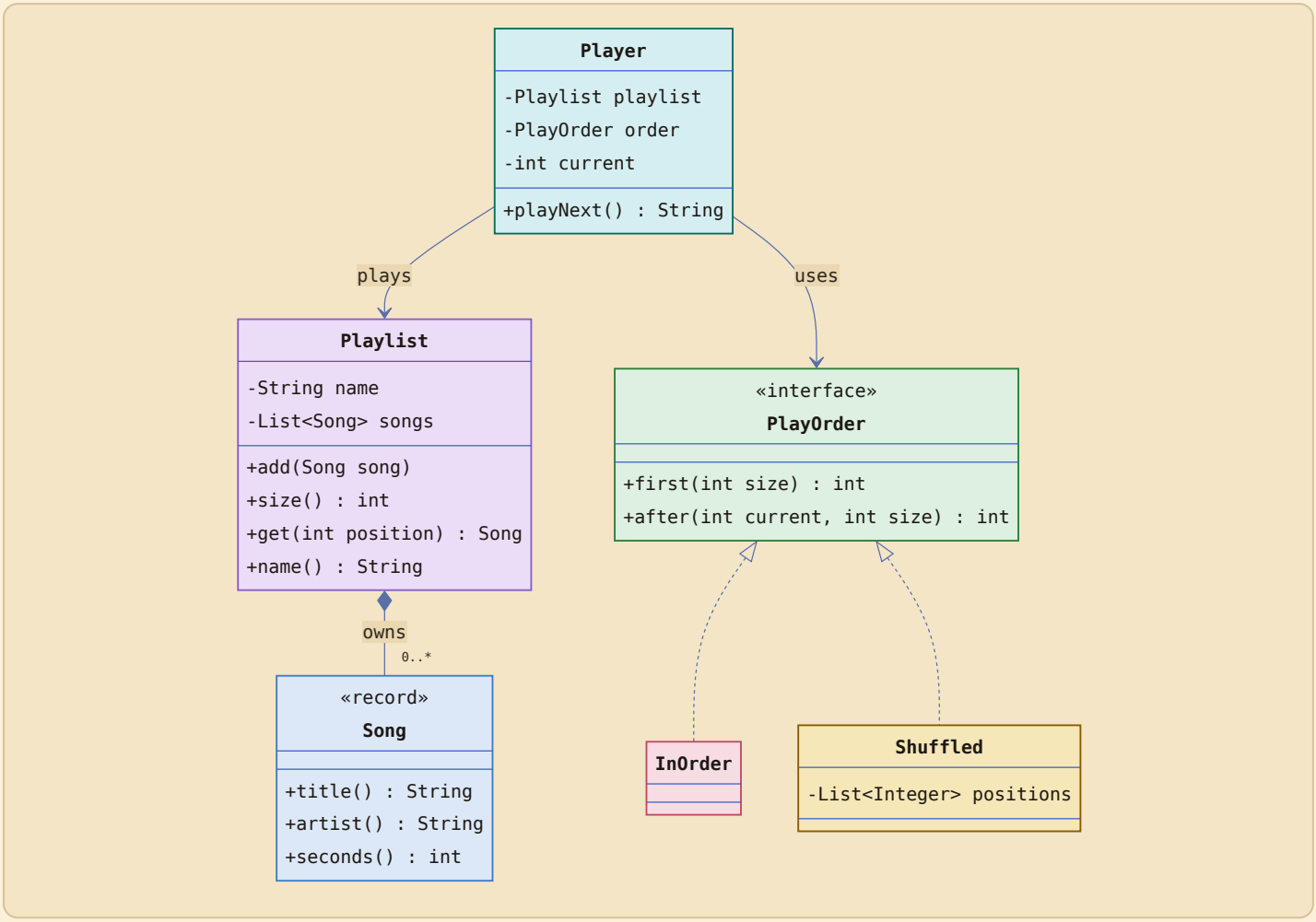
An **actor** is anyone or anything outside the application that interacts with it. Here only the **listener**: adds a song, plays the next one, switches shuffle on. Ten seconds here; three minutes for a parking lot, repaid tenfold, because **the actors reveal which classes must exist**.

Step 3: classes, one sentence each

Class	Responsibility (one sentence, no "and" where possible)
`Song`	Holds a title, an artist and a length; knows about itself and nothing else.
`Playlist`	Holds songs in order; stores them, does not play them.
`PlayOrder` (interface)	Picks the position to play after the current one; an interface because several ordering rules exist.
`InOrder`	Walks positions in insertion order.
`Shuffled`	Plays positions in a shuffled order decided once, up front.
`Player`	Holds a playlist and an order, and remembers the current position.

An **interface** names a set of methods without saying how they work. Key insight: **a rule that can vary becomes its own type.** Without shuffle, `Player` would just count upward; with shuffle, "what plays next" has two versions, so it becomes `PlayOrder`, and the player never needs to know which it holds. Test: if a responsibility sentence needs **"and"** to join two unrelated jobs, it is probably two classes.

Step 4: connect them



Three facts an interviewer checks:

- The **filled diamond** between `Playlist` and `Song`: the playlist **owns** its songs (composition); `*` / `0..*` means **many**.
- The **plain arrow** from `Player` to `PlayOrder`: the player **relies on** an ordering without owning it, and the arrow touches only the **interface**, not `InOrder` or `Shuffled`.
- The **dashed arrows with hollow triangles**: `InOrder` and `Shuffled` **implement** `PlayOrder`. A third rule later is one extra box; nothing drawn changes.

Step 5: the code

> **Key term — reading the Java below:** `package` groups related classes like a folder. `implements PlayOrder` promises every method of that interface; `@Override` marks a method fulfilling such a promise (compiler-checked). `final` on a class forbids extending it; on a field, it is set once. `private` hides a member from code outside the class.

Essential lines (full files: `code/lld/ch01/playlist/`; `Song` is a record with a validating constructor, `Playlist` wraps a private `List<Song>` with `add`/`size`/`get`/`name`):

```java
// File: code/lld/ch01/playlist/PlayOrder.java
public interface PlayOrder {
    int NONE = -1;                          // "there is no next position"
    int first(int size);                    // first position to play, or NONE if empty
    int after(int current, int size);       // position after 'current', or NONE at the end
}

// File: code/lld/ch01/playlist/InOrder.java
public final class InOrder implements PlayOrder {
    @Override public int first(int size) { return size == 0 ? NONE : 0; }
    @Override public int after(int current, int size) { return current + 1 < size ? current + 1 : NONE; }
}

// File: code/lld/ch01/playlist/Shuffled.java  — order injected, so output is deterministic = testable
public final class Shuffled implements PlayOrder {
    private final List<Integer> positions;
    public Shuffled(List<Integer> positions) { this.positions = List.copyOf(positions); }
    @Override public int first(int size) { return positions.isEmpty() ? NONE : positions.get(0); }
    @Override public int after(int current, int size) {
        int i = positions.indexOf(current);
        return (i >= 0 && i + 1 < positions.size()) ? positions.get(i + 1) : NONE;
    }
}

// File: code/lld/ch01/playlist/Player.java  — never asks WHICH order it holds
public final class Player {
    private final Playlist playlist;
    private final PlayOrder order;          // depends on the interface, not on InOrder/Shuffled
    private int current = PlayOrder.NONE;
    private boolean started = false;
    // constructor stores playlist and order
    public String playNext() {
        if (!started) {
            current = order.first(playlist.size());
            started = true;
        } else if (current != PlayOrder.NONE) {          // once at the end, stay at the end
            current = order.after(current, playlist.size());
        }
        if (current == PlayOrder.NONE) return "end of playlist";
        Song s = playlist.get(current);
        return "playing " + s.title() + " by " + s.artist();
    }
}

// File: code/lld/ch01/playlist/Main.java (driver, abridged)
Playlist mix = new Playlist("Morning mix");
mix.add(new Song("Sunrise", "Ada Lane", 201));
mix.add(new Song("Blue Line", "The Meridians", 187));
mix.add(new Song("Paper Boats", "Nori Kim", 243));
run("in order", new Player(mix, new InOrder()), mix.size());          // run() prints label, then
run("shuffled", new Player(mix, new Shuffled(List.of(2, 0, 1))), mix.size()); // playNext() size+1 times
```

Run it: `cd chapters/01-lld-interview-approach/code && javac --release 21 -d /tmp/lld_01 $(find . -name '*.java') && java -cp /tmp/lld_01 lld.ch01.playlist.Main` (`javac` compiles `.java` to runnable `.class` files; `java` runs them; `-cp`, the classpath, says where they are).

Output:

```
in order:
  playing Sunrise by Ada Lane
  playing Blue Line by The Meridians
  playing Paper Boats by Nori Kim
  end of playlist
shuffled:
  playing Paper Boats by Nori Kim
  playing Sunrise by Ada Lane
  playing Blue Line by The Meridians
  end of playlist
```

Explaining each part:

- `Song` is a Java **record**: an immutable (unchangeable after creation) data carrier whose fields, constructor, accessors (`title()`), `equals`, `hashCode` and `toString` (compare, use as map key, print) are generated; its compact constructor (no parameter list, only checks values) rejects a blank title or non-positive length.
- `Playlist` keeps its list `private` (**encapsulation**: data reachable only through the class's own methods); callers add and read but cannot replace it.
- `PlayOrder` is the **variation point** (where different behaviours plug in); `NONE = -1` is an interface constant for "nothing more to play".
- `Shuffled` is **given** its order rather than using `Random`, so output is identical every run: **testable**. In production you would **inject** (hand in from outside, usually via the constructor) a `Random` with a **seed** (a fixed starting number that makes it repeat the same sequence) or a list shuffled elsewhere.
- `Player` depends on the interface, not a **concrete class** (a real class you can instantiate). One `Player` gives both behaviours and **nothing tests which mode is active**; its only `if`s track *where* it is (first call asks `first`, later calls `after`, and at the end it stays there).

What to notice

- **One requirement turned into a type.** Shuffle was one line of requirements and produced an interface plus two implementations. **Classes come out of requirements**, never the reverse.
- **One job per class** (see the table): no class knows more than its sentence.
- **Open to changes you did not build.** "Repeat one" or "reverse" are new `PlayOrder` classes; nothing is reopened (Strategy pattern + Open/Closed Principle, Chapters 4 and 11).
- **Gaps announced on purpose:** pause, volume, queue, time. Say them yourself; every case study ends with "Extensions & follow-up questions" for this reason.

TL;DR

A complete answer = a short bounded requirement list, a few one-sentence classes, one diagram, code for the main flow. A rule that varies becomes an interface, and its user never asks which version it holds.

1.4 The step-by-step answer framework

What it is

A checklist run on **every** LLD question, expanding the five core steps into eleven, so "where do I start?" becomes small questions with known answers. Without it candidates freeze or code at once and find missing requirements at minute 35; with it your progress is **visible**, which is itself scored.

How it works

#	Step	Questions to ask yourself	Output	Core step
1	**Clarify**	Who uses it? What exactly is "a parking lot" here? One location or many?	Shared understanding	Requirements
2	**Requirements**	What must it do (functional)? How well (non-functional: thread-safe, extensible, in-memory, latency i.e. response time)? Out of scope?	8–12 numbered lines + out-of-scope list	Requirements
3	**Use cases**	For each actor, what can they do?	Actor → use case list (or diagram)	Actors & use cases
4	**Entities**	Which nouns hold state or behaviour? Which are just values (fields/enums)?	Class list, one-sentence responsibilities	Classes
5	**Relationships**	Uses, has, owns, or is-a? How many (1, 0..1, *)?	Relationship list	Connect
6	**Class diagram**	Fields, key methods, interfaces, enums	UML class diagram	Connect
7	**Key flows**	Step by step for the 2–3 main use cases? Which states does the core object go through?	Sequence / activity / state diagram (states and moves between them)	Connect
8	**Patterns & SOLID**	A rule that varies (Strategy)? A lifecycle (fixed states an object moves through: State)? Many listeners (Observer)? A class with two reasons to change?	Named patterns + rejected alternatives	Check
9	**Code**	Interfaces, enums, models, services, a driver `main`	Running code for main flows	Code
10	**Edge cases & concurrency**	Invalid input? Full capacity? Two users grabbing the same seat?	Validation, exceptions (Java's error signal that stops normal flow), locks (one thread at a time in protected code)	Code
11	**Extensions**	What changes if requirement X is added?	Short answers, maybe one extra class	Wrap-up

Trade-offs

Do this	Instead of this	Why
Spend 8–13 minutes on steps 1–3	Coding at minute 2	Unbounded scope wastes far more time later
Code 2–3 flows fully	Stub 12 classes (empty placeholders)	Depth on the chosen part is judged
Name one or two patterns where the signal appears	Apply six	Over-engineering (more flexibility than needed) reads as poor judgement
Pass time into methods (`nowMinute`); hold money as `BigDecimal` (Java's exact decimal type) or whole cents in a `long`	Call `System.currentTimeMillis()` inside logic; use `double` (binary floating point) for money	Deterministic (same input, same output) and testable; `double` cannot represent 0.10 exactly, so sums drift

Not rigidly: if the interviewer steers ("skip requirements, show the class model"), follow. In machine coding steps 5–7 are often in your head or a comment; the code **is** the diagram.

TL;DR

Clarify → requirements → use cases → entities → relationships → class diagram → key flows → patterns/SOLID → code → edge cases/concurrency → extensions. Say each step out loud as you enter it.

1.5 From one line to a list of requirements

What it is

A **requirement** states one capability so you can test whether the design delivers it ("customers can pay at the exit", not "the lot should be good").

- **Functional** — a behaviour: "a customer takes a ticket at an entry panel".
- **Non-functional** — a quality: "two entry panels never assign the same spot" (thread safety), "a new vehicle type changes no existing class" (extensibility), "data kept in memory" (storage).

The problem it solves

Real products start the same way: a short request someone turns into a specification. "Design a parking lot" is four words; the design is ~12 classes. Without a written list: **the design has no edges** (season tickets, plate recognition... and no code); **you cannot defend a choice** ("why no `Reservation`?" → "out of scope; we agreed at minute four"); **you cannot tell when you are finished**.

How it works: the four questions

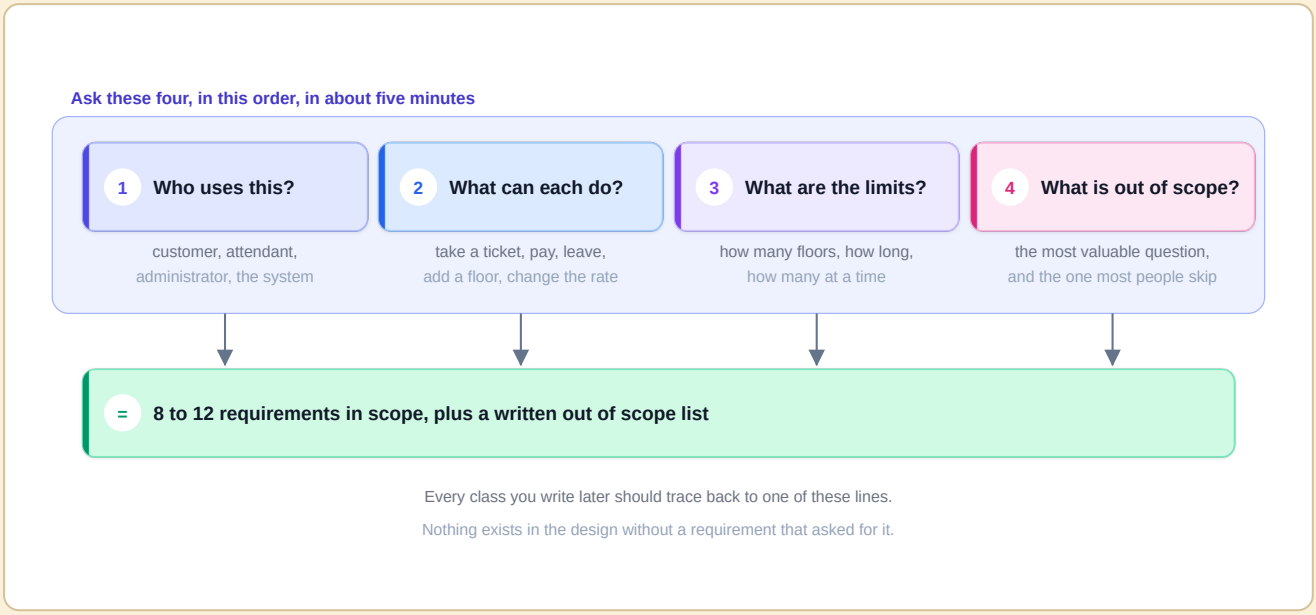


Figure — Four questions asked in order produce 8–12 in-scope requirements plus a written out-of-scope list (course diagram)

In order, ~5 minutes:

1. **Who uses this?** The **actors** — parking lot: customer, attendant, administrator, the system itself. Each brings its own requirements.
2. **What can each do?** Each action is a **use case** (customer takes ticket, parks, pays, leaves; admin adds floors, changes prices). Most of the list comes from here.
3. **What are the limits?** Floor count, spot kinds, max booking length, books per member. **Limits become constants and validation checks**; easiest to gather.
4. **What is out of scope?** Rarely asked, pays off most: the only question that makes the problem **smaller**.

Scope questions for almost any problem: "Are payments mine, or an external service I take as given?" "One site, or several sharing an account?" "Are users and permissions modelled, or may I assume a

logged-in user?" Each answer adds or removes two or three classes, so ask early. If told "your choice": **choose, say it, write it down** — a stated assumption counts like a given answer.

What good looks like: the parking lot in ten lines

In scope

1. The lot has multiple floors; each floor has many parking spots.
2. Spot types: compact, large, handicapped, motorbike, electric.
3. Vehicle types: car, van, truck, motorbike, electric car.
4. A customer takes a ticket at an entrance panel on arrival.
5. A customer pays at an exit panel, at a floor portal, or to an attendant.
6. Payment is by cash or card.
7. Charges are per hour, with the hourly rate depending on how long the stay lasts.
8. Each floor shows how many spots of each type are free.
9. The system refuses entry when no suitable spot is free.
10. Admins may add floors, spots and panels, and may update the rate.

Out of scope: season passes, licence-plate recognition, booking before arrival, refunds, multi-site reports.

Every line becomes a class, field or method. **If no line asked for it, it is not in the design.**

Make each line checkable: put a number on it ("no more than **five** books out at once" → a constant → a check in one method) and **name the actor** ("**a member** can reserve a copy currently on loan" → you know which class owns the method). Such a line is close to a test case: "requirement 5 lives here".

Use cases

A **use case** is one goal an actor achieves ("customer pays for parking"); a **use case diagram** (Chapter 3) draws actors as stick figures and use cases as ovals. In 45 minutes a list suffices:

Actor	Use cases
Customer	take ticket, park, pay (cash/card), exit
Attendant	collect cash payment, help customer
Admin	add floor, add spot, add panel, change rate
System	assign spot, update display boards, refuse entry when full

Common mistakes

- **Saying yes to every idea** — reservations, multiple sites... twenty minutes later 22 requirements and nothing designed. Declining is a **strong** move: "Let me park reservations and get ticketing and payment working end to end; I'll return if time allows."
- Forgetting non-functional requirements (concurrency, extensibility).
- Vague lines with no number and no actor.

In the interview

> **30-second answer** ("how do you start an LLD question?"): "I ask four questions: who uses it, what each actor does, what the limits are, and what is out of scope. I write 8 to 12 numbered requirements and an out-of-scope list, confirm them with you, and then every class I create must trace back to one of those lines."

- *"Why not just start designing?"* — Without a boundary the design grows forever and I cannot defend omissions. Five minutes here saves fifteen later.
- *"What if I say 'your choice'?"* — I pick the simpler option, say it, and write it as an assumption: visible and reversible.

Self-check

- **Q:** Which of the four questions makes the problem smaller? **A:** "What is out of scope?"
- **Q:** Rewrite "users can book some seats" as a checkable requirement. **A:** "A customer can hold at most 10 seats for one show; held seats are released after 10 minutes if unpaid."
- **Q:** Where do limits end up in code? **A:** Constants plus a validation check in the method that owns the data.

TL;DR

Four questions (who, what, limits, out of scope) → 8–12 checkable lines with numbers and actors, plus an out-of-scope list. Saying "no" to extra scope is a strength.

1.6 Finding the classes

What it is

A ten-minute method: **collect candidates, cut the ones that are not classes, give each survivor one job.**

How it works

1. **Underline the nouns.** Parking lot — structure: lot, floor, spot, compact spot, large spot, entrance panel, exit panel, display board, portal; vehicles: vehicle, car, van, truck, motorbike; people: customer, attendant, administrator; money/time: ticket, payment, cash, card, rate, hour. Deliberately too many: **candidates**, none made up.
2. **Underline the verbs** (park, take a ticket, pay, leave, refuse entry, show free spots, add a floor, change the rate). **Each verb becomes a method** on some class; a verb with no home means a class is missing.
3. **Cut candidates with five tests:**

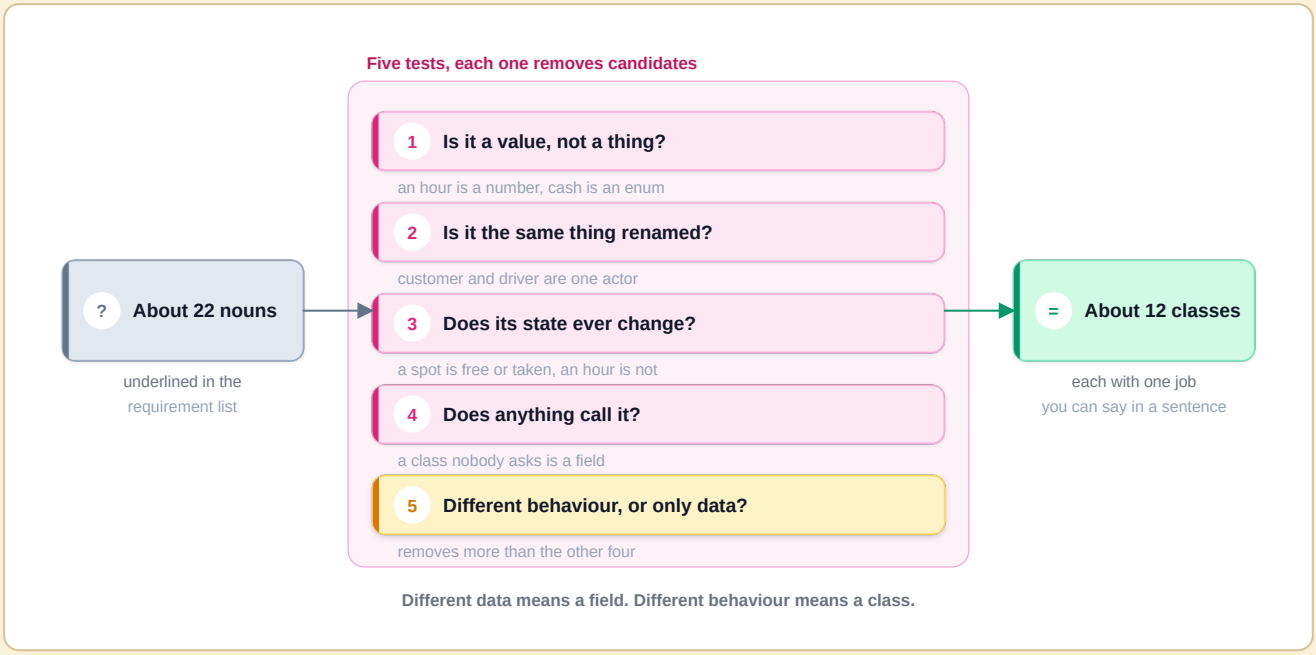


Figure — About 22 nouns pass through five tests and about 12 classes survive, each with one job (course diagram)

#	Test	Example	Result
1	Is it a **value** rather than a thing?	An hour is a number, a licence plate a string; cash/card are options of a payment type	Field or **enum** constant (a fixed set of named values)
2	Just **another name** for a listed candidate?	Customer and driver are one actor; portal and panel could be one class in several places	Merge, pick one name
3	Does it have **state that changes**?	A spot is free or occupied; an hour never changes	Changing state → class
4	Does anything **call** it?	Nobody asks an `Hour` object a question	Never called → a value
5	Different **behaviour** or only **data**?	Compact vs large spot differ only in which vehicle fits; cash vs card behave differently	**Different data → a field. Different behaviour → a class.**

Tests 1–4 cut ~22 to ~15; test 5 alone brings it to ~12. Spot kinds differ only in allowed vehicle (**data** → one `ParkingSpot` with a `type` field); cash checks the amount given while card calls a provider and waits (**behaviour** → two classes). **Smell** (a surface sign something is wrong): five subclasses (classes extending a parent) whose only content is passing a different constant to the parent — a field pretending to be classes.

4. **One responsibility each**, one sentence, no "and":

Class	Responsibility
`ParkingLot`	Issues tickets and can tell when no space is left
`ParkingFloor`	Owns a level's spots and locates an empty one
`ParkingSpot`	Tracks if it is occupied and by which vehicle
`Vehicle`	Carries its plate number and the spot types it can use
`ParkingTicket`	Notes the vehicle, the spot it took, and the entry time
`ParkingRate`	Converts a duration in hours into a price
`Payment`	Collects the money and says if that worked
`ParkingDisplayBoard`	Shows how many spots of each type are free

`ParkingLot`'s sentence admits two jobs (tickets **and** fullness/assignment); a longer design splits it, but in 45 minutes **naming the tension aloud is enough**.

5. **Put each method where the data is.** "Find a free spot" touches the floor's spots → `ParkingFloor`; "price the stay" the rate card → `ParkingRate`; "take a ticket" the ticket store → `ParkingLot`. A method needing data from three classes belongs with the one owning most of it (GRASP "Information Expert" — guidelines for which class gets which job — and the cure for *Feature Envy*, a method using another class's data more than its own; Chapters 5 and 7).

Common mistakes

- One subclass per value (`CompactSpot`, `LargeSpot`... with no behaviour).
- Missing the **record-keeping** class (ticket, loan, booking) and the **rule** class (rate, strategy): the two beginners most often forget.
- "Manager" **god classes** (one huge class owning most of the logic).

Self-check

- **Q:** Car, van and truck: subclasses or enum? **A:** If they differ only in size/which spots fit, an enum `VehicleType` (or a field); genuinely different behaviour → subclasses.
- **Q:** A verb has no class to live on? **A:** A class is missing.
- **Q:** A class has fields but no verbs? **A:** Probably a value (record/field), not a full class.

TL;DR

Nouns → candidates, verbs → methods. Cut values, duplicates, stateless and never-called nouns. **Different data = field; different behaviour = class.** One sentence per class; the method goes where its data is. Twelve clear classes beat twenty overlapping ones.

1.7 Connecting the classes (preview)

For each pair decide: does one **use** the other briefly (dependency), **hold** a reference (association), **contain** parts that can live alone (aggregation), **own** parts that die with it (composition), or **is** it a kind of the other (inheritance: a class extends another and reuses its code; realization: a class implements an interface)? Add **multiplicity** (1, 0..1, 1..*, *). Chapter 3 covers every relationship and symbol. The commonest beginner error: **inheritance** where **composition** is right. Test the sentence "A **is a** B": a car is not an engine, so `Car` has an `Engine`.

1.8 Keeping the design honest with SOLID

What it is

SOLID is five rules (one per letter) that catch the common ways a class diagram goes wrong. Chapter 4 teaches them in depth; in the interview run them as **five quick checks** over your diagram, about five minutes.

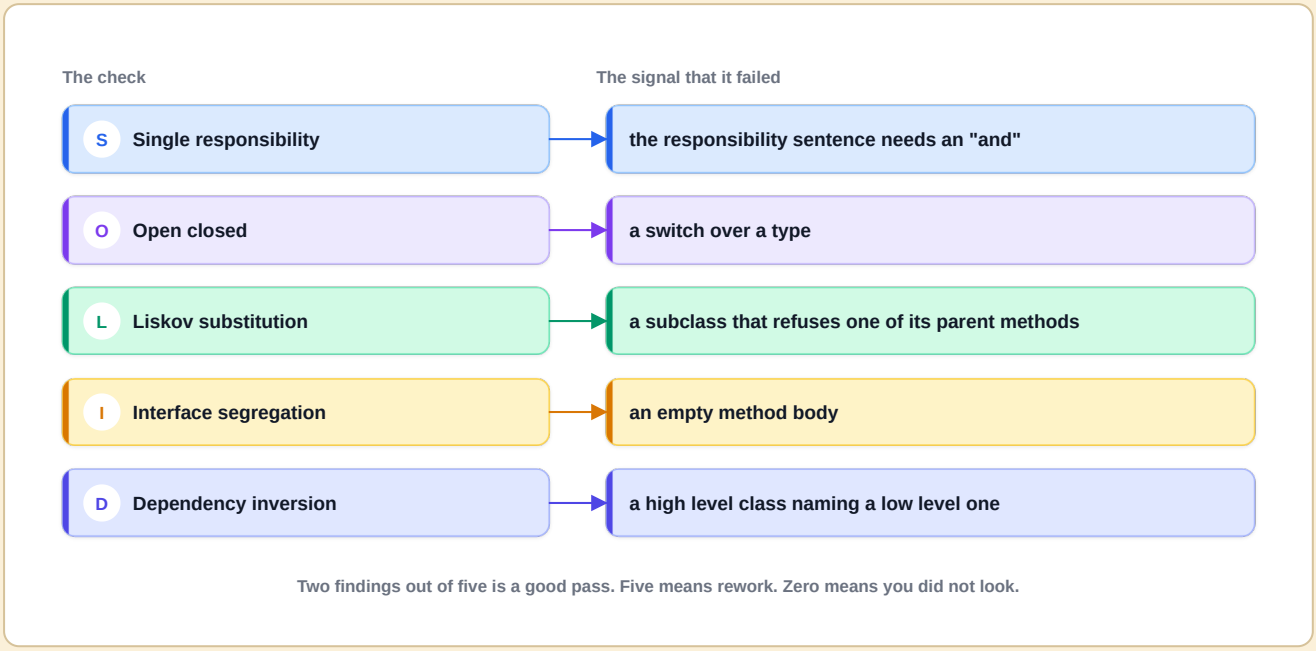


Figure — The five SOLID checks and the signal that each one has failed (course diagram)

How it works: five checks, with a pass over the parking lot

Check	Rule in one line	Signal it failed	Quick check	Parking-lot finding
S — Single responsibility	One reason to change	Responsibility sentence needs "and"	Name the change forcing an edit; two unrelated → split	`ParkingLot` issues tickets **and** finds spots (and, if it printed receipts, would change for pricing and layout): split if time allows, else say it
O — Open/closed	Add behaviour without editing code	`switch` / `if` chain over a type	Pick a likely new feature; if it edits three classes, the design is closed to the wrong thing	Pricing behind `ParkingRate`, so a weekend rate is a new class, not a switch edit. Passes
L — Liskov substitution	A subclass works anywhere its parent does	Subclass throws, does nothing, or refuses a parent method	Read each inheritance arrow: can the child go where the parent is expected?	A `ReadOnlySpot` whose `occupy` throws would fail. The five no-behaviour spot subclasses are *not* strictly a Liskov violation (they substitute fine) — it is 1.6's "different data → field" caught again; collapse to a `type` field. Re-read arrows at the end: they get added late
I — Interface segregation	No class implements methods it does not use	Empty method bodies	Split until every implementer uses everything	No wide interfaces. Passes. (A library `Search` with four methods is fine only if implementers really do all four)
D — Dependency inversion	Depend on an interface, not a concrete class	A high-level class (business rules) constructs or names a low-level one (one technical job: database, payment provider)	Could you test it without the real database/provider?	Lot depends on `Payment`; `new CreditCardTransaction()` inside it would block testing and cash. Passes

Two real findings, each explainable in a sentence, is a good pass; **five** usually means rework, **zero** that you did not look.

Most credit: S improves your class list (twelve well-named classes beat six doing three things); **O** is what interviewers probe with a late requirement ("trucks charged differently?") — "add a class" beats "add a branch to this switch".

When not to: no interface for everything — a single-implementer interface "because SOLID" is **indirection** (an extra layer between caller and code) with no benefit; add one when a second implementation exists or is clearly coming, or for a **test double** (a fake stand-in used in tests). Do not over-split: thirty two-method classes are as bad as one god class; the one-sentence test is the boundary. Apply each rule where its problem appears.

TL;DR

Run S-O-L-I-D as five quick checks on your own diagram before the interviewer does. S and O earn most credit. No single-implementer interfaces, no over-splitting.

1.9 The design patterns that come up in this round

What it is

A **design pattern** is a named solution to a recurring design problem. The Gang of Four book catalogues 23 (Chapters 8–11 cover all); **six** recur in this round. The name turns a paragraph into two words. **A misapplied pattern hurts more than an unknown one**, so learn each with its **signal**.

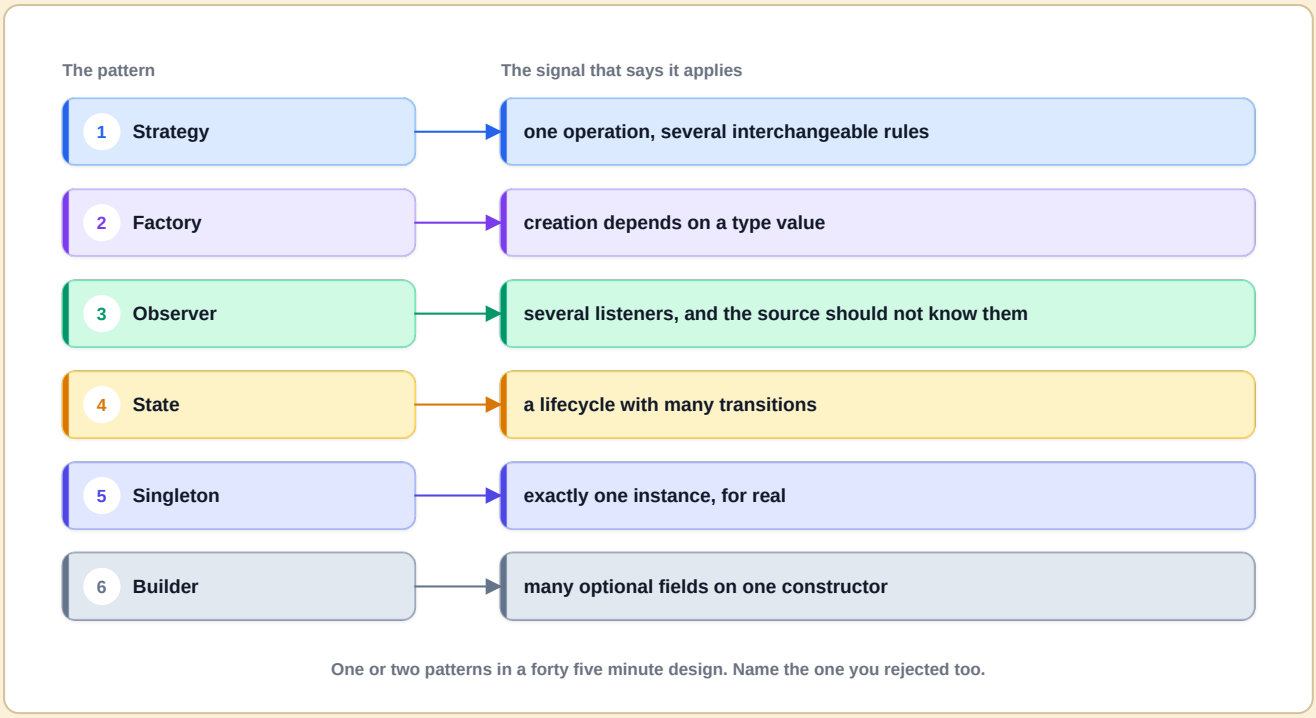


Figure — The six patterns most used in this round and the signal for each (course diagram)

Pattern	Signal → reach for it	Skip it if	Parking lot / library example and what to say
Strategy	One operation, several interchangeable rules chosen at run time	Only one rule and no second in sight	`ParkingRate` with `HourlyRate`, `WeekendRate`, `MonthlyPass`. Without it the pricing method grows a `switch` edited per rate; with it a new rate is a new class. Answers "what if the rules change?", the commonest late question. Also matching, sorting, ranking, routing
Factory	Creation depends on a type value; caller should not know the concrete class	Only one concrete class	A factory turns a type/enum into a `Vehicle`, so the entrance panel does not know every vehicle class; registering a new type happens in **one spot** (open/closed). A factory **method** is usually enough; one factory class per product is too heavy for 45 minutes
Observer	One thing changes, several others must know, source should not know them	One listener the source already holds	**Spot is the subject**; boards, entrance panels and floor portals register and update when it fills; the spot never learns who listens. With one listener a direct call is enough (e.g. notifying the member who reserved a returned book; the floor pushing to its single board)
State	Behaviour depends on current state with a fixed set of transitions	A single guarded method covers ~3–4 states	Ticket active → paid → closed; book copy available/reserved/loaned/lost. Without it every method checks status and legal transitions scatter; with it each state class knows what is allowed, so illegal moves cannot be expressed. Choose only for large transition tables; else **enum + one guarded method**, and say why
Singleton	Exactly one instance must exist	The real goal is just easy global access	`ParkingLot`, `Library`: private constructor + one accessor. Expect two follow-ups every time: **thread safety** (lock, or eager creation when the class loads) and **global state** (data any code can reach and change) hurting tests; many teams inject one shared instance instead
Builder	Many optional fields; constructor getting long	Three required fields	`BookItem` with barcode, price, format, dates, rack: eight args are easy to swap (two adjacent strings compile fine); a builder sets fields **by name**. Readability fix, not structural; mention at ~six arguments and move on

One or two patterns suit a 45-minute design; six usually means patterns drive the design rather than requirements. Best of all is a **comparison**: "The rate gets Strategy because a second rate is likely; the ticket does not get State, since one guard is enough for three statuses."

Real-world examples

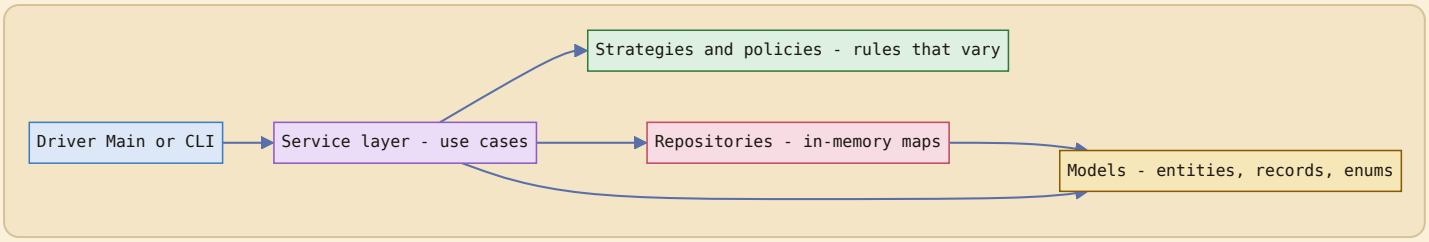
- **Strategy:** `java.util.Comparator` passed to `List.sort`; Spring Security's pluggable `PasswordEncoder` implementations.
- **Factory** (static factory methods hiding the concrete class): `Executors.newFixedThreadPool(...)` (a **thread pool** is a fixed set of reusable worker threads), `List.of(...)`, `Calendar.getInstance()`.
- **Observer:** Swing/AWT (Java's older desktop UI toolkits) listeners (`ActionListener`), `java.beans.PropertyChangeListener`, Spring `ApplicationEvent` listeners.
- **State:** e-commerce order lifecycle (placed → paid → shipped → delivered); TCP connection states (TCP: the protocol keeping a reliable connection between machines; see System Design Fundamentals).
- **Singleton:** `java.lang.Runtime.getRuntime()`; Spring beans (objects created and wired by **Spring**'s *container*) are singleton-scoped **per container** by default — not the GoF class-enforced singleton.
- **Builder:** `StringBuilder`, `java.net.http.HttpRequest.newBuilder()`, `Stream.builder()`.

TL;DR

Know six by name: Strategy, Factory, Observer, State, Singleton, Builder. Use one or two where their signal appears, and name the one you rejected.

1.10 Code, edge cases and concurrency: how to write interview code

How it works: a machine-coding skeleton that scales



1. **Models** — entities (`ParkingSpot`: identity, state changes over time), value objects (`record Vehicle`: plain data compared by value), enums (`VehicleType`). Encapsulate state; validate in constructors.
2. **Repositories** — only store and find objects: an in-memory `Map<Id, Entity>` behind a small interface, so a database could replace it.
3. **Strategies / policies** — every rule that might vary (pricing, spot selection, bill splitting).
4. **Services** — the use cases (`park`, `exit`, `bookSeats`): the **service interface** a client calls.
5. **Driver** — a `Main` running a scripted scenario (or reading commands) with clear demo output.

Habits interviewers notice:

- **Pass time in** (`long nowMinute`, `Instant now` — Java's point-in-time type), never read the clock inside logic: same input → same output → testable.
- **Money as** `BigDecimal` (built from strings: `new BigDecimal("3.50")`) or whole cents in a `long`; never `double`. This handbook uses `BigDecimal` in domain code.
- **Fail loudly**: `IllegalArgumentException` for bad arguments, `IllegalStateException` for wrong state, custom exceptions (`SpotNotAvailableException`) where clearer.
- **Return** `Optional` (a box holding a value or empty, instead of `null`) when "not found" is normal.
- **Fields** `private final` unless they must change; **domain names** (`ParkingTicket`, not `Data2`).

Edge cases and concurrency checklist

Category	Questions
Input	null (no object at all), empty, negative, duplicate IDs, unknown IDs
Capacity	Full lot, empty playlist, zero seats left, max loans reached
State	Paying twice, exiting without entering, cancelling a delivered order
Time	Exit before entry, bookings expiring, part hours (round up?)
Concurrency	Two threads booking the same seat or spot: `synchronized`, a `ReentrantLock`, or atomic structures such as `ConcurrentHashMap.computeIfAbsent`; lock the **smallest** shared thing that keeps the invariant

> **Key term — threads, locks and atomicity:** A **thread** is one independent line of execution; a server handling two panels at once runs two threads over the same objects. `synchronized` and `ReentrantLock` let only one thread at a time into a block. An **atomic** operation happens all at once from other threads' view, with no visible half-finished moment; `ConcurrentHashMap` is a thread-safe map offering such operations. An **invariant** is a rule about the data that must always hold ("one spot holds at most one vehicle").

Chapters 12–13 cover concurrency in depth.

TL;DR

Models → repositories → strategies → services → driver. Inject time, `BigDecimal` (or cents) for money, validate, return `Optional`, guard shared state.

1.11 Worked mini-example, end to end: a small parking lot

Every framework step on a deliberately small version (the full parking lot with panels, payments and boards is in the case-study chapters).

Problem & clarifying questions

Question	Answer (assumed)
Single lot or many?	Single lot, one or more floors
Which vehicles?	Motorbike, car, truck
Which spot types?	Motorbike, compact, large; a smaller vehicle may use a bigger spot
Payment?	Out of scope: we compute the fee; a payment service collects it
Pricing?	Hourly, tiered, part hours round up; trucks pay double; swappable (weekend flat rate)
Concurrency?	Several entry/exit panels may act at once

Requirements

Functional

1. The lot has floors; each floor has spots of type MOTORBIKE, COMPACT or LARGE.
2. A motorbike fits MOTORBIKE or COMPACT; a car fits COMPACT or LARGE; a truck fits LARGE only. The smallest fitting spot is tried first.
3. On entry, the system assigns a free fitting spot and issues a ticket (id, vehicle, spot, entry time).
4. If no fitting spot is free, entry is refused.
5. On exit, the ticket is closed, the spot freed and the fee returned.
6. Fee: $4.00 first hour, $3.50 for hours 2–3, $2.50 per hour after; part hours round up; minimum one hour; trucks pay double.
7. The pricing rule can be swapped at run time (e.g. a flat $10 weekend rate).
8. A floor can report how many spots of a type are free.

Non-functional: in-memory; thread-safe entry/exit; new pricing rules or vehicle types need no edits to existing services; deterministic (time passed in).

Out of scope: payment collection, reservations, display boards, number-plate recognition, multiple lots.

Use cases

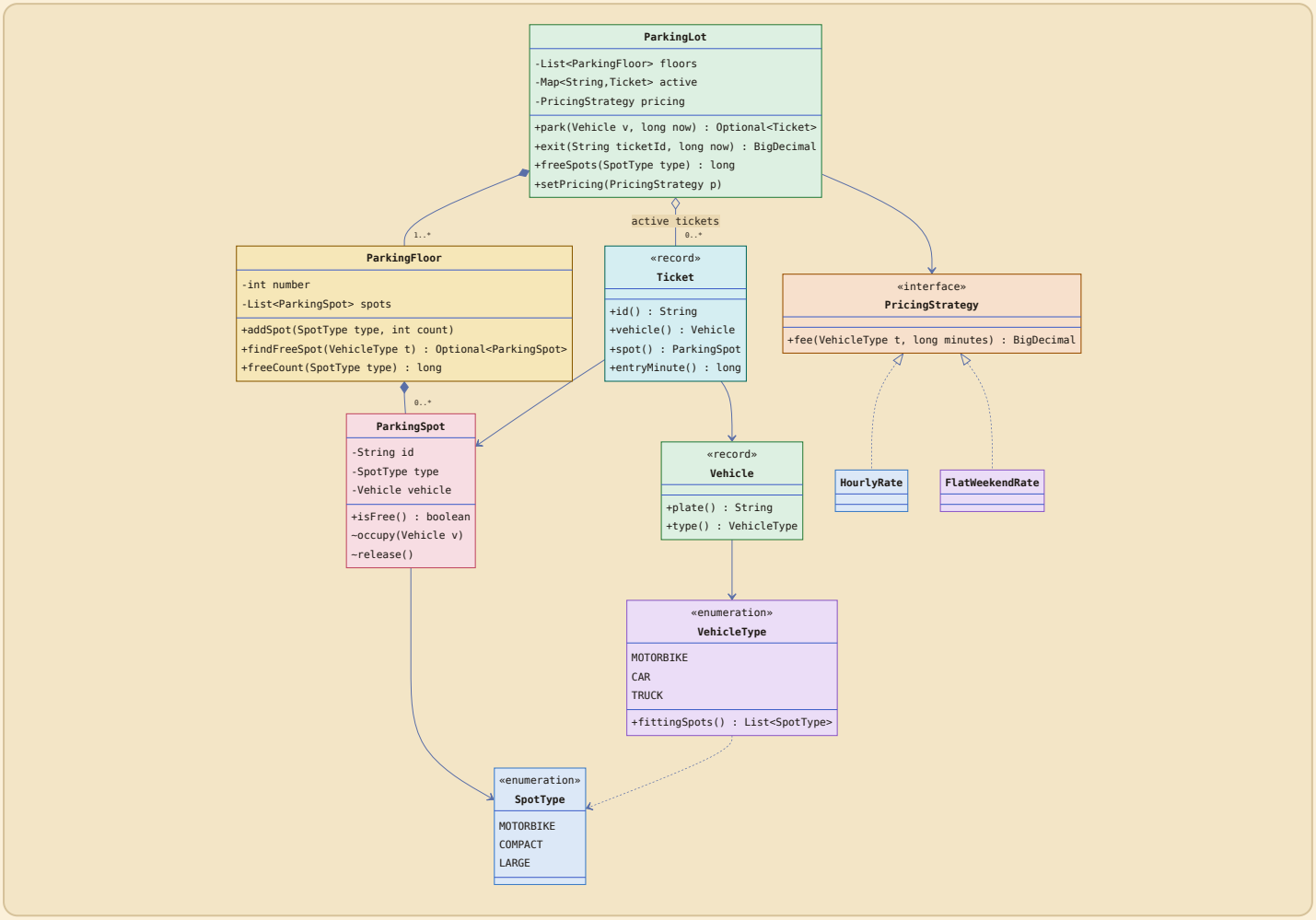
Actor	Use cases
Driver (customer)	enter and get ticket, exit and see fee
Admin	add floor/spots, change pricing rule
System	find free spot, refuse entry when full, compute fee

Core entities

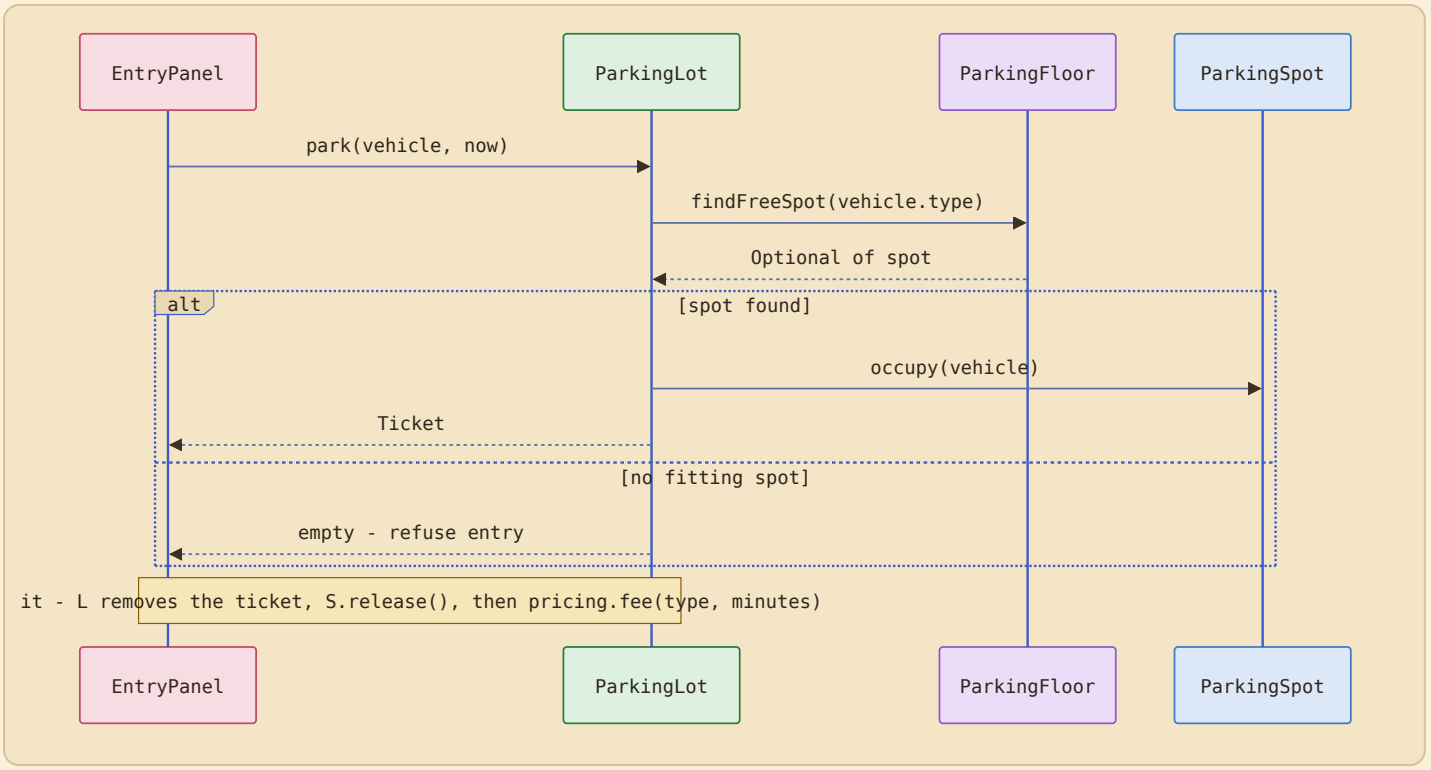
Class	Kind	Responsibility
`SpotType`	enum	The spot sizes
`VehicleType`	enum	Vehicle kinds, each knowing which spot types fit (smallest first)
`Vehicle`	record	Plate + type (pure data)
`ParkingSpot`	class	Tracks if it is occupied and by which vehicle
`ParkingFloor`	class	Holds spots on one floor and finds a free fitting one
`Ticket`	record	Which vehicle went into which spot, and when (the "connection object")
`PricingStrategy`	interface	Turns a stay into a fee (`BigDecimal`)
`HourlyRate`, `FlatWeekendRate`	classes	Two pricing rules
`ParkingLot`	class	Issues and closes tickets; the service the panels call

Spot sizes are an enum, not three subclasses: **different data → field**.

Class diagram



Key flows



Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for pricing (`PricingStrategy`)	A second rule (weekend) is required; new rules = new classes (open/closed)	A `switch` over a rate enum inside `ParkingLot`
Spot and vehicle types as **enums**	Differ in data only	One subclass per spot type
`Ticket` as a **record**	A ticket is a fact that never changes after issue. (Only *shallowly* immutable: fields cannot be reassigned, but the `ParkingSpot` it points to still changes)	Mutable ticket with setters (methods that change fields)
Money as `BigDecimal`	Exact: `3.50 + 3.50` is exactly `7.00`	`double` (binary fractions, rounding drift)
`synchronized` on `park` / `exit` / `freeSpots`	Two panels never get the same spot; a lot-wide lock is simple and correct at this size	Per-floor locks (better throughput — more requests per second — but more complex; an extension)
No Singleton for `ParkingLot`	Injecting one instance keeps it testable	Global `getInstance()`
No State pattern for tickets	Only active/closed; a map of active tickets is enough	State classes per ticket status

Java implementation

Essential lines below; full code in `code/lld/ch01/parkinglot/` (`SpotType` enum; `Vehicle(plate, type)` and `Ticket(id, vehicle, spot, entryMinute)` records; `PricingStrategy` interface with `BigDecimal fee(VehicleType type, long minutesParked)`; `FlatWeekendRate` returns a flat `10.00` — added later as a new class, no existing code edited; `ParkingFloor.findFreeSpot` tries each of the vehicle's `fittingSpots()` in order and returns the first free spot of that type, and `addSpot` names spots `"F" + floor + "-" + first letter of type + i`, e.g. `F0-C1`; `ParkingFloor` is not thread-safe itself — the lot calls it under its lock).

```java
// File: code/lld/ch01/parkinglot/VehicleType.java — each type knows which spots fit, smallest first
public enum VehicleType {
    MOTORBIKE(List.of(SpotType.MOTORBIKE, SpotType.COMPACT)),
    CAR(List.of(SpotType.COMPACT, SpotType.LARGE)),
    TRUCK(List.of(SpotType.LARGE));
    private final List<SpotType> fits;
    VehicleType(List<SpotType> fits) { this.fits = fits; }
    public List<SpotType> fittingSpots() { return fits; }
}

// File: code/lld/ch01/parkinglot/ParkingSpot.java (excerpt)
private Vehicle vehicle;                // null = free
public boolean isFree() { return vehicle == null; }
void occupy(Vehicle v) {                // package-private: only the lot may change a spot
    if (!isFree()) throw new IllegalStateException(id + " is taken");
    vehicle = v;
}
void release() { vehicle = null; }

// File: code/lld/ch01/parkinglot/HourlyRate.java — $4.00 first hour, $3.50 hours 2-3, $2.50 after; trucks x2
private static final BigDecimal FIRST_HOUR = new BigDecimal("4.00");
private static final BigDecimal HOURS_2_TO_3 = new BigDecimal("3.50");
private static final BigDecimal LATER_HOURS = new BigDecimal("2.50");
@Override
public BigDecimal fee(VehicleType type, long minutesParked) {
    long hours = Math.max(1, (minutesParked + 59) / 60);    // ceiling, minimum one hour
    BigDecimal total = BigDecimal.ZERO;
    for (long h = 1; h <= hours; h++) {
        total = total.add(h == 1 ? FIRST_HOUR : (h <= 3 ? HOURS_2_TO_3 : LATER_HOURS));
    }
    return type == VehicleType.TRUCK ? total.multiply(BigDecimal.TWO) : total;
}

// File: code/lld/ch01/parkinglot/ParkingLot.java — time PASSED IN; every method touching
// spots, tickets or pricing is synchronized, so two panels cannot hand out the same spot
public final class ParkingLot {
    private final List<ParkingFloor> floors;              // List.copyOf(floors) in the constructor
    private final Map<String, Ticket> active = new HashMap<>();
    private PricingStrategy pricing;
    private int nextTicket = 1;

    public synchronized Optional<Ticket> park(Vehicle v, long nowMinute) {
        for (ParkingFloor floor : floors) {
            Optional<ParkingSpot> spot = floor.findFreeSpot(v.type());
            if (spot.isPresent()) {
                spot.get().occupy(v);
                Ticket t = new Ticket("T" + nextTicket++, v, spot.get(), nowMinute);
                active.put(t.id(), t);
                return Optional.of(t);
            }
        }
        return Optional.empty();              // lot full for this vehicle type -> refuse entry
    }

    public synchronized BigDecimal exit(String ticketId, long nowMinute) {
        Ticket t = active.get(ticketId);
        if (t == null) throw new IllegalArgumentException("unknown or already closed ticket " + ticketId);
        if (nowMinute < t.entryMinute()) throw new IllegalArgumentException("exit before entry");
        active.remove(ticketId);               // validate first, then change state
        t.spot().release();
        return pricing.fee(t.vehicle().type(), nowMinute - t.entryMinute());
    }

    public synchronized long freeSpots(SpotType type) {    // same lock as park/exit
        return floors.stream().mapToLong(f -> f.freeCount(type)).sum();
    }
    public synchronized void setPricing(PricingStrategy pricing) { this.pricing = pricing; }
}

// File: code/lld/ch01/parkinglot/Main.java (scenario, abridged)
```

```
// one floor (floor 0) with 1 MOTORBIKE, 1 COMPACT, 1 LARGE spot; pricing = new HourlyRate()
// park car KA-01-1234 at minute 0, truck KA-02-9999 at 10, car KA-03-5555 at 20 (refused: prints orElse text)
// print lot.freeSpots(COMPACT); exit T1 at 150, T2 at 60
lot.setPricing(new FlatWeekendRate());          // swap the strategy at run time
// park KA-03-5555 at 200 (gets T3) and exit it at 500; then exit T1 again at 600 and print the caught error
```

Run: `java -cp /tmp/lld_01 lld.ch01.parkinglot.Main`

Output:

```
T1: KA-01-1234 -> spot F0-C1
T2: KA-02-9999 -> spot F0-L1
KA-03-5555 -> refused, no suitable spot
free compact spots: 0
T1 exits after 150 min, pays $11.00
T2 exits after 50 min, pays $8.00
T3: KA-03-5555 -> spot F0-C1, weekend exit pays $10.00
error: unknown or already closed ticket T1
```

Reading it: car → compact, truck → the only large spot, so the third car is refused (requirement 4). 150 min = 3 h = $4.00 + $3.50 + $3.50 = $11.00; the truck's 50 min round up to 1 h, doubled = $8.00; after the strategy swap the flat $10 applies; a double close is rejected.

Concurrency & edge cases

- `park`/`exit` are `synchronized` on the lot, so find-then-occupy is **atomic**: two panels cannot both see a spot free and both take it (a *check-then-act* **race condition** — a bug whose result depends on which thread runs first).
- `freeSpots` reads under the **same** lock, so a display never sees a half-finished `park`; `ParkingFloor` is only ever called through the lot.
- `exit` validates ticket and time **before** changing anything, so a rejected exit leaves the ticket open; exit before entry, unknown ticket and double exit all throw `IllegalArgumentException`.
- `ParkingSpot.occupy` also throws if taken (second line of defence); `occupy`/`release` are **package-private**, so outside code cannot change spot state.

Extensions & follow-up questions

Follow-up	Answer
"Add electric-vehicle spots with charging."	Add `ELECTRIC` to `SpotType` and an `ELECTRIC_CAR` vehicle type whose fitting list starts with `ELECTRIC`. If charging has **behaviour** (meter, charging fee), a `ChargingSpot` class or a charging component composed into the spot.
"Nearest spot to the entrance instead of first free."	Extract a `SpotSelectionStrategy` interface (Strategy again), injected into `ParkingLot`.
"Update display boards when spots change."	Observer: spots/floors publish "spot freed/occupied" events; boards subscribe.
"Improve throughput with many panels."	Lock per floor (or per spot type) instead of the whole lot; or a `ConcurrentLinkedQueue` (thread-safe queue) of free spots per type, `poll()`ed atomically.
"Add payment by cash or card."	`Payment` interface with `CashPayment`, `CardPayment`; `exit` returns the fee and a `PaymentService` collects it (dependency inversion).

In the interview

About 5 minutes on questions and requirements, 3 on actors, 8 on entities and the "enum vs subclass" decision, 8 on the diagram, 15 on `park`, `exit` and `HourlyRate`. Close by naming Strategy (used), Singleton and State (rejected, with reasons), and the per-floor locking extension.

Self-check

- **Q:** Why is `SpotType` an enum and not a class hierarchy? **A:** Spot types differ only in data (which vehicles fit), not behaviour.
- **Q:** What race does `synchronized` on `park` prevent? **A:** Two threads finding the same free spot and both occupying it (check-then-act).
- **Q:** How did the weekend rate respect open/closed? **A:** It was a new `PricingStrategy` class; nothing existing was edited.

TL;DR

Small, complete, honest: 8 requirements, 10 types, one Strategy, one lock, and a clear list of what was left out.

1.12 Communication tips

Do	Why
Think out loud, naming the step you are in ("Now I'll list actors")	Your process is scored, not only the result
Write things down where the interviewer sees them	Shared artefacts prevent misunderstandings
Confirm before moving on ("Does this scope look right?")	Cheap course corrections
State assumptions explicitly	Counts like an answer from the interviewer
Offer trade-offs in pairs: "X over Y because..."	Shows judgement
Welcome hints; treat the interviewer as a teammate	Collaboration is evaluated
Say what you leave out and how you would add it	Turns a gap into a strength
Keep a clock in view; announce the move to code	Prevents "no code at minute 40"
Use domain names, not `Manager`, `Helper`, `Data`	Clear names are clear thinking
When stuck, return to the requirements	Every class traces back to a line

Don't: go silent for minutes, argue with feedback, name `HashMap`s and threads in minute two (implementation before analysis and design), or recite pattern definitions without a reason to use them.

1.13 How to use this handbook, and a study plan

Attempt first, then compare

Reading case studies is pleasant but builds little skill: any finished design looks obvious. **Attempting each yourself, then comparing**, is slower and is what improves performance under pressure.

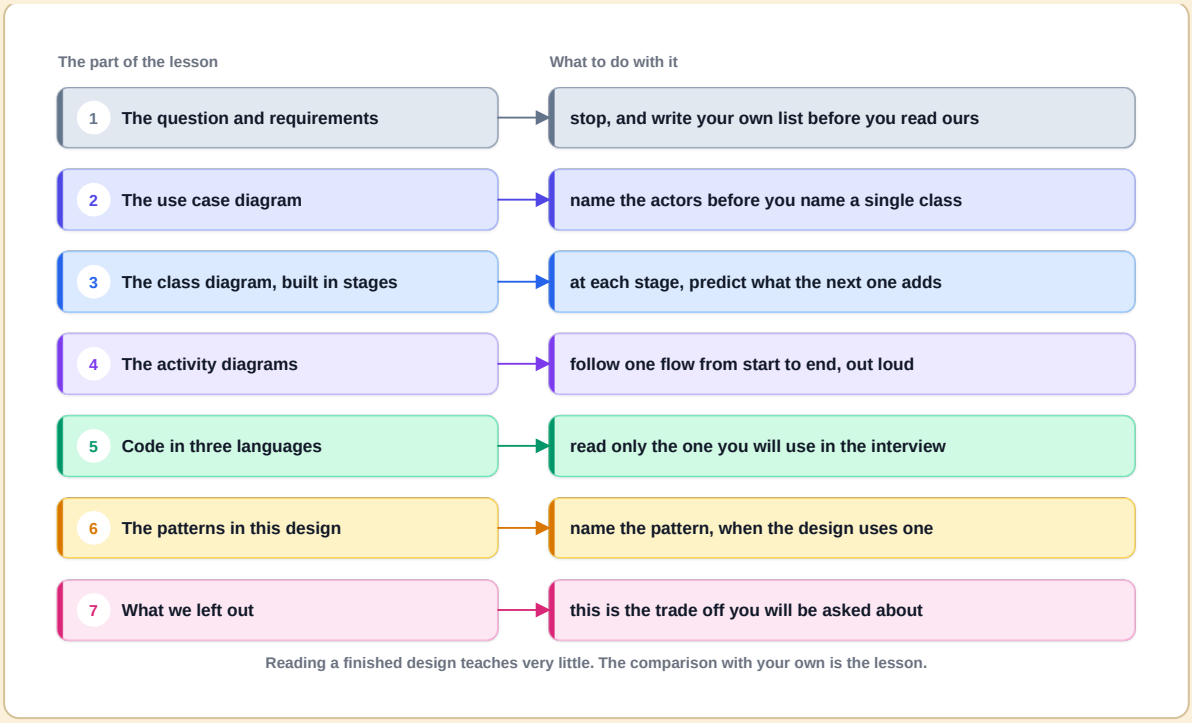


Figure — The parts of a case-study lesson and what to do at each one (course diagram)

Three stopping points matter most:

1. **After the question:** write your requirement list (8–12 lines + out of scope) and compare. A shorter list usually means **not asking enough questions**, not missing knowledge.
2. **After the requirements:** write your class list, then read the diagram. Most people are **two classes short — the record keeper** (ticket, loan, booking) and **the rule** (rate, strategy).
3. **At each stage of the class diagram:** predict the next stage; each wrong guess shows how an experienced designer thinks.

More habits: read diagrams in stages, looking symbols up in Chapter 3 as you meet them; if a decision looks wrong, write down why (several designs here exist because someone disagreed with an earlier version); read one language's code (Java) — translations teach nothing about design; **never skip "what we left out"** — interviewers ask exactly these, and "I merged these two responsibilities to save time; here is how I'd split them" beats hoping nobody asks; never invent pattern names. **After six or seven case studies, do timed runs** (a known question, 45 minutes, blank page) and measure two things: first ten minutes on requirements and actors? running code for at least one flow? They predict the result better than class count. **Compile and run every example** (`bash tools/compile_all.sh` runs all chapters; `bash` is the Unix shell, the `.sh` file a command script).

Map of this handbook

Ch	Title	Use it for
1	The LLD interview & how to approach it	The method, time budget, study plan (this chapter)
2	OOP fundamentals	Classes, objects, the four pillars, interfaces, enums, equality, immutability, generics, OOA/OOD (object-oriented analysis / design)
3	Relationships & UML	Association, aggregation, composition, dependency; class, use case, activity, sequence, state diagrams
4	SOLID principles	The five principles in depth
5	Design principles	DRY (don't repeat yourself), KISS (keep it simple), YAGNI (you aren't gonna need it), composition over inheritance, Law of Demeter, GRASP and friends
6	Clean code	Naming, functions, comments, error handling, tests
7	Refactoring	Code smells and the named refactorings that fix them
8	Creational patterns	Singleton, Factory Method, Abstract Factory, Builder, Prototype
9	Structural patterns	Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
10–11	Behavioural patterns	Strategy, Observer, State, Command, Template Method, Iterator, Chain of Responsibility, Mediator, Memento, Visitor, Interpreter; MVC (Model-View-Controller)
12	Concurrency concepts	Threads, locks, `java.util.concurrent`, thread-safety patterns
13	Concurrency problems	Producer–consumer, readers–writers, rate limiters and other classic problems
14	Case studies: everyday objects	Parking lot, vending machine, ATM, elevator, traffic signal, coffee machine
15–16	Case studies: booking and inventory	Library, movie tickets, concerts, hotels, car rental, restaurant, airline, course registration
17	Case studies: games	Chess, tic-tac-toe, snake and ladder, 2048, blackjack
18–19	Case studies: platforms	Social and Q&A sites, cricket scores, Amazon-style shopping, food delivery, auctions, stock brokerage and exchange, music streaming
20	Case studies: components	Caches (incl. LRU, evicting the least recently used item), logger, rate limiter (caps requests per time window), pub-sub (publishers send to all subscribers of a topic), notifications, Splitwise, ride sharing and similar
21	Case studies: machine coding	90-minute problems with runnable solutions: task manager, digital wallet, event calendar, coding contest platform, property listing, loan ledger, inventory
22	Cheat sheets & glossary	Last-day revision

Study plans

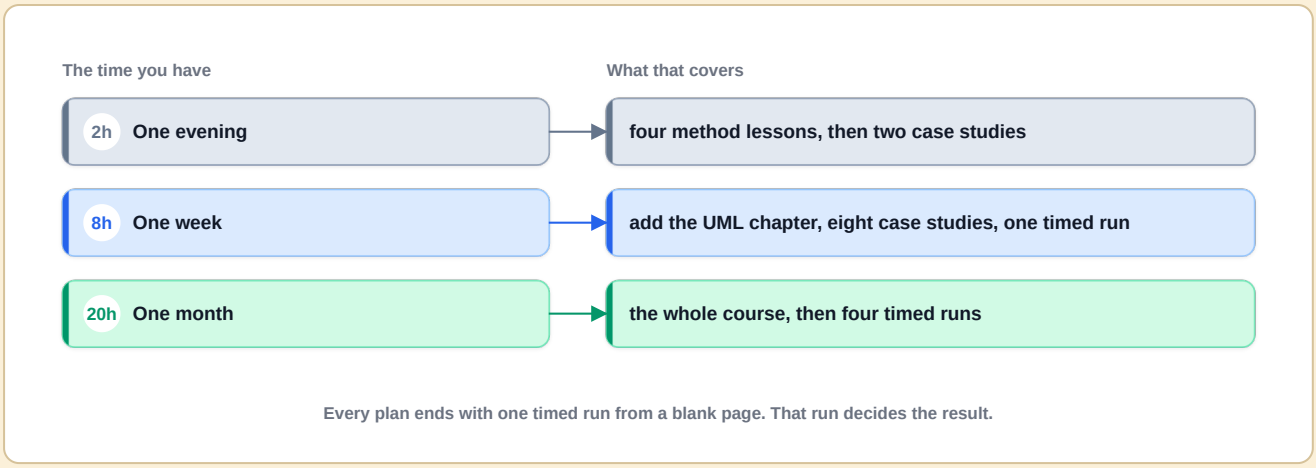


Figure — Three study plans for three amounts of time; every plan ends with one timed run from a blank page (course diagram)

Choose by the time your **calendar** really has. **The method is worth more than any single case study**, and the first six case studies more than the rest (later ones repeat earlier skills).

Plan	Do
Interview tomorrow (~2 h)	§1.2 time budget (10 min) → §1.5 requirements (15) → §1.6 finding classes (15) → Ch 3 choosing the relationship between two classes (15) → Ch 3 class diagram notation (10) → parking lot case study, whole (30) → vending machine case study, whole (25). The parking lot is the most-asked question; the vending machine shows a **lifecycle** (fixed states an object moves through), the second most common kind. Do not start a third; write the five steps on a card and rehearse them
One week (~8 h, an hour a day)	Day 1: Ch 1 fully + Ch 2's four pillars. Day 2: Ch 3 use case, class and activity diagrams (sequence later). Day 3: parking lot, then library management (your requirements first). Day 4: ATM, vending machine. Day 5: movie ticket booking, hotel management. Day 6: Splitwise, Amazon-style shopping. **Day 7: no new material** — redo the parking lot from a blank page in 45 minutes; it matters most, checking you can design against the clock
One month (~20 h)	Week 1: Ch 1–4 + first three case studies. Week 2: Ch 5, 8–11 (patterns) with case studies 4–10, two per session, always drawing your classes first. Week 3: Ch 12–13 (concurrency) + case studies 11–18; one timed run. Week 4: remaining case studies, Ch 21 machine-coding problems, three timed runs on questions you have read; Ch 22 for revision. When case studies feel repetitive by week three, the method is learned: read faster, checking your class list against the first diagram stages
Beginner (six weeks, 5–6 h/week) — OOP itself new	W1: Ch 1–2, writing and running every Ch 2 example. W2: Ch 3–5 (UML, SOLID, principles). W3: Ch 6–7 (clean code, refactoring) + Ch 8. W4: Ch 9–11 (remaining patterns) + two small case studies. W5: Ch 12–13 + four case studies incl. one with concurrency (ticket booking). W6: Ch 21 problems in 90-minute timed runs; Ch 22

If you can only do six case studies:

Case study	What it teaches
Parking lot	The canonical question, and how to bound one
Library management	Borrow, return and limits, a common family
Vending machine	A lifecycle, one class per state
Movie ticket booking	Two people wanting the same seat: the concurrency question
Splitwise	Money, and a rule with several versions
Amazon online shopping	A question too big to finish, and choosing the part you build

Add a seventh, **an in-memory cache (LRU)**, if target companies ask for a data structure rather than a product.

The night before: no new question (a half-understood design adds anxiety and will not stick). Re-read the time budget (knowing you are *allowed* eight minutes on requirements stops rushing), re-read one done case study's extensions, and write the five steps from memory: **requirements, actors and use cases, classes, relationships, code.**

1.14 Common mistakes

Mistake	Why it hurts	Fix
Coding in minute two	Missing requirements appear at minute 35	8–13 minutes on requirements and actors
No out-of-scope list	Scope creeps; omissions look like oversights	Write it down and agree it
Agreeing to every suggested feature	Nothing gets finished	"I'll leave X out and add it if time allows"
One subclass per value	Class explosion with no behaviour	Different data → field/enum
God class (`ParkingLotManager` does everything)	Fails single responsibility; untestable	One sentence per class, split on "and"
Missing the record class (ticket/loan/booking)	Data on the wrong class (`dueDate` on `Book`)	A connection that carries data = an object
Inheritance for reuse ("Car extends Engine")	Wrong "is-a"; fragile	Composition
Pattern stuffing	Looks like memorisation, not judgement	One or two patterns, with the signal
`switch` on type everywhere	Every new type edits many places	Polymorphism (one call, different behaviour per class) / Strategy
All twelve classes at equal depth	No flow works end to end	Two or three flows fully
Silent designing	Interviewer cannot credit reasoning	Narrate each step
Ignoring concurrency on a shared resource	Double booking	Lock or atomic operation on the shared state
Non-running code in machine coding	Often an automatic reject	Thin end-to-end path running by halfway
`double` for money, clock read inside logic	Rounding bugs, flaky demo	Cents/`BigDecimal`, inject time

---

In the interview

30-second answer ("How do you approach a low-level design problem?"): "First I clarify and write 8 to 12 requirements with an explicit out-of-scope list. Then I list the actors and their use cases. From the nouns I pick the classes, cutting anything that is only a value, and give each class one responsibility. I connect them with the right relationships and draw a class diagram, sketch the key flow, and run a quick SOLID check and pick at most one or two patterns where the signal is clear. Then I code the two or three core flows, handle edge cases and concurrency, and finish by discussing extensions and trade-offs."

- *"How is this different from system design?"* — System design is many machines and scale (databases, caches, queues); LLD stays inside one application: classes, interfaces, relationships, code.
- *"You won't finish; what matters?"* — Depth on the chosen part: bounded scope, clean responsibilities, correct relationships, code matching the diagram, one defensible trade-off.
- *"What changes in machine coding?"* — The code must run: in-memory storage, service interfaces and a driver, happy path by halfway, then features, validation and one extension point.
- *"How do you decide whether to use a pattern?"* — By its signal: interchangeable rules → Strategy, many-transition lifecycle → State, many listeners → Observer. No signal → plain classes.

Self-check

- **Q:** The five core steps in order? **A:** Requirements, actors & use cases, classes, relationships, code.
- **Q:** How much of a 45-minute round goes before naming a class? **A:** Roughly a quarter (8–13 minutes).
- **Q:** The five things an interviewer judges? **A:** Bounded problem, one-sentence responsibilities, correct relationships, code matching the diagram, an explained trade-off.
- **Q:** What does machine coding add? **A:** Working code in about 90 minutes, data models, service interfaces, extensibility, a demo.
- **Q:** Which two classes do beginners most often miss? **A:** The record-keeping class (ticket/loan/booking) and the rule class (rate/strategy).
- **Q:** "Different data" vs "different behaviour"? **A:** Data → field or enum; behaviour → separate class.
- **Q:** How many patterns in a 45-minute design? **A:** One or two, where the signal appears; name a rejected one too.
- **Q:** The last day of any study plan? **A:** A timed run from a blank page.

TL;DR

- LLD/OOD = design the classes inside one application and code the main flows; machine coding = the same with running code in ~90 minutes.
- Clarify → requirements → use cases → entities → relationships → diagram → flows → patterns/SOLID → code → edge cases/concurrency → extensions.
- A quarter of the time before naming a class; write an out-of-scope list.
- Different data → field; different behaviour → class; one sentence per class.
- SOLID as five quick checks; one or two patterns where the signal appears.
- Code two or three flows properly; inject time, `BigDecimal` (or cents) for money, guard shared state.
- Attempt first, then compare; finish every plan with a timed blank-page run.

Chapter 14 — Case Studies: Everyday Objects

> **Where this fits.** Chapters 1–13 gave you the tools: OOP (object-oriented programming: objects that bundle data with the methods acting on it), UML (Unified Modeling Language, the standard way to draw classes and their links), SOLID (five class-design principles), clean code, the 23 GoF patterns (the "Gang of Four" patterns, after the four authors of the 1994 book *Design Patterns*) and concurrency (several pieces of work running at once). Chapters 14–21 apply them to real interview questions, starting here with **physical machines you have used**: a parking lot, a vending machine, a coffee machine, an ATM, an elevator, a traffic light and a parcel locker. You already know how they behave, so all your attention goes to the design.

What you will learn in this chapter

#	Problem	The one idea the interviewer is really testing	Main patterns
14.1	Parking Lot	**Resource allocation** under concurrency: two cars must never get the same spot	Strategy (spot finding, pricing), Factory, Observer (display boards)
14.2	Vending Machine	A **state machine**: the order in which actions are allowed	State
14.3	Coffee Vending Machine	**Atomic, all-or-nothing** deduction from a shared inventory; recipes as data	Observer (low-stock alerts), data-driven recipes, sealed result types
14.4	ATM	**Security + money correctness**: authenticate, never lose money, never double-spend	State, Chain of Responsibility (cash dispenser), Command (transactions)
14.5	Elevator System	**Scheduling** (SCAN/LOOK: sweep orders for visiting stops, see 14.5) and **dispatching** many cars	Strategy (dispatch), State-like modes
14.6	Traffic Signal Control	**Timed state machine** with a hard safety invariant and emergency override	State, Strategy (timing), Observer (safety monitor)
14.7	Shipping Locker System (Amazon Locker)	**Best-fit allocation + deadlines + a secret**: reserve a door atomically, expire holds and uncollected parcels, single-use hashed pickup codes	Strategy (door choice), Observer (SMS), enum state machines, sealed results

> **Key terms — the pattern names in this table.** These are design patterns (proven, named shapes for arranging classes) from the earlier chapters. **Strategy**: a swappable rule (how to price, which car to send) hidden behind an interface. **Factory**: one place that creates objects, so callers need not know how they are built. **Observer**: an object keeps a list of listeners and tells each of them when something changes. **State**: each situation a machine can be in is its own class. **Chain of Responsibility**: a request is passed along a line of handlers, each doing what it can. **Command**: an action wrapped in an object, so it can be run, logged or queued. **Sealed result types**: a Java type whose possible variants form a fixed, known list (for example "served" or "refused"), so the compiler can check that callers handle each one.

What these problems test

These seven questions check three skills:

1. **State machines.** A machine's behaviour depends on what has happened so far: a vending machine must not dispense before it is paid, an ATM must not allow a withdrawal before the PIN is checked, a traffic light must never go from green straight to green on the crossing road. A **state machine** is a

fixed set of situations (states) plus rules for moving between them (transitions); the **State pattern** (Chapter 11) makes each state a class, so every button has a defined answer in every state.

2. **Resource allocation.** A limited set of things (parking spots, notes in a cassette, millilitres of milk, elevator cars, locker doors) handed out correctly: never twice, never below zero, and fairly.
3. **Concurrency.** Many actors at once — several entrance gates, two ATMs on one joint account, ten customers tapping "Latte", a timer thread and an emergency thread (a **thread** is one independent line of execution inside a program; many can run at once and touch the same objects). The design must stay correct when calls overlap (Chapters 12–13).

> **Key insight.** The hardest part of each problem is small: one method that picks a spot, makes change, deducts ingredients or picks the next elevator stop. Find it early, name it out loud, and spend your time there.

How each case study is laid out

Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns → Java implementation → Concurrency & edge cases → Extensions & follow-ups → In the interview → TL;DR. It is also a good order to speak in during an interview.

How to read the course diagrams

The course draws class diagrams in stages: classes from earlier stages in grey, new ones in colour. The legend is the same for all four course case studies in this chapter.

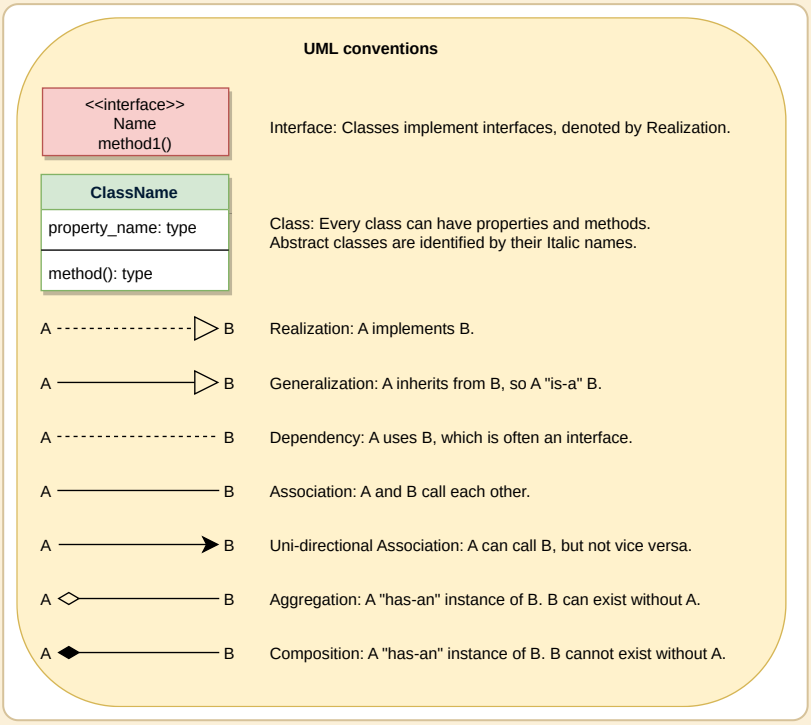


Figure — Legend for the course class diagrams: interface, class, realization, association, inheritance, composition and the other arrow kinds (course diagram)

Hollow triangle = inheritance ("is a"); **dashed line + hollow triangle** = implements an interface (an **interface** lists methods a class promises to provide, without saying how); **filled diamond** = composition (the part dies with the whole); **plain arrow** = association (refers to without owning). Chapter 3 covers these in depth.

How to run every example in this chapter

```
cd chapters/14-case-everyday-objects/code
javac --release 21 -d /tmp/lld_14 $(find . -name '*.java')
java -cp /tmp/lld_14 lld.ch14.parkinglot.Main      # or vendingmachine, coffeemachine, atm, elevator, trafficsignal, locker
```

`javac` compiles `.java` sources to bytecode (`--release 21` targets Java 21, `-d` is the output folder); `java -cp` runs a class from the given **classpath** (where the JVM — the Java Virtual Machine that runs bytecode — looks for classes). All seven `Main` classes compile with Java 21 (standard library only) and run; the output under each implementation is real. Listings below keep the essential classes and methods — the full files (imports, getters, `Main` demos) are in `code/lld/ch14/<case>/`.

> **Code-style note.** A file sometimes holds several small package-private types (declared without `public`, visible only in their package) — e.g. `Vehicles.java` holds `SpotType`, `VehicleType` and `Vehicle`. That is legal Java and compiles with no warnings under `-Xlint:all` (every optional compiler *lint* check) except `auxiliaryclass`, which exists precisely to flag this style. In production each top-level type would get its own file.

14.1 Design a Parking Lot



Figure — A large multi-level car park, the kind of building this design models (course diagram)

A **parking lot** (car park) is an area set aside for parking vehicles; malls, stadiums, airports and office towers have large multi-floor lots with gates, ticket machines and boards showing free spaces — many floors, many spot sizes, several ways in and out.

This is the most-asked LLD question (low-level design: the classes, their responsibilities and how they talk to each other). It looks easy, which is why it is useful: the interviewer watches how you **scope** it, how you **model the fit rule** (which vehicle may use which spot), and whether you notice that **two gates can hand out the same spot** at the same moment.

Problem & clarifying questions

Ask these before drawing anything. The answers in the right column are the ones this design assumes.

Question	Assumed answer
One lot or a chain of lots?	One lot, many floors.
How many entrances and exits?	Several of each, all working at the same time.
Which vehicle types?	Car, truck, van, motorcycle, electric car.
Which spot types?	Compact, large, handicapped (accessible), motorcycle, electric.
How do customers pay?	Cash or card, at the exit panel, to an attendant, or earlier at an info portal on each floor.
How is the fee computed?	Per hour, with tiers: $4 for the first hour, $3.50 for hours 2 and 3, $2.50 for each hour after. A started hour counts as a full hour.
Do EVs charge?	Yes: electric spots have a panel where the driver pays for charging.
Reservations, monthly passes, number-plate recognition?	Out of scope (mentioned as extensions).
What happens when the lot is full?	The entrance panel and the ground-floor board show a "full" message; no more vehicles are admitted.

Requirements

Functional

1. The lot has **multiple floors**.
2. It has **multiple entry and exit points**.
3. A customer **collects a ticket at the entrance** and **pays at the exit**.
4. Payment at the **automated exit panel** or to a **parking attendant**.
5. Payment by **cash or credit card**.
6. Payment is also possible at a **customer info portal on each floor**; a ticket paid there is not charged again at the exit.
7. The lot **never admits more vehicles than its capacity**. When full, a message shows on the entrance panel and on the ground-floor display board.
8. Each floor has many spots of several **types**: compact, large, handicapped, motorcycle (and electric, below).
9. Some spots are for **electric cars**; each has an **electric panel** where the customer can pay for and charge the vehicle.
10. Many **vehicle types**: car, truck, van, motorcycle (and electric car).
11. Each floor has a **display board** showing free spots per spot type.
12. A **per-hour fee model** with tiers ($4 / $3.50 / $3.50 / $2.50 …).

Non-functional

- **Correctness under concurrency:** two entrance gates working at the same instant must never be given the same spot, and a ticket must never be paid twice.
- **Low latency at the gate:** finding a spot must not scan thousands of spots; taking one from a sorted set of free spots is O(log n). (**Latency** is how long one request takes from start to finish; see System Design Fundamentals. **O(log n)** is Big-O notation for cost: the work grows with the logarithm of the number of spots n, so even 10,000 spots need only about 14 steps.)
- **Extensibility:** a new pricing rule, a new spot-finding rule or a new vehicle type should be a small, local change.
- **Testability** (how easily automated tests can check the code)**:** time must be injectable (passed in from outside rather than read inside the code, here as a fake clock), so fee tests do not wait for hours.

Use cases

Actors (an actor is anyone or anything outside the system that uses it):

- **Admin** — adds, removes and edits floors, spots, entrance/exit panels; adds and removes attendants; sets the hourly rate.
- **Customer** — takes a ticket, pays, leaves.
- **Parking attendant** — can do everything a customer can, on the customer's behalf, and can accept cash.
- **System** — shows messages on panels and boards, assigns a vehicle to a spot and frees it again.

Top use cases: add/remove/edit a floor (each floor with its own display board); add/remove/edit a spot; add/remove an attendant; **take ticket**; **scan ticket** (to find the total charge); **credit-card payment**; **cash payment**; add/modify the parking rate.

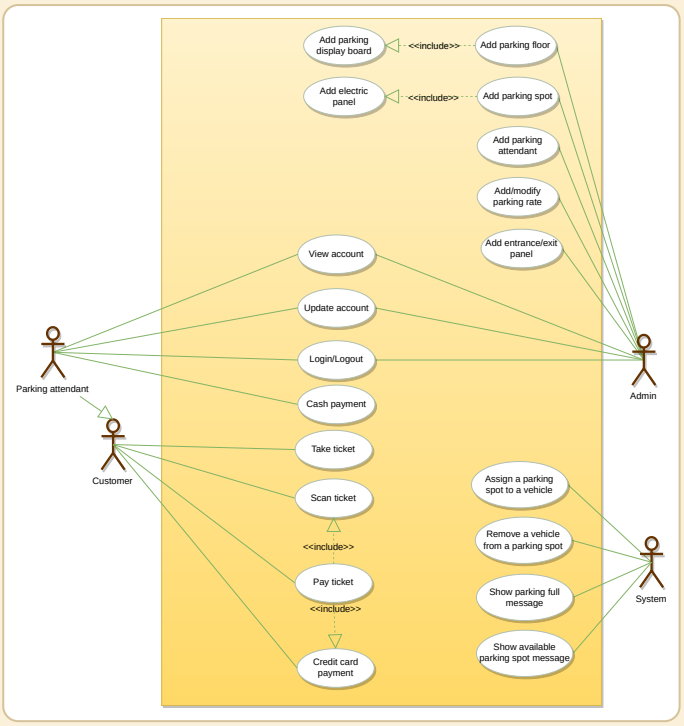


Figure — Use case diagram of the parking lot: Admin, Customer, Parking attendant and System, with their use cases (course diagram)

In the figure above, notice that the attendant's use cases overlap the customer's: the attendant is a customer "proxy" who can also take cash. The System actor is what assigns spots and updates the boards.

Core entities

Class	Responsibility
`VehicleType` (enum)	The kinds of vehicle and, for each, the spot types it fits in, **best fit first**.
`SpotType` (enum)	The kinds of spot: motorcycle, compact, large, handicapped, electric.
`Vehicle` (record)	Plate, type, accessibility permit. Knows its preferred spot types.
`ParkingSpot`	One physical spot: id, floor, type, and who occupies it (atomic).
`ChargingPanel`	The pay-and-charge panel owned by an electric spot.
`SpotFactory`	Builds spots; the only place that knows an electric spot needs a panel.
`ParkingFloor`	Holds the **free** spots per type in a thread-safe sorted set; notifies boards.
`DisplayBoard`	Shows free spots per type on one floor (an observer).
`SpotAllocationStrategy`	*How* to choose a spot (nearest floor first, least busy floor first, …).
`PricingStrategy`	*How much* a stay costs (hourly tiers, flat rate, weekend rate, …).
`Ticket`	Proof of entry: vehicle, the actual spot, entry time, status (ACTIVE → PAID → EXITED).
`PaymentMethod`	Cash or card.
`ParkingLot`	Ties it together: admit, price, pay, exit.
`EntrancePanel`, `ExitPanel`, `CustomerInfoPortal`	The devices customers touch. They hold **no rules**; they call the lot.

In the table, an **enum** is a Java type with a fixed list of named constants (such as `SpotType.COMPACT`), and a **record** is a short Java class that only carries a few values and cannot be changed after it is created. **Thread-safe** means an object stays correct even when several threads use it at the same moment.

The course also models **`Account`** with **`Admin`** and **`ParkingAttendant`** subclasses (both sign in). They carry no parking logic, so they are omitted from the code below; the attendant simply uses the same `pay` method as an exit panel.

Class diagram

The course grows the design in five steps. Every step adds a handful of classes and leaves you with a feature that runs. Use the same order in an interview: if the clock stops you early, what you have drawn already works, instead of being an unfinished picture.

Stage	What it adds	What works after it
1	Vehicle and spot types	The system can answer "may this vehicle use this spot?"
2	Floors and display boards	A floor can hand out a free spot and report counts
3	Ticket, rate card, payment	A stay can be turned into a price and settled
4	The `ParkingLot`	A full visit works: enter, park, pay, leave
5	Entrance/exit panels, info portal, electric panel	Drivers use the flow through physical devices

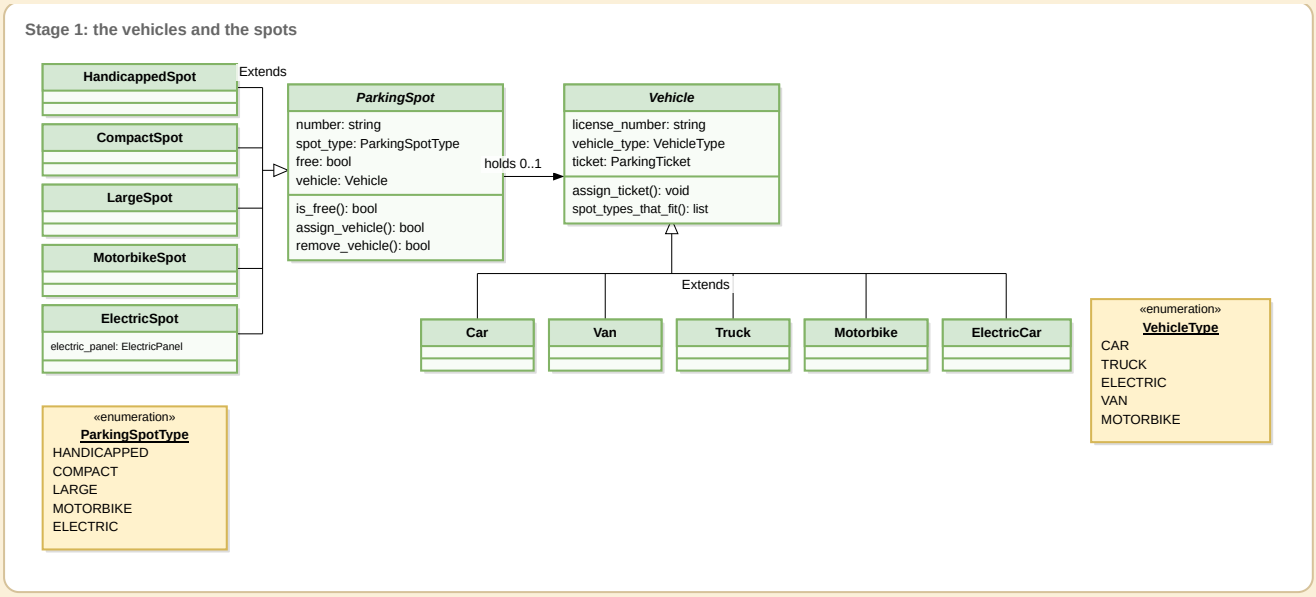


Figure — Stage 1: the vehicle and spot hierarchies, and the "holds 0..1" link from a spot to a vehicle (course diagram)

Stage 1 — vehicles and spots. Requirements 8 and 10: two small families of types, and a spot holds **at most one** vehicle (`0..1`). The most important rule — *which spot types can this vehicle use?* — lives on the **vehicle** side, so a new kind of vehicle changes only the vehicle types, never the lot. Fit rules: a motorcycle uses a motorcycle spot, trucks and vans only large spots, and an electric car tries electric, then compact, then large.

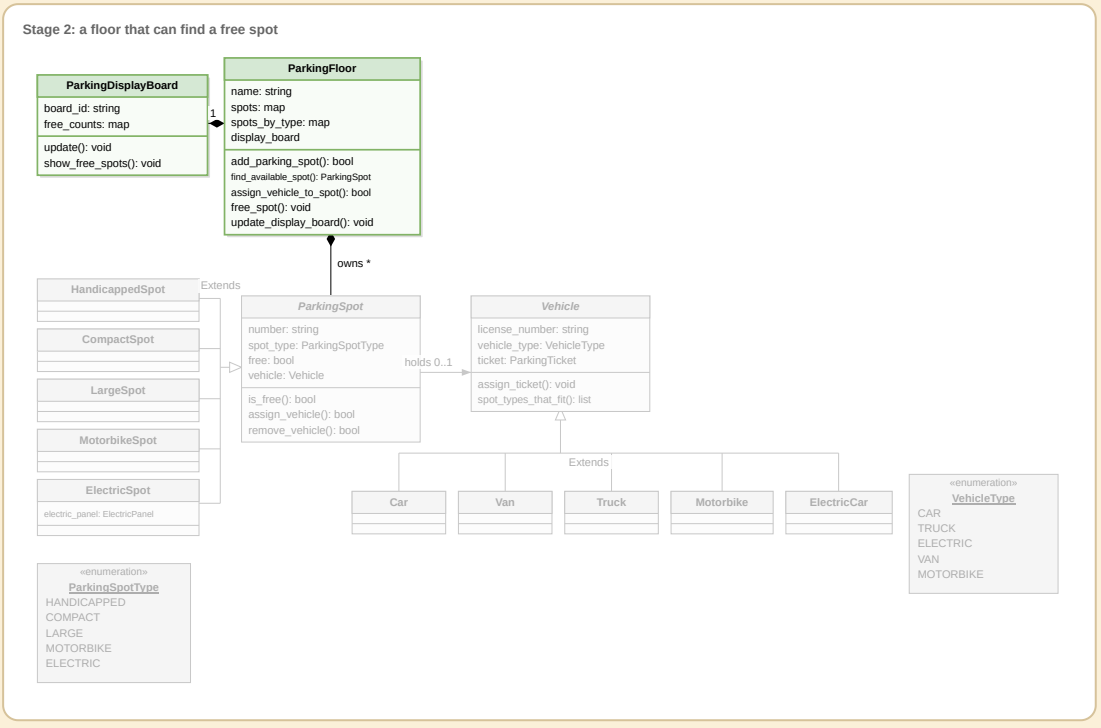


Figure — Stage 2: a floor owns its spots and its display board (composition) (course diagram)

Stage 2 — a floor that can find a free spot. Requirements 1, 8 and 11. Both links are **composition** (filled diamond): deleting a floor deletes its spots and board. The board stores **how many** spots of each type are free, not a pointer to "a sample free spot", which goes stale the moment that spot is taken. The course's floor keeps a map of all spots plus an index by type (our code keeps one map from spot

type to its free spots). Rejected: one separate collection per type — every method would need a five-way `if`, and a sixth type would touch all of them.

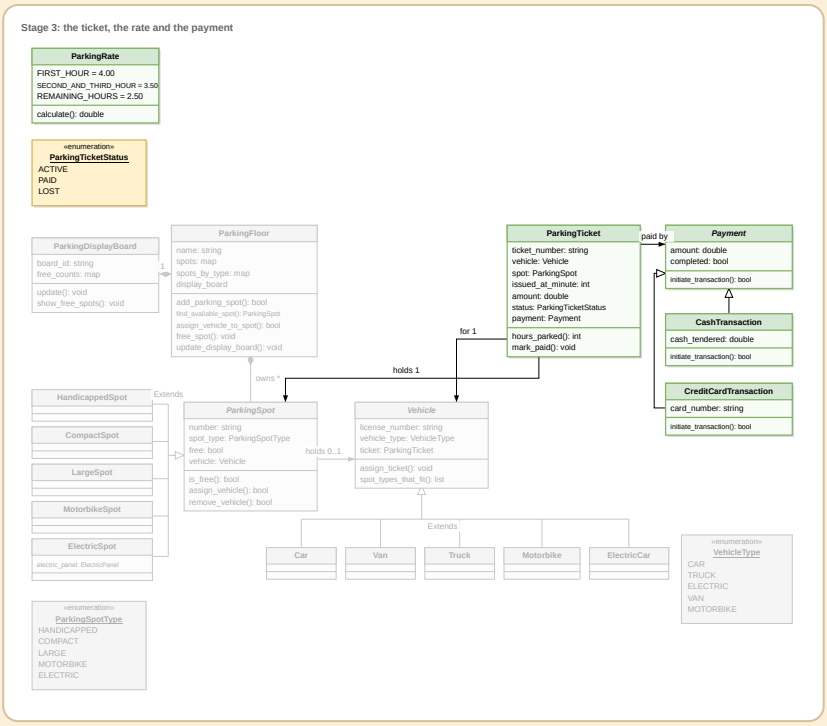


Figure — Stage 3: ticket, parking rate and the abstract payment with cash and card subclasses (course diagram)

Stage 3 — ticket, rate, payment. Requirements 3, 5 and 12. Key decision: **the ticket stores a reference to the exact spot**, not just "one more spot is taken" — at the exit, that is how the system knows which spot to free. The ticket only points at vehicle and spot (plain associations). `Payment` is abstract (cannot be created directly) with cash and card subclasses — justified because either can stand in wherever "a payment" is expected (Liskov Substitution, Chapter 4).

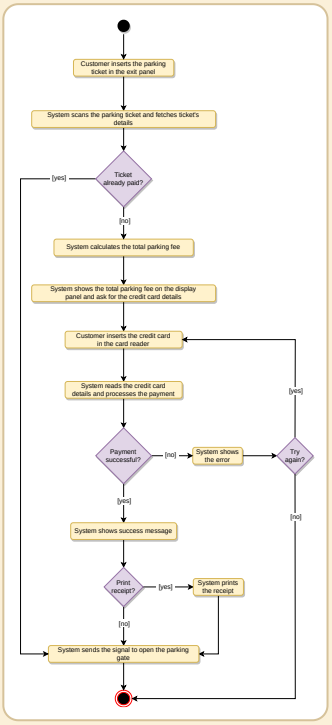


Figure — Activity diagram: the customer inserts the ticket at the exit, the system computes the fee, the customer pays by card or cash, and the barrier opens (course diagram)

The activity diagram above is the exit flow; the ticket, rate and payment classes between them handle each of its steps.

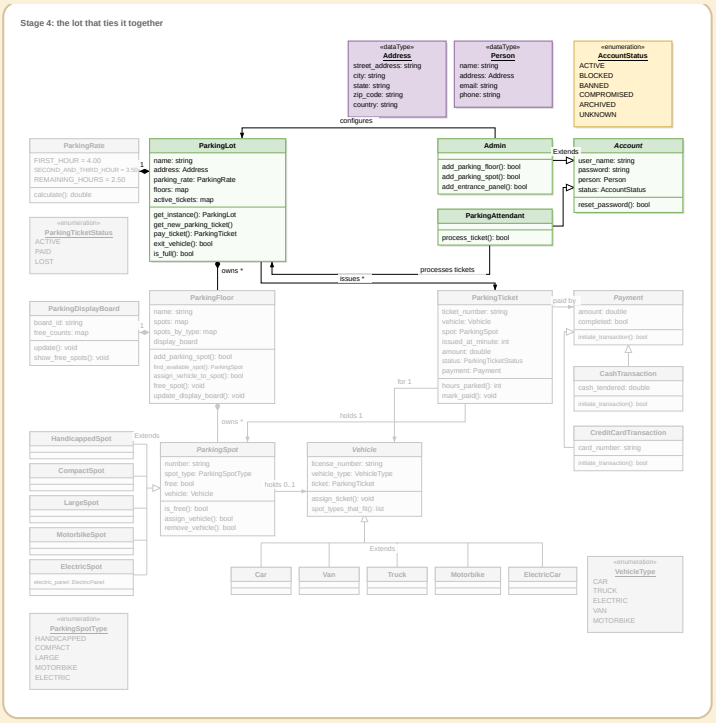


Figure — Stage 4: the parking lot owns floors and the rate; Admin and ParkingAttendant accounts connect to it (course diagram)

Stage 4 — the lot. Requirement 7 and the actors. The lot has a **name** and **address** and owns its floors and rate card (filled diamonds). `Admin` and `ParkingAttendant` are plain arrows: they work *on* the lot without owning any part of it. The course makes the lot a **Singleton** (only one instance may exist) and serialises spot assignment (**one caller at a time**), since two entrances assigning in parallel could pick the same spot.

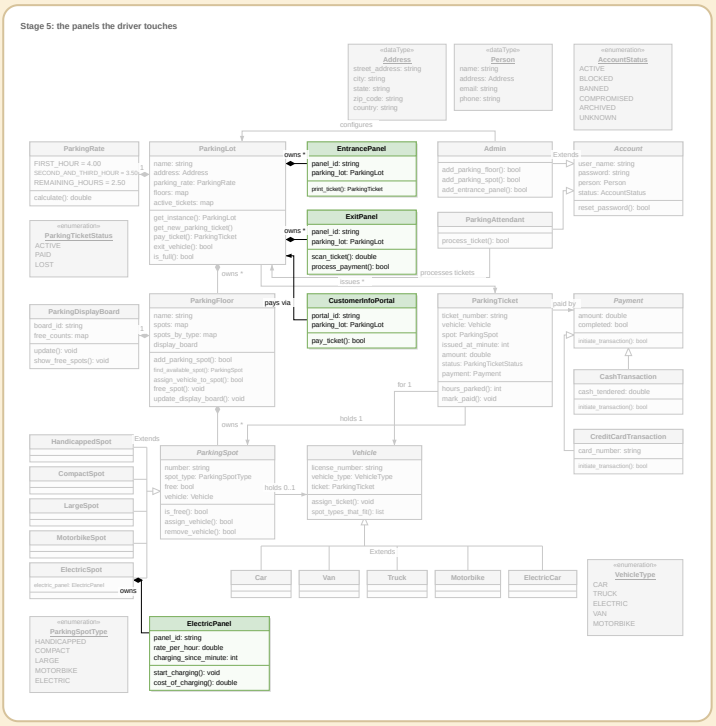


Figure — Stage 5: entrance and exit panels (owned by the lot), the customer info portal (refers to the lot) and the electric panel (owned by an electric spot) (course diagram)

Stage 5 — the panels. Requirements 2, 4, 6 and 9. Panels are thin — they forward every request to the lot, so all rules sit in one class. The entrance panel prints tickets, the exit panel scans and takes payment, the info portal settles early (marking the ticket paid), the electric panel bills

charging. The lot owns its entrance/exit panels (composition) but keeps no list of portals (a portal just refers to the lot); the electric panel is owned by its electric spot.

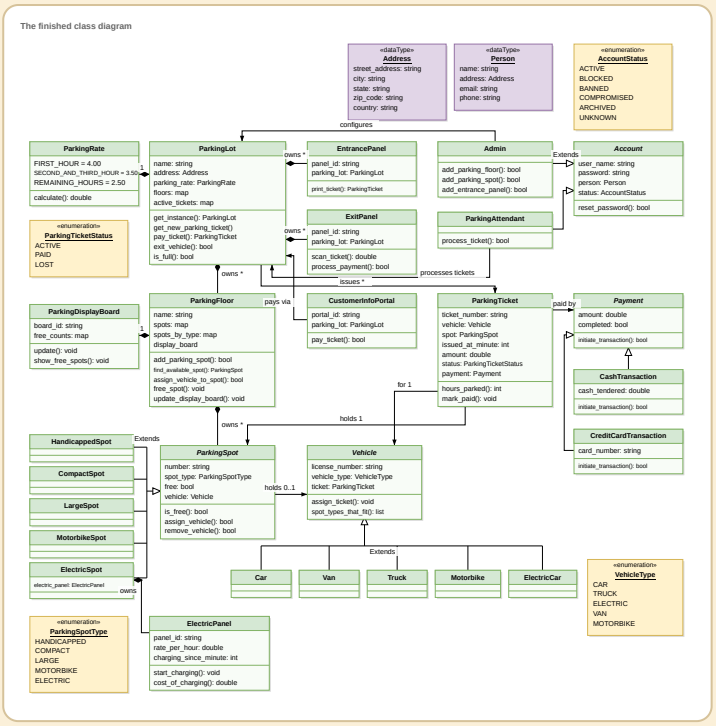
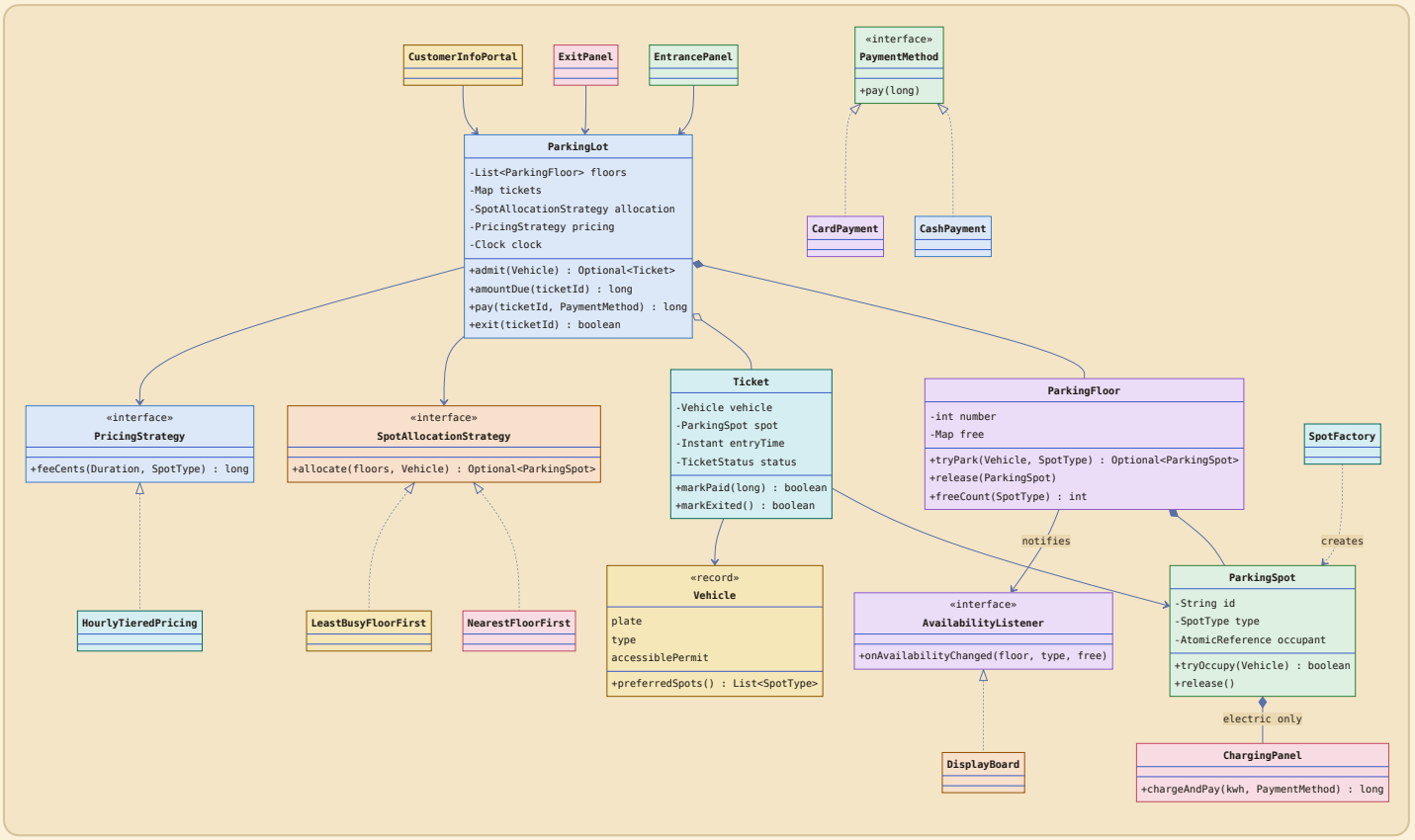


Figure — The finished class diagram of the parking lot (course diagram)

The finished diagram above is simply all five stages in colour; nothing new appears in it. Here is the class diagram of **the Java code in this chapter**. It keeps the same shape but makes three changes that interviewers usually push for: spot types are an enum instead of five empty subclasses, pricing and spot-finding are Strategies, and the display board is an Observer.

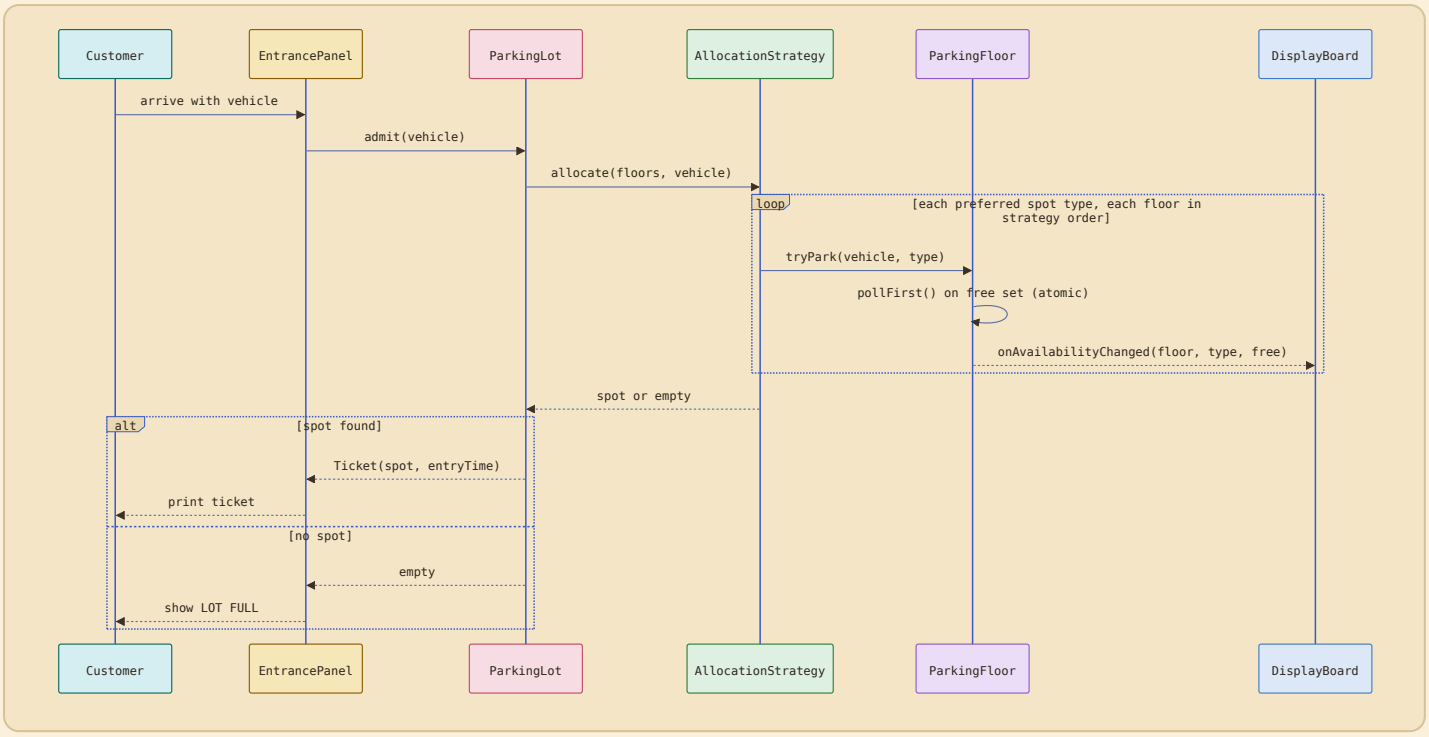


In this Mermaid diagram (Mermaid is a text format that renders as a diagram), `-` marks a private field (hidden inside its class), `+` a public method, `~T~` a generic type such as `List<ParkingFloor>`, `*--`

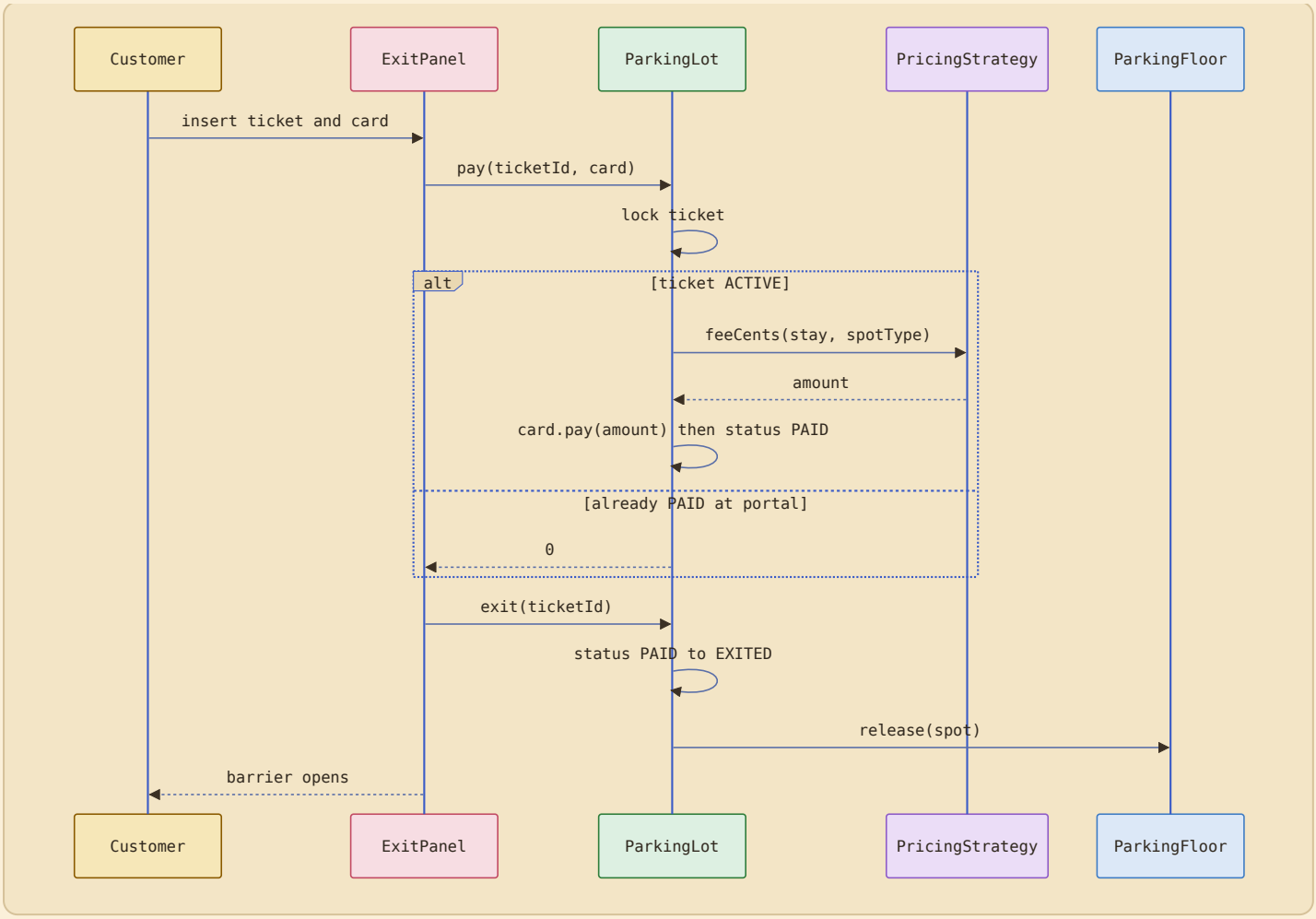
composition, `o--` aggregation (a looser "has a" where the parts can live on their own), `-->` association, `<|..` "implements", and `..>` "uses or creates".

Key flows

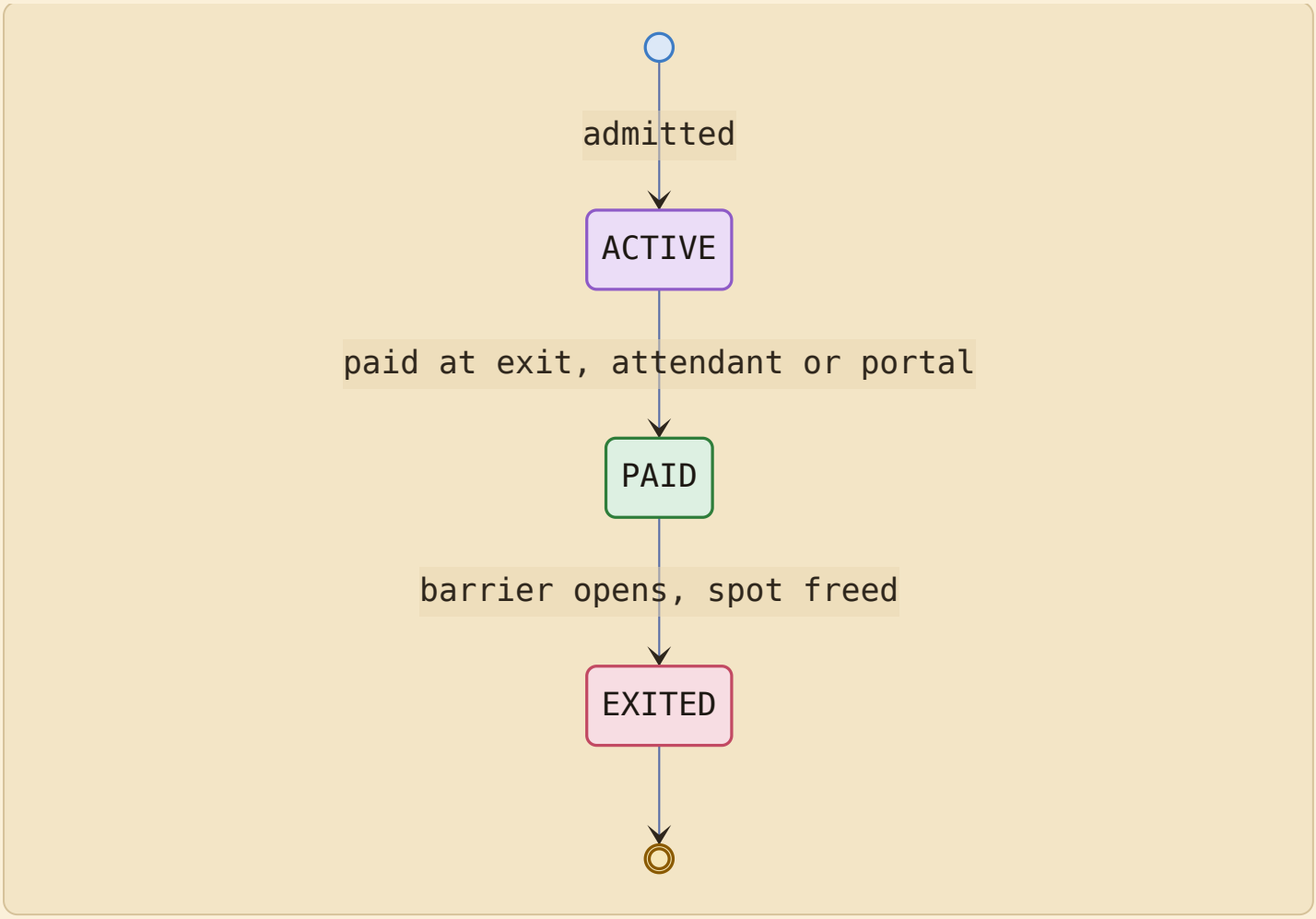
Entry (take ticket). The important step is that the spot is taken **atomically** (as one indivisible step that no other thread can interrupt halfway): `pollFirst()` on a concurrent sorted set removes the spot for exactly one caller. The **sequence diagram** below shows the messages between objects in time order, from top to bottom.



Exit (scan, pay, leave). Payment is guarded per ticket so the info portal and the exit panel cannot both charge.



Ticket life cycle.


admitted
ACTIVE
paid at exit, attendant or portal
PAID
barrier opens, spot freed
EXITED

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
The fit rule lives on `VehicleType`	Information Expert (give a job to the class that owns the data), Open/Closed (extend without editing)	Adding a vehicle type is one enum constant; the lot never changes	A big `switch(vehicleType)` inside the lot
Spot type is an **enum field**, not 5 subclasses	Composition over inheritance (hold a field instead of subclassing)	The 5 course subclasses carried no behaviour; a new type is one constant	`CompactSpot`, `LargeSpot`… (reviewers flag these as empty classes)
How to pick a spot is a `SpotAllocationStrategy`	**Strategy**	Nearest floor, least busy floor, closest to the lift, VIP floors… all swap without touching the lot	`if (mode == NEAREST) … else …` in the lot
How much to charge is a `PricingStrategy`	**Strategy**	The course's "what if pricing changes?" follow-up; admin swaps the rate at run time (the demo swaps to a flat rate)	Hard-coded tiers inside `Ticket`
Electric spots get a `ChargingPanel` in `SpotFactory`	**Factory** (simple static factory)	One place knows the construction rule "electric ⇒ has a panel"	Every caller remembers to attach a panel
Boards subscribe to floors	**Observer**	A floor change updates every board (and could update an app or a sensor light) without the floor knowing who listens	The lot pushing into boards directly
Ticket holds the **spot**, not a count	—	The exit can find and free the exact spot	Counting occupied spots
`ParkingLot` is **not** a hard-coded Singleton	Dependency injection (pass collaborators in from outside)	The course uses a Singleton ("there is one car park"). It is a global in disguise: tests cannot build two lots, and every class that calls `getInstance()` hides a dependency. We create one lot in `main` and pass it to the panels.	`ParkingLot.getInstance()`
`Clock` is injected	Testability	The demo moves time forward by hours instantly	`Instant.now()` inside the code

> **Honest note on "Strategy".** The course points out that a cash/card subclass pair is just an abstract base class, and calling it "Strategy" would be loose use of the word. Strategy means an object **holds** a behaviour and can **swap** it. In our code `ParkingLot` holds a `PricingStrategy` and a `SpotAllocationStrategy` and `setPricing(...)` swaps it at run time — that is genuinely Strategy.

The course also admits that `ParkingLot` does two jobs (finding spots and issuing tickets). Here spot-finding has moved into the strategy and the floors, so the lot is mostly a coordinator; in a longer design you would also move ticket storage into a `TicketRepository`.

Java implementation

Package `lld.ch14.parkinglot`. Amounts are in **cents** (`long`) to avoid floating-point rounding (a `double` stores numbers in binary and cannot hold 0.10 exactly, so sums drift). Essential lines:

```
// Vehicles.java - the fit rule lives on the vehicle type, best fit first
enum SpotType { MOTORCYCLE, COMPACT, LARGE, HANDICAPPED, ELECTRIC }

enum VehicleType {
    MOTORCYCLE(List.of(SpotType.MOTORCYCLE)),
    CAR(List.of(SpotType.COMPACT, SpotType.LARGE)),
    ELECTRIC_CAR(List.of(SpotType.ELECTRIC, SpotType.COMPACT, SpotType.LARGE)),
    VAN(List.of(SpotType.LARGE)),
    TRUCK(List.of(SpotType.LARGE));
    private final List<SpotType> fits;
    VehicleType(List<SpotType> fits) { this.fits = fits; }
    List<SpotType> fits() { return fits; }
}

record Vehicle(String plate, VehicleType type, boolean accessiblePermit) {
    List<SpotType> preferredSpots() {                    // a permit puts HANDICAPPED first; nobody else gets it
        List<SpotType> order = new ArrayList<>();
        if (accessiblePermit && type != VehicleType.MOTORCYCLE && type != VehicleType.TRUCK)
            order.add(SpotType.HANDICAPPED);
        order.addAll(type.fits());
        return order;
    }
}
```

```
// ParkingSpot.java - occupancy is an AtomicReference: CAS succeeds only if the spot was empty
class ParkingSpot implements Comparable<ParkingSpot> {
    private final String id;                             // e.g. "F1-C03"; lowest id = nearest
    private final SpotType type;
    private final AtomicReference<Vehicle> occupant = new AtomicReference<>();
    private final ChargingPanel chargingPanel;           // only electric spots have one (composition)
    boolean tryOccupy(Vehicle v) { return occupant.compareAndSet(null, v); }
    void release() { occupant.set(null); }
    @Override public int compareTo(ParkingSpot o) { return id.compareTo(o.id); }
}

class ChargingPanel {                                    // pay-and-charge at an electric spot
    long chargeAndPay(double kwh, PaymentMethod method) {
        long cost = Math.round(kwh * centsPerKwh);
        method.pay(cost);
        return cost;
    }
}

final class SpotFactory {                                // the only place that knows "electric => has a panel"
    static ParkingSpot create(int floor, SpotType type, int number) {
        String id = "F%d-%s%02d".formatted(floor, type.name().charAt(0), number);
        ChargingPanel panel = type == SpotType.ELECTRIC ? new ChargingPanel(EV_CENTS_PER_KWH) : null;   // 30
c/kWh
        return new ParkingSpot(id, floor, type, panel);
    }
}
```

```java
// ParkingFloor.java - one sorted, thread-safe set of FREE spots per type; boards are observers
interface AvailabilityListener { void onAvailabilityChanged(int floor, SpotType type, int freeNow); }

class ParkingFloor {
    private final Map<SpotType, ConcurrentSkipListSet<ParkingSpot>> free = new EnumMap<>(SpotType.class);
    private final List<AvailabilityListener> listeners = new CopyOnWriteArrayList<>();

    /** pollFirst() is atomic: two threads can never receive the same spot. */
    Optional<ParkingSpot> tryPark(Vehicle v, SpotType type) {
        ParkingSpot spot = free.get(type).pollFirst();
        if (spot == null) return Optional.empty();
        if (!spot.tryOccupy(v)) throw new IllegalStateException("free-set out of sync for " + spot);
        notifyListeners(type);
        return Optional.of(spot);
    }
    void release(ParkingSpot spot) {
        spot.release();
        free.get(spot.type()).add(spot);
        notifyListeners(spot.type());
    }
    int freeCount(SpotType t) { return free.get(t).size(); }
}
// DisplayBoard implements AvailabilityListener: synchronized map type -> free count, renders 0 as "FULL"
```

```java
// Strategies.java
interface SpotAllocationStrategy { Optional<ParkingSpot> allocate(List<ParkingFloor> floors, Vehicle v); }

class NearestFloorFirst implements SpotAllocationStrategy {      // fill the lowest floor first
    @Override public Optional<ParkingSpot> allocate(List<ParkingFloor> floors, Vehicle v) {
        for (SpotType type : v.preferredSpots())
            for (ParkingFloor f : floors) {
                Optional<ParkingSpot> s = f.tryPark(v, type);
                if (s.isPresent()) return s;
            }
        return Optional.empty();
    }
}
// LeastBusyFloorFirst: same loop, floors sorted by freeCount(type) descending (losing a race -> next floor)

interface PricingStrategy { long feeCents(Duration stay, SpotType spotType); }

class HourlyTieredPricing implements PricingStrategy {           // $4, $3.50, $3.50, then $2.50/hour
    static HourlyTieredPricing standard() { return new HourlyTieredPricing(List.of(400L, 350L, 350L), 250); }
    @Override public long feeCents(Duration stay, SpotType spotType) {
        long minutes = Math.max(1, stay.toMinutes());
        long hours = (minutes + 59) / 60;                         // a started hour counts in full
        long total = 0;
        for (long h = 0; h < hours; h++) total += h < firstHours.size() ? firstHours.get((int) h) :
laterHourCents;
        return total;
    }
}
```

```java
// Ticketing.java - the ticket points at the real spot so the exit can free it
enum TicketStatus { ACTIVE, PAID, EXITED }

final class Ticket {
    private final ParkingSpot spot;
    private final Instant entryTime;
    private TicketStatus status = TicketStatus.ACTIVE;
    synchronized boolean markPaid(long cents) {                  // only one payment can ever succeed
        if (status != TicketStatus.ACTIVE) return false;
        status = TicketStatus.PAID; paidCents = cents; return true;
    }
    synchronized boolean markExited() {
        if (status != TicketStatus.PAID) return false;
        status = TicketStatus.EXITED; return true;
    }
}
interface PaymentMethod { void pay(long cents); String describe(); }
// record CashPayment(long tenderedCents): throws "not enough cash"; record CardPayment(String last4)
```

```java
// ParkingLot.java - NOT a Singleton: created once in main and injected into the panels
final class ParkingLot {
    private final Map<String, Ticket> tickets = new ConcurrentHashMap<>();
    private final Map<String, String> ticketByPlate = new ConcurrentHashMap<>();
    private volatile SpotAllocationStrategy allocation;
    private volatile PricingStrategy pricing;

    Optional<Ticket> admit(Vehicle v) {
        if (ticketByPlate.putIfAbsent(v.plate(), "pending") != null)       // same car cannot enter twice
            throw new IllegalStateException(v.plate() + " is already inside");
        Optional<ParkingSpot> spot = allocation.allocate(floors, v);
        if (spot.isEmpty()) { ticketByPlate.remove(v.plate()); return Optional.empty(); }   // full for this
vehicle
        Ticket t = new Ticket("T" + nextTicket.getAndIncrement(), v, spot.get(), clock.instant());
        tickets.put(t.id(), t);
        ticketByPlate.put(v.plate(), t.id());
        return Optional.of(t);
    }
    long amountDue(String ticketId) {
        Ticket t = ticket(ticketId);
        if (t.status() != TicketStatus.ACTIVE) return 0;                // already paid (e.g. at the portal)
        return pricing.feeCents(Duration.between(t.entryTime(), clock.instant()), t.spot().type());
    }
    long pay(String ticketId, PaymentMethod method) {
        Ticket t = ticket(ticketId);
        synchronized (t) {                      // per-ticket lock: portal and exit panel cannot both charge
            long due = amountDue(ticketId);
            if (due == 0) return 0;
            method.pay(due);                    // throws if the payment fails: ticket stays ACTIVE
            t.markPaid(due);
            return due;
        }
    }
    boolean exit(String ticketId) {             // only a PAID ticket opens the barrier
        Ticket t = ticket(ticketId);
        if (!t.markExited()) return false;
        floors.get(t.spot().floor() - 1).release(t.spot());
        tickets.remove(ticketId);
        ticketByPlate.remove(t.vehicle().plate());
        return true;
    }
    void setPricing(PricingStrategy p) { this.pricing = p; }          // admin changes the rate at run time
}
// Panels.java: EntrancePanel.issueTicket, ExitPanel.scanPayAndExit (pay then exit), CustomerInfoPortal.pay -
// no rules, they only call the lot. Main.java: a TestClock (a movable java.time.Clock), then 20 threads
// released by a CountDownLatch race for the last 5 car spots.
```

A `ConcurrentSkipListSet` is a thread-safe set from `java.util.concurrent` that keeps its elements sorted; its `pollFirst()` removes the lowest id **atomically**, so if 20 threads call it at once each spot goes to exactly one of them. The lowest id stands for "nearest to the entrance". `tryOccupy` is a second safety

net: **CAS** (compare-and-set) — "set to *v* only if still *null*", one atomic step without a lock. `putIfAbsent` adds a key only if absent, atomically. `pay` locks **one ticket**, not the lot (a **lock** lets one thread at a time run a piece of code; `synchronized` is the built-in lock every object carries), so different tickets pay in parallel.

Output:

```
== Arrivals ==
[entry-A] CAR-1 -> ticket T1, spot F1-C01(COMPACT)
[entry-A] EV-1 -> ticket T2, spot F1-E01(ELECTRIC)
[entry-A] BIKE-1 -> ticket T3, spot F1-M01(MOTORCYCLE)
[entry-A] CAR-2 -> ticket T4, spot F1-H01(HANDICAPPED)
[entry-A] TRUCK-1 -> ticket T5, spot F1-L01(LARGE)
[entry-A] VAN-1 -> ticket T6, spot F2-L01(LARGE)
[entry-A] TRUCK-2 -> SORRY, LOT FULL for TRUCK
Floor 1 board: MOTORCYCLE=FULL COMPACT=1 LARGE=FULL HANDICAPPED=FULL ELECTRIC=FULL
Floor 2 board: COMPACT=3 LARGE=FULL

== EV charging at the spot ==
EV-1 charged 20 kWh, paid $6.00

== Departures ==
[exit-A] T1: paid $4.00 by cash $5.00, barrier OPEN
[portal] T2 due $16.00, charged $16.00
[exit-A] T2: already paid, barrier OPEN
Unpaid exit attempt: barrier opens? false

== Admin changes the rate: flat $5/hour ==
[exit-A] T3: paid $25.00 by card ****9999, barrier OPEN

== 20 cars race for the remaining CAR spots ==
free compact before: F1=2 F2=3, large: F1=0 F2=0
cars admitted: 5, distinct spots: 5
full for cars now? true
[entry-A] LATE-CAR -> SORRY, LOT FULL for CAR
Floor 2 board: COMPACT=FULL LARGE=FULL
```

Read the output: the truck is turned away (both large spots taken); T2 stayed 4 h 50 min → 5 hours = $4 + $3.50 + $3.50 + $2.50 + $2.50 = **$16.00**, paid at the portal, so the exit charges nothing; an unpaid ticket cannot open the barrier; after the switch to a flat $5/hour the motorcycle pays $25 for 5 hours; 20 racing cars get exactly 5 distinct spots.

Concurrency & edge cases

Situation	What could go wrong	How the design handles it
Two gates want the last compact spot	Both get it (double booking)	`pollFirst()` is atomic; `tryOccupy` uses CAS as a second check
Portal and exit panel pay the same ticket at once	Customer charged twice	`synchronized (ticket)` around price-charge-mark; a PAID ticket costs 0
Same plate enters twice (cloned ticket, sensor glitch)	Two tickets for one car	`ticketByPlate.putIfAbsent`
Payment fails (card declined)	Ticket marked paid anyway	`method.pay` throws **before** `markPaid`; ticket stays ACTIVE
Car leaves without paying	Spot never freed or fee lost	`exit` only succeeds from PAID; the barrier stays closed
Customer pays at the portal and then stays 3 more hours	Free parking	Real lots give a **grace period** (e.g. 15 min) after payment; add `paidAt` and re-price after the grace period
Display board shows a slightly old number	Two notifications arrive out of order	Acceptable for a display; if exactness matters, have the board **pull** `freeCount` when it renders
Lost ticket	No entry time	Charge a daily maximum, or look up the plate (the plate map already exists)
Stay of 0 minutes	Fee of $0	Minimum of one hour (`Math.max(1, minutes)`)
Lock scope	One global lock would serialise all gates	Locks are per spot (CAS) and per ticket; no global lock

`ConcurrentSkipListSet.size()` is not constant-time (it walks the whole set instead of reading a stored number) and is only an estimate while other threads change the set; that is fine for a display board, but never use it to decide whether a spot is free — use `pollFirst()`.

Extensions & follow-up questions

1. **"What if pricing changes (weekend rate, per spot type, surge)?"** — Write a new `PricingStrategy` (for example `WeekendPricing`, or a decorator (a wrapper object with the same interface that adds behaviour around another object) that multiplies another strategy for large spots) and call `setPricing`. No other class changes.
2. **"Two cars arrive at the same instant — what happens?"** — Spot handout is atomic (`pollFirst` + CAS). Show the 20-thread race in the output.
3. **"Add reservations."** — Add a `Reservation(plate, spotType, from, to)` and a reserved set per floor; the allocation strategy checks the reservation first. Hold the spot with a timeout (release it if the car does not arrive within, say, 30 minutes).
4. **"Add monthly passes / season tickets."** — A `PricingStrategy` that returns 0 for plates with a valid pass, or a `PassHolderPricing` decorator around the normal pricing.
5. **"Number-plate recognition (ticketless)."** — The entrance camera becomes another kind of entrance panel that calls the same `admit`; the ticket id is keyed by plate. No core class changes.
6. **"How would you store this in a database?"** — Tables `floor`, `spot(id, floor_id, type, occupied_by)`, `ticket(id, plate, spot_id, entry_time, status, paid_cents)`, `payment(id, ticket_id, method, amount)`. Claim a spot with `UPDATE spot SET occupied_by=? WHERE id=? AND occupied_by IS NULL` — the database's CAS. Candidates report that interviewers like to see **cardinalities** (how many of one thing link to another; `1..*` = one or more: a floor has 1..* spots, a spot holds 0..1 vehicles, a ticket has 1 spot), then add a new entity to see how you rewire the model.
7. **"Many lots in a city?"** — A `ParkingLot` per site plus a `ParkingNetwork` that routes drivers to the lot with free spots. This is another reason not to make the lot a Singleton.

8. **"EV idle fees?"** — Add a charging session to `ChargingPanel` (start, end, kWh) and charge a per-minute fee after charging completes.

Real-world example

Many modern malls and airports use **parking guidance systems**: a sensor above each bay drives a small green/red light, and the boards at each entrance and floor show the free count per level — exactly our `DisplayBoard` observers fed by spot changes. Airports and many city garages increasingly use **automatic number-plate recognition** instead of paper tickets, and apps such as SpotHero and ParkMobile let drivers pre-book a spot, which is the "reservations" extension. Tesla Superchargers charge **idle fees** when a car stays plugged in after charging finishes — the EV extension above.

In the interview

Plan for 45 minutes

Minutes	What you do
0–5	Clarifying questions; write the 12 requirements and an out-of-scope list
5–10	Actors and use cases (take ticket, pay, exit, admin)
10–20	Stages 1–2: vehicle/spot fit rule, floors, board. Say "the fit rule lives on the vehicle"
20–25	Stage 3–4: ticket holds the spot; lot with admit/pay/exit; Strategy for pricing
25–38	Code `admit`, `tryPark`, `pay`, `exit`, and the tiered pricing
38–45	Concurrency (atomic spot handout, per-ticket lock), extensions

30-second answer: "I'll model vehicles and spot types as enums with the fit rule on the vehicle, floors that keep a concurrent sorted set of free spots per type, and a lot that admits, prices and releases. Tickets point at the real spot. Pricing and spot-finding are Strategies so they can change without touching the lot. Handing out a spot is atomic — `pollFirst` plus a CAS on the spot — so two gates can never get the same spot, and payment locks only the one ticket."

Follow-ups with answers

- *"Why not a Singleton?"* — There is one lot in production, but a Singleton is a hidden global: it makes testing two lots impossible. I create one instance and inject it. If the interviewer insists, a lazy holder-class Singleton (the single instance lives in a nested class that Java loads only on first use, which the JVM makes thread-safe for free) is thread-safe and simple.
- *"Why is the fit rule on the vehicle?"* — Because it changes when vehicle types change; putting it there keeps the lot closed for modification (OCP).
- *"How do you make finding a spot fast?"* — Keep only the free spots, indexed by type, in a sorted set; taking one is O(log n) and needs no scan.
- *"What if the display board and the real count disagree?"* — The board is informational; the atomic `pollFirst` is the source of truth. The board can pull the count on render if needed.

TL;DR

- Fit rule on the **vehicle**, spot type as an **enum**, free spots per type in a **concurrent sorted set**.
- **Ticket → spot** reference makes the exit possible; status goes ACTIVE → PAID → EXITED.
- **Strategy** for pricing and spot-finding, **Factory** for electric spots, **Observer** for boards; inject the lot instead of a Singleton.
- Concurrency: atomic `pollFirst` + CAS for spots, per-ticket lock for payment, `putIfAbsent` for plates.

14.2 Design a Vending Machine



Figure — A vending machine: slots with codes, a keypad, a coin/note slot and a tray at the bottom (course diagram)

A **vending machine** is an unattended shop: key in a slot code, pay, pull your item from the tray. Everyone has used one, so the interviewer skips the domain and looks at one thing: **which actions are legal at which moment**. Taking a coin when nothing is chosen is a bug; two items for one payment is a bug; tidy code elsewhere does not make up for getting that order wrong.

Problem & clarifying questions

Question	Assumed answer
Select first, or pay first?	Select a slot code first, then pay.
Which money?	A fixed set of coins and notes (5¢, 10¢, 25¢, $1, $5). Anything else is returned.
Card or contactless?	Out of scope (an extension).
Can it give change?	Yes. If it cannot make **exact** change, it refuses the sale and returns everything.
Can the customer cancel?	Any time before the product is dispensed, with a full refund.
One customer at a time?	Yes (one physical machine).
Who restocks?	An operator: restocks slots, loads coins for change, collects the takings.

Requirements

Functional (the course's twelve)

1. The machine has many **slots**. A slot has a code (`A1`, `B2`, …), holds items of a **single** product, and tracks its remaining **count**.
2. A product has a **name** and a **price in cents**.
3. Order of use: **choose a slot, then pay**.
4. Only a **known set of coins and notes** is accepted; any other piece is given back.
5. After **every** customer action the **display** says what to do next.
6. When the amount paid covers the price, the machine **releases the item and the change**.
7. When **exact change is impossible**, the sale is **refused** and every cent paid is **returned**.

8. Until the item drops, the customer may **cancel** and receive a **full refund**.
9. A slot whose count is **zero cannot be chosen**.
10. One payment buys **exactly one** item, never two.
11. An **operator** refills slots, adds coins for change, and removes the takings.
12. After every sale, cancel and refusal, **both** the item counts and the cash counts must be **exactly right**.

Non-functional

- **No illegal transitions**: every button must have a defined answer in every situation.
- **Exact money**: integer cents only — no floating-point.
- **Extensible**: a new payment device (card reader) or a new state (maintenance) should not require editing every existing class.

Use cases

Actors: Customer (selects, pays, cancels, takes the product), **Operator** (restocks, loads change, collects money), **System** (decides which actions are allowed now, works out change, shows messages).

Top use cases: select a product · insert money · dispense · collect (take from the tray) · cancel · restock a slot · load change and collect takings.

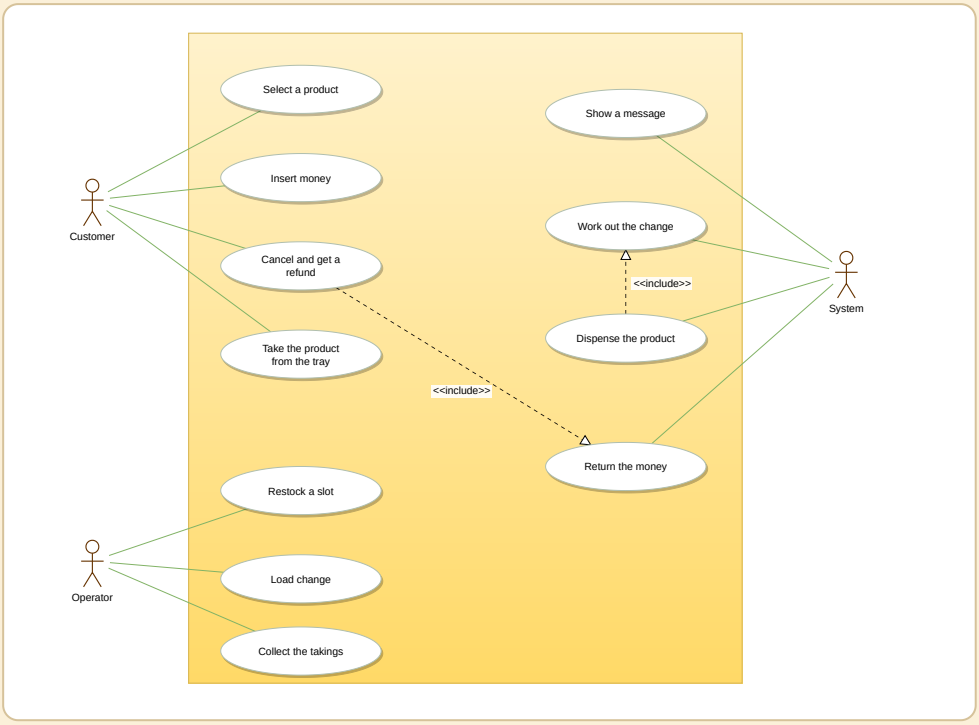


Figure — Use case diagram of the vending machine: Customer, Operator and System (course diagram)

In the figure above, "dispense" is a System use case, not a customer one: the customer only *causes* it by paying enough.

Core entities

Class	Responsibility
`Product` (record)	Name and price in cents. Describes *what* is sold.
`Slot`	One physical column: a code, one product, and the **count** left.
`Inventory`	All slots, looked up by code.
`Denomination` (enum)	The accepted coins and notes and their values in cents.
`CashBox`	Counts pieces by value; plans exact change; reports when it cannot.
`VendingState` (interface)	One method per customer button: `select`, `insert`, `cancel`, `take`.
`IdleState`, `SelectedState`, `HasMoneyState`, `DispensingState`, `OutOfStockState`	The situations the machine can be in (the course has the first four, calling `HasMoneyState` `PaidState`); each answers every button and names the next state.
`Display`, `Keypad`, `CashSlot`, `Dispenser`, `Operator`	The message shown, the hardware the customer touches (calls the machine, holds no rules) and the operator (restocks, loads change, collects takings). Our code folds them into `VendingMachine`: `show(...)`, the public button methods, `restock`, `loadChange`, `collectTakings`.
`VendingMachine`	Holds the parts, the current state, the selection and the balance; forwards each button to the state; exposes operator actions.

Why is the count on the **slot**, not the product? `Product` answers "what is it?"; `Slot` is the physical column. The same product may sit in two columns (the course even allows different prices), and emptying one must leave the other untouched — the first real design decision in this question.

Class diagram

The course builds the machine in five stages:

Stage	What it adds	What works after it
1	`Product`, `Slot`, `Inventory`	Look up a code and learn what it holds and whether it is empty
2	`Coin`, `Note`, `CashBox`	Decide whether a given amount of change can be paid exactly
3	A class for each state	Each button press gets a defined reaction in each situation
4	`Display` and `VendingMachine`	A complete sale, from choosing to picking up the item
5	`Keypad`, `CashSlot`, `Dispenser`, `Operator`	Real hardware drives the same flow

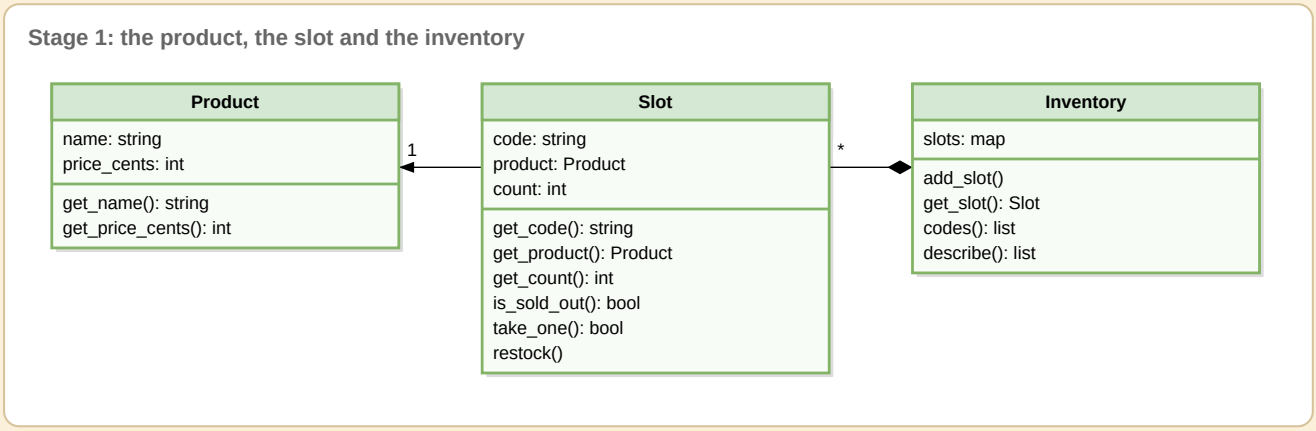


Figure — Stage 1: Product, Slot (association to Product) and Inventory (composition of Slots) (course diagram)

Stage 1. Requirements 1, 2 and 9. `Slot → Product` is an **association**: a product can appear in several slots, so removing a slot must leave the product alone. `Inventory ◆ Slot` is **composition** (filled diamond): slots are parts of the machine and do not outlive it.

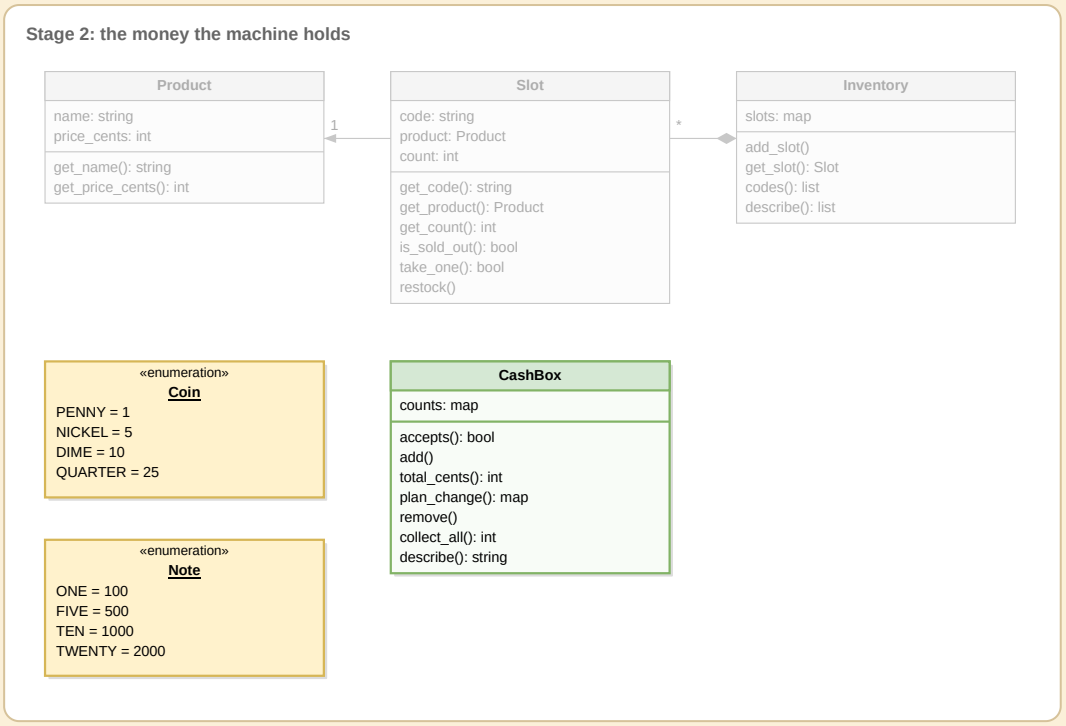


Figure — Stage 2: Coin and Note enumerations and the CashBox that counts pieces by value (course diagram)

Stage 2. Requirements 4, 6 and 7. All amounts are whole cents, so no rounding errors. `Coin` and `Note` are enumerations (fixed sets with no relationship arrows; our code merges them into one `Denomination` enum). The choice to defend: **a piece goes into the cash box the moment it is inserted**, not into a holding area — it is then available for change, and a refund is just "pay out exact change equal to what was inserted". The box stores a **count per value** (which physical quarter comes back doesn't matter); its change planner returns an exact plan or "impossible", never an approximation — that is what makes requirement 7 implementable.

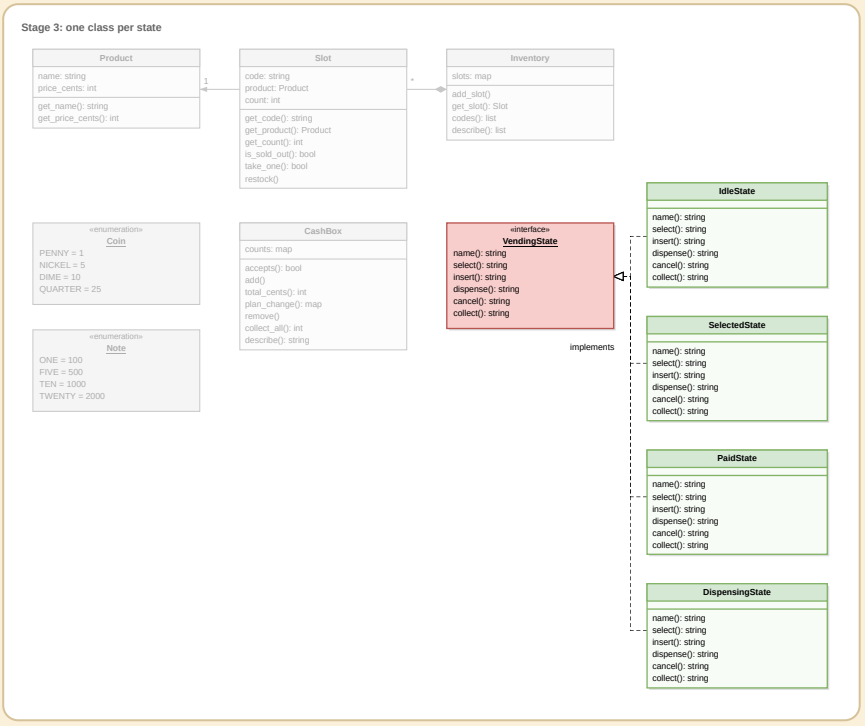


Figure — Stage 3: one class per state, all implementing the VendingState interface (course diagram)

Stage 3 — the heart of the question. Requirements 3, 8 and 10. The obvious first attempt — one class with boolean flags and nested `if`s — is exactly where double dispensing and swallowed coins come from. Use the **State pattern**: each situation is a class implementing one interface on which **each customer button is a method**, so a state that forgets a button does not compile. Picture a **grid** of buttons × situations: the design is finished when every cell has a response. State classes carry no fields and never check which state is active; each handles its buttons and hands over to the next, so a new state needs no edits to existing ones.

The course draws **four** states: `IdleState`, `SelectedState`, `PaidState` and `DispensingState`. Our code keeps those four (its `HasMoneyState` is the course's `PaidState`: some money is in, not yet enough) and adds a fifth, `OutOfStockState`, so that an empty machine refuses every customer button until an operator restocks it.

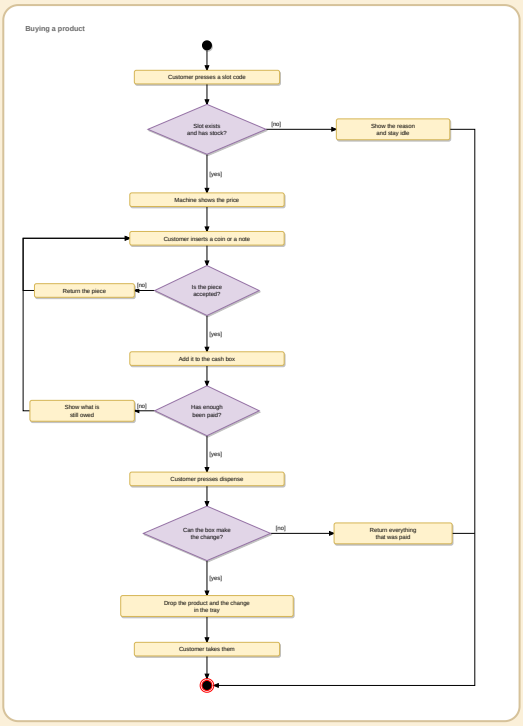


Figure — Activity diagram "Buying a product": select, check stock, insert money until enough, check change, dispense, take (course diagram)

Each decision diamond in the activity diagram above corresponds to one state choosing whether to accept an action.

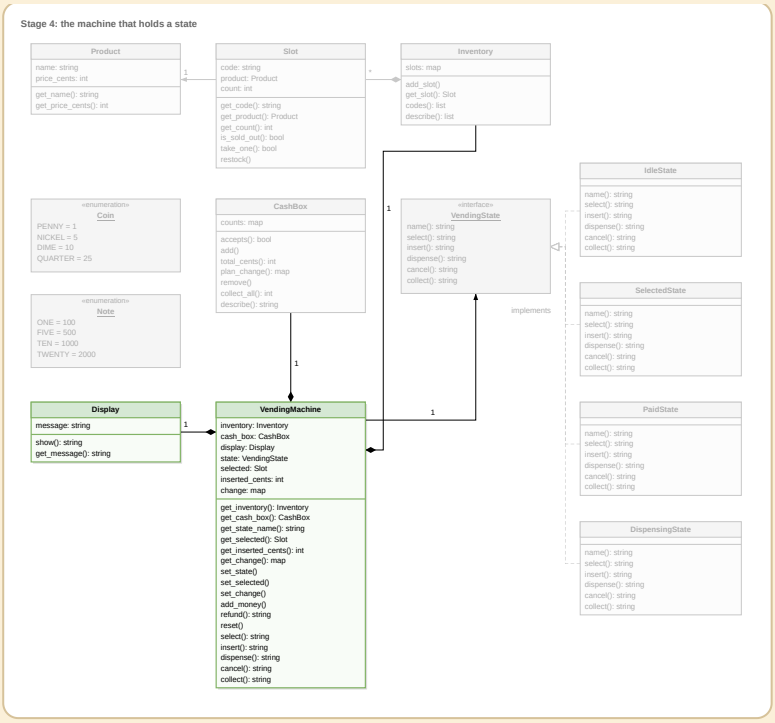


Figure — Stage 4: VendingMachine owns its display, cash box and inventory, and points at the current VendingState (course diagram)

Stage 4. Requirements 5, 6, 8 and 12. `VendingMachine` remembers the chosen slot and the money inserted, and passes every button to its current state. It **owns** its `Display`, `CashBox` and `Inventory` (composition) and merely **refers to** the current state (association). It contains no `if` testing which state is active. Because inserted money is already in the box (stage 2), a refund is just a request for exact change equal to the balance.

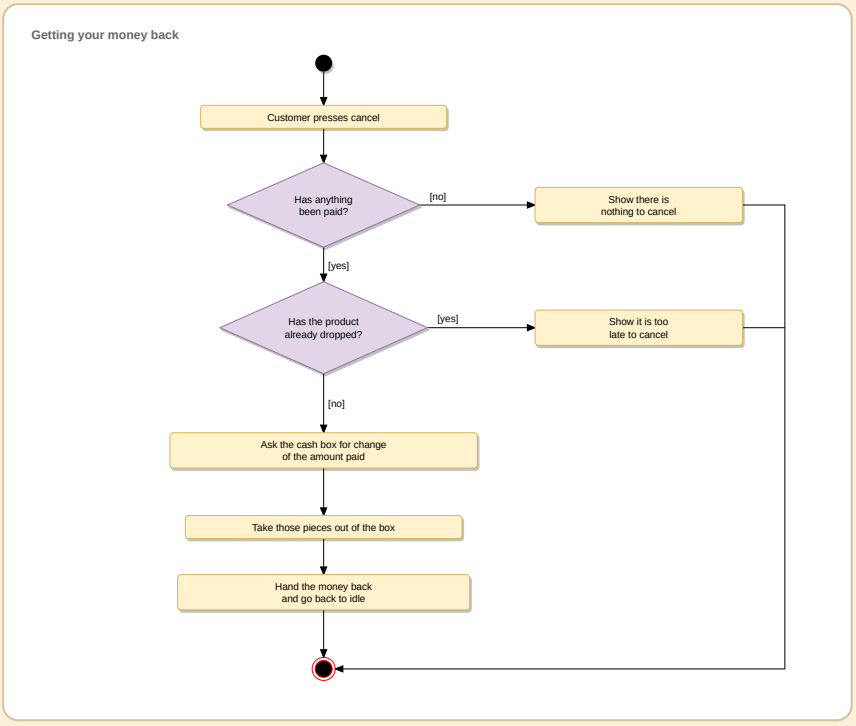


Figure — Activity diagram "Getting your money back": cancel and "cannot make change" end in the same refund path (course diagram)

As the figure above shows, a cancel and a "no exact change" refusal finish identically, so the design needs a **single** refund method.

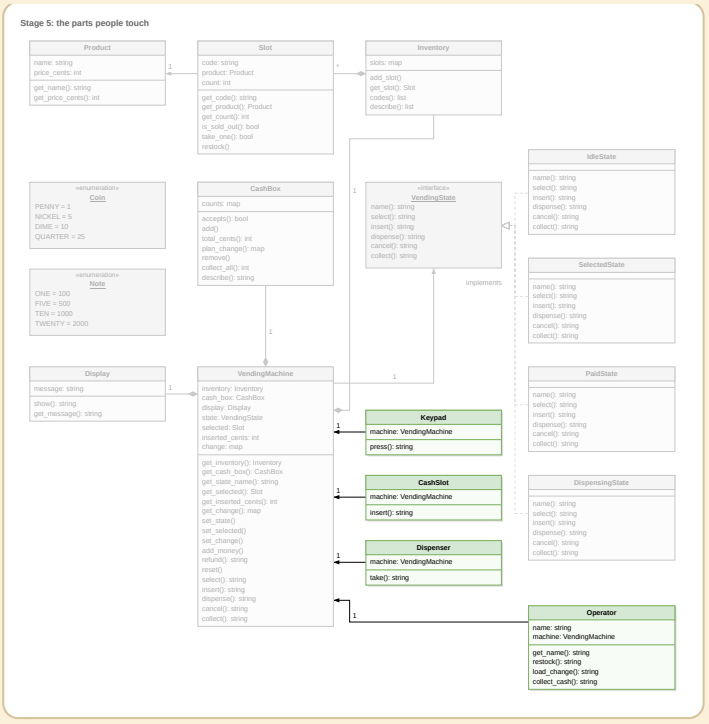


Figure — Stage 5: Keypad, CashSlot, Dispenser and Operator, all pointing at the machine (course diagram)

Stage 5. Requirements 3, 4 and 11. All four new classes hold references **to** the machine, never the reverse, so swapping the keypad for a touch screen or the coin slot for a card reader touches no other class. The `Dispenser` is the tray; pulling the item out finishes a sale, which is why `take` is a customer button. `Operator` stays **off** the state interface: refilling, loading change and emptying the box are not customer actions and are allowed in any state (otherwise every state class would have to answer them).

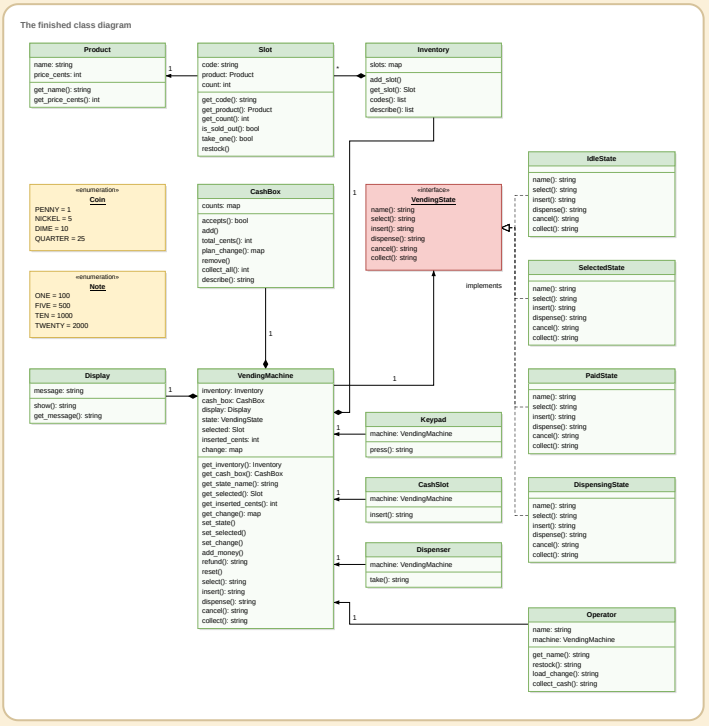
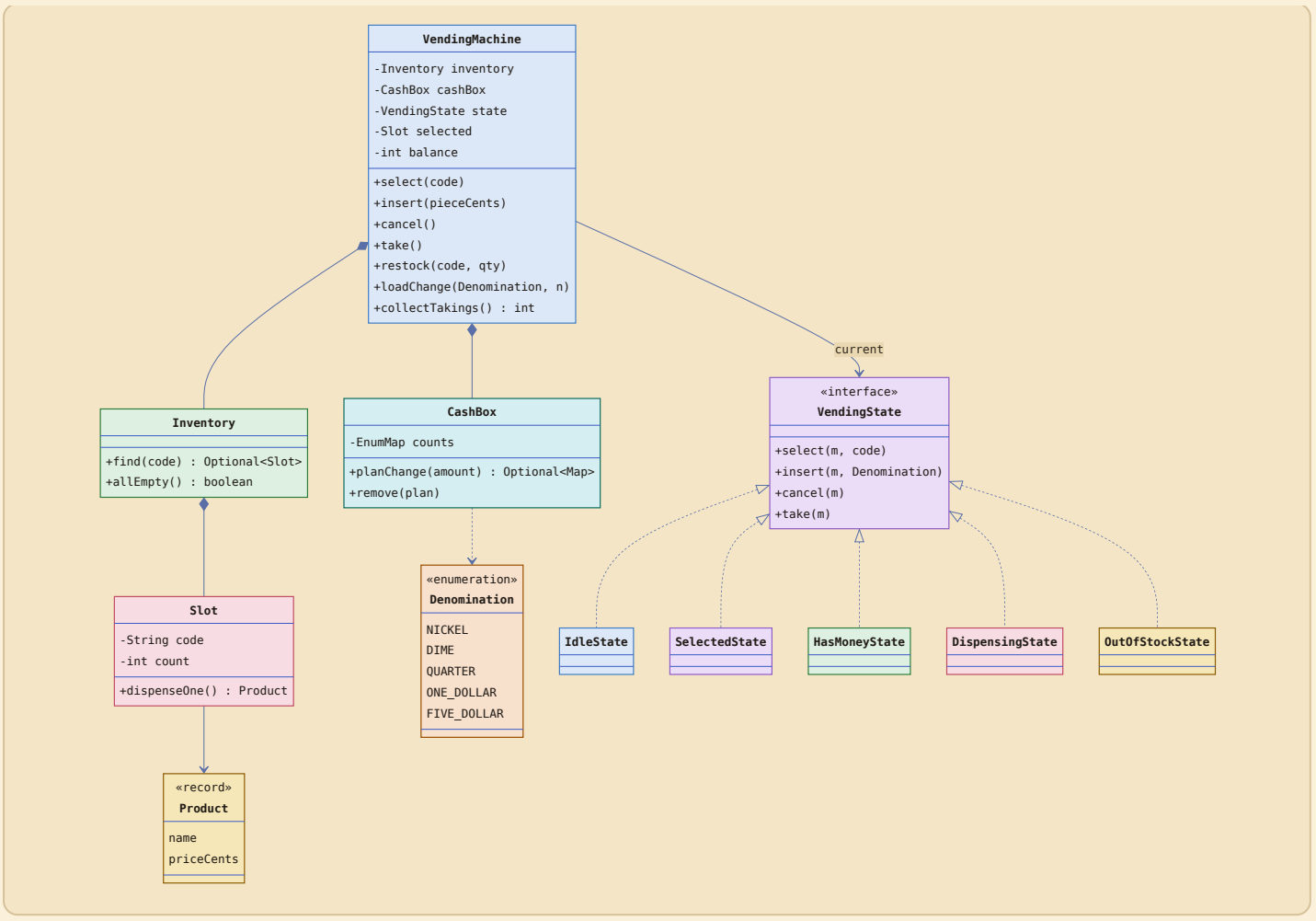


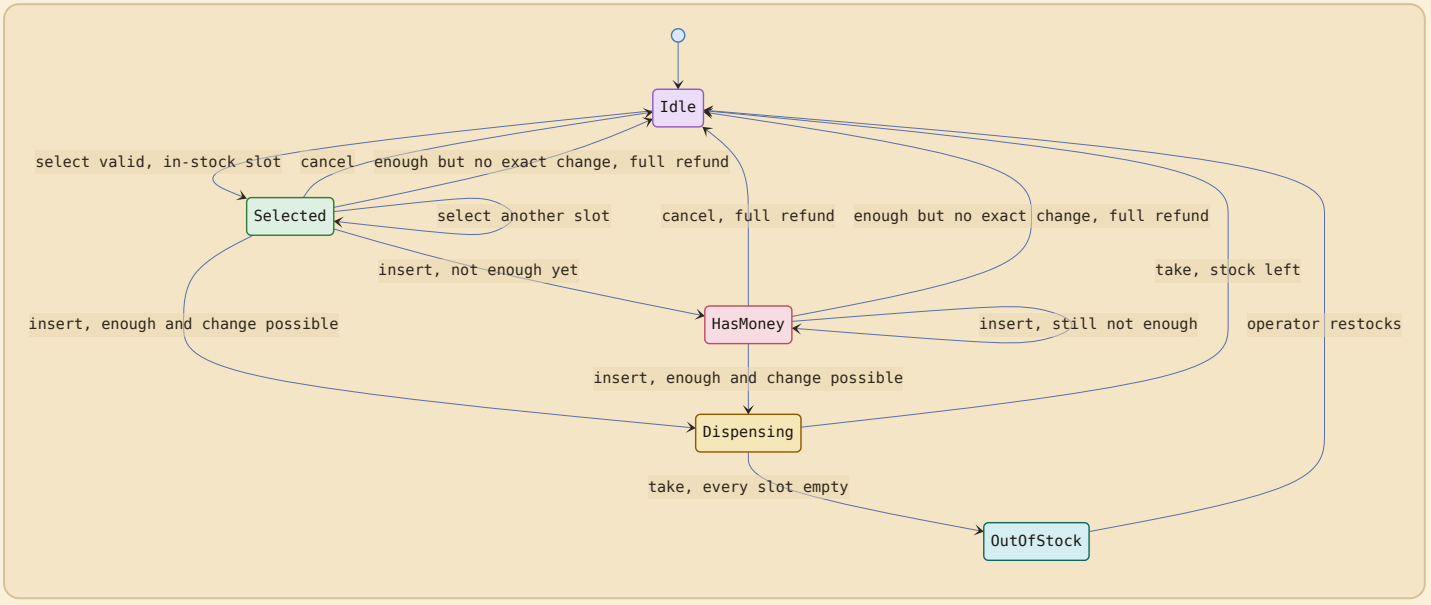
Figure — The finished class diagram of the vending machine (course diagram)

Class diagram of **our Java code** (hardware classes folded into the machine's public methods to keep the code short):



Key flows

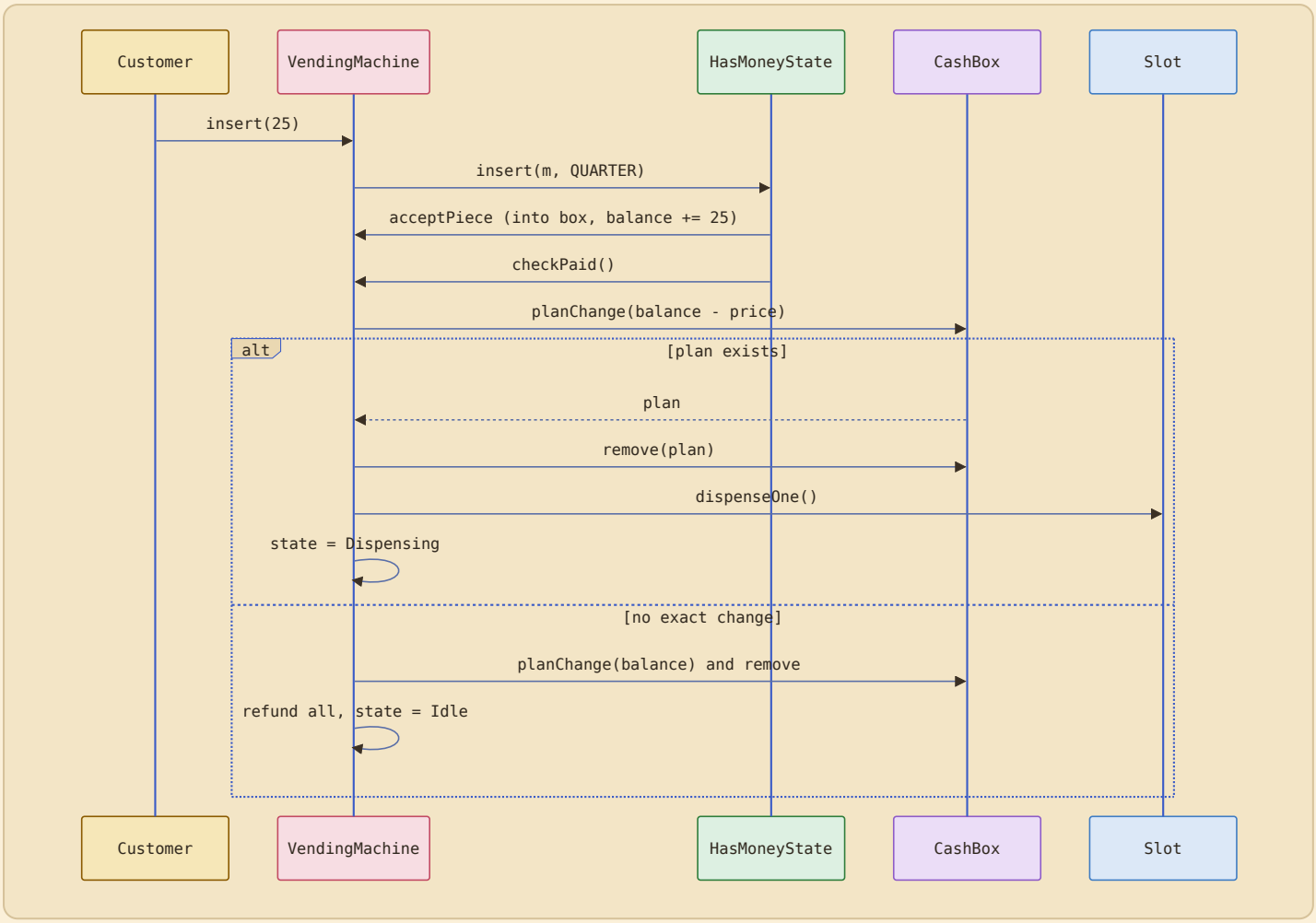
State machine. This diagram *is* the design; draw it before writing code.



The state table (rows = buttons, columns = states). Every cell has an answer:

Button ↓ / State →	Idle	Selected	HasMoney	Dispensing	OutOfStock
`select`	choose slot (reject empty/unknown)	change choice	"cancel first"	"take your product first"	"out of stock"
`insert`	return piece	accept; check paid	accept; check paid	return piece	return piece
`cancel`	"nothing to cancel"	clear choice → Idle	**refund all** → Idle	"too late"	"nothing to cancel"
`take`	"tray empty"	"pay first"	"pay the rest"	give product + change → Idle/OutOfStock	"tray empty"

Paying enough.



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
One class per situation behind `VendingState`	**State**	Every button × situation cell is answered; the compiler enforces completeness; no `if (state == …)`	Flags + `if/else` chains (the classic source of double-dispense bugs)
States are shared **stateless singletons** (`INSTANCE`; no fields, so one copy serves everyone)	Flyweight-style sharing (one shared object instead of many identical ones)	The course notes that creating a new state object on each transition is readable but wasteful; the states hold no data, so one instance each is enough	`new IdleState()` on every transition
Operator actions are **not** on the state interface	Interface Segregation (no class is forced to implement methods it does not need)	Restocking is allowed in every state; forcing five classes to answer it would be noise	Adding `restock` to `VendingState`
Money counted by value in an `EnumMap` (a fast map keyed by enum constants)	—	The machine never cares which physical quarter it returns	A list of coin objects
Inserted money goes straight into the box	—	Refund = "make change for the balance"; one refund path	A separate escrow pile (a holding area for inserted money)
Exact change with dynamic programming	Correctness	Greedy can fail when the box is short of one coin (see output part 6)	Greedy only
Refuse the sale if exact change is impossible	Requirement 7	Never short-change the customer	Give "close enough" change

> **About the change algorithm.** The course's planner is **greedy** (largest piece first, never looks back) and honestly says so: greedy is correct for standard coin sets when the box is well stocked, but when the box is short of one piece it can fail for an amount that smaller pieces could make. The exact fix is **dynamic programming** — the "coin change with limited coins" problem, where you build the answer for each amount from the answers for smaller amounts. Our `CashBox.planChange` implements that (bounded coin change, fewest pieces), and `greedyPlan` is kept only to show the difference: for 30¢ from {1 quarter, 3 dimes}, greedy takes the quarter and gets stuck at 5¢; DP returns three dimes.

Java implementation

Package `lld.ch14.vendingmachine`. Essential lines:

```
// Inventory.java
record Product(String name, int priceCents) {}

final class Slot {                                   // the COUNT lives on the slot, not on the product
    private final String code;
    private final Product product;
    private int count;
    boolean isEmpty() { return count == 0; }
    Product dispenseOne() {
        if (count == 0) throw new IllegalStateException(code + " is empty");
        count--;
        return product;
    }
}
// Inventory: LinkedHashMap code -> Slot; find(code), allEmpty()
```

```java
// CashBox.java
enum Denomination {                                     // anything else is returned
    NICKEL(5, false), DIME(10, false), QUARTER(25, false), ONE_DOLLAR(100, true), FIVE_DOLLAR(500, true);
    final int cents; final boolean note;
}

final class CashBox {                                   // inserted money goes in at once: a refund is just "make
change"
    private final EnumMap<Denomination, Integer> counts = new EnumMap<>(Denomination.class);

    /** Exact change with the FEWEST pieces, respecting stock (bounded coin change, DP). Empty = impossible. */
    Optional<Map<Denomination, Integer>> planChange(int amount) {
        Denomination[] ds = Denomination.values();
        int[][] best = new int[amount + 1][];           // best[a] = pieces used per denomination to make a
        best[0] = new int[ds.length];
        for (int i = 0; i < ds.length; i++) {
            int v = ds[i].cents;
            int copies = Math.min(counts.get(ds[i]), amount / v);
            for (int c = 0; c < copies; c++)            // each copy may be used once (0/1)
                for (int a = amount; a >= v; a--) {
                    if (best[a - v] == null) continue;
                    int pieces = sum(best[a - v]) + 1;
                    if (best[a] == null || pieces < sum(best[a])) { best[a] = best[a - v].clone(); best[a][i]++; }
                }
        }
        if (best[amount] == null) return Optional.empty();
        // ... convert best[amount] into an EnumMap plan
    }
    // greedyPlan(amount): largest piece first - kept only to compare; remove(plan);
    // collectNotes(): operator empties the notes, coins stay as the float for change
}
```

```java
// States.java - every button is on the interface, so every state must answer every button
interface VendingState {
    void select(VendingMachine m, String code);
    void insert(VendingMachine m, Denomination d);
    void cancel(VendingMachine m);
    void take(VendingMachine m);
    String name();
}

final class IdleState implements VendingState {
    static final IdleState INSTANCE = new IdleState();   // states hold no data, so share one instance
    @Override public void select(VendingMachine m, String code) {
        m.inventory().find(code).ifPresentOrElse(slot -> {
            if (slot.isEmpty()) { m.show(code + " is sold out. Choose another."); return; }
            m.choose(slot);
            m.show("%s costs %d c. Insert money.".formatted(slot.product().name(), slot.product().priceCents()));
            m.setState(SelectedState.INSTANCE);
        }, () -> m.show("Unknown code " + code));
    }
    @Override public void insert(VendingMachine m, Denomination d) { m.rejectPiece(d, "select a product first"); }
    @Override public void cancel(VendingMachine m) { m.show("Nothing to cancel."); }
    @Override public void take(VendingMachine m) { m.show("The tray is empty."); }
}

final class HasMoneyState implements VendingState {      // some money in, not enough yet
    @Override public void select(VendingMachine m, String code) { m.show("Cancel first to change your choice."); }
    @Override public void insert(VendingMachine m, Denomination d) { m.acceptPiece(d); m.checkPaid(); }
    @Override public void cancel(VendingMachine m) { m.refundAll("Cancelled."); }
    @Override public void take(VendingMachine m) { m.show("Please pay the rest first."); }
}
// SelectedState: select = re-select via Idle, insert = acceptPiece + checkPaid, cancel = clear -> Idle, take =
"pay first"
// DispensingState: rejects select/insert/cancel ("too late"); take -> m.emptyTray()
// OutOfStockState: refuses every button; only an operator restock leaves it
```

```java
// VendingMachine.java - forwards every button to the state; no "if (state == ...)" anywhere
final class VendingMachine {
    private VendingState state;
    private Slot selected;
    private int balance;                                  // cents inserted in this sale

    synchronized void select(String code) { state.select(this, code); }
    synchronized void insert(int pieceCents) {            // unknown piece returned before reaching a state
        Denomination.of(pieceCents).ifPresentOrElse(d -> state.insert(this, d),
                () -> show("Unknown piece of " + pieceCents + "c returned."));
    }
    // cancel(), take() likewise

    // operator actions: allowed in every state, so NOT on the state interface
    synchronized void restock(String code, int qty) {
        inventory.find(code).orElseThrow().restock(qty);
        if (state == OutOfStockState.INSTANCE) setState(IdleState.INSTANCE);
    }
    // loadChange(d, n), collectTakings()

    void acceptPiece(Denomination d) { cashBox.add(d, 1); balance += d.cents; }

    void checkPaid() {                                    // after every accepted piece in SELECTED / HAS_MONEY
        int price = selected.product().priceCents();
        if (balance < price) {
            show("Paid %d c, %d c to go.".formatted(balance, price - balance));
            setState(HasMoneyState.INSTANCE);
            return;
        }
        var change = cashBox.planChange(balance - price);
        if (change.isEmpty()) { refundAll("Cannot make exact change."); return; }   // refuse, never short-change
        cashBox.remove(change.get());
        tray.add(selected.dispenseOne().name());
        if (!change.get().isEmpty()) tray.add("change " + change.get());
        show("Dispensed. Please take: " + tray);
        balance = 0;
        selected = null;
        setState(DispensingState.INSTANCE);
    }

    void refundAll(String reason) {                       // ONE refund path: cancel and "cannot make change"
        Map<Denomination, Integer> back = cashBox.planChange(balance)
                .orElseThrow(() -> new IllegalStateException("inserted pieces must be refundable"));
        cashBox.remove(back);
        show(reason + " Refunded " + balance + " c as " + back);
        balance = 0;
        selected = null;
        setState(IdleState.INSTANCE);
    }

    void emptyTray() {                                    // the sale ends when the customer takes the product
        tray.clear();
        setState(inventory.allEmpty() ? OutOfStockState.INSTANCE : IdleState.INSTANCE);
    }
}
// Main.java: A1 Cola 125c x2, B1 Chips 50c x1, C1 Candy 65c x0; float of 2 quarters + 1 dime; six scenarios below.
```

`refundAll` can never fail: the pieces just inserted are in the box, so an exact plan always exists. All public methods are `synchronized`: one physical machine has one customer, but a networked controller (remote price change, telemetry — status data sent automatically to a remote server — or a second input device) must not interleave two actions.

Output:

```
state=IDLE cash=60c stock=[A1:Colax2 B1:Chipsx1 C1:Candyx0]

1) Happy path: Cola with $1 + 25c. Extra coins and a late cancel are refused
 > select A1   (IDLE)
   [display] Cola costs 125 c. Insert money.
 > insert 100c   (SELECTED)
   [display] Paid 100 c, 25 c to go.
 > insert 25c   (HAS_MONEY)
   [display] Dispensed. Please take: [Cola]
 > insert 25c   (DISPENSING)
   [display] QUARTER returned: take your product first.
 > insert 10c   (DISPENSING)
   [display] DIME returned: take your product first.
 > cancel   (DISPENSING)
   [display] Too late to cancel: product already dispensed.
 > take   (DISPENSING)
   [display] Customer took [Cola]

2) Wrong order and bad input
 > insert 25c   (IDLE)
   [display] QUARTER returned: select a product first.
 > select C1   (IDLE)
   [display] C1 is sold out. Choose another.
 > select Z9   (IDLE)
   [display] Unknown code Z9
 > select B1   (IDLE)
   [display] Chips costs 50 c. Insert money.
 > insert 3c   (SELECTED)
   [display] Unknown piece of 3c returned.

3) Cancel with money in: full refund
 > insert 25c   (SELECTED)
   [display] Paid 25 c, 25 c to go.
 > select A1   (HAS_MONEY)
   [display] Cancel first to change your choice.
 > cancel   (HAS_MONEY)
   [display] Cancelled. Refunded 25 c as {QUARTER=1}

4) Cannot make exact change -> sale refused, everything refunded
 > select B1   (IDLE)
   [display] Chips costs 50 c. Insert money.
 > insert 500c   (SELECTED)
   [display] Cannot make exact change. Refunded 500 c as {FIVE_DOLLAR=1}
state=IDLE cash=185c stock=[A1:Colax1 B1:Chipsx1 C1:Candyx0]

5) Last items sold -> OUT_OF_STOCK -> operator restocks
 > select B1   (IDLE)
   [display] Chips costs 50 c. Insert money.
 > insert 25c   (SELECTED)
   [display] Paid 25 c, 25 c to go.
 > insert 25c   (HAS_MONEY)
   [display] Dispensed. Please take: [Chips]
 > take   (DISPENSING)
   [display] Customer took [Chips]
 > select A1   (IDLE)
   [display] Cola costs 125 c. Insert money.
 > insert 100c   (SELECTED)
   [display] Paid 100 c, 25 c to go.
 > insert 25c   (HAS_MONEY)
   [display] Dispensed. Please take: [Cola]
 > take   (DISPENSING)
   [display] Customer took [Cola]
state=OUT_OF_STOCK cash=360c stock=[A1:Colax0 B1:Chipsx0 C1:Candyx0]
 > select A1   (OUT_OF_STOCK)
   [display] Machine is out of stock.
   [display] Operator restocked C1 +5
 > select C1   (IDLE)
   [display] Candy costs 65 c. Insert money.
 > insert 100c   (SELECTED)
   [display] Dispensed. Please take: [Candy, change {DIME=1, QUARTER=1}]
```

```
  > take   (DISPENSING)
    [display] Customer took [Candy, change {DIME=1, QUARTER=1}]

6) Why the change planner is not greedy
30c from {1 quarter, 3 dimes}: greedy=Optional.empty dp=Optional[{DIME=3}]

Operator collects notes: 300 c
state=IDLE cash=125c stock=[A1:Colax0 B1:Chipsx0 C1:Candyx4]
```

Read the output: part 1 — coins inserted while the product waits are returned and a late cancel is refused; part 4 — 450¢ change cannot be made, so the $5 note comes straight back; part 5 — `OUT_OF_STOCK` and back to `IDLE` after a restock; part 6 — greedy fails where DP succeeds.

Concurrency & edge cases

Edge case	Handling
Money inserted before selecting	`IdleState.insert` returns the piece
Unknown coin or foreign note	Returned before reaching the state
Selecting an empty slot	`IdleState.select` refuses; the state stays Idle
Changing the choice after paying	`HasMoneyState.select` says "cancel first" (prevents price confusion)
Paying more than the price	Change is planned **before** dispensing; if impossible, the sale is refused
Cancel after dispensing	"Too late" — the product is already in the tray
Two products for one payment	Impossible: after dispensing, the balance is 0 and the state is Dispensing, which accepts no money
Last item sold	`take` moves to `OutOfStock`; only a restock leaves it
Two threads pressing buttons	`synchronized` public methods; states run under the machine's lock
A bank of machines sharing one stock count	The course notes the design is not safe for that; the shared count on `Slot` would then need a lock or an atomic decrement
Power failure mid-sale	Real machines keep a transaction log (credit held) in non-volatile memory (memory that keeps its contents without power); out of scope

Extensions & follow-up questions

1. **"Where would you add contactless card payment?"** — One new input device beside the cash slot that calls the same machine (for example a `payByCard(amount)` that sets the balance to the price). No state class changes if the states only look at the balance.
2. **"What if it cannot give change?"** — The planner returns empty; the sale is refused and everything is refunded (requirement 7). Mention greedy vs DP.
3. **"Add a maintenance state."** — A new `MaintenanceState` class; the operator switches it on. The existing five states are untouched (Open/Closed).
4. **"Different price per slot for the same product?"** — Move the price from `Product` to `Slot` (the slot already owns the count).
5. **"Report empty slots to headquarters."** — Make `Inventory` notify an observer when a slot reaches 0; a telemetry client sends it over the network.
6. **"What did you leave out on purpose?"** — Card payment, reporting empty slots over a network, refrigeration (chilling) control and per-customer discounts. Say so explicitly: they are scope decisions, not oversights.
7. **"Timeout if the customer walks away with money in?"** — A timer that calls `cancel()` after, say, 60 seconds of inactivity; it goes through the same state machine.

Real-world example

Modern vending machines are often retro-fitted with cashless readers such as **Nayax** or **Cantaloupe (ePort)** units, which plug into the machine's controller as just another payment device — exactly extension 1: a new input beside the coin mechanism, with the dispensing logic unchanged. Coin mechanisms in real machines keep "change tubes" of each coin and show an **"exact change only"** light when a tube runs low — the physical version of our planner reporting that it cannot make change.

In the interview

Plan for 45 minutes: 5 min questions and requirements → 5 min entities (product, slot, inventory, cash box) → **10 min: draw the state diagram and the state table** → 15 min code the interface, two or three states and `checkPaid` / `refundAll` → 10 min edge cases and follow-ups.

> **30-second answer:** "The whole question is the order of events, so I'll use the State pattern: one class per situation — Idle, Selected, HasMoney, Dispensing, OutOfStock — all implementing an interface that lists every button. The machine just forwards to its current state. Money goes into the cash box as it is inserted, so a refund is just 'make exact change for the balance'; if exact change is impossible, the sale is refused and everything is returned. Amounts are integer cents."

Follow-ups with answers

- *"Why not an enum with a switch?"* — It works for tiny machines, but every new state means editing every switch; with the State pattern a new state is a new class, and the compiler forces it to answer every button.
- *"Is greedy change correct?"* — Only for well-stocked canonical coin sets (coin systems such as 1/5/10/25 where taking the largest coin first always gives the fewest coins). With limited coins it can fail; use bounded coin-change DP.
- *"Where does the operator fit?"* — Outside the state interface; restocking is allowed in any state and can move OutOfStock → Idle.

TL;DR

- **State pattern is the whole design**: five state classes, one interface listing every button, no `if` on state in the machine.
- Count lives on the **slot**; money in **integer cents**; inserted money goes **into the box immediately**.
- Change: **exact** or refuse; greedy can fail, **DP** (bounded coin change) is the correct planner.
- One refund path for cancel and "no change"; operator actions stay off the state interface.

14.3 Design a Coffee Vending Machine

A **coffee vending machine** makes drinks on demand — espresso, americano, latte, cappuccino, mocha, hot chocolate — from a shared stock of **ingredients** (beans, water, milk, chocolate, sugar). It looks like the previous problem, but the hard part is different. A snack machine hands out a finished item from a slot. A coffee machine **consumes several ingredients at once** from shared containers, and many drinks draw on the same milk. The interviewer wants to see three things:

1. **Recipes as data** — adding a drink should be a data change, not a new class.
2. **Atomic, all-or-nothing deduction** — a latte must take its beans, water *and* milk together, or take nothing; stock must never go negative, even when ten people order at once.
3. **Low-stock alerts** — the operator hears about milk running low *before* it runs out (Observer).

Problem & clarifying questions

Question	Assumed answer
Which drinks?	Espresso, Americano, Latte, Cappuccino, Mocha, Hot Chocolate — loaded from a menu file.
Customisation?	Extra espresso shot (+50¢) and sugar (grams).
Payment?	Amount paid in cents; change returned (coin handling was covered in 14.2).
Several screens / concurrent orders?	Yes: a large machine or a kiosk with several touch screens, so orders can overlap.
Who refills?	An operator, alerted when an ingredient falls to a threshold.
One alert or one per drink?	One alert per crossing of the threshold (re-armed after a refill).

Requirements

Functional

1. Several drink types, each with a **price** and a **recipe** (amount of each ingredient).
2. Show a **menu** with prices and availability.
3. Accept a selection and a **payment**; return **change**; refund in full if the drink cannot be made.
4. Track **ingredient inventory**; deduct exactly the recipe amounts per drink.
5. Raise **low-stock notifications** when an ingredient drops to its threshold.
6. Allow **refills**.
7. Support **customisation** (extra shot, sugar) without changing the menu.

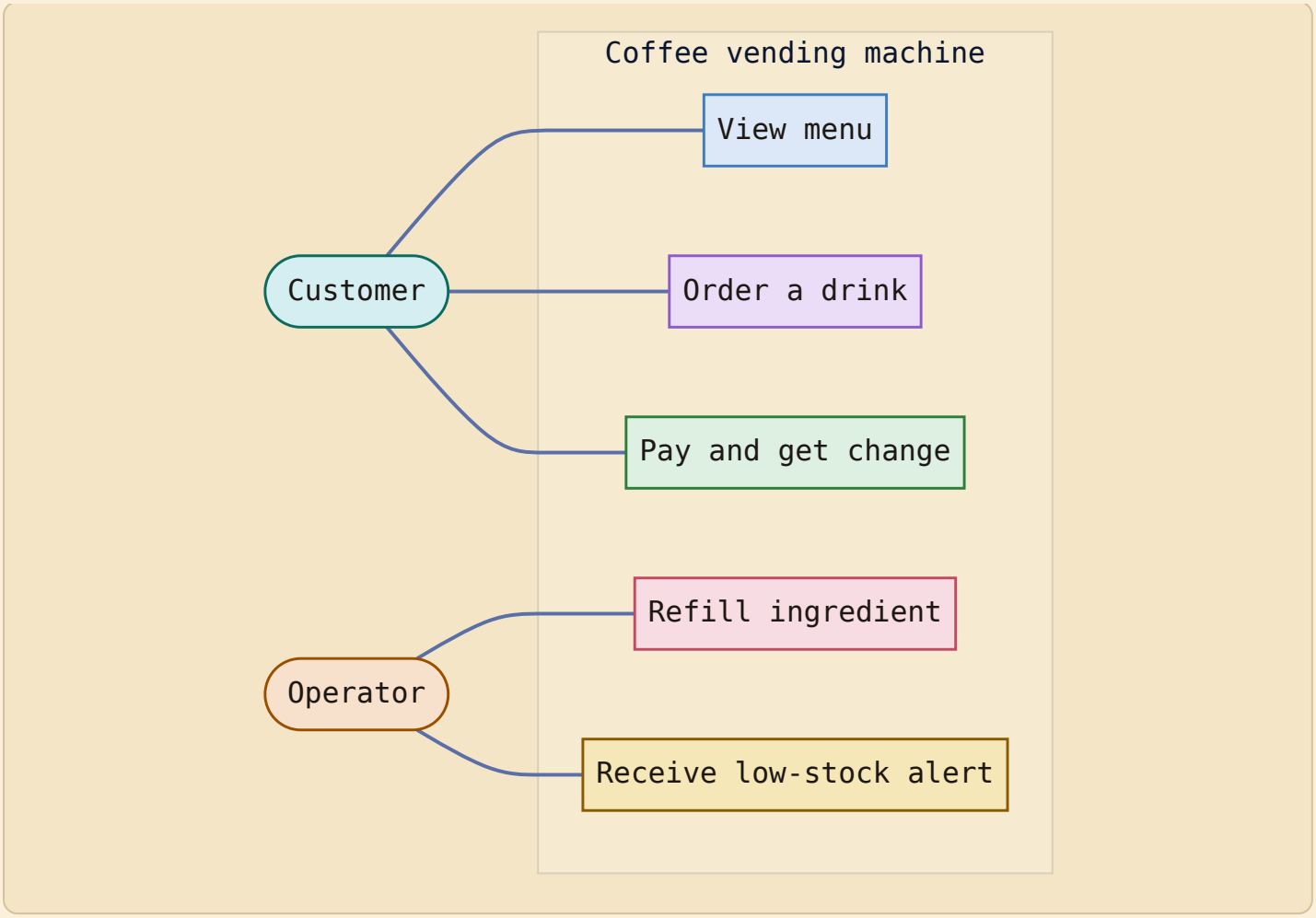
Non-functional

- **Thread-safe**: concurrent orders must never over-draw an ingredient or leave a half-deducted recipe.
- **Data-driven**: new drinks and recipe tweaks without code changes.
- **Observable**: operators, dashboards or a telemetry client can subscribe to alerts without the machine knowing about them.

Use cases

Actors: **Customer** (views menu, orders, pays, takes drink and change), **Operator** (refills, receives alerts), **System** (checks stock, deducts, brews, sends alerts).

Use cases: view menu · order drink (with customisation) · pay / get change / get refund · refill ingredient · receive low-stock alert.



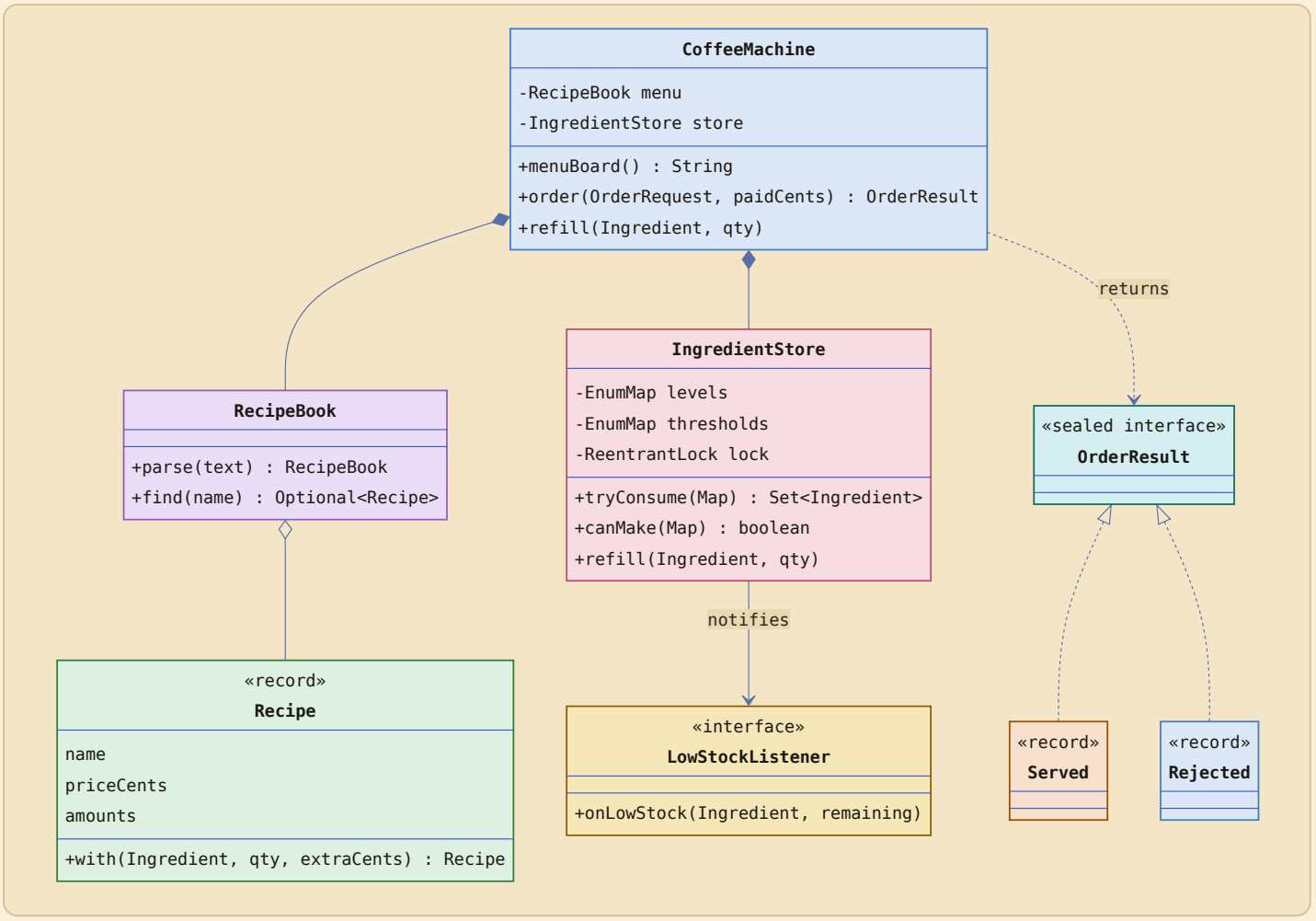
Core entities

Class	Responsibility
`Ingredient` (enum)	Beans, water, milk, chocolate, sugar — with the unit (g or ml).
`Recipe` (record)	**Data**: name, price, `Map<Ingredient, Integer>` amounts. Immutable (cannot change after creation). `with(...)` returns a customised copy.
`RecipeBook`	Parses the menu text into recipes; looks drinks up by name.
`IngredientStore`	Levels, thresholds, **one lock**; `tryConsume` = check all + deduct all atomically; alerts listeners.
`LowStockListener` (interface)	Observer called once when an ingredient crosses its threshold.
`OrderRequest` (record)	Drink name + extra shots + sugar grams.
`OrderResult` (sealed)	Either `Served(drink, price, change)` or `Rejected(reason, refund)`.
`CoffeeMachine`	Facade (one simple front object hiding the parts behind it): menu board, `order(...)`, `refill(...)`.

Mapping to the common problem statement. The widely used version (e.g. the open-source *awesome-low-level-design* collection) asks for coffee types with price and recipe, a menu, selection and payment, dispensing with change, ingredient tracking with low-stock notifications, and thread-safe concurrent requests. Its `Coffee` (name, price, recipe) → our `Recipe`; `Ingredient` (name + quantity with `synchronized` updates) → our `Ingredient` enum plus the levels in `IngredientStore`; `Payment` → the `paidCents` argument and change/refund in `OrderResult`; `CoffeeMachine` (a Singleton there) → our injected `CoffeeMachine`; concurrent users via an `ExecutorService` (a pool of worker threads running submitted tasks) → our 10-thread demo. Two deliberate differences: we lock the **whole recipe**, not each ingredient

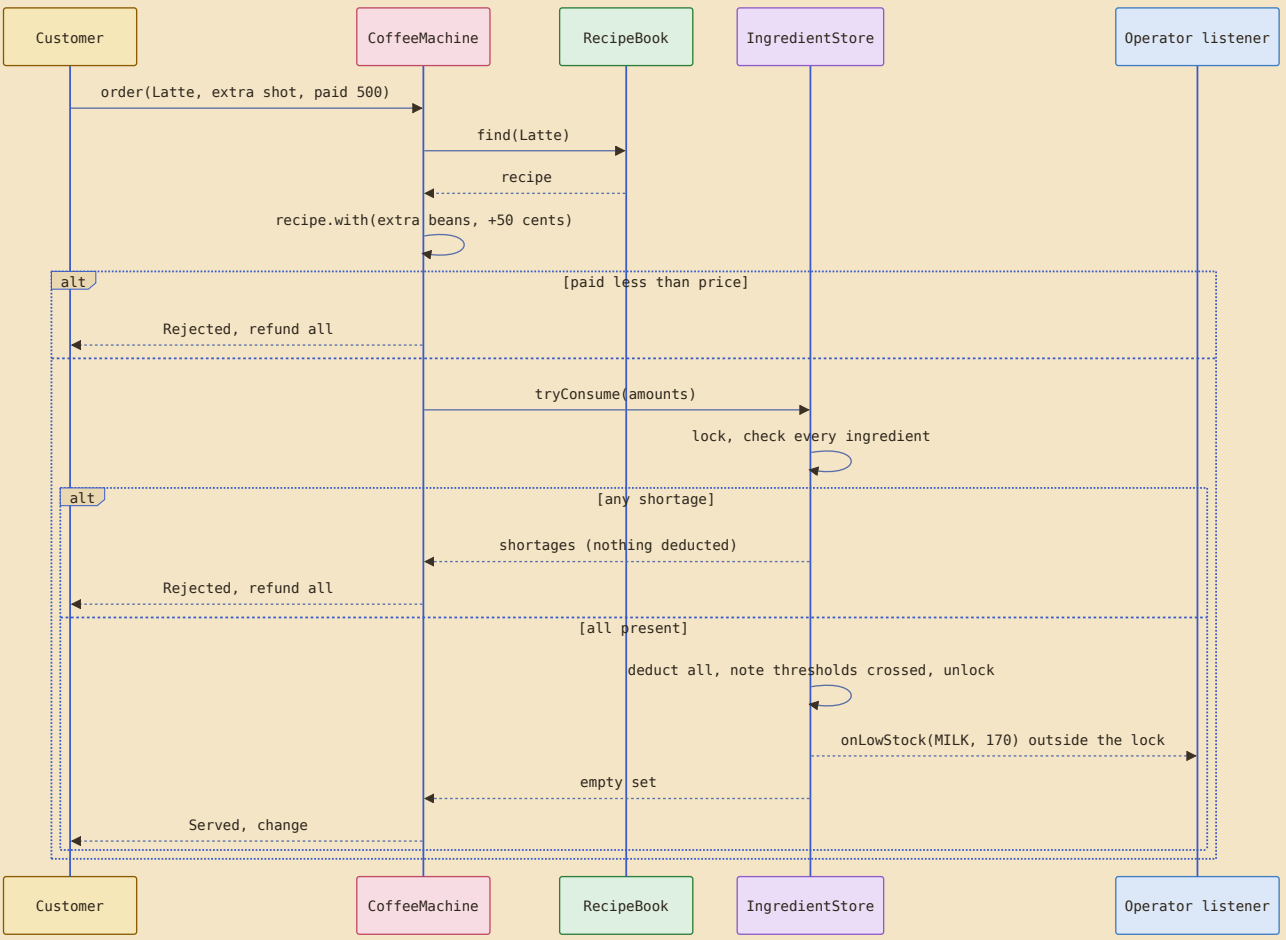
(per-ingredient locking cannot make a multi-ingredient deduction all-or-nothing), and we avoid the Singleton.

Class diagram

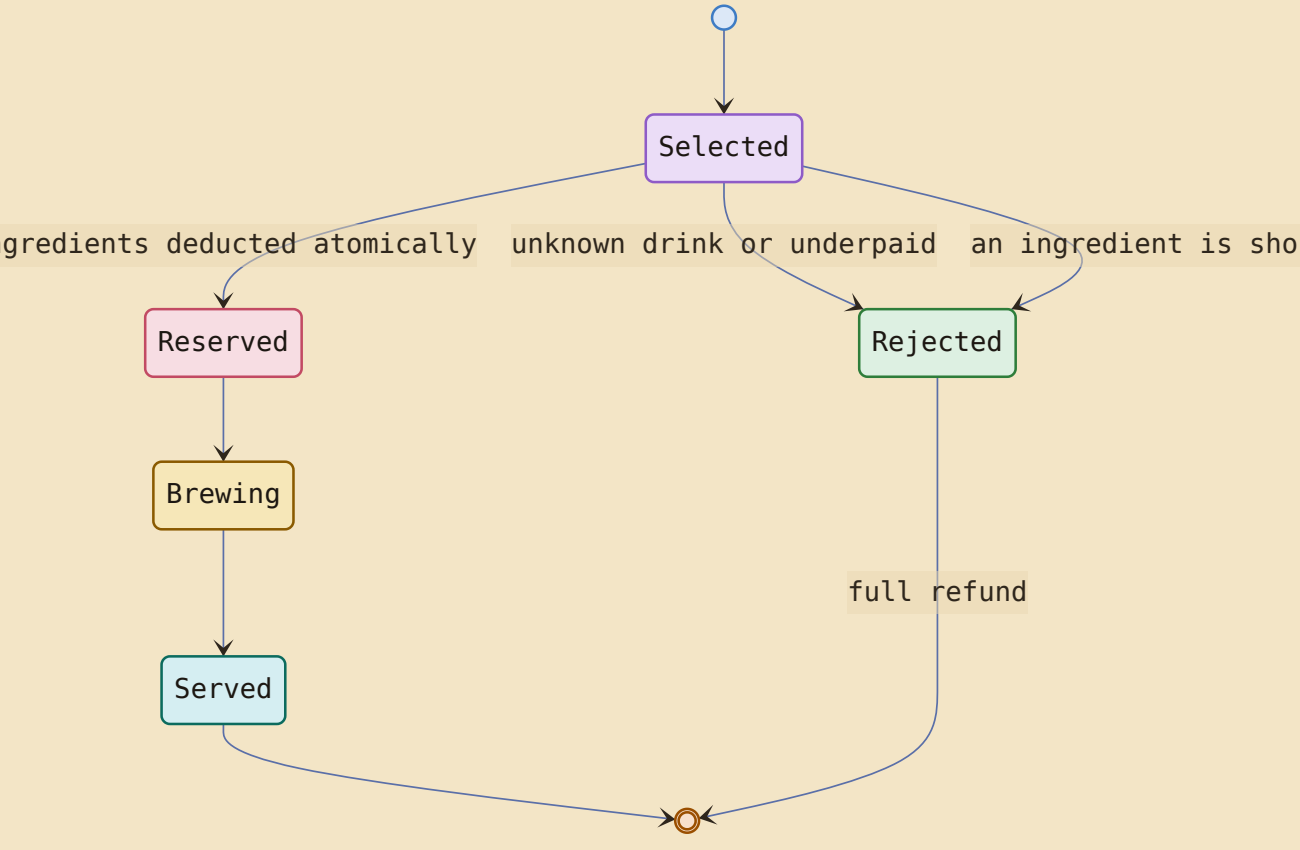


Key flows

Ordering a drink. The key point is that the stock check and the deduction happen **inside one critical section** (a piece of code that only one thread may run at a time), and alerts are sent **after** the lock is released.



Order life cycle.



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Recipes are **data** parsed from a menu text	Data-driven design, Open/Closed	A new drink is a new line in the menu; no new class, no recompile of logic	One subclass per drink (`Latte extends Coffee`) — class explosion, and recipe tweaks need code changes
Check-and-deduct under **one lock**	Atomicity (Ch. 12)	Prevents two orders both seeing "enough milk" and both deducting (a check-then-act **race**: both check, then both act on an answer that is already stale)	Checking in one call and deducting in another; per-ingredient locks (a multi-ingredient recipe would need all of them, in order)
All-or-nothing	Transaction semantics (all steps take effect together, or none do)	Never leave a half-made recipe's ingredients missing	Deduct one by one and "roll back" on failure
Alerts via `LowStockListener`	**Observer**	Operator app, dashboard, telemetry can subscribe independently	The store calling a specific SMS class
Alert **once per crossing**, re-armed by refill	—	Prevents alert storms (one per drink)	Alerting on every order below threshold
Listeners called **outside** the lock	Concurrency hygiene	A slow listener (network call) must not block every order; it also avoids deadlocks (two threads each waiting forever for a lock the other holds) if a listener calls back into the store	Notifying inside the lock
Customisation returns a **new** immutable `Recipe`	Immutability	The menu recipe can never be corrupted by one order	Mutating the menu recipe; a Decorator chain (works, but heavier than needed for "extra X")
`OrderResult` is a **sealed** interface with records	Algebraic data type (a value that is exactly one of a fixed set of shapes; Java 17+ sealed, Java 21 pattern switch)	Callers must handle both Served and Rejected; `switch` is checked for exhaustiveness (the compiler complains if a case is missing)	Returning `null` or throwing for normal refusals
No Singleton	Dependency injection	The problem-set solutions often make `CoffeeMachine` a Singleton; we create one and pass it around, so tests can build fresh machines (the demo builds two)	`CoffeeMachine.getInstance()`

> **Decorator or data?** A classic textbook example (*Head First Design Patterns*) models coffee add-ons with the **Decorator** pattern: `new Whip(new Mocha(new DarkRoast()))`, each wrapper adding cost and description. That is a good fit when each add-on has **behaviour**. Here add-ons only change **quantities and price**, so a data change (`recipe.with(...)`) is simpler. Say both in the interview and pick one for a reason.

Java implementation

Package `lld.ch14.coffeemachine`. Essential lines:

```
// Recipes.java
enum Ingredient { COFFEE_BEANS("g"), WATER("ml"), MILK("ml"), CHOCOLATE("g"), SUGAR("g"); final String unit; }

record Recipe(String name, int priceCents, Map<Ingredient, Integer> amounts) {   // a recipe is DATA
    Recipe { amounts = Collections.unmodifiableMap(new EnumMap<>(amounts)); }   // compact constructor: defensive copy
    Recipe with(Ingredient extra, int qty, int extraCents) {     // customisation returns a NEW recipe
        if (qty == 0) return this;
        Map<Ingredient, Integer> m = new EnumMap<>(Ingredient.class);
        m.putAll(amounts);
        m.merge(extra, qty, Integer::sum);
        return new Recipe(name, priceCents + extraCents, m);
    }
}
// RecipeBook.parse(text): each non-comment line "name, price_cents, INGREDIENT=qty, ..." -> a Recipe;
// find(name) is case-insensitive. Main.MENU is a text block, e.g.  Latte, 400, COFFEE_BEANS=18, WATER=40, MILK=180
```

```
// IngredientStore.java - the whole check-then-deduct runs under ONE lock: all-or-nothing, never negative
interface LowStockListener { void onLowStock(Ingredient ingredient, int remaining); }

final class IngredientStore {
    private final Map<Ingredient, Integer> levels = new EnumMap<>(Ingredient.class);
    private final Map<Ingredient, Integer> thresholds = new EnumMap<>(Ingredient.class);
    private final Set<Ingredient> alerted = EnumSet.noneOf(Ingredient.class);
    private final List<LowStockListener> listeners = new CopyOnWriteArrayList<>();
    private final ReentrantLock lock = new ReentrantLock();

    /** Returns the ingredients that are short. Empty set = success, everything was deducted. */
    Set<Ingredient> tryConsume(Map<Ingredient, Integer> need) {
        Set<Ingredient> shortages = EnumSet.noneOf(Ingredient.class);
        List<Runnable> alerts = new ArrayList<>();
        lock.lock();
        try {
            need.forEach((i, q) -> { if (levels.get(i) < q) shortages.add(i); });
            if (!shortages.isEmpty()) return shortages;               // nothing touched
            need.forEach((i, q) -> levels.merge(i, -q, Integer::sum));
            for (Ingredient i : need.keySet()) {
                int left = levels.get(i);
                if (left <= thresholds.get(i) && alerted.add(i))      // alert once per crossing
                    alerts.add(() -> listeners.forEach(l -> l.onLowStock(i, left)));
            }
        } finally {
            lock.unlock();
        }
        alerts.forEach(Runnable::run);                                 // listeners OUTSIDE the lock
        return shortages;
    }
    void refill(Ingredient i, int qty) {
        lock.lock();
        try {
            levels.merge(i, qty, Integer::sum);
            if (levels.get(i) > thresholds.get(i)) alerted.remove(i); // re-arm the alert
        } finally { lock.unlock(); }
    }
    // canMake(need) and level(i) read under the same lock
}
```

```java
// CoffeeMachine.java
record OrderRequest(String drink, int extraShots, int sugarGrams) {}
sealed interface OrderResult permits Served, Rejected {}            // callers must handle both
record Served(String drink, int pricedCents, int changeCents) implements OrderResult {}
record Rejected(String reason, int refundCents) implements OrderResult {}

final class CoffeeMachine {                       // EXTRA_SHOT_CENTS = 50, SHOT_BEANS_G = 18
    OrderResult order(OrderRequest req, int paidCents) {             // thread-safe: many touch screens
        var base = menu.find(req.drink());
        if (base.isEmpty()) return new Rejected("no such drink: " + req.drink(), paidCents);
        Recipe r = base.get()
                .with(Ingredient.COFFEE_BEANS, req.extraShots() * SHOT_BEANS_G, req.extraShots() *
EXTRA_SHOT_CENTS)
                .with(Ingredient.SUGAR, req.sugarGrams(), 0);
        if (paidCents < r.priceCents()) return new Rejected("insufficient payment, need " + r.priceCents() + " c",
paidCents);
        Set<Ingredient> short_ = store.tryConsume(r.amounts());     // atomic reserve-and-deduct
        if (!short_.isEmpty()) return new Rejected("out of " + short_, paidCents);
        return new Served(r.name(), r.priceCents(), paidCents - r.priceCents());
    }
    // menuBoard(): marks a drink "(unavailable)" when !store.canMake(recipe.amounts()); refill(i, qty)
}
// Main.show uses a Java 21 pattern switch over the sealed type:
//   case Served s -> ...;  case Rejected x -> ...
// Last demo: 10 threads released by a CountDownLatch each order a Latte with milk for exactly 4.
```

A Java text block is a multi-line string between `"""` marks; a compact constructor is a record constructor without a parameter list, used for checks and defensive copies. `ReentrantLock` is an explicit lock with `lock()`/`unlock()` (*re-entrant*: the holder may take it again); `EnumSet` is a compact set of enum constants.

Output:

```
Menu:
  Espresso        250 c
  Americano       300 c
  Latte           400 c
  Cappuccino      380 c
  Mocha           450 c
  Hot Chocolate   300 c

== Orders ==
SERVED   Espresso for 250 c, change 50 c
SERVED   Latte for 450 c, change 50 c
REJECTED no such drink: Flat White, refund 400 c
REJECTED insufficient payment, need 450 c, refund 200 c
SERVED   Mocha for 450 c, change 0 c
   ALERT to operator: MILK low, 170ml left
   ALERT to operator: CHOCOLATE low, 5g left
SERVED   Hot Chocolate for 300 c, change 0 c
SERVED   Cappuccino for 380 c, change 20 c
REJECTED out of [MILK], refund 400 c
   beans left 410g (unchanged by the rejected latte)
Menu now:
  Espresso        250 c
  Americano       300 c
  Latte           400 c  (unavailable)
  Cappuccino      380 c  (unavailable)
  Mocha           450 c  (unavailable)
  Hot Chocolate   300 c  (unavailable)

== Operator refills milk ==
SERVED   Latte for 400 c, change 0 c

== 10 customers order a Latte at the same instant (milk for exactly 4) ==
   ALERT to operator: MILK low, 80ml left
served=4 rejected=6 milk left=80ml (never negative)
```

Read the output: the rejected latte left the beans at 410 g (nothing deducted when any ingredient is short); milk and chocolate alerts fire once each; ten simultaneous lattes with milk for four → four served, six refused, milk ends at 80 ml (never negative), one alert.

Concurrency & edge cases

Situation	What could go wrong	How the design handles it
Ten concurrent orders, milk for four	Over-draw (negative stock) — classic check-then-act race	Check and deduct under one lock
Recipe needs several ingredients, one is short	Beans and water taken, milk missing	All-or-nothing: shortages are found before anything is deducted
Listener is slow (sends an SMS)	Every order waits on the network	Listeners run after `unlock()`
Many orders below the threshold	Alert storm	One alert per crossing; refill re-arms
Underpayment	Drink made, money short	Payment checked before reserving ingredients
Brewing fails after deduction (hardware fault)	Customer pays, gets nothing	Extension: refund and optionally `refill` the reserved amounts back (compensation: an action that undoes an earlier step)
Menu displays "available" and then the order fails	A race between display and order	The menu is advisory; `tryConsume` is the source of truth

A **single lock** is right here: the critical section is tiny (five map operations) and the machine brews one drink at a time anyway. If contention (many threads waiting for the same lock) mattered (a cloud-backed inventory shared by many machines), you would move to optimistic concurrency (work without a lock, then check at write time that nobody changed the data, and retry if they did — e.g. version numbers / compare-and-set on a snapshot) or a database transaction (a group of database changes that succeed or fail together).

Extensions & follow-up questions

1. **"Add a new drink."** — Add one line to the menu data. No code change.
2. **"Sizes (small/medium/large)?"** — A `Size` enum with a multiplier; `recipe.scaled(size)` returns a new recipe with scaled amounts and price.
3. **"Several brewing units?"** — A pool of `Brewer` workers (a `BlockingQueue` — a thread-safe queue where a taker waits while it is empty — of orders consumed by N threads). Inventory stays in the one `IngredientStore`.
4. **"Remote monitoring?"** — Another `LowStockListener` that posts to a telemetry endpoint. The store does not change.
5. **"Brewing steps differ per drink (grind, tamp, steam milk)?"** — Template Method (a base class fixes the order of the steps and subclasses fill in some of them): a `BrewProcess` with fixed steps and hooks (optional steps a subclass may override), or a list of `Step` objects in the recipe data (Command).
6. **"Card payments and change?"** — Reuse the `CashBox` from 14.2 for cash; card payment is another payment device.

Real-world example

Professional espresso machines such as the Thermoplan Mastrena used in Starbucks stores store **drink recipes as configuration**, so a chain can roll out a new beverage without new hardware. Starbucks and Microsoft publicly described (in 2019) connecting store equipment, including espresso machines, to the cloud through Microsoft Azure Sphere: each shot's data is collected as telemetry and new recipes are pushed from the cloud instead of carried to stores on USB drives — "recipes as data" plus a remote-

monitoring listener. Many connected self-serve kiosks report stock remotely so a service visit is scheduled before the milk runs out — the low-stock Observer.

In the interview

Plan for 40 minutes: 5 min questions → 5 min entities (ingredient, recipe, store, machine) → 5 min "recipes are data" and the menu format → 15 min code `tryConsume` (lock, all-or-nothing, alert outside the lock) and `order` → 10 min concurrency demo and extensions.

> **30-second answer:** "Recipes are data: a map from ingredient to amount, loaded from a menu, so a new drink is a new row. The ingredient store checks and deducts **every** ingredient of a recipe under one lock, so an order is all-or-nothing and stock never goes negative. When an ingredient crosses its threshold, the store notes it and, after releasing the lock, notifies observers once. Orders return a sealed Served/Rejected result, and a rejected order is refunded in full."

Follow-ups with answers

- *"Why not one class per drink?"* — Drinks differ only in data. Classes would multiply with every size and add-on (class explosion) and every recipe tweak would be a code change.
- *"Why not an `AtomicInteger` per ingredient?"* — Each atomic is safe alone, but a recipe updates several at once; without a common lock, two orders could each take the last milk and beans in different orders, leaving partial deductions.
- *"Why notify outside the lock?"* — A slow or re-entrant listener would block all orders or deadlock.

TL;DR

> - **Recipes are data** (`Map<Ingredient,Integer>` + price), parsed from a menu; customisation creates a new immutable recipe.
> - **One lock, check-then-deduct, all-or-nothing**: stock never goes negative under concurrent orders.
> - **Observer** for low-stock alerts, fired **once per crossing** and **outside** the lock.
> - A **sealed** `OrderResult` makes callers handle Served and Rejected; refund in full on rejection.

14.4 Design an ATM



Figure — An automated teller machine with its card reader, keypad, screen, cash dispenser, deposit slot and printer (course diagram)

An **automated teller machine (ATM)** is a cashier-less machine in a public spot where a bank's customers do everyday banking, because branches keep limited hours. The interviewer checks whether you keep **money correct** (never dispense without debiting, never debit without dispensing, never let two machines overdraw one account), **authenticate** before anything else, and model the **session states** cleanly.

Problem & clarifying questions

Question	Assumed answer
Which transactions?	Balance inquiry, cash withdrawal, cash deposit, check deposit, transfer.
Which account types?	Checking and savings. Withdrawal is from checking (and savings within a daily cap).
How is the user identified?	ATM card + PIN (personal identification number, a short secret code). The card is the main security element.
Does the ATM talk to the bank?	Yes, on every transaction (assume the network is available).
Are deposits available at once?	A check is **not**: it waits for verification. We keep a *total* and an *available* balance.
Which notes are loaded?	$100, $50, $20, $10 cassettes.
Wrong PIN?	Three wrong attempts block the card and the ATM keeps it. (The course deliberately leaves out PIN retry limits and card capture, along with receipt-paper levels, an audit journal and other banks' networks; we add the first two and the journal.)
How many customers at a time?	One per machine; the ATM must not shut down mid-session.

Requirements

Parts of the machine (from the course): **card reader** (reads the card), **keypad** (PIN, amounts), **screen** (messages; also used to pick a transaction), **cash dispenser**, **deposit slot** (cash or checks), **printer** (receipts), and a **network link** to the bank.

Functional

1. The customer inserts a card and enters a PIN; the ATM asks the **bank** to authenticate. No transaction is possible without authentication.
2. Five transactions: **balance inquiry** (each account), **deposit cash**, **deposit check**, **withdraw cash** (from checking), **transfer funds** to another account.
3. The ATM **keeps the card** for the whole session; the customer can **cancel** at any time and the card is ejected.
4. The ATM keeps an **internal log (journal)** of transactions, including hardware failures, for the operator to resolve issues.
5. A deposited **check is not added at once**: it waits for manual verification and bank approval. (The course assumes deposits in general are verified first; our code credits counted cash immediately and holds checks — see the trade-off below.)
6. The **bank manager** can see the ATM's system information (reports).

Actors' operations

- **Operator:** turn the ATM on/off with the key switch; refill cash; refill receipt paper and ink; take out deposited cash and checks.
- **Customer:** balance inquiry, withdraw, deposit (cash or check), transfer.
- **Bank manager:** reports of total deposits and total withdrawals; print them; check the cash remaining in the ATM.

Non-functional

- **Security:** PINs never stored or compared in clear on the ATM; limit PIN attempts.
- **Consistency:** debit and dispense happen together or not at all (compensate on failure).
- **Concurrency:** many ATMs act on the same account at once without overdrawing it or deadlocking.
- **Auditability** (every action can be traced afterwards)**:** every action is journaled.

Use cases

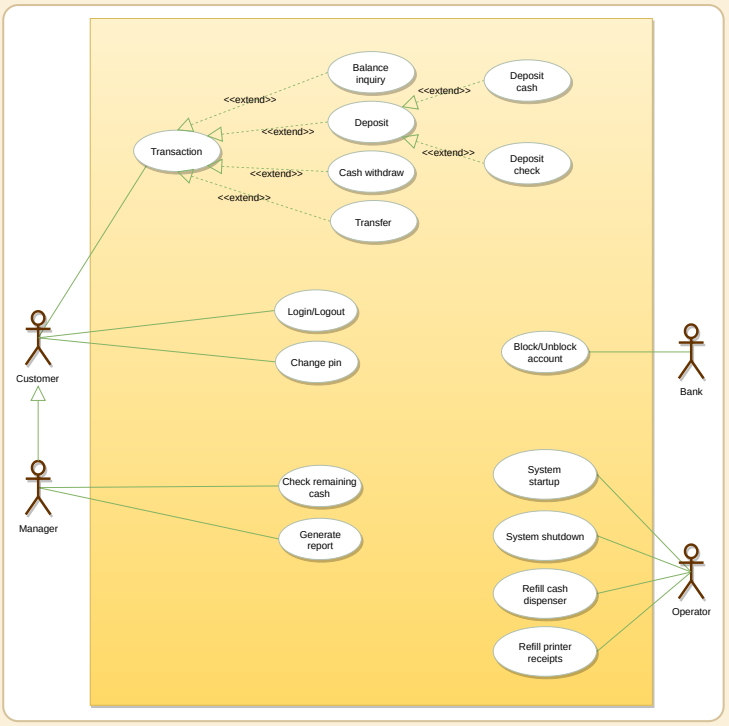


Figure — ATM use case diagram: Customer (balance, withdraw, deposit, transfer), Operator (on/off, refill cash, paper, ink, empty deposits) and Bank Manager (reports, check remaining cash) (course diagram)

The figure above shows three actors. Only the Customer's use cases go through PIN authentication; the Operator works with a physical key and the Bank Manager reads data held at the bank.

Customer authentication is the entry point of every session:

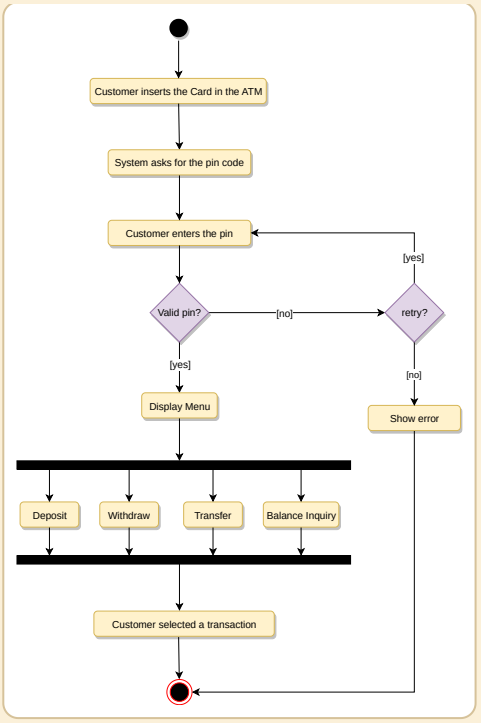


Figure — Activity diagram: insert card, the system asks for the PIN, the customer enters it; an invalid PIN shows an error and offers a retry; a valid PIN displays the menu (balance inquiry, deposit, withdraw, transfer) (course diagram)

Withdraw:

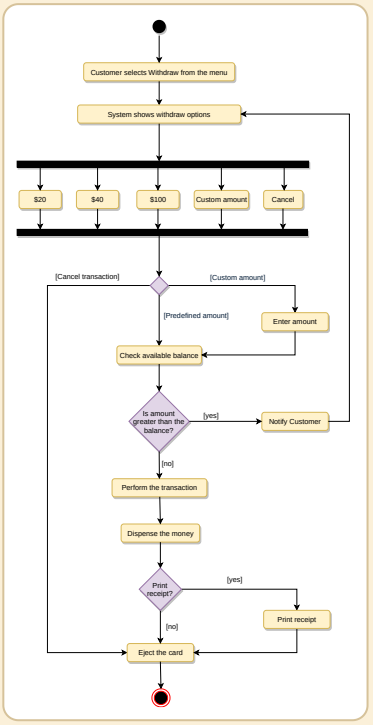


Figure — Activity diagram: choose Withdraw, pick a predefined amount ($20, $40, $100) or enter a custom one, check the available balance (notify the customer if the amount is greater), perform the transaction, dispense, optional receipt, eject the card (course diagram)

Deposit check:

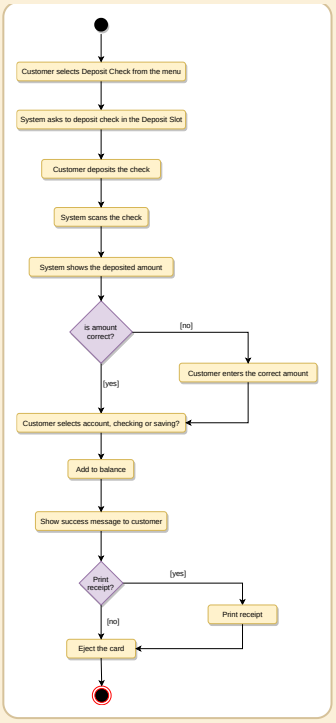


Figure — Activity diagram: choose Deposit Check, pick checking or savings, put the check in the deposit slot, the system scans it and shows the amount; if wrong the customer enters the correct amount; the amount is added, a success message and optional receipt follow, and the card is ejected (course diagram)

Transfer:

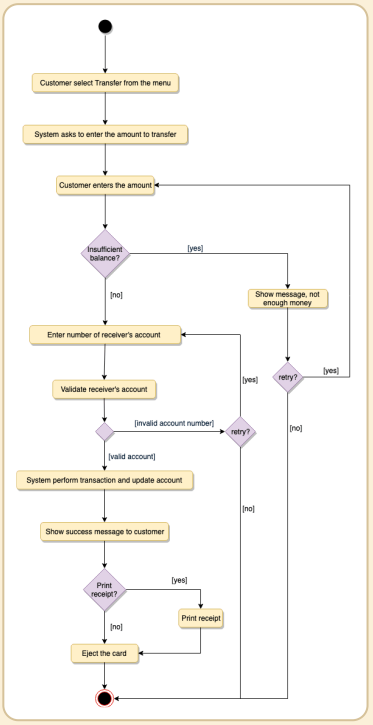


Figure — Activity diagram: enter the amount (retry if balance is insufficient), enter and validate the receiver's account (retry if invalid), perform the transfer, show success, optional receipt, eject card (course diagram)

Note the two retry loops in the transfer diagram ("insufficient balance", "invalid account number"). If the customer declines to retry, the diagram ends without "Eject the card"; in a real design every exit path must return the card — the card-reader rule in stage 2 below.

Core entities

Class	Responsibility
`Card` (record)	Card number and holder name. (The course also has a `Customer` with name and email owning exactly one card; we fold the holder's name into the card.)
`Account` (abstract)	Id, **total** and **available** balance, a lock; the withdrawal-rule hook.
`CheckingAccount`, `SavingsAccount`	Savings overrides the rule with a **daily withdrawal cap** — the only behavioural difference, hence two classes.
`BankService` (interface)	What the ATM needs from the bank: authenticate, balances, withdraw, reverse, deposits, transfer.
`InMemoryBank`	A stand-in bank: PIN hashes (one-way fingerprints of PINs), failed-attempt counts, account locks, ordered-lock transfers.
`AuthResult` (sealed)	`Ok(accounts)`, `WrongPin(attemptsLeft)` or `Blocked`.
`NoteHandler` + 4 subclasses	**Chain of Responsibility** for $100 → $50 → $20 → $10.
`CashDispenser`	Plans a withdrawal on the chain (dry run: a trial that changes nothing), then dispenses; can jam (fault injection: a deliberately triggered failure, for testing).
`Transaction` (sealed) + 5 records	**Command**: each transaction has one `execute(ctx)` holding its rules.
`AtmState` + 4 states	**State**: Idle → CardInserted → Authenticated → Idle, plus OutOfService.
`Atm`	Has an id (the course adds a location); owns its parts, holds the session, forwards buttons to the state, keeps the journal.
`CardReader`, `Keypad`, `Screen`, `Printer`, `DepositSlot` (cash / check)	The course's hardware classes, folded into `Atm`: the `card` field is the reader's slot, `screen(...)` the screen, method arguments the keypad; a receipt would format a journal line.

Class diagram

The course builds the ATM in four stages:

Stage	What it adds	What works after it
1	`Customer`, `Card`, `Account` (checking, savings)	Verify a PIN and read a balance
2	Card reader, dispenser, keypad, screen, printer, deposit slots	Work out and count out a set of notes
3	`Transaction` and its five kinds	Move money, with each rule written in exactly one class
4	`Bank` and `ATM`	Run a full session for a single customer at a time

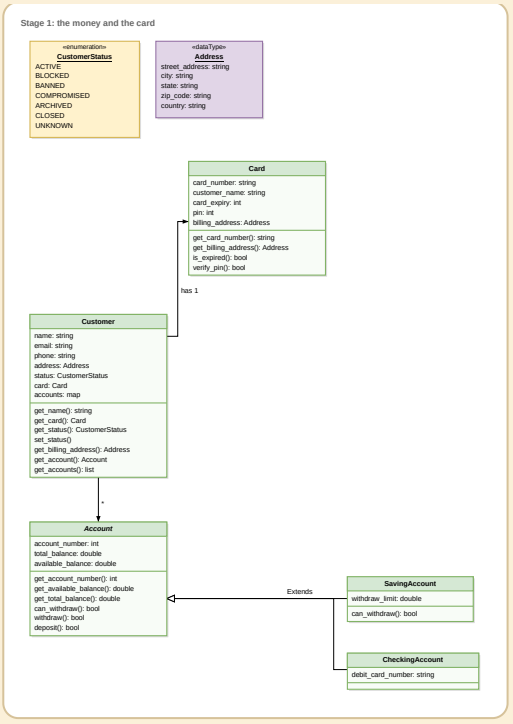


Figure — Stage 1: enumerations, Customer, Card and the abstract Account with Checking and Saving subclasses (course diagram)

Stage 1 — money and card. An account keeps **two balances**. The *total balance* is everything credited to it; the *available balance* is the part the customer may spend right now. A check deposit is the moment they diverge: the check raises the total at once, but the money becomes available only after the check clears. `Account` is abstract with `CheckingAccount` and `SavingsAccount` beneath it; savings overrides a single method to impose its withdrawal cap, and because that is a difference in **behaviour** (not just data) inheritance is justified here instead of a type field. `Customer` (name, email; each customer has **one** card) is linked to its card and accounts by plain associations, since none of them is deleted along with another.

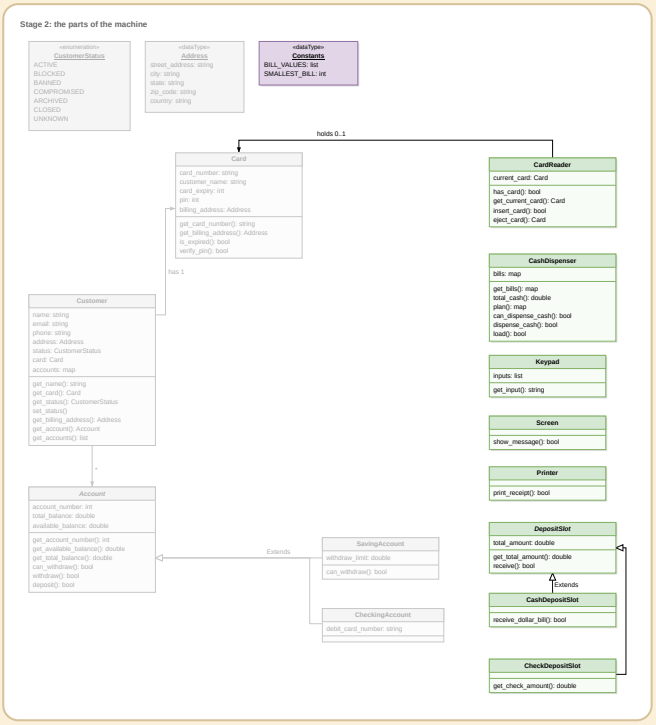


Figure — Stage 2: CardReader, CashDispenser, Keypad, Screen, Printer and the abstract DepositSlot with cash and check slots (course diagram)

Stage 2 — the parts. Six of the seven hardware parts become one class each (the network link is assumed, not modelled), and none knows what a transaction is. Whether an amount (say $85) can be paid from the notes **currently loaded** is the **dispenser's** decision, not the withdrawal's: it tries the largest

note first and reports failure when the exact amount is impossible. The card reader **keeps the card for the whole session** and releases it when the session ends, so every failed sign-in path must eject (or retain) the card; its link to `Card` is the only arrow leaving this stage. `DepositSlot` is abstract (cash slot, check slot) — the physical opening, not the deposit *transaction* of stage 3.

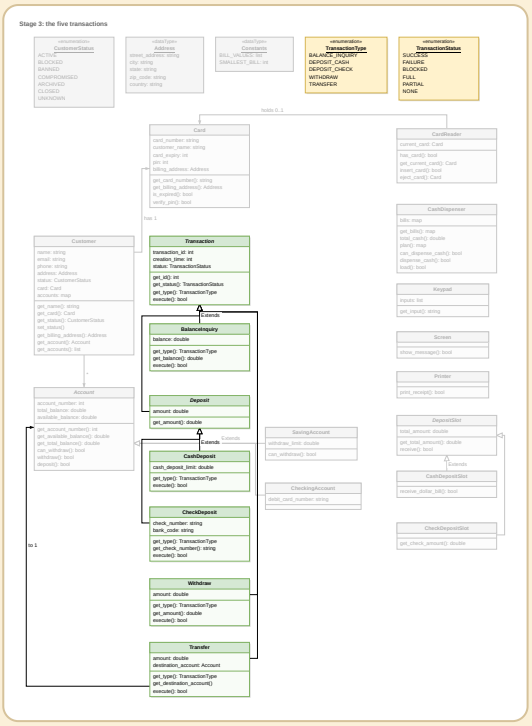


Figure — Stage 3: the abstract Transaction with BalanceInquiry, Deposit (CashDeposit, CheckDeposit), Withdraw and Transfer (course diagram)

Stage 3 — the five transactions. Each transaction type is a class with one `execute` method, so each rule has exactly one home. `Deposit` is intermediate: cash and check deposits share an amount but differ in when the money becomes spendable. `Transfer` is the only one with an outgoing arrow (to the destination account). The course's `Withdraw` checks **note denominations**, then the **account**, then the **dispenser**, and debits only once the notes are counted out — if you take the money first and dispensing fails, the customer has paid for cash never received.

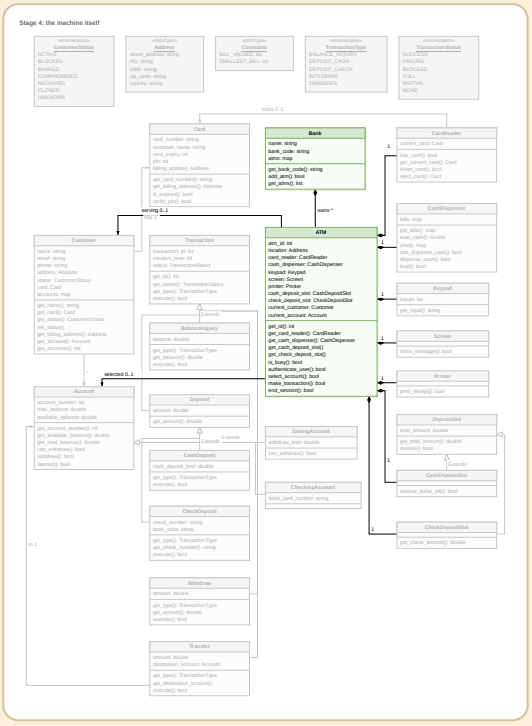


Figure — Stage 4: the ATM owns its parts (composition), refers to the current Customer and Account, and the Bank owns its ATMs (course diagram)

Stage 4 — the machine. The ATM has an **id** (`atmID`) and a **location** and **owns** its parts (composition: scrap the machine and its reader, dispenser and screen go too). Its links to the current `Customer` and `Account` are plain associations that change per person, so "one customer at a time" is enforced by structure, not a comment. The `Bank` owns its ATMs and all account data; the ATM asks the bank to carry out every customer transaction.

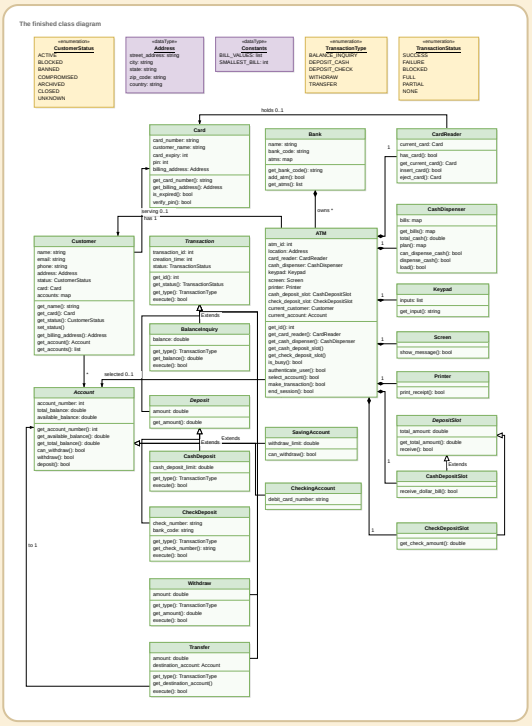
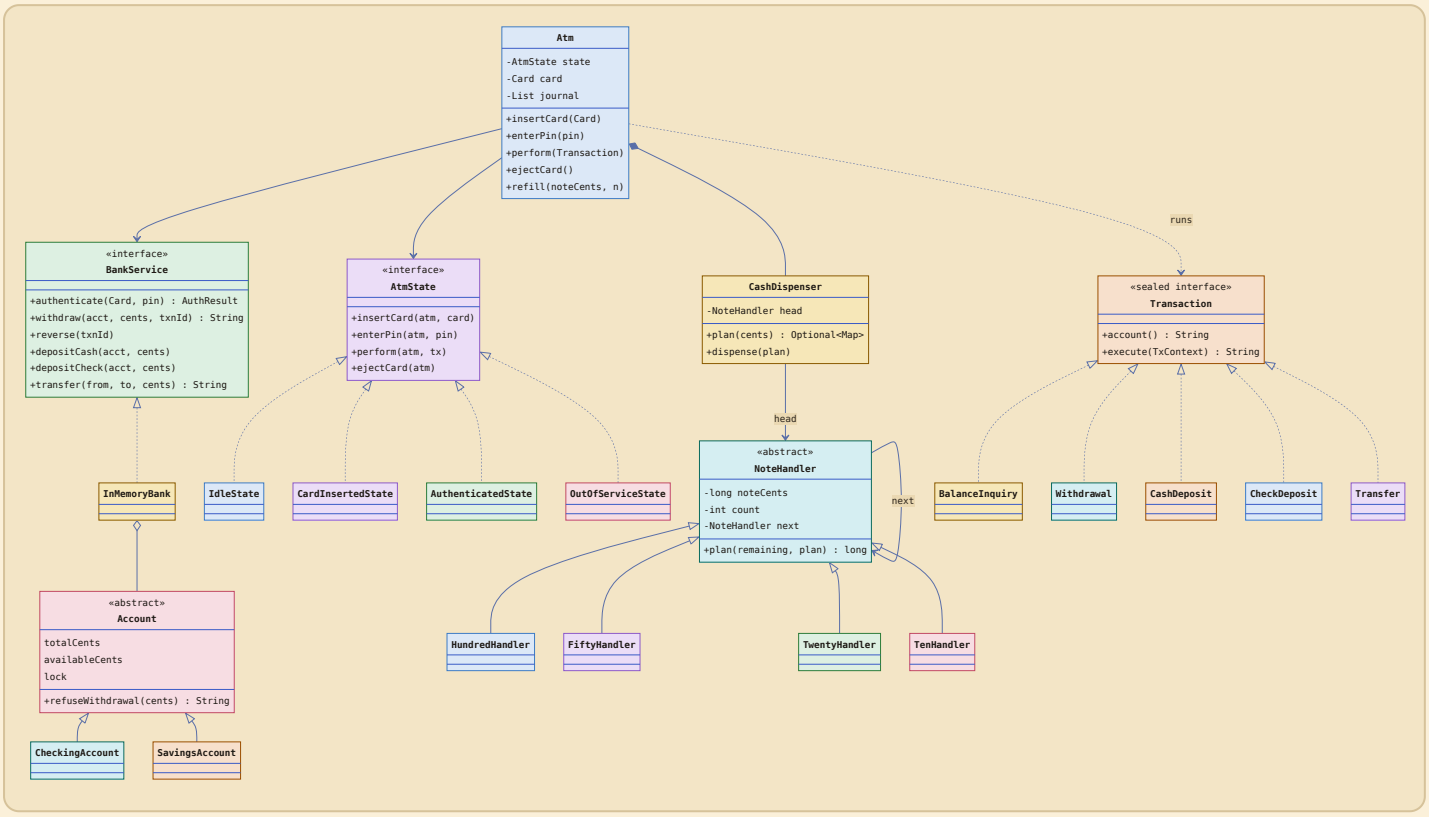


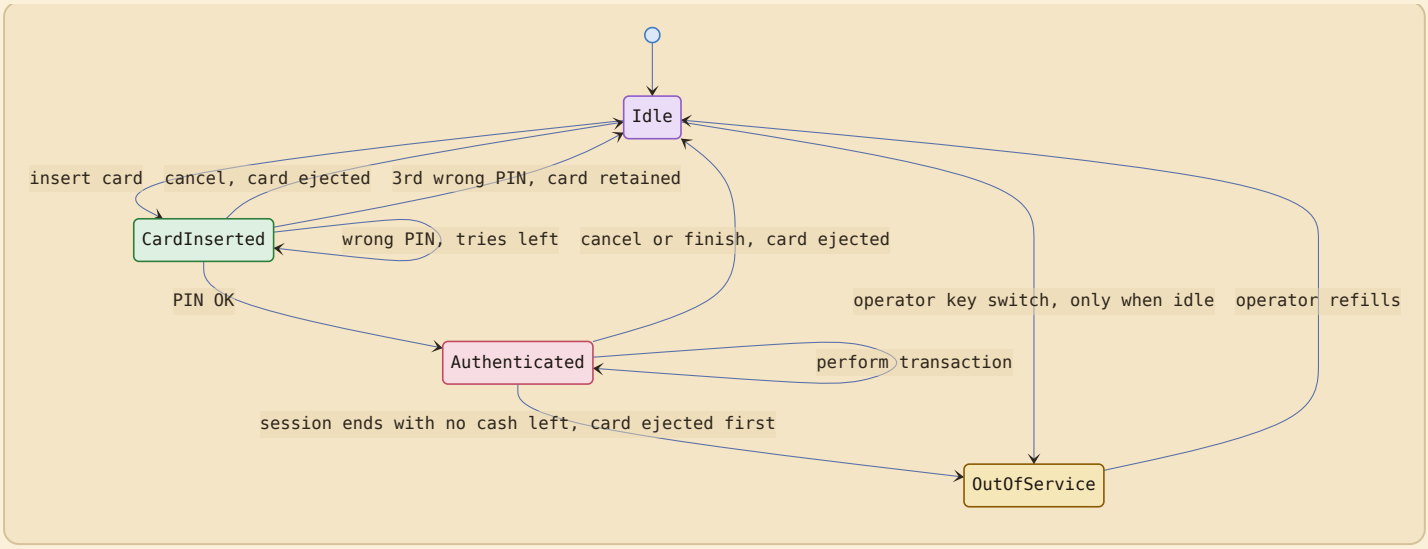
Figure — The finished class diagram of the ATM system (course diagram)

Class diagram of **our Java code**:



Key flows

Session states.



Balance inquiry (the course's sequence diagram):

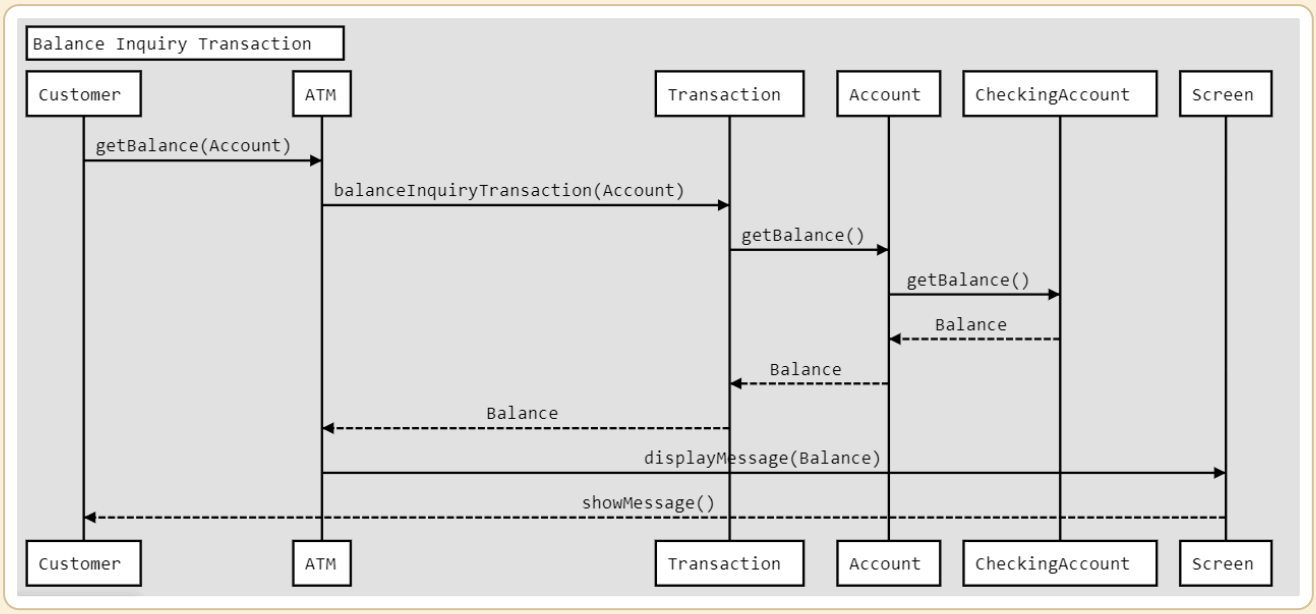
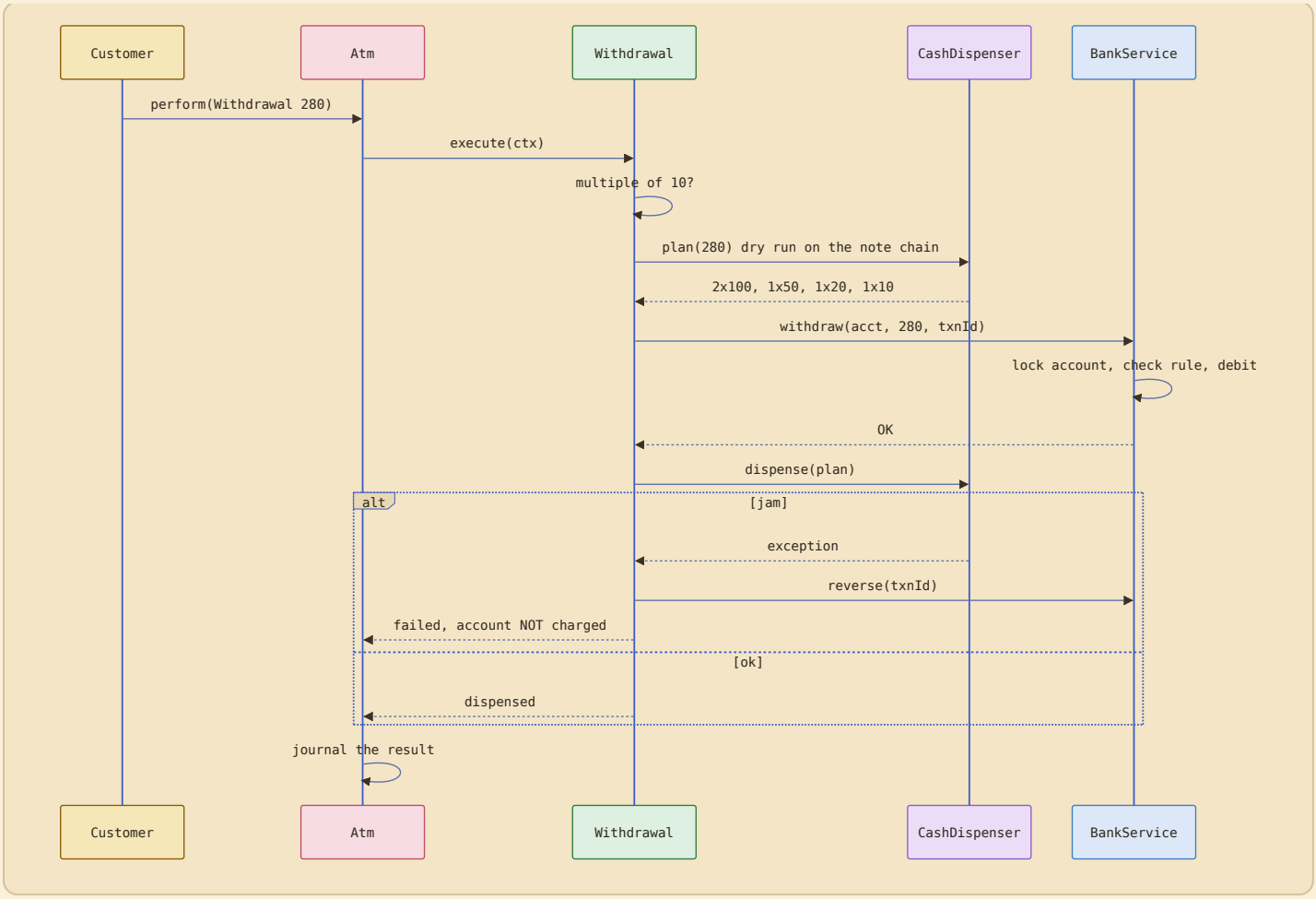


Figure — Sequence diagram of a balance inquiry: Customer → ATM → Transaction → Account → CheckingAccount, and the balance travels back to the Screen (course diagram)

In the sequence above, the ATM never reads the balance itself: it creates a transaction, the transaction asks the account, and the result is shown on the screen.

Withdrawal with compensation (our code):


Customer
Atm
Withdrawal
CashDispenser
BankService
perform(Withdrawal 280)
execute(ctx)
multiple of 10?
plan(280) dry run on the note chain
2x100, 1x50, 1x20, 1x10
withdraw(acct, 280, txnId)
lock account, check rule, debit
OK
dispense(plan)
alt
[jam]
exception
reverse(txnId)
failed, account NOT charged
[ok]
dispensed
journal the result
Customer
Atm
Withdrawal
CashDispenser
BankService

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Session flow as state classes	**State**	Insert-card/enter-PIN/perform/eject each mean something different per state; e.g. `perform` in Idle is simply refused	Booleans `cardInserted`, `authenticated` and nested `if`s
Dispenser as a chain of note handlers	**Chain of Responsibility**	Each handler takes what it can and passes the rest on; adding a $5 cassette is one new handler linked into the chain	One method with a hard-coded list of notes
Plan first, then dispense	—	The chain's dry run touches nothing; a failed plan leaves the cassettes intact	Removing notes while searching
Each transaction = one class with `execute`	**Command**	The rules of each transaction live in one place; transactions can be journaled, queued or replayed	A big `switch (type)` in the ATM
`Transaction` and `AuthResult` are **sealed**	Java 17+ sealed types	The compiler knows the full list; `switch` over `AuthResult` is exhaustive	Open hierarchies with `instanceof` chains
ATM talks to a `BankService` interface	Dependency Inversion (depend on an interface, not on a concrete class)	The ATM does not care whether the bank is in-memory, ISO 8583 (the standard card-payment message format) over a network, or a test fake	ATM changes balances directly
Total vs available balance	Domain modelling	Makes the check-hold rule correct, not merely convenient	A single balance
Savings is a subclass	Inheritance for **behaviour**	Its withdrawal rule differs (daily cap), so it overrides one hook	A `type` field and `if (type == SAVINGS)`

> **Debit before or after dispensing?** The course debits **after** the notes are counted out — the safer order for the customer — and admits the weakness: a crash between the two leaves the bank short. Our code does the opposite (debit, then dispense) and adds a **compensating action**: if the dispenser jams, `reverse(txnId)` puts the money back. Both are simplifications of what real systems do: make the pair atomic (a hold/authorisation, then a confirmation after dispensing), and **reconcile** (compare two sets of records afterwards and fix any difference) from the ATM's journal when something goes wrong in between. Say this out loud — it is the "power fails halfway through" follow-up.

> **Cash deposits.** The course assumes all deposits are verified before they count. Many banks make cash that the machine itself has counted available immediately (or up to a limit) and hold checks. Our `depositCash` credits both balances and `depositCheck` credits only the total; changing the policy is a one-line change in the bank, not the ATM.

Java implementation

Package `lld.ch14.atm`. Amounts are cents (`long`); notes are $100/$50/$20/$10. Essential lines:

```java
// Bank.java - total = everything credited; available = spendable now (a check has not cleared)
abstract class Account {
    final String id;
    final ReentrantLock lock = new ReentrantLock();   // one lock per account, shared by every ATM
    long totalCents, availableCents;
    /** Hook, called with the lock held. null = allowed, else the reason. */
    String refuseWithdrawal(long cents) { return cents > availableCents ? "insufficient funds" : null; }
    void recordWithdrawal(long cents) {}
    void reverseWithdrawal(long cents) {}
}
class CheckingAccount extends Account { /* no extra rule */ }
class SavingsAccount extends Account {              // the one behavioural difference: a daily cap
    @Override String refuseWithdrawal(long cents) {
        if (withdrawnTodayCents + cents > dailyLimitCents) return "savings daily limit " +
Money.fmt(dailyLimitCents);
        return super.refuseWithdrawal(cents);
    }
    @Override void recordWithdrawal(long cents) { withdrawnTodayCents += cents; }
    @Override void reverseWithdrawal(long cents) { withdrawnTodayCents -= cents; }
}

record Card(String number, String holder) {}
sealed interface AuthResult permits AuthResult.Ok, AuthResult.WrongPin, AuthResult.Blocked {
    record Ok(List<String> accountIds) implements AuthResult {}
    record WrongPin(int attemptsLeft) implements AuthResult {}
    record Blocked() implements AuthResult {}
}

interface BankService {                                 // the ATM never touches balances directly
    AuthResult authenticate(Card card, String pin);
    long availableBalance(String accountId);
    long totalBalance(String accountId);
    String withdraw(String accountId, long cents, String txnId);   // null = OK, else reason
    void reverse(String txnId);
    void depositCash(String accountId, long cents);   // credits total AND available
    void depositCheck(String accountId, long cents);  // credits total only: on hold
    void clearCheck(String accountId, long cents);
    String transfer(String from, String to, long cents);
}

final class InMemoryBank implements BankService {
    private static final int MAX_PIN_TRIES = 3;
    @Override public AuthResult authenticate(Card card, String pin) {
        String n = card.number();
        if (failedTries.getOrDefault(n, 0) >= MAX_PIN_TRIES) return new AuthResult.Blocked();
        if (hash(n, pin).equals(pinHashByCard.get(n))) { failedTries.remove(n); return new
AuthResult.Ok(accountsByCard.get(n)); }
        int tries = failedTries.merge(n, 1, Integer::sum);
        return tries >= MAX_PIN_TRIES ? new AuthResult.Blocked() : new AuthResult.WrongPin(MAX_PIN_TRIES - tries);
    }
    @Override public String withdraw(String id, long cents, String txnId) {
        Account a = acc(id);
        a.lock.lock();
        try {                                           // check-then-act under ONE lock: atomic
            String refusal = a.refuseWithdrawal(cents);
            if (refusal != null) return refusal;
            a.availableCents -= cents;
            a.totalCents -= cents;
            a.recordWithdrawal(cents);
            withdrawals.put(txnId, new Object[]{id, cents});   // remembered so it can be reversed
            return null;
        } finally { a.lock.unlock(); }
    }
    // reverse(txnId): compensating action - puts the money back under the same lock
    @Override public String transfer(String from, String to, long cents) {
        if (cents <= 0) return "amount must be positive";
        if (from.equals(to)) return "same account";
        Account a = acc(from), b = acc(to);
        Account first = from.compareTo(to) < 0 ? a : b, second = first == a ? b : a;   // fixed (id) order
        first.lock.lock();
```

```
        try {
            second.lock.lock();
            try {
                if (cents > a.availableCents) return "insufficient funds";
                a.availableCents -= cents; a.totalCents -= cents;
                b.availableCents += cents; b.totalCents += cents;
                return null;
            } finally { second.lock.unlock(); }
        } finally { first.lock.unlock(); }
    }
    /** Demo only: SHA-256(card + ":" + pin). Real banks never see the clear PIN (PIN pad encrypts, an HSM
verifies). */
    private static String hash(String card, String pin) { /* MessageDigest SHA-256 -> hex */ }
}
```

```
// CashDispenser.java - Chain of Responsibility: each handler takes what it can and passes the rest on
abstract class NoteHandler {
    private final long noteCents;
    private int count;
    private NoteHandler next;
    NoteHandler linkTo(NoteHandler next) { this.next = next; return next; }

    /** Dry run: fills 'plan', returns the amount nobody in the chain could cover. */
    long plan(long remaining, Map<Long, Integer> plan) {
        int use = (int) Math.min(count, remaining / noteCents);
        if (use > 0) plan.put(noteCents, use);
        long left = remaining - use * noteCents;
        return (left == 0 || next == null) ? left : next.plan(left, plan);
    }
    void take(Map<Long, Integer> plan) {
        count -= plan.getOrDefault(noteCents, 0);
        if (next != null) next.take(plan);
    }
}
final class HundredHandler extends NoteHandler { HundredHandler(int n) { super(100_00, n); } }
// FiftyHandler, TwentyHandler, TenHandler likewise

final class CashDispenser {
    CashDispenser(int hundreds, int fifties, int twenties, int tens) {
        head = new HundredHandler(hundreds);
        head.linkTo(new FiftyHandler(fifties)).linkTo(new TwentyHandler(twenties)).linkTo(new TenHandler(tens));
    }
    synchronized Optional<Map<Long, Integer>> plan(long cents) {      // touches nothing
        Map<Long, Integer> plan = new LinkedHashMap<>();
        return head.plan(cents, plan) == 0 ? Optional.of(plan) : Optional.empty();
    }
    synchronized void dispense(Map<Long, Integer> plan) {             // may jam: then nothing leaves
        if (jamNext) { jamNext = false; throw new IllegalStateException("note jam"); }   // fault injection
        head.take(plan);
    }
    // refill(noteCents, n), total()
}
```

```java
// Transactions.java - Command: each transaction is one record with one execute(); sealed = full list known
record TxContext(BankService bank, CashDispenser dispenser, String txnId) {}
sealed interface Transaction permits BalanceInquiry, Withdrawal, CashDeposit, CheckDeposit, Transfer {
    String account();
    String execute(TxContext ctx);
}

record Withdrawal(String account, long cents) implements Transaction {
    public String execute(TxContext c) {
        if (cents <= 0 || cents % 10_00 != 0) return "Withdrawal refused: amount must be a multiple of $10";
        var plan = c.dispenser().plan(cents);                                  // 1. can the notes make it?
        if (plan.isEmpty()) return "Withdrawal refused: ATM cannot make " + Money.fmt(cents) + " from its notes";
        String refusal = c.bank().withdraw(account, cents, c.txnId());         // 2. atomic debit at the bank
        if (refusal != null) return "Withdrawal refused: " + refusal;
        try {
            c.dispenser().dispense(plan.get());                                // 3. hand out the cash
        } catch (IllegalStateException hardwareFault) {
            c.bank().reverse(c.txnId());                                       // compensate: undo the debit
            return "Withdrawal failed (" + hardwareFault.getMessage() + "), account NOT charged";
        }
        return "Dispensed " + Money.fmt(cents) + " as " + describe(plan.get());
    }
}
// BalanceInquiry, CashDeposit, CheckDeposit ("on hold until the bank clears it"), Transfer: one bank call each
```

```java
// States.java - default methods refuse; each state overrides only the buttons it accepts
interface AtmState {
    default void insertCard(Atm atm, Card card) { atm.screen("Card slot is not ready."); }
    default void enterPin(Atm atm, String pin) { atm.screen("Please insert your card first."); }
    default void perform(Atm atm, Transaction tx) { atm.screen("Please sign in first."); }
    default void ejectCard(Atm atm) { atm.screen("No card to eject."); }
}
final class CardInsertedState implements AtmState {
    @Override public void enterPin(Atm atm, String pin) {
        switch (atm.bank().authenticate(atm.card(), pin)) {                    // exhaustive over the sealed type
            case AuthResult.Ok ok -> {
                atm.startSession(ok.accountIds());
                atm.screen("PIN OK. Accounts: " + ok.accountIds());
                atm.setState(AuthenticatedState.INSTANCE);
            }
            case AuthResult.WrongPin w -> atm.screen("Wrong PIN. Tries left: " + w.attemptsLeft());
            case AuthResult.Blocked b -> {
                atm.screen("Too many wrong PINs. Card retained. Contact your bank.");
                atm.retainCard();
                atm.finishSession();
            }
        }
    }
    @Override public void ejectCard(Atm atm) { atm.returnCard(); atm.finishSession(); }
}
final class AuthenticatedState implements AtmState {
    @Override public void perform(Atm atm, Transaction tx) {
        if (!atm.sessionAccounts().contains(tx.account())) {                   // only this card's accounts
            atm.screen("Account " + tx.account() + " is not linked to this card.");
            return;
        }
        atm.run(tx);
    }
    @Override public void ejectCard(Atm atm) { atm.returnCard(); atm.finishSession(); }
}
// IdleState.insertCard -> CardInserted; OutOfServiceState.insertCard -> "Out of service. Card returned."
```

```
// Atm.java - owns its parts, one session at a time, forwards buttons to its state, keeps the journal
final class Atm {
    private final List<String> journal = new ArrayList<>();   // operator / bank-manager audit log
    private AtmState state = IdleState.INSTANCE;
    private Card card;                                        // the card reader's slot

    synchronized void insertCard(Card c) { state.insertCard(this, c); }       // enterPin, perform, ejectCard
alike

    synchronized boolean setOutOfService() {                  // key switch: never while serving a customer
        if (state != IdleState.INSTANCE && state != OutOfServiceState.INSTANCE) {
            journal("operator: switch-off refused, session in progress");
            return false;
        }
        state = OutOfServiceState.INSTANCE;
        return true;
    }
    synchronized void refill(long noteCents, int n) {
        dispenser.refill(noteCents, n);
        if (state == OutOfServiceState.INSTANCE) state = IdleState.INSTANCE;
    }
    void run(Transaction tx) {
        String txnId = id + "-" + TXN_SEQ.getAndIncrement();
        String result = tx.execute(new TxContext(bank, dispenser, txnId));
        screen(result);
        journal(txnId + " " + tx.getClass().getSimpleName() + ": " + result);
    }
    void finishSession() {                                    // after every eject/retain: card is already out
        if (dispenser.total() == 0) { state = OutOfServiceState.INSTANCE; journal("cash empty: out of service"); }
        else state = IdleState.INSTANCE;
    }
    // journal also records "card in/out ****1111" (masked) and "card RETAINED"
}
// Main.java: Alice (CHK-1 $1000, SAV-1 $2000 with a $100 daily cap), Bob (CHK-2 $50); a jam injected;
// two ATMs race $300 each on a $500 JOINT account; 2,000 opposite A<->B transfers on 8 threads.
```

The demo PIN hash is SHA-256, a standard one-way function producing a fixed-length fingerprint that cannot practically be reversed. (The course compares the PIN inside `Card` and flags that as a simplification. Real banks never see the clear PIN: the PIN pad encrypts it into a PIN block and a hardware security module (HSM) verifies it; a plain hash of a 4-digit PIN is also easy to brute-force — try every value until one matches.) Transfers lock both accounts in **id order**, so opposite transfers (A→B, B→A) can't deadlock (Chapter 13). The bank refuses zero or negative transfers and deposits.

Output:

```
Cash loaded: $430.00

== Alice: wrong PIN, then right PIN ==
   [ATM-7 screen] Please sign in first.
   [ATM-7 screen] Hello Alice. Enter PIN.
   [ATM-7 screen] Wrong PIN. Tries left: 2
   [ATM-7 screen] PIN OK. Accounts: [CHK-1, SAV-1]
   [ATM-7 screen] Balance CHK-1: available $1000.00, total $1000.00
   [ATM-7 screen] Dispensed $280.00 as 2x$100 + 1x$50 + 1x$20 + 1x$10
   [ATM-7 screen] Withdrawal refused: amount must be a multiple of $10
   [ATM-7 screen] Withdrawal refused: savings daily limit $100.00
   [ATM-7 screen] Check $500.00 received: on hold until the bank clears it
   [ATM-7 screen] Balance CHK-1: available $720.00, total $1220.00
   [ATM-7 screen] Transferred $150.00 to CHK-2
   [ATM-7 screen] Account CHK-2 is not linked to this card.
   (dispenser jams on the next withdrawal)
   [ATM-7 screen] Withdrawal failed (note jam), account NOT charged
   [ATM-7 screen] Balance CHK-1: available $720.00, total $1220.00
   [ATM-7 screen] Card ejected. Goodbye.

== Bank clears the check overnight ==
   CHK-1 available now $1220.00

== Bob forgets his PIN three times ==
   [ATM-7 screen] Hello Bob. Enter PIN.
   [ATM-7 screen] Wrong PIN. Tries left: 2
   [ATM-7 screen] Wrong PIN. Tries left: 1
   [ATM-7 screen] Too many wrong PINs. Card retained. Contact your bank.
   [ATM-9 screen] Hello Bob. Enter PIN.
   [ATM-9 screen] Too many wrong PINs. Card retained. Contact your bank.

== Two ATMs, one account, same instant: $300 each from $500 ==
   cash handed out $300.00, JOINT left $200.00

== 2,000 opposite transfers in parallel (ordered locking: no deadlock) ==
   successful=2000, A+B=$2000.00

== ATM-9 runs out of cash; the operator refills it ==
   [ATM-9 screen] Hello Alice. Enter PIN.
   [ATM-9 screen] PIN OK. Accounts: [CHK-1, SAV-1]
   operator key switch mid-session accepted? false
   [ATM-9 screen] Dispensed $180.00 as 1x$100 + 1x$50 + 1x$20 + 1x$10
   state after the last notes: AUTHENTICATED (card still inside)
   [ATM-9 screen] Card ejected. Goodbye.
   state after the session: OUT_OF_SERVICE
   [ATM-9 screen] Out of service. Card returned.
   state after refill: IDLE, cash $200.00

== Journal of ATM-7 (for the operator and bank manager) ==
   card in ****1111
   ATM-7-1 BalanceInquiry: Balance CHK-1: available $1000.00, total $1000.00
   ATM-7-2 Withdrawal: Dispensed $280.00 as 2x$100 + 1x$50 + 1x$20 + 1x$10
   ATM-7-3 Withdrawal: Withdrawal refused: amount must be a multiple of $10
   ATM-7-4 Withdrawal: Withdrawal refused: savings daily limit $100.00
   ATM-7-5 CheckDeposit: Check $500.00 received: on hold until the bank clears it
   ATM-7-6 BalanceInquiry: Balance CHK-1: available $720.00, total $1220.00
   ATM-7-7 Transfer: Transferred $150.00 to CHK-2
   ATM-7-8 Withdrawal: Withdrawal failed (note jam), account NOT charged
   ATM-7-9 BalanceInquiry: Balance CHK-1: available $720.00, total $1220.00
   card out ****1111
   card in ****2222
   card RETAINED ****2222
```

Read the output: $280 = 2×$100 + $50 + $20 + $10; $85 is refused; the savings cap refuses $140; after the $500 check *total* grows but *available* waits for clearing; the jam leaves the balance unchanged; Bob's card is blocked at the bank, so another ATM refuses even the right PIN; two ATMs racing on a $500 joint account (one account shared by two people) hand out only $300; 2,000 opposite transfers finish with no deadlock and the total conserved; ATM-9 refuses the key switch mid-session, pays its last $180, ejects the card, and only then goes out of service until refilled.

Concurrency & edge cases

Situation	Risk	Handling
Two ATMs withdraw from one account at once	Overdraft (check-then-act race)	Check and debit under the **account lock** at the bank
A→B and B→A transfers at once	**Deadlock** (each holds one lock, waits for the other)	Lock in a global order (account id)
Dispenser jams after the debit	Customer charged, no cash	Compensating `reverse(txnId)`; journal entry for reconciliation
Amount not makeable from loaded notes	Partial dispense	Dry-run `plan` first; refuse before touching the bank
Greedy chain misses a combination	e.g. $60 with one $50 and three $20 but no $10: greedy takes $50, then cannot make $10, though 3×$20 works	Acceptable for standard cassettes; mention a DP planner (as in 14.2) if asked
Wrong PIN many times	Brute force	3 tries, then block at the bank (so another ATM also refuses) and retain the card
Customer walks away mid-session	Next person uses the open session	Session timeout that ejects/retains the card (extension)
Transaction on an account not linked to the card	Unauthorised access	`AuthenticatedState` checks `sessionAccounts`
Cash runs out	Customer's card trapped, or the next customer gets refused mid-session	The session finishes normally (card ejected), then `finishSession` moves the ATM to `OutOfService`; an operator refill returns it to Idle
Operator turns the key mid-session	The machine shuts down while serving	`setOutOfService` is refused unless the ATM is idle (requirement: never shut down while serving)

Extensions & follow-up questions

1. **"What if the machine cannot make the amount?"** — The dispenser's plan fails and the withdrawal is refused before the bank is called.
2. **"What if the power fails halfway through a withdrawal?"** — Use a two-phase approach: authorise (hold) at the bank, dispense, then confirm; on restart the ATM reads its journal and either confirms or reverses. The bank reconciles against the journal.
3. **"Add a daily withdrawal limit for all accounts."** — Move the cap into `Account` or into a policy object the bank consults; the ATM does not change.
4. **"Receipts and PIN change?"** — Two more `Transaction` records (`PinChange`, and a `Printer` part that formats the journal entry).
5. **"Other banks' cards?"** — `BankService` gets a routing implementation that forwards to a card network (such as Visa or Mastercard, which passes requests between banks) for foreign cards (a Proxy — a stand-in object — or an Adapter — a translator between two interfaces — in front of the network).
6. **"Cardless / mobile withdrawals?"** — A new state path: `enterOneTimeCode` instead of `insertCard` → `enterPin`. The transactions do not change.
7. **"Bank manager reports?"** — Aggregate the journal (total deposits, total withdrawals, cash left = `dispenser.total()`).

Real-world example

Real ATMs (NCR, Diebold Nixdorf and others) control their devices through the **CEN/XFS** standard, which gives the card reader, PIN pad, dispenser and printer each a standard software interface — the same "parts behind interfaces" idea as our design. Messages between ATM networks and banks commonly use the **ISO 8583** message format, and PIN blocks are defined by **ISO 9564** and verified inside HSMs. Cash sits in

cassettes of one denomination each, and the dispenser works out a note mix from what the cassettes hold, like our handler chain. After a dispense error, ATMs routinely reverse the transaction, and banks reconcile disputed withdrawals from the machine's electronic journal.

In the interview

Plan for 45 minutes: 5 min parts, actors and transactions → 5 min total vs available balance → 10 min state diagram (Idle → CardInserted → Authenticated) → 15 min code: `Withdrawal.execute` (plan → debit → dispense → compensate) and the dispenser chain → 10 min concurrency (account lock, ordered transfer locks) and follow-ups.

30-second answer: "The ATM has parts behind interfaces, a session modelled with the State pattern, and five transactions as Command objects with one `execute` each. It never touches balances: it calls a `BankService`. Accounts have total and available balances so checks can be held. Withdrawal plans the notes first through a Chain of Responsibility over the cassettes, debits the account atomically at the bank, dispenses, and reverses the debit if the hardware fails. The bank locks per account, and transfers lock in id order to avoid deadlock."

Follow-ups with answers

- *"Where is the PIN checked?"* — At the bank (in reality in an HSM, from an encrypted PIN block). The ATM never stores or compares it.
- *"How do you stop two ATMs overdrawing a joint account?"* — The check and the debit happen together under the account's lock at the bank (or in one database transaction).
- *"Why Chain of Responsibility for notes?"* — Each denomination handler decides how much it can cover and passes the rest on; adding or removing a cassette changes the chain, not the algorithm.

TL;DR

- **State** for the session, **Command** for the five transactions, **Chain of Responsibility** for the note cassettes, `BankService` interface for the bank.
- **Total vs available** balance makes check deposits correct.
- Withdrawal = **plan notes → atomic debit at the bank → dispense → reverse on failure**, all journaled.
- Concurrency: **per-account lock**, transfers lock in **id order**, PIN attempts limited at the bank.

14.5 Design an Elevator System

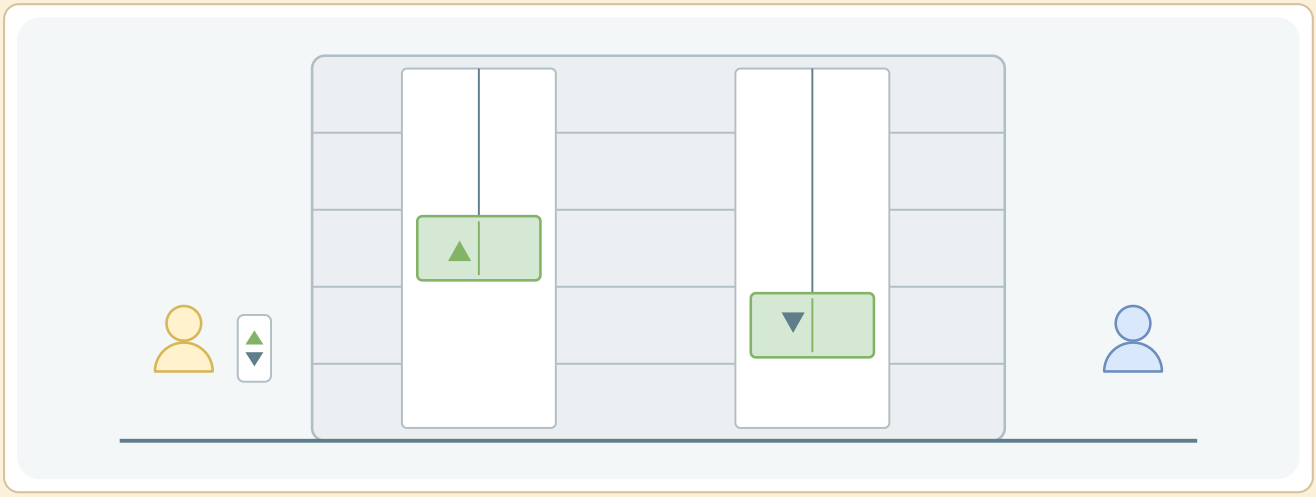


Figure — An elevator bank in a multi-floor building: several cars, hall buttons on each floor and displays (course diagram)

An **elevator** (lift) moves people up and down a building. A real building rarely has just one, and its cars are run as a **group** by a shared controller. That group control is what turns this from a small coding task into a design question. Two decisions carry the interview: **which car should answer a call from a floor** (dispatching) and **in what order a single car should visit its stops** (scheduling).

Problem & clarifying questions

Question	Assumed answer
How many floors and cars?	Many floors (0 … N-1), several cars serving all floors.
Which buttons?	Hall panel on each floor with UP and DOWN (no UP on the top floor, no DOWN on the ground floor); a car panel with one button per floor plus door-open and door-close.
Displays?	Inside each car: floor, direction, load. On each floor: one display per car showing where it is.
Scheduling rule?	A car serves every stop ahead in its direction before turning (the elevator algorithm; we use **LOOK**).
Dispatching rule?	Nearest suitable car; swappable.
Capacity?	Each car has a maximum load; an overloaded car does not move and says so.
Maintenance and emergencies?	A car can be taken out of service; fire alarm recalls all cars to the lobby.
Real time or simulated?	Simulated: one `step()` = one action. This makes the design testable without a clock.

Requirements

Functional (the course's twelve, plus emergency)

1. The building has **multiple floors** and **more than one elevator** serving all of them.
2. Each floor has a **hall panel** with UP and DOWN buttons (top floor: no UP; ground floor: no DOWN).
3. Each car has an **inside panel** with a button per floor, plus **door open** and **door close**.
4. Each car has a **display** showing its floor, direction and the weight it carries.
5. Each floor has a **display for every elevator** so a waiting rider can see where each car is.
6. When a rider presses a hall button, the system **picks one elevator** and sends it.
7. When a rider presses a floor button inside a car, the car **adds that floor** to its stops.
8. A car **serves the stops ahead of it** in its direction before turning around.
9. A car **does not move while its doors are open**.
10. Each car has a **maximum load**; an overloaded car does not move and **says so** on its display.
11. A car with no stops left **waits on the floor where it finished**.
12. An elevator can be **taken out of service** for maintenance; the system must not send riders to it.
13. *(Extension the course leaves out; we add it)* **Fire recall**: all cars go to the lobby, open their doors and ignore car calls.

Non-functional

- **Efficiency:** low average wait and ride time; no car runs to the end of the shaft for nothing.
- **Fairness / no starvation** (starvation = a request that waits forever because others keep being chosen first)**:** every accepted call is eventually served.
- **Safety:** doors, load limit and emergency modes always override scheduling.
- **Thread safety:** riders press buttons concurrently while a controller thread moves the cars.
- **Testability:** deterministic stepping (the same inputs always give the same results).

Use cases

Actors: Rider (calls a car, rides, chooses a destination), **Maintenance engineer** (takes a car out of service and back), **System** (chooses the car, moves cars, opens/closes doors, updates displays).

Top use cases: call an elevator · choose a destination · dispatch a car · move one floor · open and close the doors · report an overload · take a car out of service.

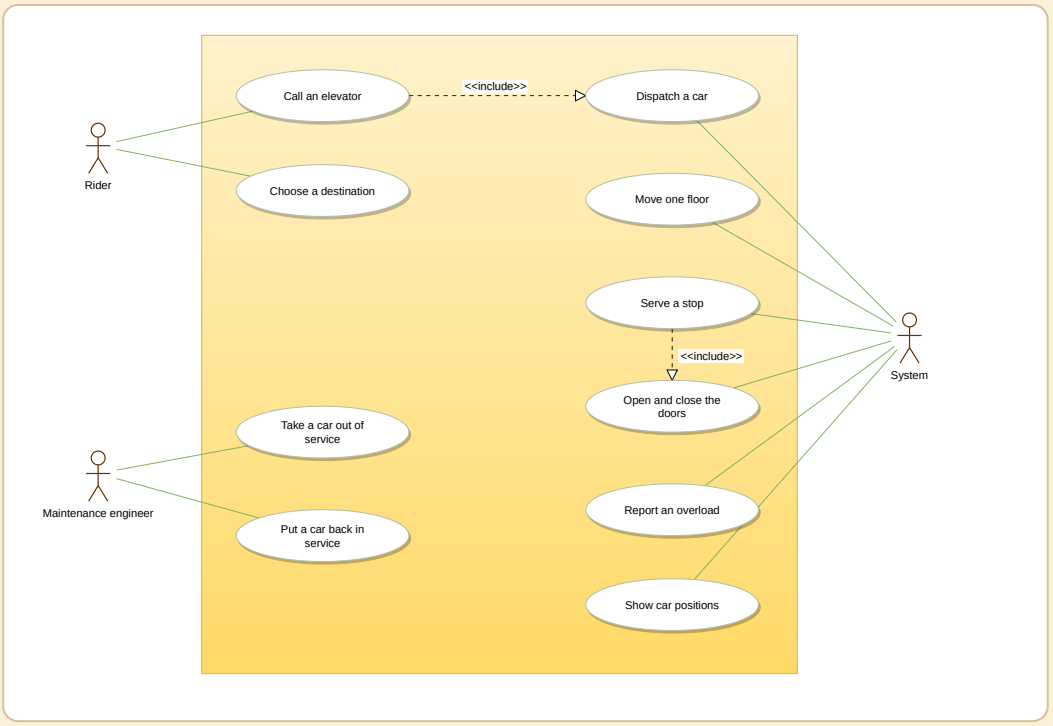


Figure — Use case diagram of the elevator system: Rider, Maintenance engineer and System (course diagram)

In the figure above, "dispatch a car" and "move one floor" belong to the System — the rider only triggers them.

Core entities

Class	Responsibility
`Direction` (enum)	UP, DOWN, IDLE.
`Stop` (record)	A floor **plus** the direction the car will travel when it leaves — two riders on floor 5 (one up, one down) are two stops.
`StopList`	The accepted stops in **two sorted sets** (`up`, `down`); `next(...)` implements LOOK.
`Door`	Open or closed.
`Elevator`	One car: floor, direction, mode, load, door, stop list. `step()` does exactly one action.
`Mode` (enum)	NORMAL, MAINTENANCE, FIRE_RECALL.
`Event` (record)	What one step did (moved, arrived, doors closed, overload, nothing).
`DispatchStrategy`	*Which car* answers a hall call. `NearestCarStrategy` is one rule.
`Dispatcher`	Holds the cars and one strategy; queues calls no car can take now.
`Building`	Owns hall-button lights, the cars and the dispatcher; ticks every car; clears hall lights on arrival; fire alarm.

The course also models the physical parts one-to-one — `Button` (`HallButton`, `ElevatorButton`, `DoorButton`), `HallPanel`, `ElevatorPanel`, `Display` (`ElevatorDisplay`, `FloorDisplay`) and `Floor`. Our code keeps their behaviour (lights, displays, door) in fewer classes; either level of detail is fine in 45 minutes if you say which you chose.

Class diagram

The course builds the design in five stages:

Stage	What it adds	What works after it
1	`Button` family and `Door`	Buttons light and reset; a door knows if it is open
2	`HallPanel`, `ElevatorPanel`, `Display` family, `Floor`	A floor accepts a call and shows each car's position
3	`Stop` and `StopList`	Given a floor and a direction, compute the next stop
4	`Elevator`	A car travels to all the stops it has accepted, one step at a time
5	`DispatchStrategy`, `Dispatcher`, `Building`	A call on floor 7 is handed to the most suitable car

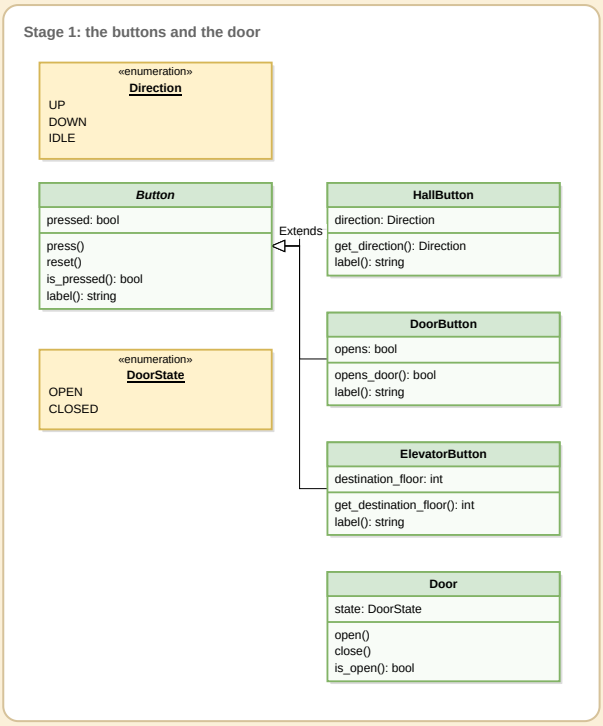


Figure — Stage 1: the Direction enum, the abstract Button with HallButton, ElevatorButton and DoorButton, and the Door (course diagram)

Stage 1. Requirements 2, 3 and 9. All buttons share one behaviour (light up on press, go dark once handled) and differ only in the value they carry: a hall button stores a **direction**, a car button a **floor number**, a door button **open or close**. That gives one abstract `Button` parent with three subclasses

instead of three unrelated classes. `Door` has no owner yet; the car will own it in stage 4. Nothing can move at this point.

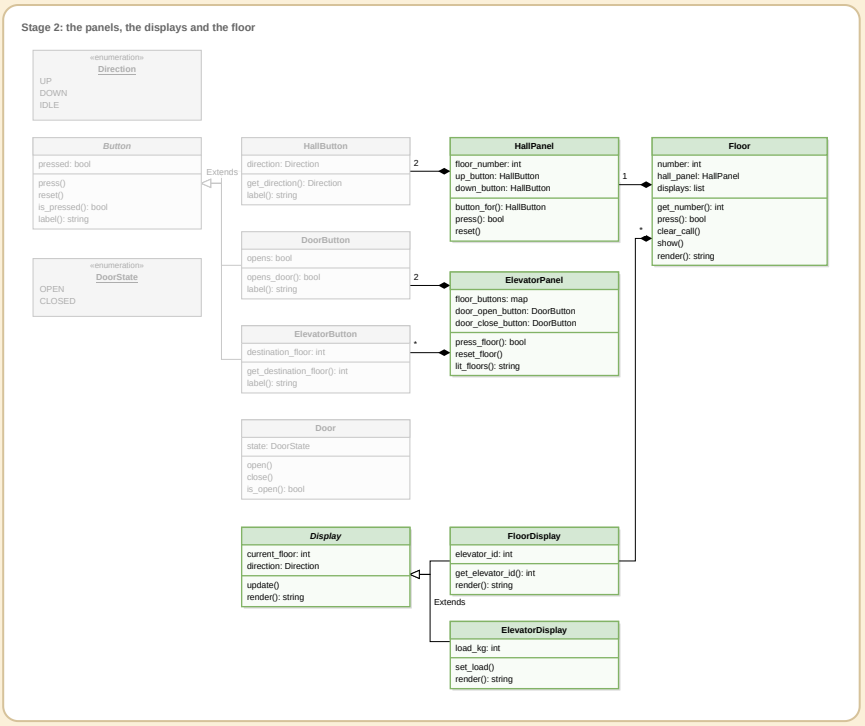


Figure — Stage 2: HallPanel, ElevatorPanel, the abstract Display with ElevatorDisplay and FloorDisplay, and Floor, all joined by composition (course diagram)

Stage 2. Requirements 2, 3, 4 and 5. Every new link is **composition**: a hall panel owns its buttons, a floor owns its hall panel and displays. A hall panel has **at most two** buttons, each optional (top and bottom floors lack one). The car panel creates one button per floor from the floor list it receives, so adding a floor needs no edit here. `Display` is abstract: both subclasses show a floor and a direction; `ElevatorDisplay` adds the load, the wall-mounted `FloorDisplay` adds which car it describes — either can render wherever a `Display` is expected (the substitution test, Chapter 4). A floor can now register a call and show where cars are, but nothing moves yet.

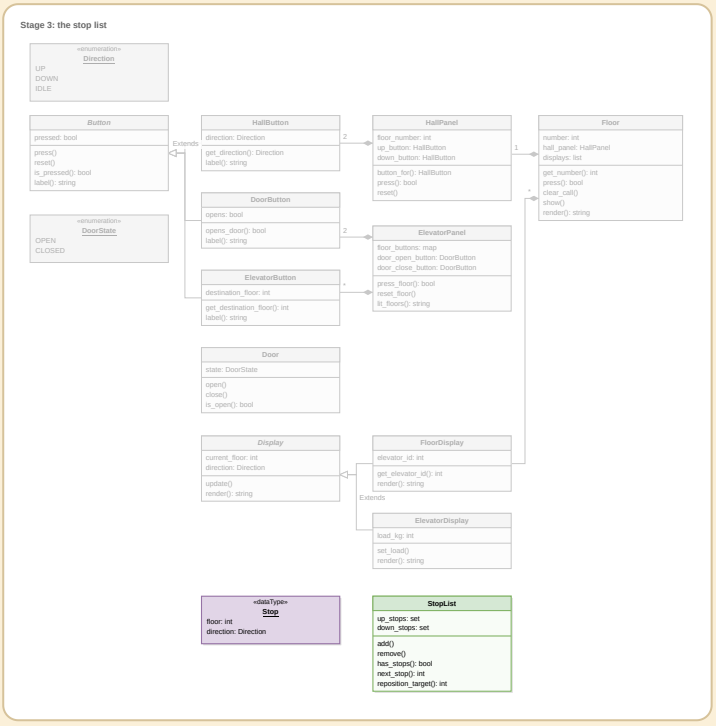


Figure — Stage 3: the Stop data type and the StopList with its up and down sets (course diagram)

Stage 3 — the most important stage. Requirement 8: *knowing a car's floor and direction, which floor should it stop at next?* Store stops in **two sets** (to serve going up, going down), not one list: with two

people waiting on floor 5, one going up and one down, a plain list records "5" once, so the car stops twice for nothing or picks up the wrong person. `Stop` is a small data type (a floor plus the direction the car will be moving there). The next stop returned is always reachable **without reversing**; when nothing lies ahead, the car heads to the **farthest** waiting request, sweeping everyone up on the return trip. `Elevator` **moves**, `StopList` **chooses**.

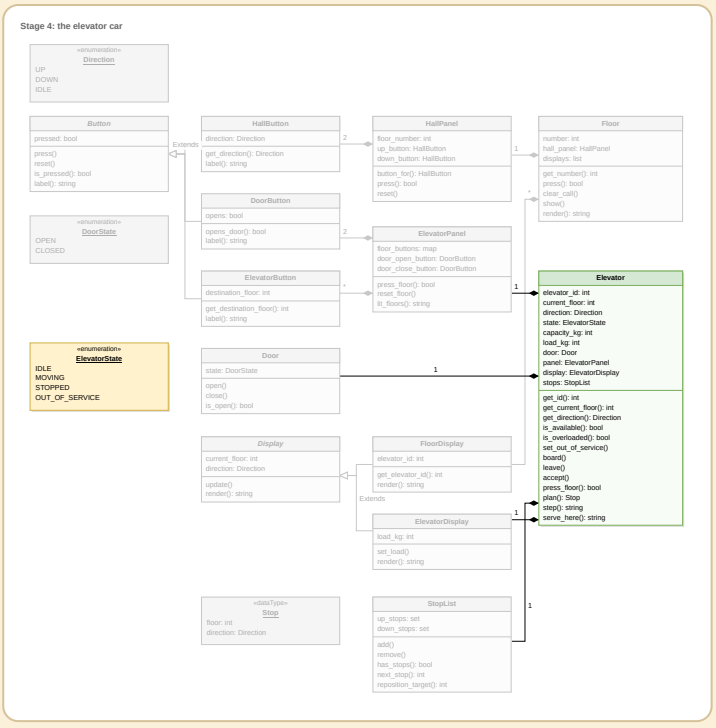


Figure — Stage 4: the Elevator owns its door, panel, display and stop list (course diagram)

Stage 4. Requirements 7, 9, 10 and 11. The car owns its door, panel, display and stop list (composition). One `step` does exactly **one** of four things — close the doors, refuse to move because of overload, move one floor, or stop and open — so a test can drive the car step by step with **no clock**. Only `step` changes the position, and an out-of-service car simply does nothing on `step` (our `NOTHING` event), so it drops out of the rounds without special cases. The car never touches a `Floor` or another car, leaving the choice of car to the dispatcher.

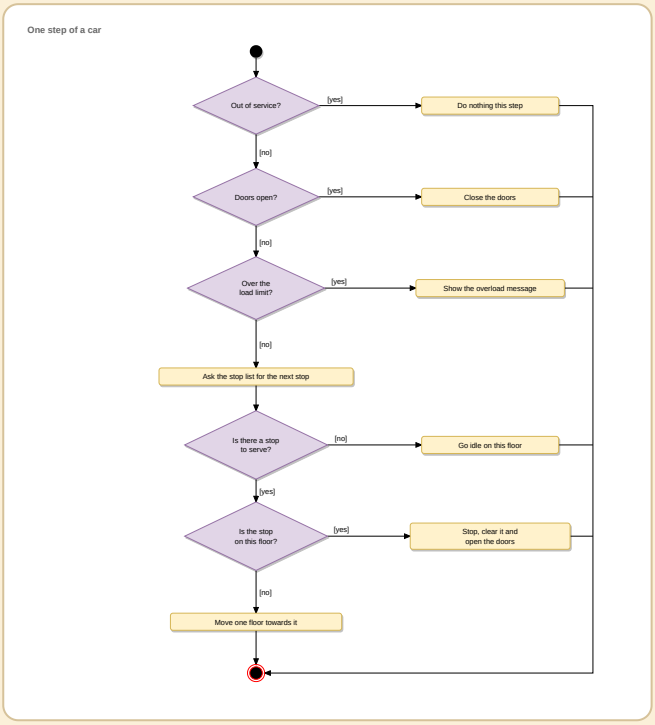


Figure — Activity diagram "One step of a car": out of service → nothing; doors open → overload check or close; otherwise ask the stop list, go idle, stop and open, or move one floor (course diagram)

The activity diagram above is our `Elevator.step()` drawn as a picture: every path ends in exactly one outcome, and only one outcome moves the car.

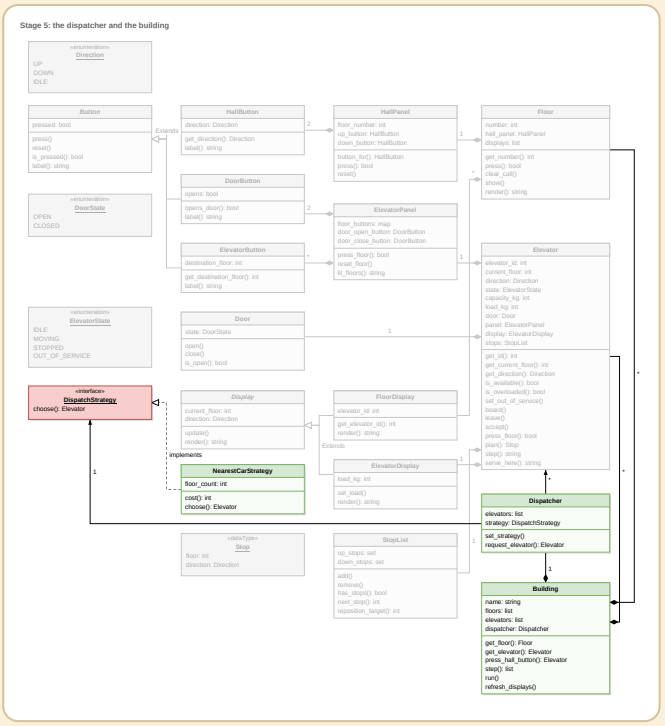


Figure — Stage 5: DispatchStrategy interface with NearestCarStrategy, the Dispatcher, and the Building that owns floors, cars and the dispatcher (course diagram)

Stage 5. Requirements 1, 6 and 12. The rule that picks a car lives **behind an interface** (`DispatchStrategy`); the dispatcher depends only on it, and `NearestCarStrategy` implements it. Morning rush in an office and a quiet hotel night want different rules; swapping touches one class. The building owns its floors, cars and dispatcher, and is the only class that knows both cars and floors — so switching off the hall-button light when a car opens its doors is its job.

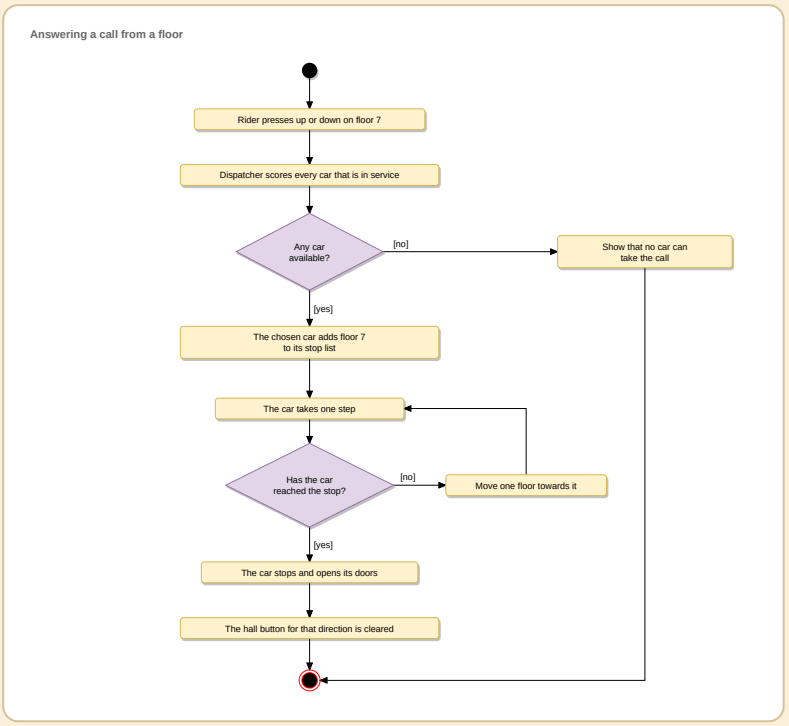


Figure — Activity diagram "Answering a call from a floor": rider presses up/down on floor 7, the dispatcher scores every in-service car, the chosen car adds floor 7, steps until it reaches the stop, opens its doors, and the hall button is cleared (course diagram)

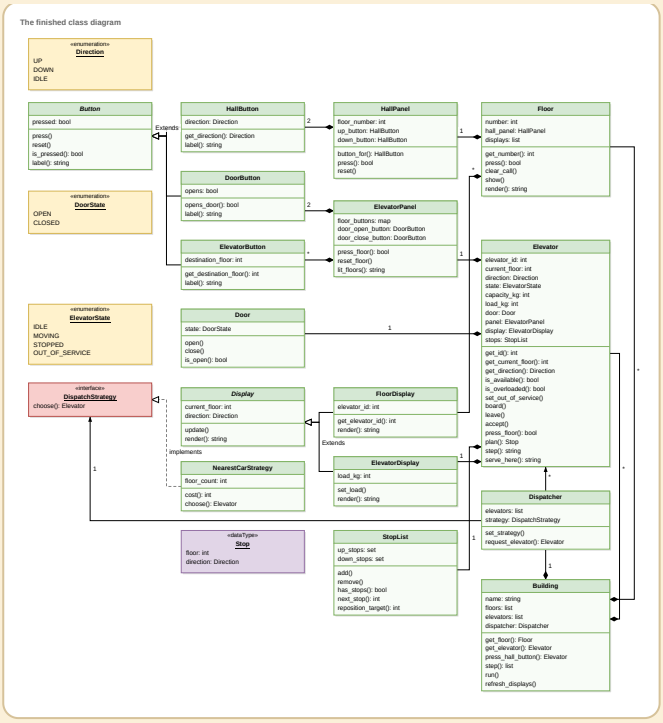
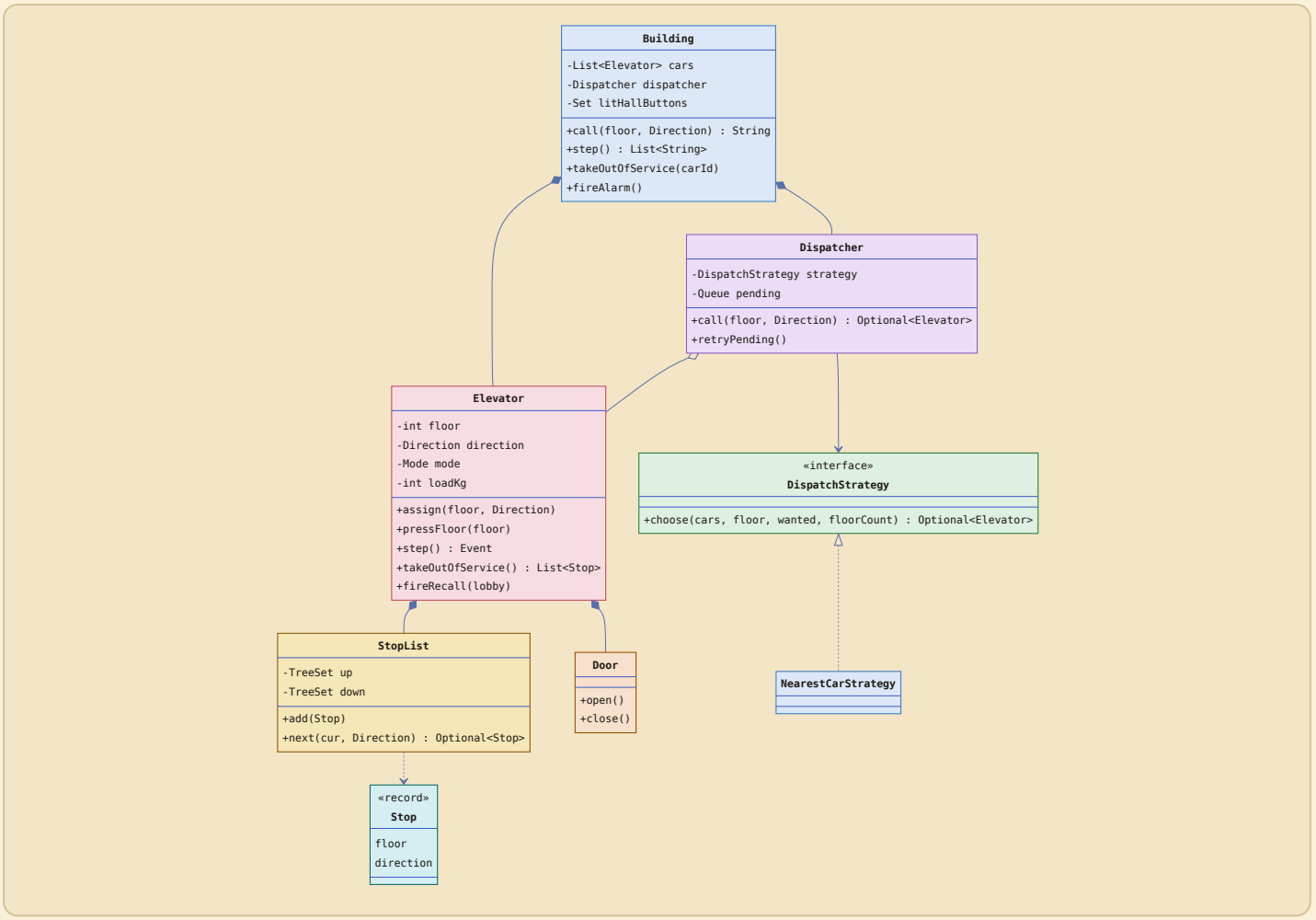


Figure — The finished class diagram of the elevator system (course diagram)

Class diagram of **our Java code**:



Key flows

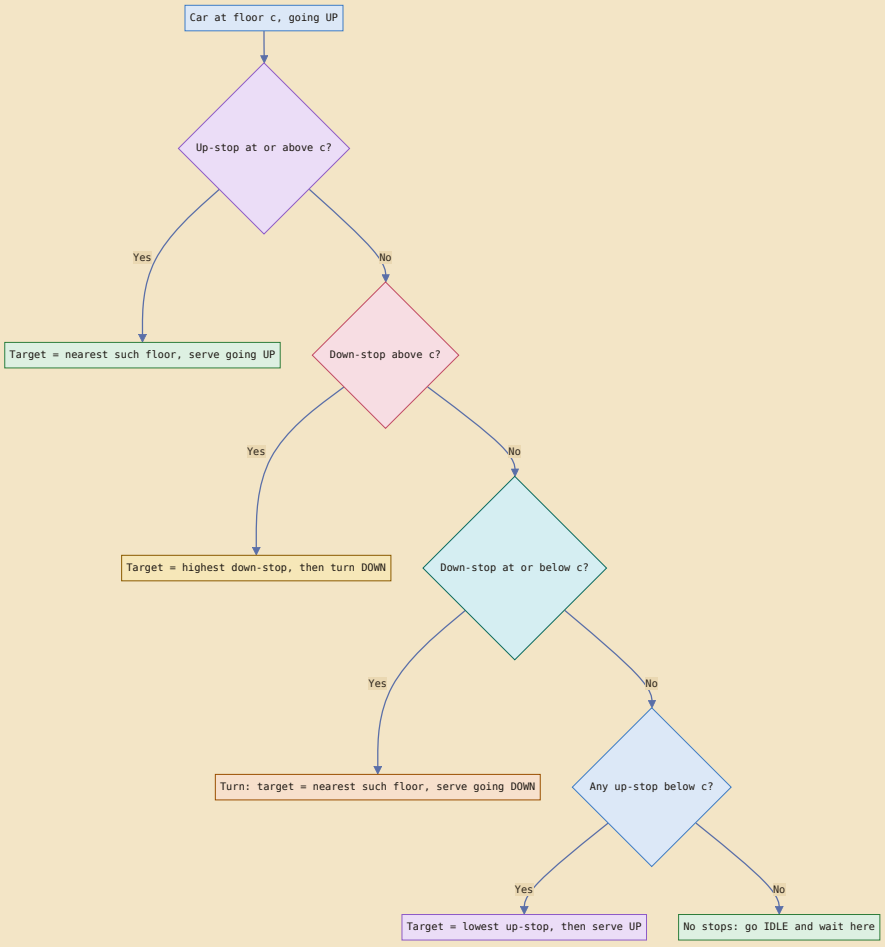
Scheduling algorithms compared. "Scheduling" means choosing the order in which one car serves its stops. These algorithms come from disk-arm scheduling in operating systems (the order in which a hard

disk's read head visits requested tracks), which has the same shape (a head moving along a line of requests).

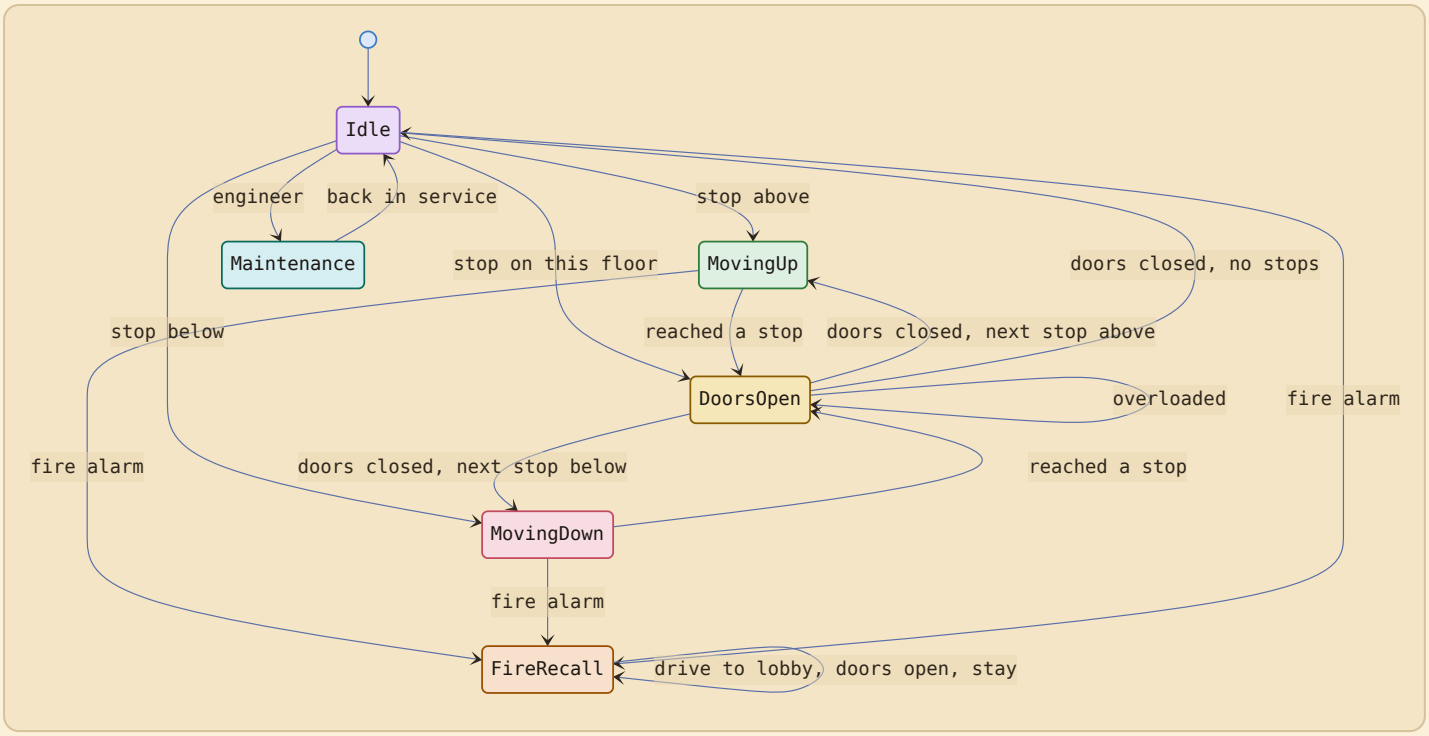
Algorithm	Rule	Good	Bad
FCFS (first come, first served)	Serve calls in arrival order	Simple, fair in order	Zig-zags up and down; very inefficient
SSTF (shortest seek time first)	Always go to the nearest stop	Short average trips	**Starvation**: a far floor may wait forever while nearby calls keep coming
SCAN ("elevator algorithm")	Sweep in one direction to the **end of the shaft**, then reverse	No starvation, predictable	Wastes travel going to the top/bottom floor even when nobody is there
LOOK	Sweep in one direction only **as far as the last request**, then reverse	No starvation, no wasted travel	Slightly more bookkeeping
C-SCAN / C-LOOK	Serve in one direction only; jump back to the start	More uniform waits	Empty return trips

Our `StopList.next` is **LOOK** with direction-aware stops:

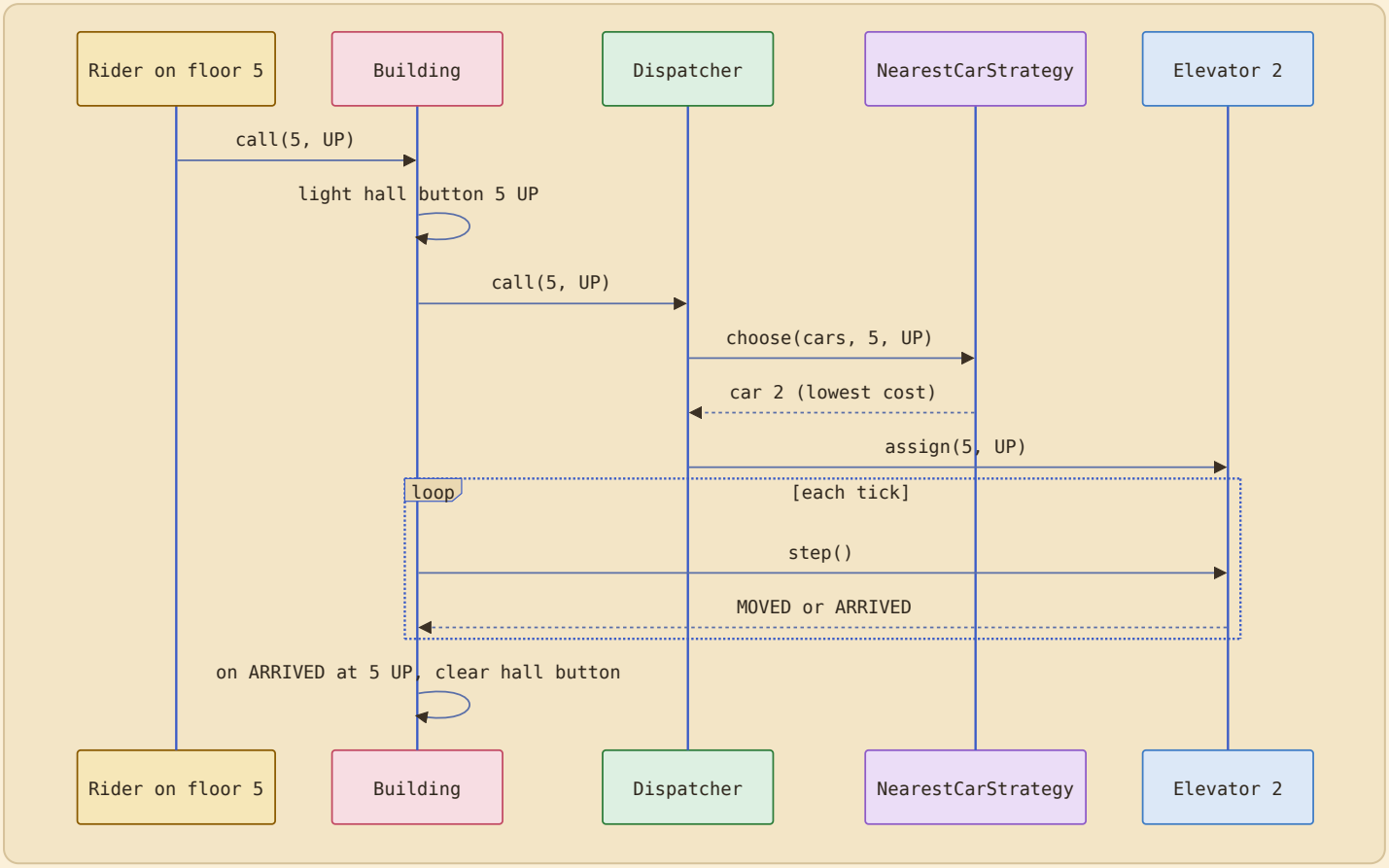
1. Going **UP**: the nearest up-stop at or above the car (`up.ceiling(cur)`).
2. None? If someone *above* wants to go **down**, go up to the **highest** such floor, then turn (collect the rest on the way down).
3. Otherwise turn: the nearest down-stop at or below the car (`down.floor(cur)`).
4. Otherwise the only work left is up-stops **below** the car: go down to the **lowest** one, then serve upward.
5. Going **DOWN** is the mirror image. An **IDLE** car picks the side of the nearest stop.



Car states.



A hall call.



Dispatch cost (`NearestCarStrategy`, from the course): an **idle** car costs its distance; a car **already coming this way in the same direction** costs its distance; any other car pays distance **+ two building heights**, so it wins only when nothing better is free. Cars out of service are skipped.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Choosing a car is a `DispatchStrategy`	**Strategy**	Different buildings and times of day want different rules; the course admits one implementation normally doesn't justify an interface, but here the *requirement* (swap the rule) does	`if (officeMode) … else …` in the dispatcher
Sweep rule in `StopList`, not in `Elevator`	Single Responsibility	The car drives; the stop list decides. Without this split, `Elevator` would do two jobs	Next-stop logic inside `step()`
Two sets of stops (up/down)	Data modelling	Distinguishes "5 going up" from "5 going down"	One list of floors
`step()` = one action	Deterministic simulation	Every scenario is a list of calls and an expected list of events; no clock, no flakiness (tests that pass or fail at random)	A thread per car with `sleep`
Modes as an enum inside the car	Lightweight State	Only three modes and a few checks; full State classes would add more code than they save here. If modes grew (fire phase II, independent service, VIP), switch to the State pattern	State classes from day one
Hall calls re-dispatched when a car leaves service	Robustness	A call accepted by a car must not be lost when that car is taken out	Silently clearing the car's stops
Queue calls when no car is available	—	"No car can take the call right now" is not the same as "rejected"	Dropping the call

The course names three weak points worth saying: (1) one step per call means **no threads** — a real controller runs each car on its own thread and needs a lock around the stop list (our methods are `synchronized` for that); (2) the dispatcher **decides once** and never re-evaluates, though real systems do (why a busy building sometimes changes your car number); (3) `Elevator` nearly does two jobs, hence the sweep rule in `StopList`. The course names only **one** pattern here (Strategy): `Button` and `Display` are ordinary abstract bases and `StopList` is Single Responsibility. Its scope leaves out emergency calls, fire service mode, express cars and door sensors that reopen for a hand (we add fire recall; the rest are extensions below).

Java implementation

Package `lld.ch14.elevator`. Essential lines:

```java
// StopList.java - LOOK over TWO sorted sets; a TreeSet's ceiling/floor/first/last answer
// "nearest stop at or above/below" in O(log n)
enum Direction { UP, DOWN, IDLE }
record Stop(int floor, Direction direction) {}        // floor + direction the car leaves in

final class StopList {
    private final TreeSet<Integer> up = new TreeSet<>();
    private final TreeSet<Integer> down = new TreeSet<>();
    void add(Stop s) { (s.direction() == Direction.DOWN ? down : up).add(s.floor()); }
    void remove(Stop s) { (s.direction() == Direction.DOWN ? down : up).remove(s.floor()); }
    // isEmpty(), clear(), drain() (removes and returns every stop - used when a car leaves service)

    Optional<Stop> next(int cur, Direction dir) {
        if (isEmpty()) return Optional.empty();
        if (dir == Direction.IDLE) dir = nearestSide(cur);   // idle: the side of the nearest stop
        return Optional.of(dir == Direction.UP ? goingUp(cur) : goingDown(cur));
    }
    private Stop goingUp(int cur) {
        Integer f = up.ceiling(cur);
        if (f != null) return new Stop(f, Direction.UP);                                        // work ahead
        if (!down.isEmpty() && down.last() > cur) return new Stop(down.last(), Direction.DOWN); // go up, then
turn
        f = down.floor(cur);
        if (f != null) return new Stop(f, Direction.DOWN);                                      // turn around now
        return new Stop(up.first(), Direction.UP);                                              // down to lowest
up-call
    }
    private Stop goingDown(int cur) {                                                           // mirror image
        Integer f = down.floor(cur);
        if (f != null) return new Stop(f, Direction.DOWN);
        if (!up.isEmpty() && up.first() < cur) return new Stop(up.first(), Direction.UP);
        f = up.ceiling(cur);
        if (f != null) return new Stop(f, Direction.UP);
        return new Stop(down.last(), Direction.DOWN);
    }
}
```

```java
// Elevator.java - it drives, its StopList decides; one step() = one action; synchronized because
// riders press buttons from other threads while the controller thread calls step()
enum Mode { NORMAL, MAINTENANCE, FIRE_RECALL }
record Event(Kind kind, int car, int floor, Direction direction, String note) {
    enum Kind { MOVED, ARRIVED, DOORS_CLOSED, OVERLOAD, NOTHING }
}

final class Elevator {
    synchronized void assign(int f, Direction wanted) { stops.add(new Stop(f, wanted)); }   // from the dispatcher
    synchronized void pressFloor(int f) {                    // from the panel inside the car
        if (mode != Mode.NORMAL) return;                     // car buttons are dead during fire recall
        if (f == floor && door.isOpen()) return;
        Direction d = f > floor ? Direction.UP : f < floor ? Direction.DOWN
                : (direction == Direction.IDLE ? Direction.UP : direction);
        stops.add(new Stop(f, d));
    }

    synchronized Event step() {
        if (mode == Mode.MAINTENANCE) return nothing();      // parked: never moves
        if (door.isOpen()) {
            if (mode == Mode.FIRE_RECALL && stops.isEmpty()) return nothing();   // parked at lobby, doors open
            if (loadKg > capacityKg)
                return new Event(Event.Kind.OVERLOAD, id, floor, direction,
                        "OVERLOAD " + loadKg + "kg > " + capacityKg + "kg, doors stay open");
            door.close();
            return new Event(Event.Kind.DOORS_CLOSED, id, floor, direction, "doors close");
        }
        var next = stops.next(floor, direction);
        if (next.isEmpty()) { direction = Direction.IDLE; return nothing(); }
        Stop target = next.get();
        if (target.floor() == floor) {                       // arrived
            stops.remove(target);
            direction = stops.isEmpty() ? Direction.IDLE : target.direction();
            door.open();
            return new Event(Event.Kind.ARRIVED, id, floor, target.direction(), "doors open");
        }
        direction = target.floor() > floor ? Direction.UP : Direction.DOWN;   // never moves with doors open
        floor += direction == Direction.UP ? 1 : -1;
        return new Event(Event.Kind.MOVED, id, floor, direction, "");
    }

    synchronized List<Stop> takeOutOfService() {             // returns accepted stops for re-dispatch
        mode = Mode.MAINTENANCE;
        direction = Direction.IDLE;
        door.open();                                         // parked with doors open so riders can leave
        return stops.drain();
    }
    synchronized void fireRecall(int lobby) {                // drop every stop, go to the lobby, stay there
        if (mode == Mode.MAINTENANCE) return;
        mode = Mode.FIRE_RECALL;
        stops.clear();
        stops.add(new Stop(lobby, Direction.DOWN));
        direction = Direction.IDLE;
    }
    // board(kg)/alight(kg), returnToService(), available() = mode == NORMAL, display()
}
```

```java
// Dispatch.java
interface DispatchStrategy {                                  // office at 9am vs hotel at midnight
    Optional<Elevator> choose(List<Elevator> cars, int floor, Direction wanted, int floorCount);
}

final class NearestCarStrategy implements DispatchStrategy {
    @Override public Optional<Elevator> choose(List<Elevator> cars, int floor, Direction wanted, int floorCount) {
        return cars.stream().filter(Elevator::available)      // out-of-service cars skipped
                .min(Comparator.comparingInt((Elevator c) -> cost(c, floor, wanted, floorCount))
                        .thenComparingInt(Elevator::id));
    }
    private int cost(Elevator c, int floor, Direction wanted, int n) {
        int distance = Math.abs(c.floor() - floor);
        Direction d = c.direction();
        if (d == Direction.IDLE) return distance;
        boolean onTheWay = (d == Direction.UP && c.floor() <= floor) || (d == Direction.DOWN && c.floor() >=
floor);
        if (onTheWay && d == wanted) return distance;
        return distance + 2 * n;                              // penalty: two building heights
    }
}

final class Dispatcher {                                      // calls no car can take wait in a queue
    private volatile DispatchStrategy strategy;
    private final Queue<Stop> pending = new ArrayDeque<>();
    synchronized Optional<Elevator> call(int floor, Direction wanted) {
        Optional<Elevator> car = strategy.choose(cars, floor, wanted, floorCount);
        car.ifPresentOrElse(c -> c.assign(floor, wanted), () -> pending.add(new Stop(floor, wanted)));
        return car;
    }
    // retryPending(): called every tick, re-asks the strategy for each queued call
}
```

```java
// Building.java - owns hall buttons, cars and dispatcher; the only class that knows cars AND floors
final class Building {
    private final Set<Stop> litHallButtons = ConcurrentHashMap.newKeySet();

    String call(int floor, Direction d) {                     // top floor has no UP, ground floor no DOWN
        if ((floor == floors - 1 && d == Direction.UP) || (floor == 0 && d == Direction.DOWN))
            throw new IllegalArgumentException("no " + d + " button on floor " + floor);
        if (!litHallButtons.add(new Stop(floor, d))) return "hall %d %s already lit".formatted(floor, d);
        return dispatcher.call(floor, d).map(c -> "hall %d %s -> car%d".formatted(floor, d, c.id()))
                .orElse("hall %d %s -> queued (no car available)".formatted(floor, d));
    }
    List<String> step() {                                     // one tick: retry queued calls, every car steps once
        tick++;
        dispatcher.retryPending();
        for (Elevator c : cars) {
            Event e = c.step();
            if (e.kind() == Event.Kind.ARRIVED) litHallButtons.remove(new Stop(e.floor(), e.direction()));
            // ... collect "car1->2" / "car1@1 doors open" lines
        }
    }
    List<String> takeOutOfService(int carId) {                // accepted hall calls go back to the dispatcher
        for (Stop s : car(carId).takeOutOfService())
            if (litHallButtons.contains(s)) dispatcher.call(s.floor(), s.direction());   // re-dispatch or queue
    }
    void fireAlarm() { litHallButtons.clear(); cars.forEach(c -> c.fireRecall(0)); }
}
// Main.java: part 1 runs LOOK on one StopList; parts 2-5 a two-car building (floors 0-9, 680 kg);
// part 6: 8 threads released by a CountDownLatch press hall buttons on a three-car building.
```

Output:

```
== 1. LOOK on one car: at floor 4 going up; stops 6(up) 9(car) 2(down) 1(up) ==
service order [6, 9, 2, 1]  (turns at 9 and at 1: never runs to 0 or to a floor above 9)

== 2. Building: floors 0-9, car1 at 0, car2 at 8 ==
hall 5 UP -> car2
hall 1 UP -> car1
  t1: car1->1, car2->7
  t2: car1@1 doors open, car2->6
rider enters car1 at floor 1, presses 7
  t3: car1@1 doors close, car2->5
  t4: car1->2, car2@5 doors open
rider enters car2 at floor 5, presses 9
hall 3 UP -> car1
  t5: car1->3, car2@5 doors close
  t6: car1@3 doors open, car2->6
  t7: car1@3 doors close, car2->7
  t8: car1->4, car2->8
  t9: car1->5, car2->9
  t10: car1->6, car2@9 doors open
  t11: car1->7, car2@9 doors close
  t12: car1@7 doors open
  t13: car1@7 doors close
display: car1@7 IDLE 80kg | car2@9 IDLE 75kg

== 3. Overload: at floor 9, 10 more people squeeze into car2 ==
hall 9 DOWN -> car2
  t14: car2@9 doors open
  t15: car2@9 OVERLOAD 775kg > 680kg, doors stay open
  t16: car2@9 OVERLOAD 775kg > 680kg, doors stay open
two people step out
  t17: car2@9 doors close
  t18: car2->8
  t19: car2->7
  t20: car2->6
  t21: car2->5
  t22: car2->4
  t23: car2->3
  t24: car2->2
  t25: car2->1
  t26: car2->0
  t27: car2@0 doors open
  t28: car2@0 doors close

== 4. Maintenance: calls move to another car ==
hall 4 DOWN -> car1
re-dispatch hall 4 DOWN -> car2
re-dispatch hall 4 DOWN -> queued
hall 2 UP -> queued (no car available)
engineer returns car1 to service; queued calls are dispatched on the next tick
  t29: car1@7 doors close
  t30: car1->6
  t31: car1->5
  t32: car1->4
  t33: car1@4 doors open
  t34: car1@4 doors close
  t35: car1->3
  t36: car1->2
  t37: car1@2 doors open
  t38: car1@2 doors close

== 5. Fire alarm: every car to the lobby, doors open, car buttons dead ==
  t39: car2@0 doors close
  t40: car2->1
  t41: car2->2
  t42: car1->1, car2->1
  t43: car1->0, car2->0
  t44: car1@0 doors open, car2@0 doors open
display: car1@0 IDLE [open] 80kg FIRE_RECALL | car2@0 IDLE [open] 0kg FIRE_RECALL
```

```
== 6. 8 riders press hall buttons at the same instant (new building) ==
all 8 calls served: true (ticks=11 < 200)
```

Read the output: part 1 — the car serves 6 and 9 going up, turns at 9 (the highest request, not the top of the shaft), then 2 and 1: LOOK, not SCAN. Part 2 — the hall call on 3 goes to car 1, below 3 and already going up. Part 3 — the overloaded car keeps its doors open until two people step out. Part 4 — car 1's call moves to car 2, is queued when no car is available, and is served once car 1 returns. Part 5 — car 2 reverses mid-trip and both cars park at the lobby with doors open.

Concurrency & edge cases

Situation	Handling
Riders press buttons while the controller steps cars	`Elevator` methods are `synchronized`; the dispatcher is `synchronized`; hall lights are a concurrent set
Same hall button pressed twice	`litHallButtons.add` returns false: "already lit", no second dispatch
Up and down requests on the same floor	Two different `Stop`s in two sets
Car call for the floor the car is on, doors open	Ignored (doors already open)
Overload	Doors stay open, display shows OVERLOAD, car does not move
Car taken out of service with accepted hall calls	Calls re-dispatched or queued; never lost
No car available	Call queued; retried every tick
Fire alarm	All stops dropped, lobby added, car buttons ignored, doors stay open at the lobby
Starvation	LOOK serves every accepted stop within one round trip (one sweep each way)
Invalid button (UP on the top floor)	`IllegalArgumentException` — that button does not exist

Extensions & follow-up questions

1. **"How do you choose which car?"** — `DispatchStrategy`; show `NearestCarStrategy` and mention alternatives: zoning (car 1 serves floors 0–10), **up-peak** mode (send idle cars to the lobby at 9 a.m.), and energy-saving rules.
2. **"What if two riders press at the same instant?"** — The dispatcher is synchronized, so calls are dispatched one at a time; each car's stop list is guarded by the car's lock. (The course's design leaves this lock out and says so.)
3. **"How would you test this?"** — Because one call = one step, every test is: a list of button presses, N steps, and an expected list of events. No sleeps, no flakiness.
4. **"Destination dispatch?"** — Riders enter their destination in the lobby keypad; the system groups riders with nearby destinations into the same car. Implement as another `DispatchStrategy` that receives (from, to) instead of (floor, direction).
5. **"Re-evaluate assignments?"** — Each tick, for unserved hall calls, ask the strategy again and move the call if a much better car is free now.
6. **"Real time?"** — A `ScheduledExecutorService` (a Java thread pool that runs tasks on a timer) calls `building.step()` every second (or per car on its own thread, with the car lock guarding the stop list).
7. **"Emergency stop button inside the car?"** — A new mode that halts movement immediately and raises an alarm; with more modes, refactor modes into State classes.
8. **"Express cars?"** — A car with a set of floors it may serve (e.g. lobby plus 30–40); the dispatch strategy skips cars whose served set does not include the call floor.

9. **"Door sensor that reopens for a hand?"** — A `DoorSensor` input; while it reports an obstruction, `step()` reopens the doors instead of closing them (same slot as the overload check).

Real-world example

Modern high-rise buildings use **destination dispatch** systems (for example Schindler's PORT, Otis CompassPlus and KONE's destination control): riders choose their floor in the lobby and the system tells them which car to take, which cuts stops per trip — extension 4. Controllers switch between traffic modes such as **up-peak** in the morning. The fire behaviour in this design follows the idea of **Phase I emergency recall** in elevator safety codes such as ASME A17.1: on a fire alarm, cars return to the designated landing, open their doors and are removed from normal service.

In the interview

Plan for 45 minutes: 5 min requirements and scope (how many cars, which modes) → 5 min entities → **15 min on the stop list and LOOK** (two sets, the four cases) → 10 min car `step()` and dispatcher strategy → 10 min concurrency, testing and follow-ups.

> **30-second answer:** "Each car keeps its stops in two sorted sets, up-stops and down-stops, and uses LOOK: keep serving stops ahead in the current direction, turn at the last request, never run to the end of the shaft for nothing. A car's `step()` does one action — close doors, refuse to move if overloaded, move one floor, or stop and open — which makes the whole system testable without a clock. A dispatcher assigns hall calls through a `DispatchStrategy`, nearest suitable car by default, and queues calls when no car is available. Doors, load, maintenance and fire recall always override scheduling."

Follow-ups with answers

- *"Why LOOK and not SSTF?"* — SSTF can starve a distant floor; LOOK serves every accepted stop within one round trip (one sweep up, one sweep down).
- *"Why two sets and not one?"* — A floor can be requested going up and going down; one set cannot tell which rider to serve on which sweep.
- *"Where do you put the lock?"* — Around each car's stop list and state (the car's monitor — the built-in lock every Java object carries, taken by `synchronized`), and around the dispatcher's assignment. No global lock across cars.

TL;DR

- **Two sorted sets** (up/down) of stops + **LOOK** = efficient, starvation-free scheduling.
- **`step()` = one action**; doors, load, maintenance and fire recall override scheduling.
- **Strategy** for dispatch (nearest car by default), with a pending queue and re-dispatch when a car leaves service.
- Thread safety: per-car lock, synchronized dispatcher, concurrent hall-light set.

14.6 Design a Traffic Signal Control System

A **traffic signal controller** decides which approaches of an intersection get green, for how long, and in what order. An **approach** is one road entering the intersection (north, south, east, west). A **phase** is a set of approaches that may be green together because their paths do not cross (for example north and south going straight). The controller cycles through phases, inserting **yellow** (warning) and **all-red** (clearance) intervals between them.

This question tests a **timed state machine** with a **hard safety rule** — *two crossing flows must never be green at the same time* — plus an **emergency override** that must interrupt the normal cycle *safely* (never by cutting a yellow short).

Problem & clarifying questions

Question	Assumed answer
How many roads?	A four-way intersection (N, S, E, W); phases are configurable.
Which signals?	Red, yellow, green; flashing red in fault mode.
Fixed or adaptive timing?	Configurable: fixed durations, or adaptive based on queue sensors.
Pedestrian signals, turn arrows?	Out of scope (extensions).
Emergency vehicles?	Yes: an approaching ambulance or fire truck gets green as soon as safely possible, held until it passes.
Many intersections?	Each intersection has its own controller; a central system can coordinate them (extension).
Real time?	A timer ticks once per second; the logic is tick-driven so it can also be simulated.

Requirements

Functional

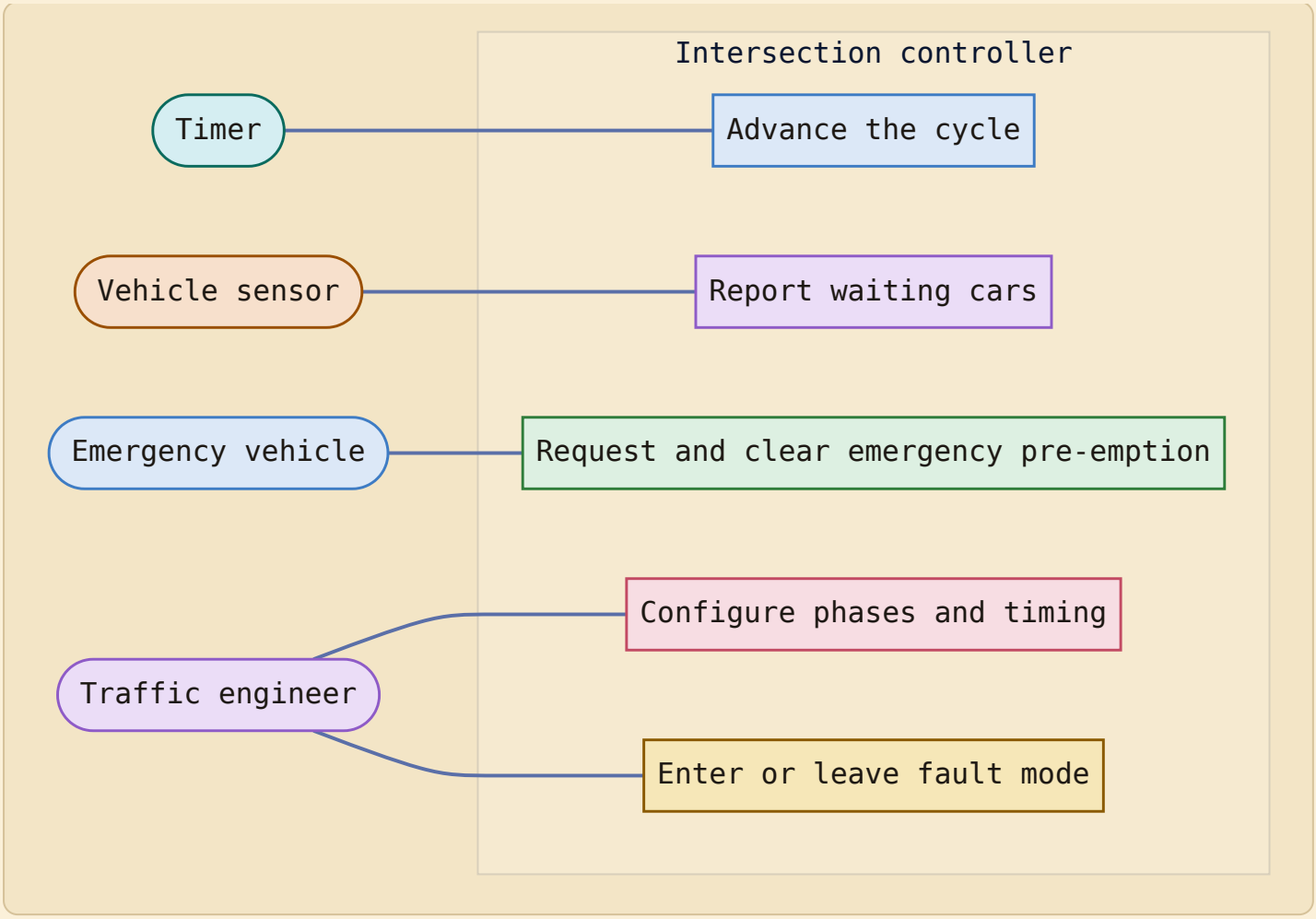
1. Control traffic at an intersection with **multiple roads**.
2. Support **red, yellow and green** signals.
3. Signal **durations are configurable** and can **adapt to traffic** conditions.
4. Transitions are **smooth and safe**: green → yellow → all-red → next green.
5. Handle **emergencies**: an approaching ambulance or fire truck gets priority.
6. Be **scalable and extensible** (more phases, more intersections, new features).
7. On a controller fault, fall back to a **safe mode** (all flashing red = treat as an all-way stop).

Non-functional

- **Safety invariant** (an invariant is a rule that must hold at every moment)**:** never conflicting greens, not even for an instant during a switch.
- **Deterministic timing:** a tick = one second; testable with a simulated clock.
- **Thread safety:** a timer thread ticks while sensor and emergency inputs arrive from other threads.
- **Observability** (the system's behaviour can be watched from outside)**:** displays, logs and monitors can subscribe to light changes.

Use cases

Actors: Timer (ticks every second), **Vehicle sensors** (report queue lengths), **Emergency vehicle / dispatch** (request and clear pre-emption — interrupting the normal cycle to serve a priority request), **Traffic engineer** (configures phases and timing), **Drivers** (see the lights).

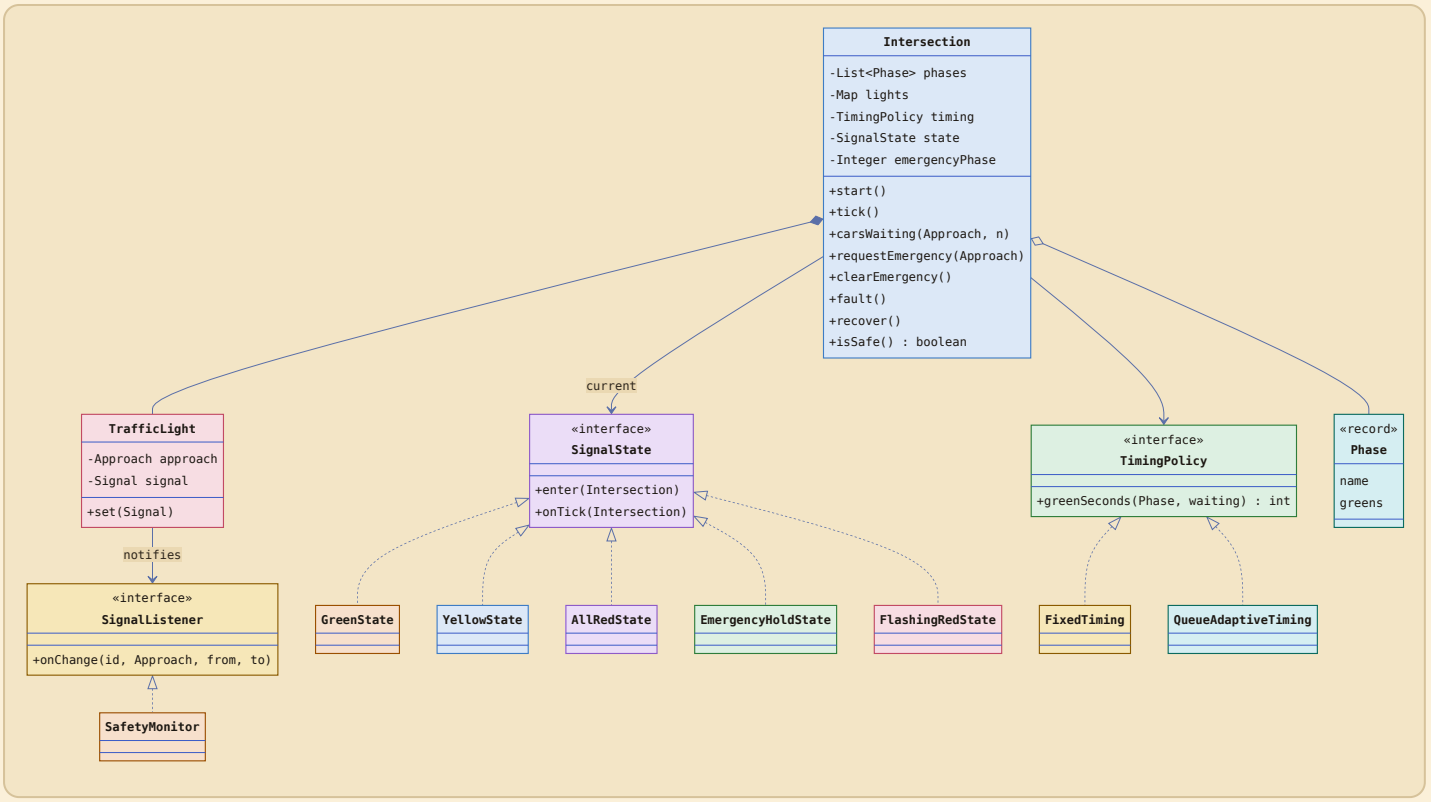


Core entities

Class	Responsibility
`Signal` (enum)	RED, YELLOW, GREEN, FLASHING_RED.
`Approach` (enum)	NORTH, SOUTH, EAST, WEST.
`Phase` (record)	A name and the set of approaches that may be green together.
`TrafficLight`	One signal head for one approach; notifies listeners on every change.
`SignalListener`	**Observer** interface for light changes.
`SafetyMonitor`	A listener that checks the safety invariant after every change to green.
`TimingPolicy`	**Strategy** for green duration: `FixedTiming`, `QueueAdaptiveTiming`.
`SignalState` + 5 states	**State**: `GreenState`, `YellowState`, `AllRedState`, `EmergencyHoldState`, `FlashingRedState`.
`Intersection`	The State context (the object that holds the current state and hands it each call): phases, lights, timing, sensors, emergency flag; `tick()`.

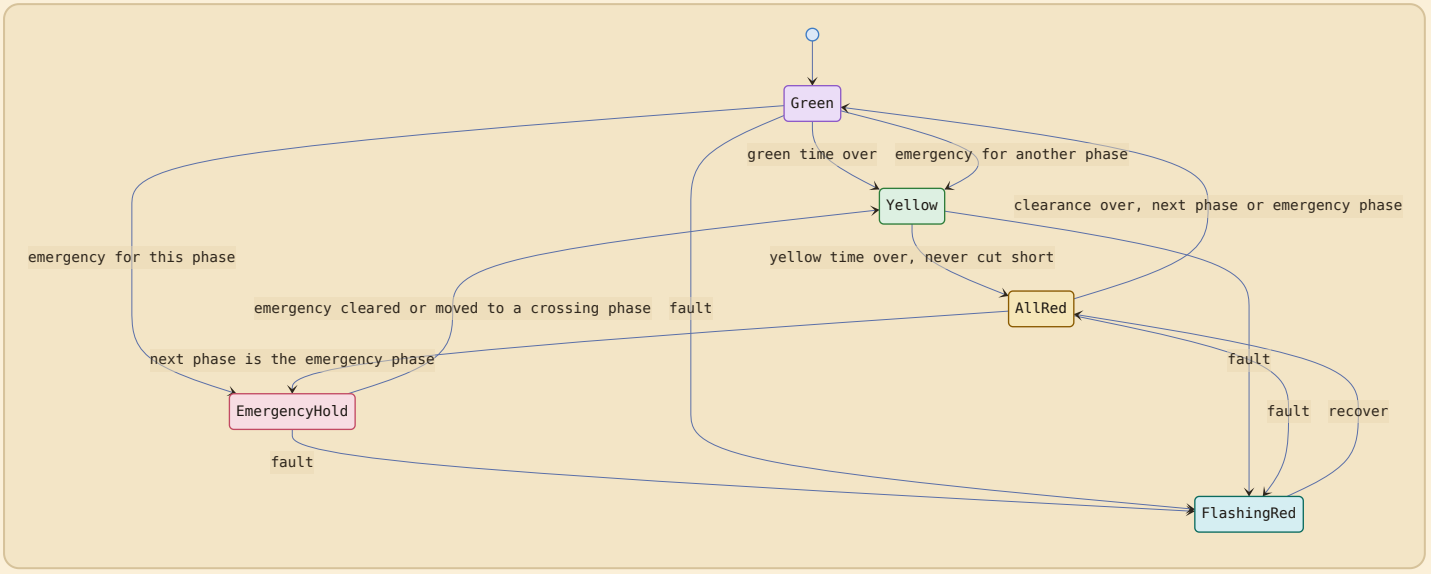
Mapping to the common problem statement. The widely used version (e.g. the open-source *awesome-low-level-design* collection) asks for an intersection with multiple roads, red/yellow/green signals, configurable durations that adjust to traffic, smooth transitions, emergency-vehicle handling, and an extensible design. Its `Signal` enum → our `Signal`; `Road` (id, name, its traffic light) → our `Approach` plus the `TrafficLight` stored for it; `TrafficLight` (signal, durations, notifies observers) → our `TrafficLight` (signal + listeners), with durations moved into `TimingPolicy` and the yellow/all-red settings; `TrafficController` (a Singleton managing all roads) → our `Intersection`, deliberately **not** a Singleton. What the reference leaves implicit, and we make explicit: the phase model (which roads may be green together), the all-red clearance interval, and the safety check.

Class diagram

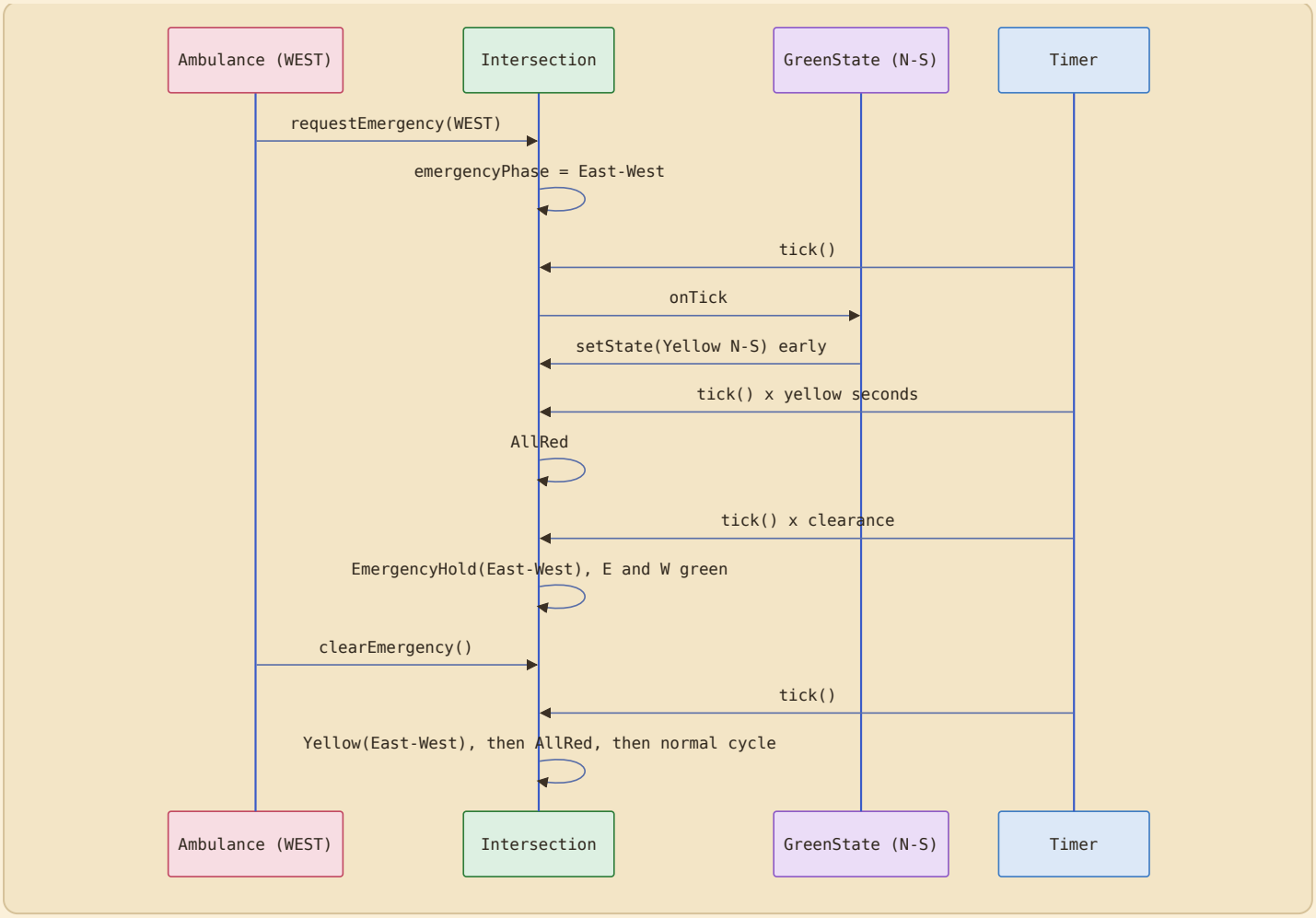


Key flows

The controller's state machine.



Emergency pre-emption.



One full fixed cycle (green 5 s, yellow 2 s, all-red 1 s): `N-S green (5) → N-S yellow (2) → all red (1) → E-W green (5) → E-W yellow (2) → all red (1) → …` — a 16-second cycle.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Each interval is a state class with `enter` and `onTick`	**State**	What a tick means depends on the interval; the transitions (and the "yellow is never cut short" rule) live in one small class each	One `switch` over an enum with counters for every case
States **carry data** (remaining seconds) and are created per transition	State with per-instance data	Unlike the vending machine's stateless states, these hold a countdown, so they cannot be shared	Shared singletons with counters on the context
Green duration comes from a `TimingPolicy`	**Strategy**	Fixed, adaptive (queue sensors), time-of-day plans all swap at run time	Durations hard-coded in `GreenState`
Light changes are published to listeners	**Observer**	Displays, logs, a central monitoring system and the safety monitor subscribe independently	The intersection calling each consumer directly
`showPhaseGreen` turns others **red first**, then turns the phase green	Safety by construction	There is never an instant with conflicting greens	Setting lights in arbitrary order
A separate `SafetyMonitor` checks the invariant	Defence in depth (several independent safety layers)	Mirrors real hardware conflict monitors that force flash if a bug ever causes conflicting greens	Trusting the controller logic alone
Emergency is a **flag** the states read, not a direct jump	Safety	A request only *shortens* green; yellow and all-red still run in full	Jumping straight to the emergency green (a crash risk)
The awesome-lld reference makes the controller a **Singleton**	Rejected	A city has many intersections, each with its own controller; the demo runs two. A per-city `TrafficControlCenter` can own them	`TrafficController.getInstance()`

Java implementation

Package `lld.ch14.trafficsignal`. Essential lines:

```java
// Model.java
enum Signal { RED, YELLOW, GREEN, FLASHING_RED }
enum Approach { NORTH, SOUTH, EAST, WEST }
record Phase(String name, Set<Approach> greens) {}            // approaches that may be green together

interface SignalListener { void onChange(String intersection, Approach approach, Signal from, Signal to); }

final class TrafficLight {                                     // one head facing one approach
    private Signal signal = Signal.RED;
    void set(Signal to) {
        if (to == signal) return;
        Signal from = signal;
        signal = to;
        listeners.forEach(l -> l.onChange(intersection, approach, from, to));
    }
}

interface TimingPolicy { int greenSeconds(Phase phase, Map<Approach, Integer> waitingCars); }
record FixedTiming(int green) implements TimingPolicy {
    public int greenSeconds(Phase p, Map<Approach, Integer> waiting) { return green; }
}
record QueueAdaptiveTiming(int min, int max, int secondsPerCar) implements TimingPolicy {
    public int greenSeconds(Phase p, Map<Approach, Integer> waiting) {    // more cars = longer, clamped
        int cars = p.greens().stream().mapToInt(a -> waiting.getOrDefault(a, 0)).sum();
        return Math.max(min, Math.min(max, cars * secondsPerCar));
    }
}

final class SafetyMonitor implements SignalListener {          // the one rule that must never break
    @Override public void onChange(String id, Approach a, Signal from, Signal to) {
        if (to != Signal.GREEN || watched[0] == null) return;
        if (!watched[0].isSafe()) violations.add(id + ": " + a + " went green while a crossing flow was moving");
    }
}
```

```java
// States.java - each state knows what one second means for it and which state is next
interface SignalState {
    default void enter(Intersection x) {}
    void onTick(Intersection x);
    String name();
}

final class GreenState implements SignalState {
    private final int phase;
    private int remaining;                                  // per-instance countdown: not shareable
    @Override public void enter(Intersection x) {
        if (Integer.valueOf(phase).equals(x.emergencyPhase())) { x.setState(new EmergencyHoldState(phase));
return; }
        remaining = x.greenSecondsFor(phase);               // from the TimingPolicy
        x.showPhaseGreen(phase);
    }
    @Override public void onTick(Intersection x) {
        Integer em = x.emergencyPhase();
        if (em != null) {                                   // pre-emption: end this green early
            x.setState(em == phase ? new EmergencyHoldState(phase) : new YellowState(phase));
            return;
        }
        if (--remaining <= 0) x.setState(new YellowState(phase));
    }
}

final class YellowState implements SignalState {
    @Override public void enter(Intersection x) { remaining = x.yellowSeconds(); x.showPhase(phase,
Signal.YELLOW); }
    @Override public void onTick(Intersection x) {          // never reads the emergency flag:
        if (--remaining <= 0) x.setState(new AllRedState(x.nextPhaseAfter(phase)));   // yellow is never cut short
    }
}

final class AllRedState implements SignalState {            // clearance: the junction empties
    @Override public void enter(Intersection x) { remaining = x.allRedSeconds(); x.showAllRed(); }
    @Override public void onTick(Intersection x) {
        if (--remaining <= 0) {
            Integer em = x.emergencyPhase();
            x.setState(new GreenState(em != null ? em : next));   // emergency phase first, else the next one
        }
    }
}

final class EmergencyHoldState implements SignalState {     // hold the phase green until cleared
    @Override public void enter(Intersection x) { x.showPhaseGreen(phase); }
    @Override public void onTick(Intersection x) {
        Integer em = x.emergencyPhase();
        if (em == null || em != phase) x.setState(new YellowState(phase));   // cleared or crossing request: leave
safely
    }
}

final class FlashingRedState implements SignalState {       // fault: treat as a four-way stop
    @Override public void enter(Intersection x) { x.showAll(Signal.FLASHING_RED); }
    @Override public void onTick(Intersection x) {}
}
```

```
// Intersection.java - the State context; tick() once per second from a timer thread, other inputs from
// other threads; every public method synchronized, so a transition is never observed half-done
final class Intersection {
    private SignalState state;
    private Integer emergencyPhase;                          // null = no override

    synchronized void start() { setState(new GreenState(0)); }
    synchronized void tick() { clock++; state.onTick(this); }
    synchronized void carsWaiting(Approach a, int n) { waitingCars.put(a, n); }
    synchronized void requestEmergency(Approach from) {
        for (int i = 0; i < phases.size(); i++) if (phases.get(i).greens().contains(from)) emergencyPhase = i;
    }
    synchronized void clearEmergency() { emergencyPhase = null; }
    synchronized void fault() { setState(new FlashingRedState()); }
    synchronized void recover() { emergencyPhase = null; setState(new AllRedState(0)); }

    void setState(SignalState s) { state = s; s.enter(this); }   // the new state sets lights and loads its timer

    void showPhaseGreen(int p) {                             // red first, then green: never two crossing greens
        Phase phase = phases.get(p);
        lights.forEach((a, l) -> { if (!phase.greens().contains(a)) l.set(Signal.RED); });
        phase.greens().forEach(a -> lights.get(a).set(Signal.GREEN));
    }
    synchronized boolean isSafe() {                          // every green/yellow approach belongs to ONE phase
        return phases.stream().anyMatch(p -> lights.entrySet().stream()
                .filter(e -> e.getValue().signal() == Signal.GREEN || e.getValue().signal() == Signal.YELLOW)
                .allMatch(e -> p.greens().contains(e.getKey())));
    }
}
// Main.java: phases North-South and East-West; FixedTiming(5), yellow 2, all-red 1; part 5 drives a second
// intersection with a ScheduledExecutorService (1 ms stands for 1 s) while a virtual thread (a lightweight
// Java 21 thread, cheap to create by the thousand) fires and clears 20 emergency requests.
```

Output:

```
== 1. Fixed cycle: green 5s, yellow 2s, all-red 1s (printed when lights change) ==
t=0   N:G S:G E:R W:R  GREEN(0)
t=5   N:Y S:Y E:R W:R  YELLOW(0)
t=7   N:R S:R E:R W:R  ALL_RED
t=8   N:R S:R E:G W:G  GREEN(1)
t=13  N:R S:R E:Y W:Y  YELLOW(1)
t=15  N:R S:R E:R W:R  ALL_RED
t=16  N:G S:G E:R W:R  GREEN(0)

== 2. Adaptive timing: 12 cars waiting East-West, 2 North-South ==
t=21  N:Y S:Y E:R W:R  YELLOW(0)
t=23  N:R S:R E:R W:R  ALL_RED
t=24  N:R S:R E:G W:G  GREEN(1)
t=34  N:R S:R E:Y W:Y  YELLOW(1)
t=36  N:R S:R E:R W:R  ALL_RED

== 3. Ambulance from the WEST while North-South is green ==
t=37  N:G S:G E:R W:R  GREEN(0)
t=38  emergency request: WEST
t=39  N:Y S:Y E:R W:R  YELLOW(0)
t=41  N:R S:R E:R W:R  ALL_RED
t=42  N:R S:R E:G W:G  EMERGENCY_HOLD(1)
t=45  ambulance has passed: clear
t=46  N:R S:R E:Y W:Y  YELLOW(1)
t=48  N:R S:R E:R W:R  ALL_RED
t=49  N:G S:G E:R W:R  GREEN(0)

== 4. Controller fault -> flashing red, then recovery ==
t=50  N:F* S:F* E:F* W:F*  FLASHING_RED
t=51  N:G S:G E:R W:R  GREEN(0)

== 5. Real timer thread (1 ms = 1 s) + emergency from another thread ==
ticks run: true, safety violations: 0

light changes seen by observer: 48, safety violations: 0
```

Read the output: fixed cycle 5 s green, 2 s yellow, 1 s all-red; with 12 cars waiting east-west the adaptive policy gives E-W its 10-second maximum; the ambulance request at t=38 ends N-S green at the next tick, but yellow (2 s) and all-red (1 s) still run before E-W goes green and is **held**; after the clear the cycle resumes; a fault flashes every head red; no safety violation.

Concurrency & edge cases

Situation	Handling
Emergency request arrives between two ticks	Stored in a field under the intersection's lock; the next tick acts on it
Emergency arrives during yellow	Yellow runs in full; `AllRedState` then picks the emergency phase
Emergency for the phase that is already green	`GreenState.onTick` switches to `EmergencyHold` — no needless yellow
Two emergency vehicles from crossing directions	The latest request wins: if it arrives during a hold, `EmergencyHoldState` leaves through yellow and all-red and the new phase is then held. A real system queues requests by priority and arrival and serves them in turn (extension)
Emergency never cleared (lost message)	Add a maximum hold time after which the controller resumes the cycle
Timer thread and sensor thread overlap	All public methods are `synchronized`; listeners are in a `CopyOnWriteArrayList` (a thread-safe list that copies itself on every change)
Adaptive timing starves a quiet road	Minimum green time in `QueueAdaptiveTiming`; every phase still gets its turn in the cycle
Controller bug causes conflicting greens	`SafetyMonitor` detects it; real controllers then force flashing red
Timer drift (small delays adding up so ticks slowly fall behind)	`scheduleAtFixedRate` keeps a fixed rate; for real deployments use the controller's hardware clock

Extensions & follow-up questions

1. **"Pedestrian crossings?"** — Add pedestrian signal heads per phase (WALK during the phase's green, flashing DON'T WALK before yellow) and a push-button input that requests the phase.
2. **"Left-turn arrows?"** — More phases (e.g. N-S protected left, then N-S through); phases are data, so the state machine does not change.
3. **"Coordinate many intersections (green wave)?"** — A `TrafficControlCenter` sets each intersection's cycle length and **offset** so that a platoon (group travelling together) of cars driving at the speed limit meets greens in sequence.
4. **"Emergency corridor?"** — The centre pre-empts the next intersections along the ambulance's route, each through `requestEmergency`.
5. **"Night mode?"** — A time-of-day `TimingPolicy`, or a flashing-yellow state for the main road with flashing red for the side road.
6. **"How to test?"** — The tick-driven design is deterministic: call `tick()` N times and assert the snapshot, as `Main` does.

Real-world example

Adaptive systems such as **SCATS** (developed in Sydney) and **SCOOT** (developed in the UK) adjust green times and cycle lengths from road sensors across whole city networks — the `QueueAdaptiveTiming` strategy scaled up. **Emergency vehicle pre-emption** systems (for example infrared or GPS-based emitters on fire trucks) request green for the vehicle's approach; the controller still times out the current yellow and red clearance before serving it, exactly like our `EmergencyHoldState`. In North American cabinets, a **conflict monitor / malfunction management unit** watches the outputs and forces the intersection into flash if it ever sees conflicting greens — our `SafetyMonitor` observer.

In the interview

Plan for 40 minutes: 5 min requirements (phases, timing, emergency, fault) → 5 min entities → 10 min draw the state machine (green → yellow → all-red → next; emergency and fault edges) → 15 min code `GreenState` / `YellowState` / `AllRedState` and `tick()` → 5 min concurrency and follow-ups.

30-second answer: "An intersection has phases — sets of approaches that can be green together — and a controller that is a timed State machine: Green, Yellow, All-Red, plus Emergency-Hold and Flashing-Red. A timer calls `tick()` once a second; each state counts down and names the next one. Green duration comes from a swappable timing Strategy, fixed or queue-adaptive. An emergency request only shortens the current green: yellow and all-red always run in full, then the emergency phase is held green until cleared. Lights publish changes to observers, including a safety monitor that checks no crossing flows are ever green together."

Follow-ups with answers

- *"Why not jump straight to the emergency green?"* — Cars already in the intersection need the yellow and all-red intervals to clear it; skipping them causes crashes.
- *"Why is the controller not a Singleton?"* — There is one per intersection, and a city has thousands.
- *"How do you make the timing adaptive?"* — Sensors report queue lengths; a `TimingPolicy` turns them into green seconds within a min/max.

TL;DR

- **Timed State machine**: Green → Yellow → All-Red → next Green; plus Emergency-Hold and Flashing-Red.
- **Safety first**: red before green, yellow never cut short, conflict-monitor observer.
- **Strategy** for timing (fixed/adaptive), **Observer** for lights, emergency as a flag the states read.
- Tick-driven and synchronized: deterministic tests, safe with a timer thread plus emergency threads.

14.7 Design a Shipping Locker System (Amazon Locker)

Also asked as: "Design Amazon Locker", "Design an Amazon Hub Locker system", "Design a parcel/pickup locker". It is the same problem.

A **parcel locker** (also called a *pickup locker* or *shipping locker*) is a wall of numbered steel doors in a shop, station or apartment lobby. A courier puts a parcel behind one door; the customer later types a **pickup code** on the station's screen and that one door opens. Amazon Hub Locker, InPost's Paczkomat network in Poland and similar networks run by couriers and online shops in many countries all work this way. Lockers save the courier a failed doorstep delivery and let the customer collect at any hour.

This question looks like the parking lot (§14.1) — hand out a limited set of spaces of different sizes — but adds three things interviewers like to probe: **a lifecycle with deadlines** (a reservation that times out, a parcel that expires if nobody collects it), **a secret** (the pickup code, which must be hard to guess and never shown to the courier), and **a network** of stations (if the nearest one is full, suggest another).

Problem & clarifying questions

Question	Assumed answer
What sizes do doors come in?	A few fixed sizes (small, medium, large). A parcel may go into a bigger door if no door of its own size is free.
When is a door chosen — at the station, or before?	Before: the carrier's system **reserves** a door when the parcel is sorted for delivery, so the courier never arrives to a full station. The hold lasts 2 hours; if nothing is deposited, the door is released.
How does the customer open the door?	A 6-digit **one-time code** sent by SMS/app when the parcel is deposited. The courier never sees it.
How long can a parcel stay?	3 days after deposit. After that it **expires** and the carrier takes it back to the sender on the next visit.
What stops someone guessing codes?	Codes are random and single-use; after 5 wrong codes in a row the keypad locks for 15 minutes.
Returns (customer drops a parcel off)?	Yes. The customer starts a return online; a door is held for 2 days and a **drop-off code** is sent. The code opens the empty door once, and the courier collects the parcel on the next visit.
Many stations?	Yes, a city-wide network. If the chosen station is full, offer the nearest station with space.
Broken doors?	A technician can take a free door **out of service**.
Payment at the locker (cash on delivery)?	Out of scope (extension).

Requirements

Functional

1. A station has numbered compartments of fixed sizes.
2. **Reserve** the best-fitting free compartment for a parcel (all-or-nothing); a reservation expires if the parcel is not deposited in time.
3. **Deposit**: the parcel goes in, a one-time pickup code is generated and **sent to the customer only**.
4. **Pickup**: a correct code opens exactly one door, frees the compartment and stops working.
5. **Expiry**: an uncollected parcel expires after the pickup window; the customer is told; the carrier **collects expired parcels** for return.
6. **Brute-force protection**: lock the keypad after too many wrong codes.
7. **Network search**: nearest station within a radius that has a free door this parcel fits.
8. Compartments can be taken out of service.
9. **Customer return**: reserve a door for a return, send a single-use drop-off code, open the empty door on that code, and let the courier collect dropped-off returns. An unused drop-off hold is cancelled after its window.

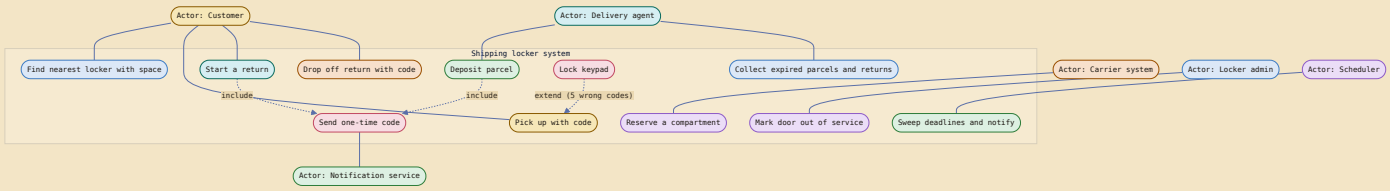
Non-functional

- **Never give one door to two parcels**, even when many carriers reserve at the same moment.
- **Security**: codes are unpredictable, single-use, stored only as a hash (the one-way fingerprint described in the ATM section).
- **Correct deadlines** without depending on a background timer being on time.
- **Testable**: time and randomness are injected (a fake clock and a seeded random generator in the demo — one started from a fixed seed, so it produces the same numbers every run).

Use cases

Actors: Customer (picks up, drops off returns), **Delivery agent** (the courier: deposits parcels, collects expired parcels and returns), **Carrier system** (the shipper's software that books doors), **Locker admin** (a technician who takes doors out of service), **Scheduler** (a component that runs a job at set times; here

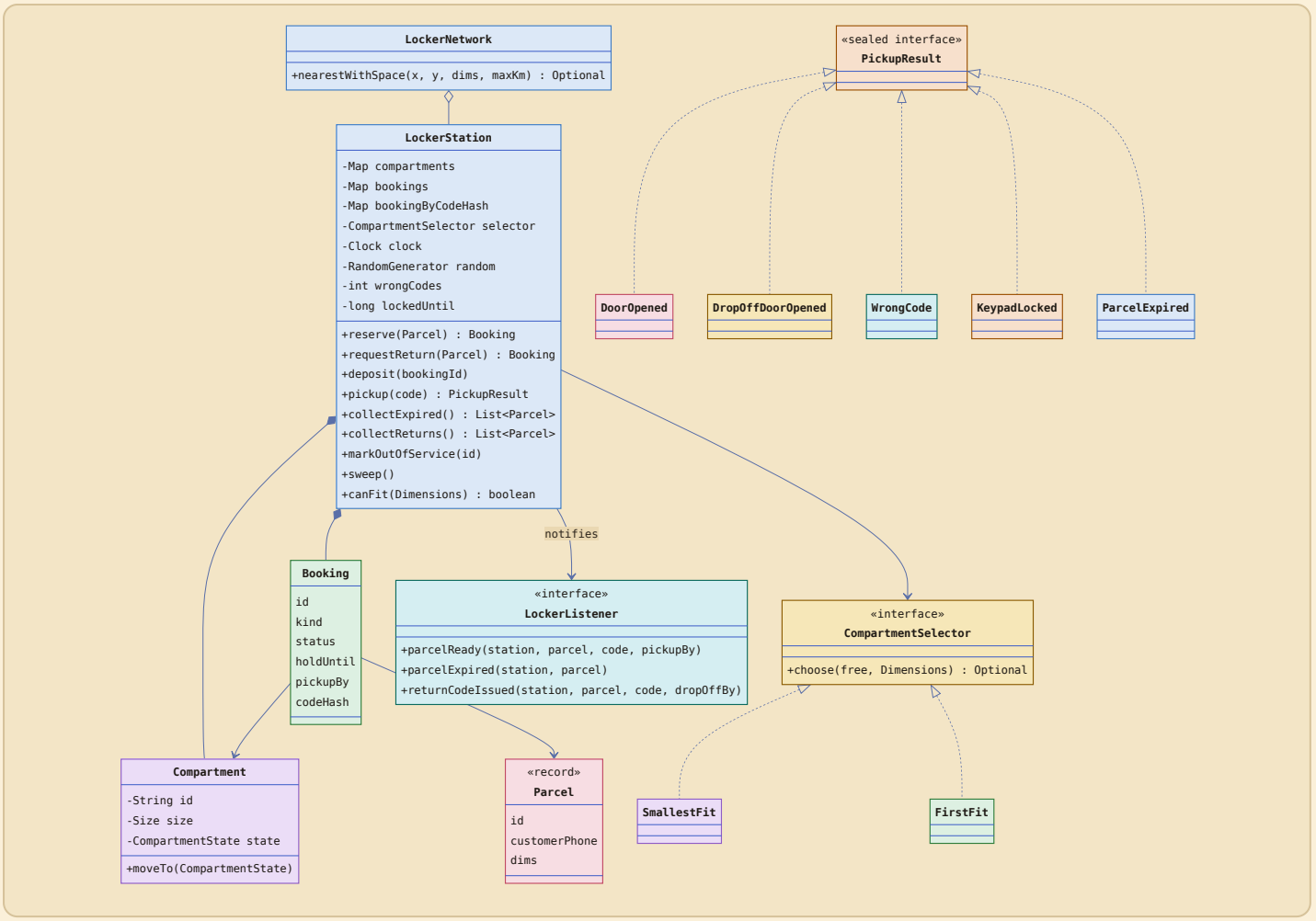
the periodic sweep) and **Notification service** (SMS/app). Notation (system box, include/extend arrows) as in §3.10.



Core entities

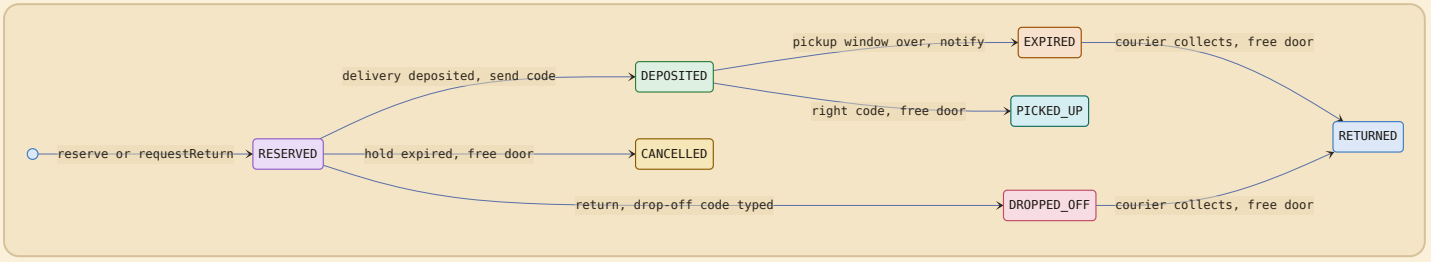
Class	Responsibility
`Size` (enum)	SMALL / MEDIUM / LARGE with inner dimensions; `fits(Dimensions)` checks a box against a door after sorting both sides.
`Dimensions` (record)	A parcel's three sides in cm (validated positive).
`Parcel` (record)	Parcel id, customer phone, dimensions.
`Compartment`	One door: id, size, `CompartmentState` (FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE) with the legal moves inside `moveTo`.
`Booking` + `BookingKind` + `BookingStatus`	One parcel's stay in one door, either a DELIVERY or a CUSTOMER_RETURN (statuses in the state diagram under Key flows). Holds the deadlines and the code **hash**.
`PickupResult` (sealed)	`DoorOpened`, `DropOffDoorOpened`, `WrongCode`, `KeypadLocked`, `ParcelExpired` — every keypad outcome is a type, not a magic string (a bare text value whose meaning is known only by convention).
`CompartmentSelector`	**Strategy**: `SmallestFit` (default), `FirstFit`.
`LockerListener`	**Observer**: parcel ready (with the code), parcel expired, return drop-off code issued.
`LockerStation`	The station: doors, bookings, code index, lockout counter, clock, random generator; every state change under one lock.
`LockerNetwork`	All stations; nearest station with space.

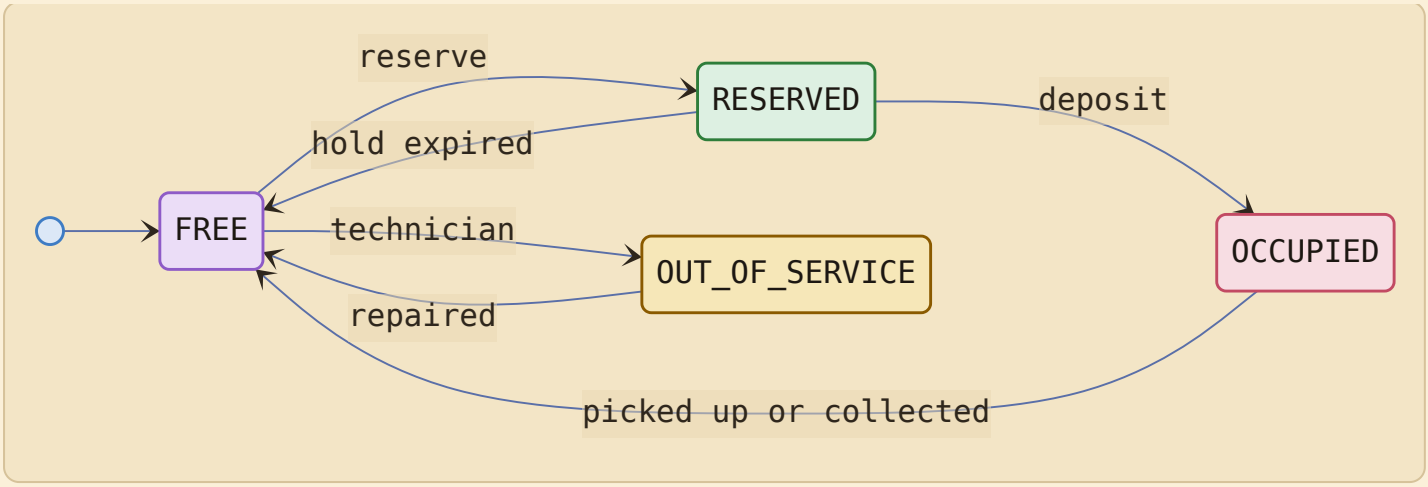
Class diagram



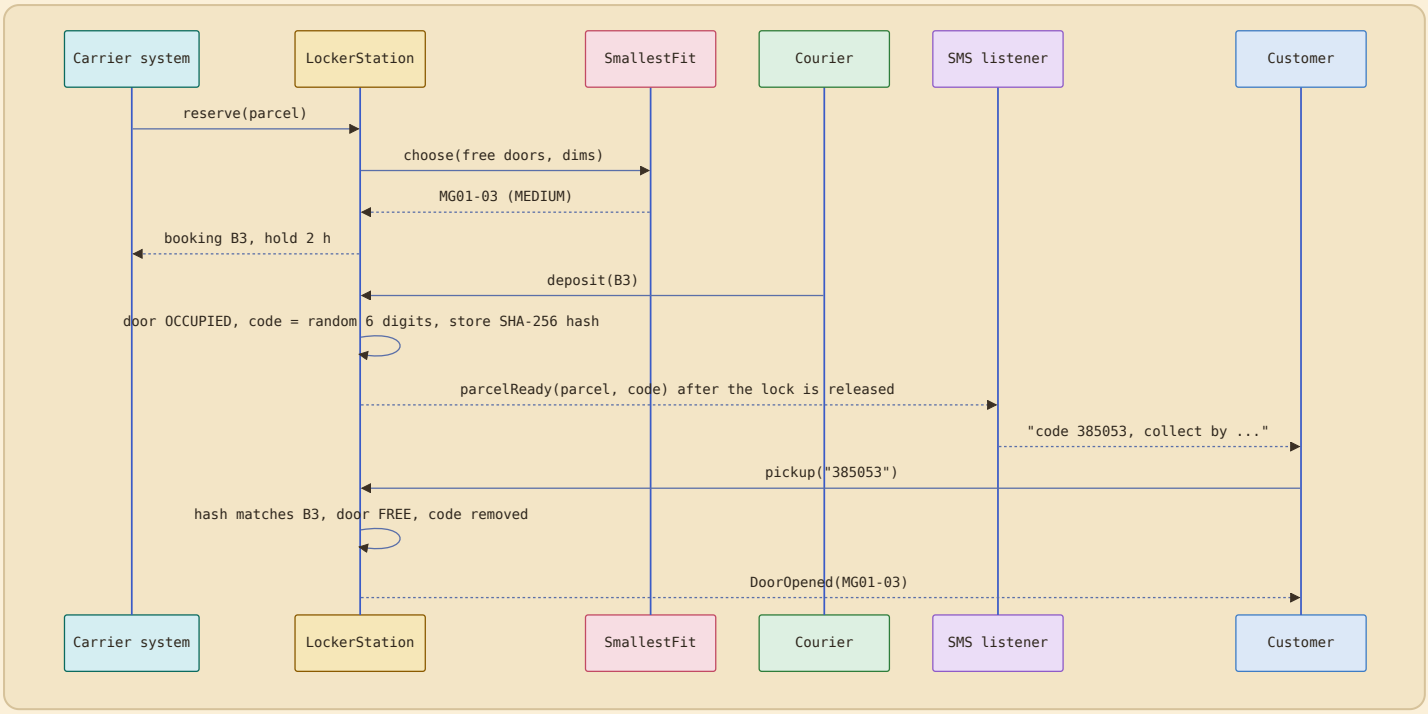
Key flows

The two state machines — first the booking, then the door. A compartment only knows whether it is free; a booking knows the parcel's story. Keeping them separate means a door can be reused while the old booking stays in the history. A customer return uses the same booking machine with one extra status, DROPPED_OFF.



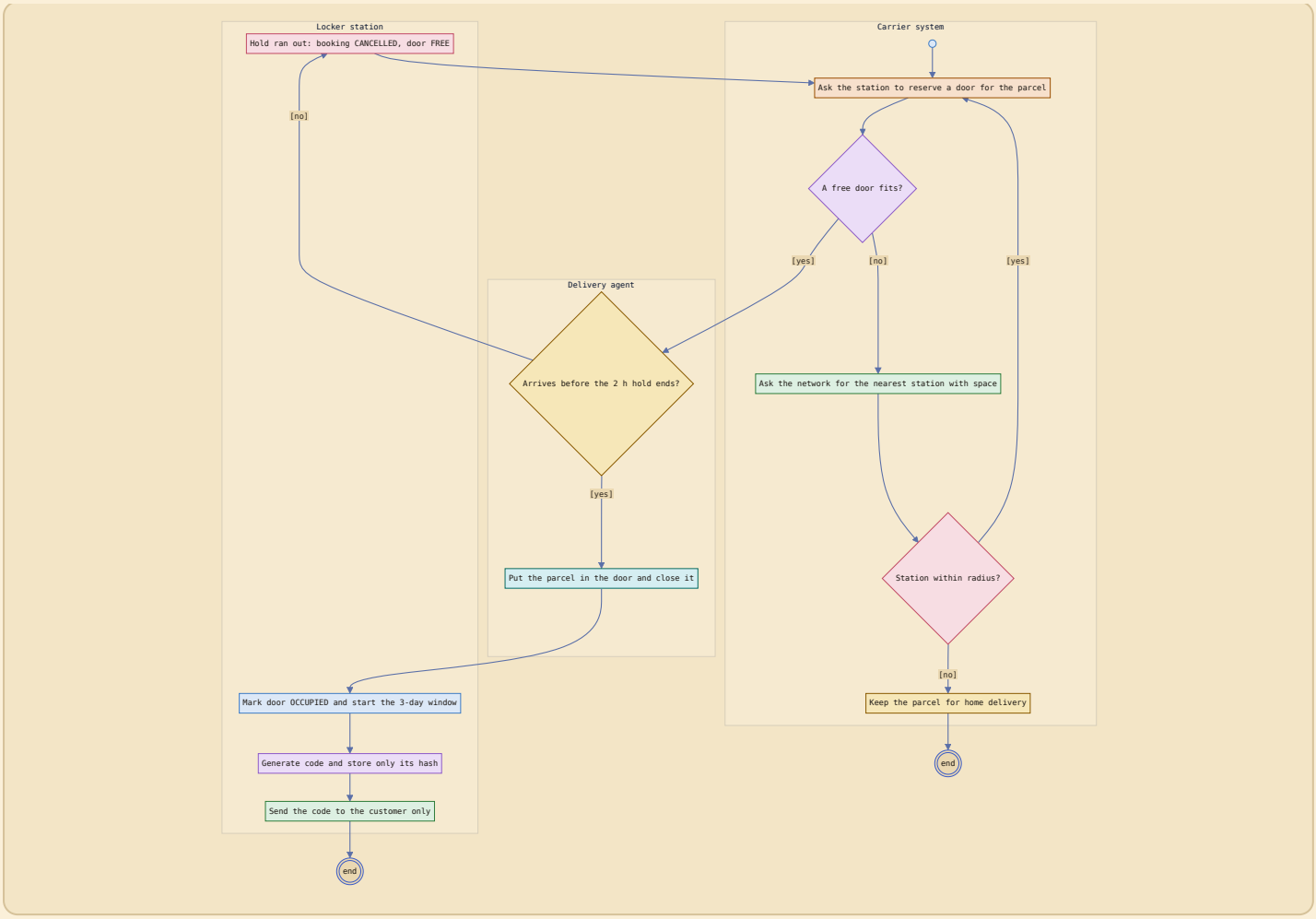


Delivery and pickup.

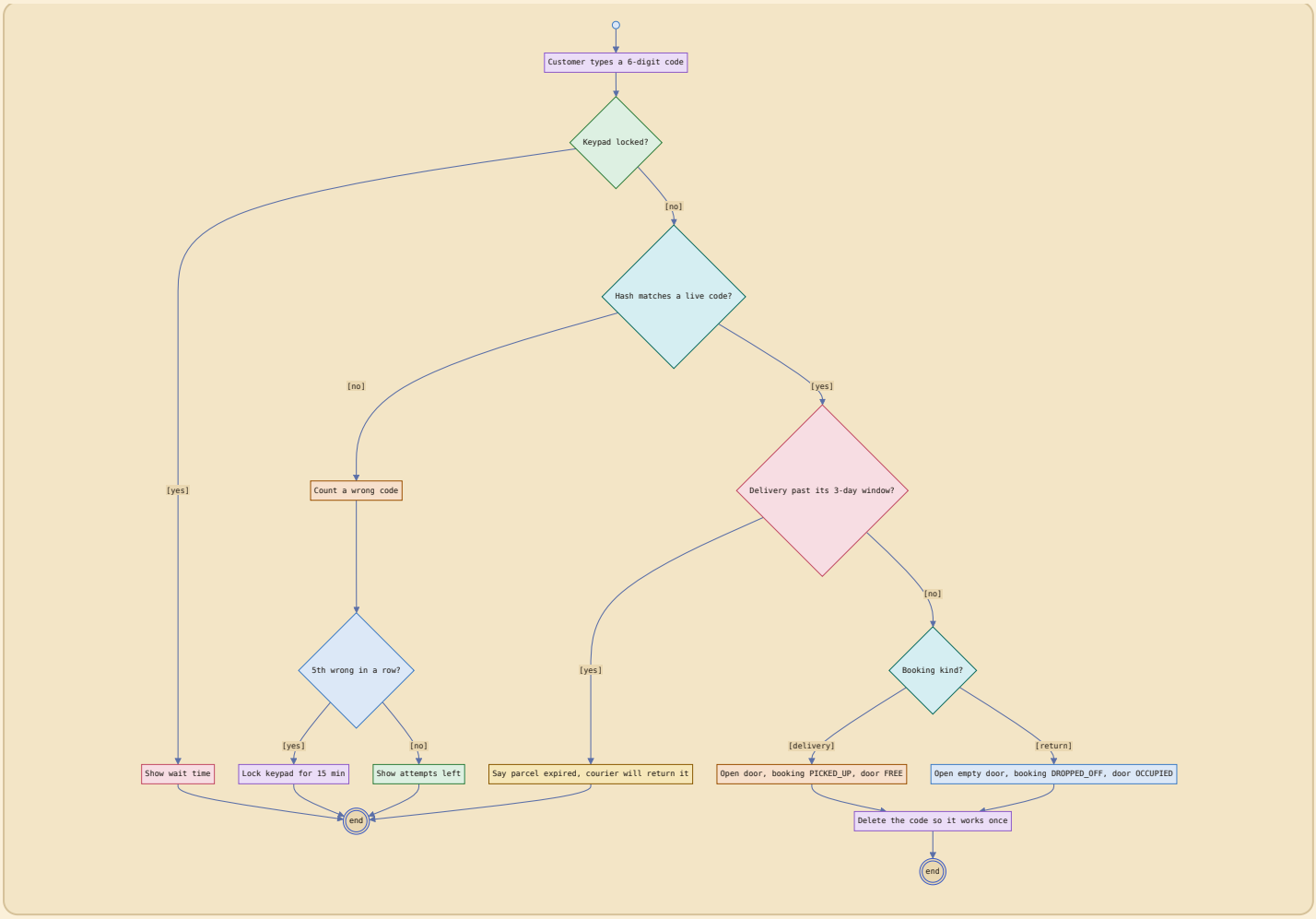


The sequence diagram shows the happy path. The two activity diagrams below (§3.13) add the branches: a full station or a late courier, and every outcome of typing a code.

Activity — deliver a parcel.



Activity — type a code at the keypad (pickup or return drop-off). One keypad serves both: the code's hash finds the booking, and the booking's kind decides whether the door opens to take a parcel out or to put a return in.



Expired deliveries and dropped-off returns leave the station the same way: on the delivery agent's next visit, `collectExpired()` and `collectReturns()` open those doors, mark the bookings RETURNED and free the doors.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Door choice is a `CompartmentSelector`	**Strategy**	"Smallest door that fits" keeps large doors for large parcels; a station may prefer "lowest door first" (fast for the courier) or "eye-level first" (accessibility). Each is a small class.	An `if (mode == ...)` inside `reserve`
Separate `Compartment` state and `Booking` status	Two small state machines	A door is reused many times; the booking keeps each parcel's history and deadlines	One status field on the door that mixes "free" with "expired parcel inside"
Transitions checked inside `moveTo` (a `switch` over the current state)	Enum state machine	An illegal move (e.g. taking an occupied door out of service) throws at once	Setting the field anywhere
Keypad outcomes as a **sealed** `PickupResult`	Sealed result types	The UI must handle every outcome; the compiler checks the `switch`	Returning `null`, `-1` or strings
Codes: random, single-use, unique per station, stored as a salted (mixed with a random value before hashing) **SHA-256** hash	Security by design	A database leak does not reveal live codes; the courier never sees them	Storing the plain code, or deriving it from the parcel id
Notify through `LockerListener` **after** releasing the lock	**Observer**	SMS, app push and audit log are independent; a slow SMS gateway (the service that actually sends text messages) must not block the keypad	Calling the SMS gateway inside `deposit` while holding the lock
Lazy expiry (deadlines checked whenever the data is next touched) at the start of every operation, plus a `sweep()` for notifications	Deadline checking on read	Correct even if the scheduler is late or down; the scheduler only sends messages	A reaper thread (a background thread that cleans up expired items) as the *only* thing that frees doors
`Clock` and `RandomGenerator` are injected	Dependency injection	Deterministic demo and tests; production passes `Clock.systemUTC()` and a `SecureRandom` (Java's cryptographically strong random generator)	`Instant.now()` and `new Random()` inside the class
One lock per station (`synchronized`)	Monitor	A station has tens of doors and each call is microseconds, so one lock is simple and never the bottleneck (the one part that limits overall speed); stations never share a lock	Per-door CAS like the parking lot — more code for no gain at this size

Java implementation

Package `lld.ch14.locker` (imports omitted).

```java
// Model.java
/** Inner size of a compartment door, in centimetres (height x width x depth). */
enum Size {
    SMALL(10, 40, 60), MEDIUM(20, 40, 60), LARGE(40, 40, 60);

    private final int[] sorted;

    Size(int h, int w, int d) { sorted = new int[]{h, w, d}; Arrays.sort(sorted); }

    /** A box fits if, after sorting both sides, each parcel side is no longer than the matching compartment side.
*/
    boolean fits(Dimensions p) {
        int[] q = p.sorted();
        return q[0] <= sorted[0] && q[1] <= sorted[1] && q[2] <= sorted[2];
    }
}

/** Parcel box size in centimetres. */
record Dimensions(int a, int b, int c) {
    Dimensions {
        if (a <= 0 || b <= 0 || c <= 0) throw new LockerException("Dimensions must be positive");
    }
    int[] sorted() { int[] s = {a, b, c}; Arrays.sort(s); return s; }
    @Override public String toString() { return a + "x" + b + "x" + c + "cm"; }
}

record Parcel(String id, String customerPhone, Dimensions dims) { }

enum CompartmentState { FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE }

/** One door in a locker station. Its state is changed only by the station, under the station's lock. */
final class Compartment {
    private final String id;
    private final Size size;
    private CompartmentState state = CompartmentState.FREE;

    Compartment(String id, Size size) { this.id = id; this.size = size; }

    String id() { return id; }
    Size size() { return size; }
    CompartmentState state() { return state; }
    boolean isFree() { return state == CompartmentState.FREE; }

    void moveTo(CompartmentState next) {
        boolean ok = switch (state) {
            case FREE -> next == CompartmentState.RESERVED || next == CompartmentState.OUT_OF_SERVICE;
            case RESERVED -> next == CompartmentState.OCCUPIED || next == CompartmentState.FREE;
            case OCCUPIED -> next == CompartmentState.FREE;
            case OUT_OF_SERVICE -> next == CompartmentState.FREE;
        };
        if (!ok) throw new LockerException("Compartment " + id + ": illegal move " + state + " -> " + next);
        state = next;
    }

    @Override public String toString() { return id + "(" + size + ")"; }
}

/** Why a door is booked: a delivery to the customer, or a parcel the customer sends back. */
enum BookingKind { DELIVERY, CUSTOMER_RETURN }

/** Lifecycle of one parcel's stay in a locker. */
enum BookingStatus {
    RESERVED,      // door held; delivery: parcel not yet inside / return: drop-off code sent
    DEPOSITED,     // delivery parcel inside, pickup code sent
    PICKED_UP,     // customer opened the door with the code (final)
    EXPIRED,       // not collected in time; waiting for the carrier to take it back
    DROPPED_OFF,   // customer put a return inside; waiting for the carrier
    RETURNED,      // carrier removed the parcel: an expired delivery or a customer return (final)
    CANCELLED      // hold ran out before the deposit or the drop-off (final)
}
```

```
/** One parcel's use of one compartment. */
final class Booking {
    final String id;
    final Parcel parcel;
    final Compartment compartment;
    final BookingKind kind;
    BookingStatus status = BookingStatus.RESERVED;
    long holdUntil;          // epoch seconds: deposit (or customer drop-off) must happen before this
    long pickupBy;           // epoch seconds: set at deposit
    String codeHash;         // only a hash of the pickup (or drop-off) code is stored

    Booking(String id, Parcel parcel, Compartment compartment, BookingKind kind, long holdUntil) {
        this.id = id; this.parcel = parcel; this.compartment = compartment; this.kind = kind; this.holdUntil =
holdUntil;
    }
}

/** What the customer's keypad entry produced. */
sealed interface PickupResult permits DoorOpened, DropOffDoorOpened, WrongCode, KeypadLocked, ParcelExpired { }
record DoorOpened(String compartmentId, String parcelId) implements PickupResult { }
record DropOffDoorOpened(String compartmentId, String parcelId) implements PickupResult { }   // empty door for a
return
record WrongCode(int attemptsLeft) implements PickupResult { }
record KeypadLocked(long secondsLeft) implements PickupResult { }
record ParcelExpired(String parcelId) implements PickupResult { }

/** Observer: SMS/app notifications, carrier dashboards, audit logs. */
interface LockerListener {
    void parcelReady(String stationId, Parcel parcel, String pickupCode, long pickupBy);
    default void parcelExpired(String stationId, Parcel parcel) { }
    default void returnCodeIssued(String stationId, Parcel parcel, String dropOffCode, long dropOffBy) { }
}

final class LockerException extends RuntimeException {
    LockerException(String message) { super(message); }
}
```

```
// Selection.java
/** Strategy: which free compartment should this parcel get? */
interface CompartmentSelector {
    Optional<Compartment> choose(List<Compartment> free, Dimensions parcel);
}

/** Smallest door that fits: keeps large doors free for large parcels (the usual default). */
final class SmallestFit implements CompartmentSelector {
    @Override public Optional<Compartment> choose(List<Compartment> free, Dimensions parcel) {
        return free.stream()
                .filter(c -> c.size().fits(parcel))
                .min(Comparator.comparing(Compartment::size).thenComparing(Compartment::id));
    }
}

/** Lowest door number that fits, whatever its size: simple and fast, but wastes large doors. */
final class FirstFit implements CompartmentSelector {
    @Override public Optional<Compartment> choose(List<Compartment> free, Dimensions parcel) {
        return free.stream()
                .filter(c -> c.size().fits(parcel))
                .min(Comparator.comparing(Compartment::id));
    }
}
```

```java
// LockerStation.java
/**
 * One locker station (a bank of doors at one address). Every state change happens under the
 * station's monitor lock: a station has tens of doors and each operation takes microseconds,
 * so one lock is simple and never the bottleneck.
 */
final class LockerStation {
    static final int MAX_WRONG_CODES = 5;

    private final String id;
    private final double x, y;                                      // location in km on a city grid
    private final Map<String, Compartment> compartments = new LinkedHashMap<>();
    private final Map<String, Booking> bookings = new HashMap<>();
    private final Map<String, String> bookingByCodeHash = new HashMap<>();
    private final List<LockerListener> listeners = new CopyOnWriteArrayList<>();
    private final CompartmentSelector selector;
    private final Clock clock;
    private final RandomGenerator random;
    private final Duration reservationHold, pickupWindow, lockout, dropOffWindow;
    private final List<Parcel> newlyExpired = new ArrayList<>();
    private int nextBooking = 1, wrongCodes = 0;
    private long lockedUntil = 0;

    /** layout = the size of each door, in door-number order (door 01 is the first element). */
    LockerStation(String id, double x, double y, List<Size> layout, CompartmentSelector selector,
                  Clock clock, RandomGenerator random) {
        this.id = id; this.x = x; this.y = y;
        this.selector = selector; this.clock = clock; this.random = random;
        this.reservationHold = Duration.ofHours(2);
        this.pickupWindow = Duration.ofDays(3);
        this.lockout = Duration.ofMinutes(15);
        this.dropOffWindow = Duration.ofDays(2);
        for (int n = 1; n <= layout.size(); n++) {
            String cid = "%s-%02d".formatted(id, n);
            compartments.put(cid, new Compartment(cid, layout.get(n - 1)));
        }
    }

    /** Convenience layout: small doors first, then medium, then large. */
    static List<Size> layout(int small, int medium, int large) {
        List<Size> l = new ArrayList<>();
        for (int i = 0; i < small; i++) l.add(Size.SMALL);
        for (int i = 0; i < medium; i++) l.add(Size.MEDIUM);
        for (int i = 0; i < large; i++) l.add(Size.LARGE);
        return l;
    }

    String id() { return id; }
    double distanceTo(double px, double py) { return Math.hypot(x - px, y - py); }
    void addListener(LockerListener l) { listeners.add(l); }

    /** Carrier books a door before driving to the station. All-or-nothing: a door, or an exception. */
    synchronized Booking reserve(Parcel parcel) {
        expireOverdue();
        return allocate(parcel, BookingKind.DELIVERY, reservationHold);
    }

    /** Customer starts a return online: an empty door is held for 2 days and a drop-off code goes to the
customer. */
    Booking requestReturn(Parcel parcel) {
        Booking b;
        String code;
        synchronized (this) {
            expireOverdue();
            b = allocate(parcel, BookingKind.CUSTOMER_RETURN, dropOffWindow);
            code = issueCode(b);
        }
        for (LockerListener l : listeners) l.returnCodeIssued(id, b.parcel, code, b.holdUntil);   // outside the
lock
        return b;
```

```
    }

    /** Carrier puts the parcel in and closes the door. The code goes to the customer, never to the carrier. */
    void deposit(String bookingId) {
        Booking b;
        String code;
        synchronized (this) {
            expireOverdue();
            b = booking(bookingId);
            if (b.status != BookingStatus.RESERVED) throw new LockerException("Booking " + bookingId + " is " +
b.status);
            b.compartment.moveTo(CompartmentState.OCCUPIED);
            b.status = BookingStatus.DEPOSITED;
            b.pickupBy = now() + pickupWindow.toSeconds();
            code = issueCode(b);
        }
        for (LockerListener l : listeners) l.parcelReady(id, b.parcel, code, b.pickupBy);   // outside the lock
    }

    /** Customer types a code on the station keypad: a pickup code, or a drop-off code for a return. */
    synchronized PickupResult pickup(String code) {
        expireOverdue();
        long t = now();
        if (t < lockedUntil) return new KeypadLocked(lockedUntil - t);
        String bookingId = bookingByCodeHash.get(hash(code));
        if (bookingId == null) {
            if (++wrongCodes >= MAX_WRONG_CODES) {                  // slow down code guessing
                lockedUntil = t + lockout.toSeconds();
                wrongCodes = 0;
                return new KeypadLocked(lockout.toSeconds());
            }
            return new WrongCode(MAX_WRONG_CODES - wrongCodes);
        }
        wrongCodes = 0;
        Booking b = bookings.get(bookingId);
        if (b.status == BookingStatus.EXPIRED) return new ParcelExpired(b.parcel.id());
        if (b.kind == BookingKind.CUSTOMER_RETURN) {                // drop-off: open the empty door once
            b.status = BookingStatus.DROPPED_OFF;
            b.compartment.moveTo(CompartmentState.OCCUPIED);
            bookingByCodeHash.remove(b.codeHash);
            return new DropOffDoorOpened(b.compartment.id(), b.parcel.id());
        }
        b.status = BookingStatus.PICKED_UP;
        b.compartment.moveTo(CompartmentState.FREE);
        bookingByCodeHash.remove(b.codeHash);                       // a code works once
        return new DoorOpened(b.compartment.id(), b.parcel.id());
    }

    /** Carrier's daily visit: open every door holding an expired parcel and take them for return. */
    synchronized List<Parcel> collectExpired() {
        expireOverdue();
        List<Parcel> taken = new ArrayList<>();
        for (Booking b : bookings.values())
            if (b.status == BookingStatus.EXPIRED) {
                b.status = BookingStatus.RETURNED;
                b.compartment.moveTo(CompartmentState.FREE);
                bookingByCodeHash.remove(b.codeHash);
                taken.add(b.parcel);
            }
        return taken;
    }

    /** Same visit: take the parcels customers dropped off for return. */
    synchronized List<Parcel> collectReturns() {
        expireOverdue();
        List<Parcel> taken = new ArrayList<>();
        for (Booking b : bookings.values())
            if (b.status == BookingStatus.DROPPED_OFF) {
                b.status = BookingStatus.RETURNED;
                b.compartment.moveTo(CompartmentState.FREE);
                taken.add(b.parcel);
```

```
            }
        return taken;
    }

    synchronized void markOutOfService(String compartmentId) {
        Compartment c = compartments.get(compartmentId);
        if (c == null) throw new LockerException("Unknown compartment " + compartmentId);
        c.moveTo(CompartmentState.OUT_OF_SERVICE);                // only a FREE door can be taken out
    }

    synchronized Map<Size, Integer> freeBySize() {
        expireOverdue();
        Map<Size, Integer> m = new EnumMap<>(Size.class);
        for (Size s : Size.values()) m.put(s, 0);
        compartments.values().stream().filter(Compartment::isFree).forEach(c -> m.merge(c.size(), 1,
Integer::sum));
        return m;
    }

    synchronized boolean canFit(Dimensions d) {
        expireOverdue();
        return compartments.values().stream().anyMatch(c -> c.isFree() && c.size().fits(d));
    }

    /** Called by a scheduler (e.g. every minute): applies deadlines and tells customers, outside the lock. */
    void sweep() {
        List<Parcel> expired;
        synchronized (this) { expireOverdue(); expired = List.copyOf(newlyExpired); newlyExpired.clear(); }
        for (Parcel p : expired) for (LockerListener l : listeners) l.parcelExpired(id, p);
    }

    synchronized BookingStatus status(String bookingId) { expireOverdue(); return booking(bookingId).status; }

    /** Lazy expiry: every operation first applies the deadlines that have passed, so correctness never waits for
a timer. */
    private void expireOverdue() {
        long t = now();
        for (Booking b : bookings.values()) {
            if (b.status == BookingStatus.RESERVED && t >= b.holdUntil) {
                b.status = BookingStatus.CANCELLED;
                b.compartment.moveTo(CompartmentState.FREE);
                if (b.codeHash != null) bookingByCodeHash.remove(b.codeHash);   // an unused drop-off code dies
too
            } else if (b.status == BookingStatus.DEPOSITED && t >= b.pickupBy) {
                b.status = BookingStatus.EXPIRED;                    // door stays OCCUPIED until the carrier
collects
                newlyExpired.add(b.parcel);                          // notified by sweep(), outside the lock
            }
        }
    }

    private static boolean isActive(Booking b) {
        return b.status == BookingStatus.RESERVED || b.status == BookingStatus.DEPOSITED
                || b.status == BookingStatus.EXPIRED || b.status == BookingStatus.DROPPED_OFF;
    }

    /** Caller holds the lock. All-or-nothing: a door marked RESERVED and a booking, or an exception. */
    private Booking allocate(Parcel parcel, BookingKind kind, Duration hold) {
        if (bookings.values().stream().anyMatch(b -> b.parcel.id().equals(parcel.id()) && isActive(b)))
            throw new LockerException("Parcel " + parcel.id() + " already has a locker");
        List<Compartment> free = compartments.values().stream().filter(Compartment::isFree).toList();
        Compartment c = selector.choose(free, parcel.dims())
                .orElseThrow(() -> new LockerException("No free compartment fits " + parcel.dims() + " at " +
id));
        c.moveTo(CompartmentState.RESERVED);
        Booking b = new Booking(id + "-B" + nextBooking++, parcel, c, kind, now() + hold.toSeconds());
        bookings.put(b.id, b);
        return b;
    }

    /** Caller holds the lock. A random 6-digit code, unique among live codes here; only its hash is kept. */
```

```
    private String issueCode(Booking b) {
        String code;
        do { code = "%06d".formatted(random.nextInt(1_000_000)); }
        while (bookingByCodeHash.containsKey(hash(code)));
        b.codeHash = hash(code);
        bookingByCodeHash.put(b.codeHash, b.id);
        return code;
    }

    private Booking booking(String bookingId) {
        Booking b = bookings.get(bookingId);
        if (b == null) throw new LockerException("Unknown booking " + bookingId);
        return b;
    }

    private long now() { return clock.instant().getEpochSecond(); }

    /** Store only a salted hash of each code, so a leaked database does not reveal live codes. */
    private String hash(String code) {
        try {
            byte[] h = MessageDigest.getInstance("SHA-256").digest((id + ":" +
code).getBytes(StandardCharsets.UTF_8));
            return HexFormat.of().formatHex(h);
        } catch (NoSuchAlgorithmException e) { throw new IllegalStateException(e); }
    }
}
```

```
// LockerNetwork.java
/** All stations in a city. Used at checkout ("pick a locker near you") and when a station is full. */
final class LockerNetwork {
    private final List<LockerStation> stations = new CopyOnWriteArrayList<>();

    void add(LockerStation s) { stations.add(s); }

    /** Nearest station within maxKm that has a free door this parcel fits in right now. */
    Optional<LockerStation> nearestWithSpace(double x, double y, Dimensions parcel, double maxKm) {
        return stations.stream()
                .filter(s -> s.distanceTo(x, y) <= maxKm)
                .sorted(Comparator.comparingDouble(s -> s.distanceTo(x, y)))
                .filter(s -> s.canFit(parcel))
                .findFirst();
    }
}
```

```
// Main.java (excerpt): a movable TestClock (production passes Clock.systemUTC()) and a fake SMS gateway
static final class SmsInbox implements LockerListener {   // remembers the last code sent to each phone
    final Map<String, String> lastCode = new ConcurrentHashMap<>();
    @Override public void parcelReady(String station, Parcel p, String code, long pickupBy) {
        lastCode.put(p.customerPhone(), code);
        System.out.printf("   SMS to %s: parcel %s is in locker %s, code %s, collect by %s%n",
                p.customerPhone(), p.id(), station, code, Instant.ofEpochSecond(pickupBy));
    }
    // parcelExpired and returnCodeIssued print the other two SMS lines
}
// Station MG01 = layout(2, 2, 1) with SmallestFit and new SplittableRandom(7); ten numbered scenarios:
// reserve 4 parcels, deposit, pickup, guessing lockout, hold expiry, 3-day expiry, network search,
// FirstFit vs SmallestFit, 50 threads reserving on a 20-door HUB, and a customer return at RT04.
```

How the code works, part by part

- `Size.fits` sorts the three sides of box and door and compares smallest-to-smallest — handles a parcel on its side without trying all six rotations.
- `reserve` applies overdue deadlines, rejects a parcel that already has a door, asks the selector, marks the door RESERVED and creates a `Booking` with a 2-hour hold. It is `synchronized`, so "find a free door" + "mark it reserved" is one atomic step.
- `deposit` changes state inside `synchronized (this)`, generates a 6-digit code (retrying on the rare clash with another live code at this station), stores only its hash, and calls listeners **after** the

block.

- `pickup` checks the lockout, looks the code up by hash, counts wrong attempts (5 in a row lock the keypad for 15 minutes), refuses an expired parcel, and on success frees the door and deletes the code.
- `expireOverdue` runs at the top of every public method: RESERVED past its hold → CANCELLED (door freed); DEPOSITED past its window → EXPIRED (door stays OCCUPIED until the courier collects). `sweep()`, called by a scheduler every minute, runs the same check and sends "not collected" messages outside the lock.
- `requestReturn` reuses `allocate` (2-day hold instead of 2 hours) and issues a drop-off code at once; in `pickup`, a CUSTOMER_RETURN code opens the empty door → DROPPED_OFF; `collectReturns` frees those doors; a lapsed return hold is cancelled and its code deleted.
- `LockerNetwork.nearestWithSpace` sorts stations by distance, keeps those within the radius, returns the first that can fit the parcel now.

Output:

```
== 1. Carrier reserves doors (smallest door that fits) ==
   P1 3x10x18cm    -> MG01-01(SMALL)
   P2 4x16x24cm    -> MG01-02(SMALL)
   P3 9x12x12cm    -> MG01-03(MEDIUM)
   P4 14x22x34cm   -> MG01-04(MEDIUM)
   free now: {SMALL=0, MEDIUM=0, LARGE=1}  (the mug got a MEDIUM: both SMALL doors were taken)
   error: Parcel P1 already has a locker
   error: No free compartment fits 50x50x70cm at MG01

== 2. Deposit: the code goes to the customer by SMS, never to the carrier ==
   SMS to +91-98450-00001: parcel P1 is in locker MG01, code 694696, collect by 2026-10-04T09:00:00Z
   SMS to +91-98450-00002: parcel P2 is in locker MG01, code 607553, collect by 2026-10-04T09:00:00Z
   SMS to +91-98450-00003: parcel P3 is in locker MG01, code 385053, collect by 2026-10-04T09:00:00Z
   error: Booking MG01-B1 is DEPOSITED

== 3. Pickup ==
   typo 123456 -> WrongCode[attemptsLeft=4]
   right code  -> DoorOpened[compartmentId=MG01-01, parcelId=P1]
   same code again -> WrongCode[attemptsLeft=4]  (a code works once)

== 4. Someone guesses codes: keypad locks after 5 wrong tries in a row ==
   guess 000001 -> WrongCode[attemptsLeft=3]
   guess 000002 -> WrongCode[attemptsLeft=2]
   guess 000003 -> WrongCode[attemptsLeft=1]
   guess 000004 -> KeypadLocked[secondsLeft=900]
   real owner during lockout -> KeypadLocked[secondsLeft=900]
   15 min later -> DoorOpened[compartmentId=MG01-02, parcelId=P2]

== 5. Reservation hold runs out (carrier never came with the shoes) ==
   booking MG01-B4 is CANCELLED, free now: {SMALL=2, MEDIUM=1, LARGE=1}

== 6. Mug not collected within 3 days ==
   SMS to +91-98450-00003: parcel P3 was not collected and will be returned
   customer types the code -> ParcelExpired[parcelId=P3]
   carrier collects for return: [P3], free now: {SMALL=2, MEDIUM=2, LARGE=1}

== 7. Station full -> suggest the nearest station with space ==
   customer at (1.0, 0.4), box 8x20x30cm -> MG01
   same box, max 0.2 km -> none within 0.2 km

== 8. FirstFit vs SmallestFit: doors 01 = LARGE, 02 = SMALL ==
   station F: small parcel -> F-01(LARGE), then large parcel -> REJECTED
   station S: small parcel -> S-02(SMALL), then large parcel -> S-01(LARGE)

== 9. 50 carriers reserve at once on a 20-door station ==
   reserved=20 rejected=30 distinct doors=20

== 10. Customer return: drop-off code opens an empty door, courier collects ==
   SMS to +91-98450-00010: drop return R1 at locker RT04 with code 490825 by 2026-10-06T11:15:00Z
   return RT04-B1 holds RT04-02(MEDIUM), free now: {SMALL=1, MEDIUM=0, LARGE=0}
   customer types the drop-off code -> DropOffDoorOpened[compartmentId=RT04-02, parcelId=R1]
   booking RT04-B1 is DROPPED_OFF
   courier collects returns: [R1], free now: {SMALL=1, MEDIUM=1, LARGE=0}
   SMS to +91-98450-00011: drop return R2 at locker RT04 with code 988592 by 2026-10-06T11:15:00Z
   2 days later, never dropped off: booking RT04-B2 is CANCELLED, old code -> WrongCode[attemptsLeft=4]
```

Reading the output: the mug (9 × 12 × 12 cm) fits a small door but both were taken, so it was upgraded to a medium one; a second reservation for the same parcel and an oversized box are refused. Codes went only to the customers' phones. The keypad counted the earlier typo, so the fourth guess in section 4 hit the limit of five and locked it — even the real owner had to wait 15 minutes. The shoes' reservation lapsed after 2 hours and the door came back. After 3 days the mug expired, the customer's code now says so, and the courier's visit freed the door. `FirstFit` put a tiny parcel in the only large door and then had to refuse a large parcel; `SmallestFit` served both. Fifty concurrent reservations on a 20-door station produced exactly 20 bookings on 20 different doors. In section 10 the kettle return got the medium door, the drop-off code opened it once, and the courier's visit freed it; the lamp was never dropped off, so after 2 days its booking was cancelled and its code stopped working.

Concurrency & edge cases

Situation	Handling
Two carriers reserve the last medium door at once	`reserve` is `synchronized`: choose + mark RESERVED is one step; the loser gets "no free compartment" (or a larger door)
Courier arrives after the hold ran out	`deposit` sees CANCELLED and refuses; the carrier system reserves again (possibly at another station)
Same parcel reserved twice (retry after a timeout)	Rejected while it has an active booking — make the carrier's reserve call idempotent (repeating it has the same effect as calling it once) by parcel id
Two live parcels get the same random code	Generation retries until the code is unique among live codes at this station
Brute-force guessing	10^6 possible codes, 5 wrong tries then a 15-minute lockout per station; codes also expire
Scheduler down for an hour	Lazy expiry still frees doors on the next operation; only the "expired" SMS is late
Slow SMS gateway	Listeners run outside the lock; in production they hand off to a queue (a buffer that a separate worker drains)
`canFit` says yes but `reserve` then fails (someone else took the door)	Expected: `nearestWithSpace` is a hint; the caller reserves and, on failure, tries the next station
Door jammed / broken	Technician marks a **free** door OUT_OF_SERVICE; an occupied door must be emptied first (`moveTo` enforces it)
Power cut or network loss at the station	Real lockers keep a local copy of live code hashes and sync events later; the design already keeps the station self-contained

Extensions & follow-up questions

1. **"Let the customer extend the pickup window?"** — A method that moves `pickupBy` once, only while DEPOSITED; a policy object can decide how many extensions are allowed.
2. **"Cash on delivery?"** — Payment at the station screen before the door opens: a `PaymentStep` inserted into `pickup` (Chain of Responsibility or a small state machine per session).
3. **"Fairness when the station is nearly full?"** — Keep a few large doors for large parcels only (a `ReservedCapacitySelector` that refuses to upgrade small parcels when fewer than k large doors are free).
4. **"Scale to thousands of stations?"** — Stations are independent, so shard by station id (sharding = splitting the data across several servers, each owning a slice; see System Design Fundamentals); the network search uses a geo index (an index for fast "what is near this point?" queries: a geohash turns a map cell into a short string, an R-tree is a tree of nested rectangles) instead of sorting every station.
5. **"How would you store this in a database?"** — `compartment(id, station_id, size, state)`, `booking(id, parcel_id, compartment_id, status, hold_until, pickup_by, code_hash)`; reserve with `UPDATE compartment SET state='RESERVED' WHERE id=? AND state='FREE'` and check that one row changed.

Real-world example

Amazon Hub Locker sends the customer a pickup code by email or app and gives a limited number of days to collect before the parcel goes back — the `DEPOSITED → EXPIRED → RETURNED` path. InPost's Paczkomat lockers in Poland let couriers load many parcels in one visit and customers open a door with a code or a phone app; the network is dense enough that "nearest locker with space" matters. In both, the door is chosen by the system, not the courier, which is our `CompartmentSelector`.

In the interview

Plan for 40 minutes: 5 min questions (sizes, holds, codes, expiry) → 5 min entities → 8 min the two state machines → 15 min code `reserve`, `deposit`, `pickup` with the lock and the listener call outside it → 7 min security, concurrency and follow-ups.

> **30-second answer:** "A station has doors of a few sizes. A carrier reserves one ahead of time; a Strategy picks the smallest free door that fits, and the choose-and-mark step runs under the station's lock, so two carriers never get the same door. On deposit I generate a random single-use code, store only its hash, and notify the customer through an observer after releasing the lock. Pickup matches the hash, frees the door and deletes the code; five wrong codes lock the keypad. Holds and pickup windows are checked lazily on every call, so doors come back even if the scheduler is late, and a network object suggests the nearest station with space."

Follow-ups with answers

- *"Why not let the courier choose any empty door?"* — Couriers would use big doors for small parcels; the system also needs to know the door before the courier arrives.
- *"Why store only a hash of the code?"* — Anyone who reads the database or logs could otherwise open doors.
- *"Why a separate Booking instead of putting the parcel on the Compartment?"* — The door is reused; the booking keeps deadlines and history per parcel.

Self-check

1. **Q:** How does `Size.fits` handle a box lying on its side? **A:** It sorts the three sides of both box and door and compares them in order.
2. **Q:** What happens to an expired parcel's door? **A:** It stays OCCUPIED until the courier runs `collectExpired`, which frees it.
3. **Q:** Why is the listener called after the `synchronized` block in `deposit`? **A:** A slow or failing SMS call must not hold the station lock.
4. **Q:** What frees a door if the scheduler never runs? **A:** Lazy expiry at the start of every public method.
5. **Q:** Which pattern lets you switch from smallest-fit to first-fit? **A:** Strategy (`CompartmentSelector`).
6. **Q:** Why is one lock per station enough here but not in a parking lot of 5,000 spots with 20 gates? **A:** A station has tens of doors and few concurrent callers; the lock is held for microseconds.

TL;DR

- Doors of fixed sizes; a **Strategy** picks the smallest free door that fits; choose-and-mark is **atomic** under the station lock.
- Two small **state machines**: door (FREE/RESERVED/OCCUPIED/OUT_OF_SERVICE) and booking (RESERVED → DEPOSITED → PICKED_UP / EXPIRED → RETURNED / CANCELLED; a customer return goes RESERVED → DROPPED_OFF → RETURNED).
- Codes are **random, single-use, hashed**, sent only to the customer via an **Observer** outside the lock; lockout after 5 wrong tries.
- Deadlines are enforced **lazily** on every call; a sweep only sends messages. Inject clock and randomness.

Chapter summary

Problem	Core difficulty	Key data structure	Key patterns	Concurrency tool
Parking lot	Allocate spots without double booking	Concurrent sorted set of free spots per type	Strategy, Factory, Observer	`pollFirst` + CAS, per-ticket lock, `putIfAbsent`
Vending machine	Legal order of actions; exact change	`EnumMap` of coin counts; DP table	State	`synchronized` machine
Coffee machine	All-or-nothing multi-ingredient deduction	`EnumMap` levels, recipes as data	Observer, sealed results	One `ReentrantLock`; notify outside the lock
ATM	Money correctness and authentication	Total vs available balance; note chain	State, Chain of Responsibility, Command	Per-account lock; ordered locking for transfers
Elevator	Scheduling and dispatching	Two `TreeSet`s (up/down stops)	Strategy (+ modes)	Per-car lock, synchronized dispatcher
Traffic signal	Timed transitions with a safety invariant	Phases as data	State, Strategy, Observer	Synchronized tick; scheduled executor
Shipping locker	Best-fit door, holds and pickup windows, secret codes	Doors per station; bookings; code-hash index	Strategy, Observer, enum state machines, sealed results	One lock per station; lazy expiry; notify outside the lock

Chapter self-check

1. **Q:** Which two problems use the State pattern with *stateless* states, and which uses states *with data*? **A:** Vending machine and ATM share stateless singletons; the traffic signal's states hold countdowns, so a new instance is created per transition.
2. **Q:** Name the check-then-act race in each of parking, coffee and ATM, and its fix. **A:** Two gates take one spot (atomic `pollFirst`/CAS); two orders take the last milk (one lock around check+deduct); two ATMs overdraw one account (check+debit under the account lock).
3. **Q:** Where did we reject a Singleton, and why? **A:** Parking lot, coffee machine, traffic controller: a Singleton is a hidden global, blocks testing several instances, and real deployments have many (lots, machines, intersections).
4. **Q:** Where does greedy fail and what replaces it? **A:** Change-making with limited coins (vending) and note selection with limited cassettes (ATM); bounded coin-change dynamic programming.
5. **Q:** Which designs are tick-driven, and why is that useful? **A:** Elevator and traffic signal: deterministic, clock-free tests; a real timer just calls `tick()`/`step()`.
6. **Q:** Which case enforces deadlines *lazily*, and why? **A:** The shipping locker: every call first expires overdue holds and parcels, so doors come back even if the scheduler is late; the scheduled sweep only sends messages.

Chapter TL;DR

- Find the **one hard method** in each problem (spot allocation, change-making, ingredient deduction, withdrawal, next stop, next phase) and spend your time there.
- **State** for machines whose buttons mean different things over time; **Strategy** for rules that change (pricing, dispatch, timing); **Observer** for boards, alerts and monitors; **Chain of Responsibility** and **Command** for the ATM.
- Make every **check-then-act** atomic, lock at the **smallest** useful scope, lock in a **fixed order**, and call listeners **outside** locks.
- Prefer injected instances over Singletons, integer cents over doubles, and tick-driven logic over sleeps.
- For anything with a **deadline** (holds, pickup windows), check it on every call and keep secrets (pickup codes) **hashed and single-use**.

Chapter 15 — Case Studies: Booking & Inventory (Part 1)

> **Where this fits.** Chapter 14 designed small, self-contained objects. This chapter is the first of two on **booking systems**: software that sells or lends a limited thing (a book copy, a cinema seat, a concert place, a hotel night) to many people who all want it at once. Chapter 16 continues with more booking problems. Everything here uses the case-study template from Chapter 1 and the concurrency tools from Chapters 12–13.

Why booking problems are interview favourites

A booking problem looks like simple CRUD ("create, read, update, delete") until two people want the same thing. Then five hard questions appear, and every problem in this chapter is a different mix of them:

Theme	The question the interviewer is really asking	Where you will see it
Inventory	What exactly is the thing being sold or lent? A title or a copy? A seat or a seat *for one show*? A room or a room *on one night*?	`Book` vs `BookItem` (15.1), `HallSeat` vs `ShowSeat` (15.2), GA (general admission) count vs numbered seat (15.3), `Room` + booked nights (15.4)
Holds / locks	How do you keep something for a customer while they pay, without keeping it forever?	Seat lock with a TTL (time-to-live) (15.2), CAS holds + reaper (15.3)
Double-booking prevention	What stops two threads (or two servers) from both succeeding?	`synchronized` per item (15.1, 15.4), one monitor per show (15.2), lock-free compare-and-set (15.3)
Payments	What happens if payment fails? If it succeeds but the hold was lost meanwhile?	Release on decline, refund as **compensation** (15.2, 15.3)
Cancellations	Who gets the thing next, and how much money comes back?	Waitlists (15.1, 15.3), refund policies (15.2, 15.4)

Three words you will meet in every section:

- **Race condition** — two threads read the same data, both decide "it is free", and both act. One result silently overwrites the other.
- **Check-then-act** — the pattern "if free, then take it". It is only safe if the check and the act happen as one indivisible (atomic) step.
- **Hold** (also called a *lock*, *soft lock* or *reservation*) — a temporary claim with an expiry time, so an abandoned checkout does not block the item forever.

> **Key terms — the other words in the table above.** A **thread** is one independent line of execution inside a program; a server runs many at once, and they can all touch the same objects. A **lock** lets only one thread at a time run a piece of code; Java's `synchronized` keyword uses the built-in lock that every object carries, called its **monitor**. **Compare-and-set (CAS)** is a single atomic step that says "change this value to *new*, but only if it still equals *expected*"; code built on CAS instead of locks is called **lock-free**, because no thread ever waits for another. **Contention** means many threads competing for the same item or lock at the same moment. A **reaper** is a background job that sweeps away expired holds. **Compensation** is an action that undoes an earlier step after the fact — for money, a refund.

What you will learn

#	Problem	The one idea that carries the design
15.1	Library Management System	A *title* (`Book`) is not a *copy* (`BookItem`). Limits live with the member; the waitlist lives with the title.
15.2	Movie Ticket Booking (BookMyShow)	You book a `ShowSeat`, not a physical seat. Hold with TTL → pay → commit; refund if the hold was lost.
15.3	Concert Ticket Booking	Extreme contention: lock-free CAS, a counter for general admission, a fair virtual queue, per-user limits, a waitlist.
15.4	Hotel Management System	A status field is not availability. Each room remembers which nights are already sold.

How to run every example in this chapter

```
cd chapters/15-case-booking-1/code
javac --release 21 -d /tmp/lld_15 $(find . -name '*.java')
java -cp /tmp/lld_15 lld.ch15.library.Main        # or moviebooking, concert, hotel
```

All four `Main` classes compile with Java 21 (standard library only) and run; the output under each implementation is real. Listings below keep the essential classes and methods — imports, getters and the `Main` demos are in `code/lld/ch15/<case>/`. In the concurrency demos, *which* thread wins a race changes from run to run; the *number* of winners never does.

> **Code-style note.** A file sometimes holds several small package-private types (declared without `public`, visible only in their package) — e.g. the library's `Model.java` holds its enums and records. Legal Java; production code would give each public type its own file. Compiling with `-Xlint:all` (every optional warning) makes javac print `[auxiliaryclass]` style warnings for exactly this — not errors.

> **Money.** Every amount (prices, fines, deposits, refunds) is a `java.math.BigDecimal` (Java's exact decimal-number type) built from a string such as `new BigDecimal("0.50")`, never a `double`: binary floating point cannot store 0.10 exactly, so sums drift. Comparisons use `compareTo`/`signum()`, not `equals` (which also compares the number of decimal places), and every rounding step names its `RoundingMode` (the rounding rule, for example half-up).

> **A time trick used in every demo.** Holds expire and fines grow with time. Instead of `Thread.sleep` (pausing for real time), every service takes a `java.time.Clock`. The demos pass a tiny `SimClock` (a `Clock` subclass whose `advance(Duration)` moves time forward). This is standard practice for testable time-dependent code: production passes `Clock.systemUTC()`, tests pass a controllable clock.

15.1 Design a Library Management System



Figure — Cover image for the library case study (course diagram)

Problem & clarifying questions

A **Library Management System** (LMS) is the software a library uses for its daily work: it tracks every book and every physical copy, who borrowed which copy and when it is due, each member's account and card, reservations, and fines. New stock is entered into it, and every loan is recorded with a due date.

Always clarify scope first. Good questions, with the answers we will design for:

Question	Answer we assume
Can a title have several physical copies?	Yes. Each copy is a **book item** with its own barcode and rack position.
How many books can one member hold?	At most **5** at a time.
How long can a member keep a book?	At most **10 days**, then a fine per day late.
Can a member reserve a book that is out?	Yes — and be told when a copy comes back.
How are people and books identified at the desk?	Every book item and every library card has a **unique barcode**; the system scans both.
Do we collect the fine money?	We *calculate and record* fines. Taking payment is out of scope (we say where it would go).
E-books, inter-library loans, lost-copy replacement?	Out of scope. Name them as extensions.

Requirements

Functional

1. Search the catalog by **title, author, subject category and publication date**.
2. Each book has a unique identification number (ISBN, the International Standard Book Number printed on every published book) and details; each copy has a **rack number** to find it physically.
3. A title can have **many copies** (book items); members check out and reserve copies.
4. Answer **"who took this copy?"** and **"which books does this member have?"**.

5. Limit: **max 5 books** per member.
6. Limit: **max 10 days** per loan.
7. **Fines** for books returned after the due date.
8. **Reserve** books that are not currently available.
9. **Notify** a member when a reserved book becomes available, and when a book is overdue.
10. **Barcodes** on every book item and every library card.

Non-functional

- **Correctness under concurrency:** two members at two desks can never both borrow the last copy; a member can never exceed 5 books even if two librarians scan for them at once.
- **Fast search:** searches are by far the most common operation, so they should be index lookups, not scans (an **index** is a lookup structure, such as a map from author name to books, that jumps straight to the matches instead of checking every book).
- **Extensibility:** fine rules and notification channels change; they should be swappable without editing the lending code.

Use cases

Three actors:

- **Librarian** — adds, edits and removes books, book items and members; can also issue, reserve and return items on a member's behalf.
- **Member** — searches the catalog; checks out, reserves, renews and returns books.
- **System** — sends notifications (overdue books, a reserved book is back, cancelled reservations).

Top use cases: add/remove/edit book (and book item); search catalog; register new account / cancel membership; **check out** a book; **reserve** a book that is out; **renew** (borrow the same copy again); **return** a book.

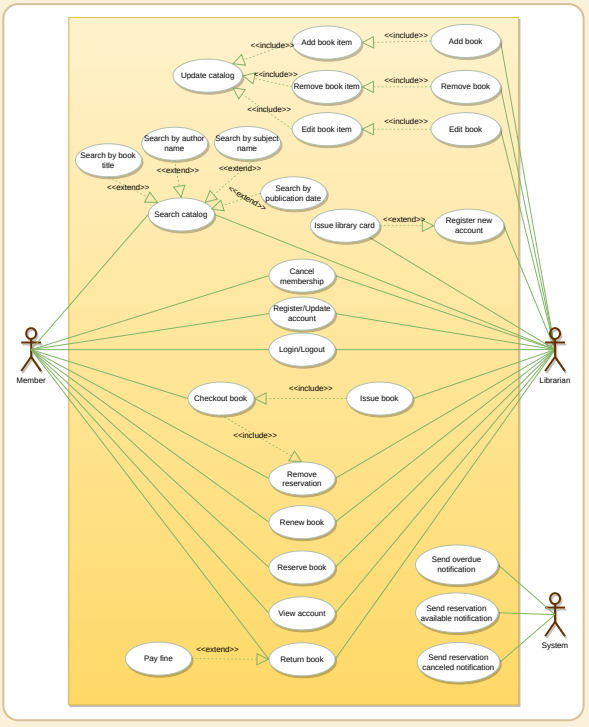


Figure — Library use cases: Member and Librarian share most of them (log in, search, check out, renew, reserve, return); the four "search by" use cases extend "Search catalog"; add/remove/edit book and book item feed "Update catalog"; "Checkout book" includes "Remove reservation"; the librarian's "Issue book" includes "Checkout book"; "Issue library card" is attached to "Register new account" (course diagram)

In the figure above, notice the two relationship kinds. `<<include>>` means a use case *always* performs another one as part of it — checking out a book always clears the member's reservation for it, if they had one. `<<extend>>` means an *optional variation* of a base use case — searching by author is one particular way a catalog search can go.

Core entities

Class	Responsibility
`Book` (record)	The **title**: ISBN, title, subject, publisher, publication date, authors. Immutable.
`BookItem`	One **physical copy**: barcode, format, rack, price, status (`AVAILABLE`, `RESERVED`, `LOANED`, `LOST`), and who it is held for.
`Author`, `Rack` (records)	An author's name; a rack's number and location.
`Account` (sealed, abstract) → `Member`, `Librarian`	People. An account **owns** its `LibraryCard` (composition). A member carries the loan count.
`LibraryCard` (record)	Card number, barcode, issue date.
`BookLending`	One loan: copy barcode, member id, creation date, **due date**, return date.
`Fine` (record)	Amount owed for one late return.
`Search` (interface) / `Catalog`	Four search methods; the catalog keeps one index per search and knows every copy of every title.
`LendingService`	The rules: check out, return, renew, reserve, overdue notices.
`FinePolicy` / `PerDayFine`	How much a late return costs (Strategy: a swappable rule behind an interface).
`Notifier` / `ConsoleNotifier`	Where messages go (email, SMS… here: console).
`Library`	Facade (one simple front object for the whole subsystem): owns the catalog and the lending service; only librarians may add stock.

In the table, a **record** is a short Java class that only carries values; **immutable** means it cannot change after it is created; an **enum** is a type with a fixed list of named constants; an **abstract** class cannot be created directly, only its subclasses can; and a **sealed** type lists exactly which subclasses it allows, so the compiler knows them all.

The course lists the same core classes: **Library** (name, address), **Book**, **BookItem**, **Account** (member or librarian), **LibraryCard**, **BookReservation**, **BookLending**, **Catalog** (searching by title, author, subject, publish date), **Fine**, **Author**, **Rack** (number + location identifier) and **Notification**. We keep every one of them; the only renames are `BookReservation` → a per-title waitlist inside `LendingService` (explained under *Design decisions*) and `Notification` → the `Notifier` interface.

Class diagram

The course builds its diagram in five stages, adding a few classes each time and greying out earlier ones. Walking through the stages is a good way to *present* a design in an interview, because every box arrives with a reason.

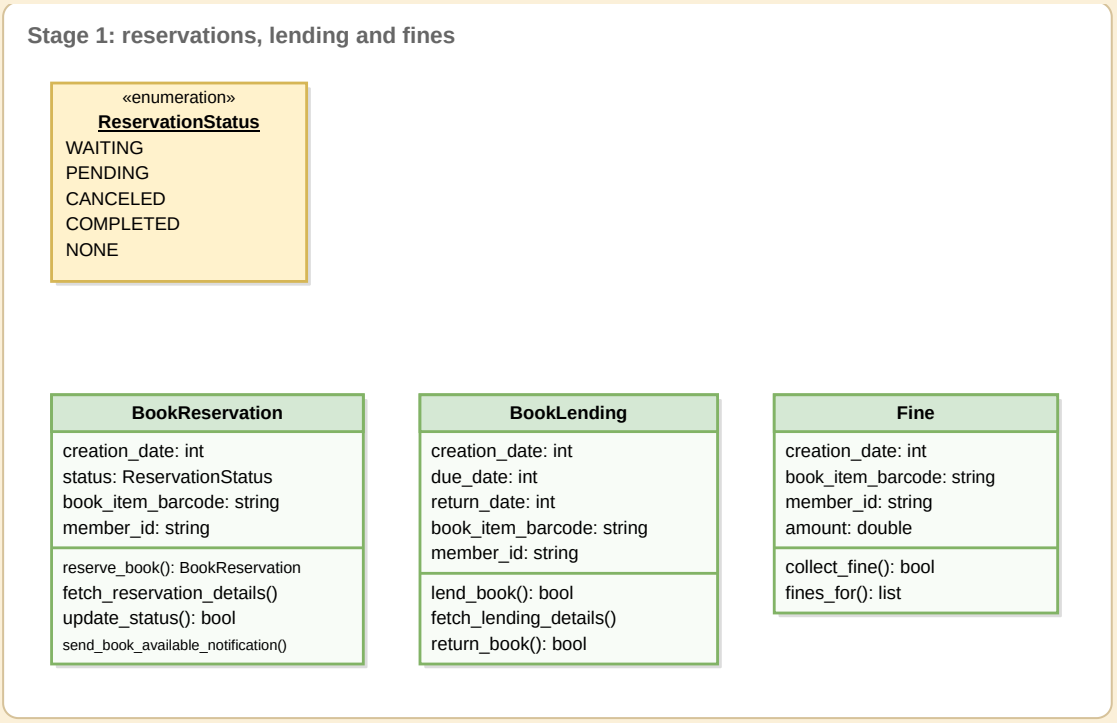


Figure — Stage 1: the three "what happened" records — `BookReservation`, `BookLending` and `Fine` — each keeping a copy's barcode and a member id (course diagram)

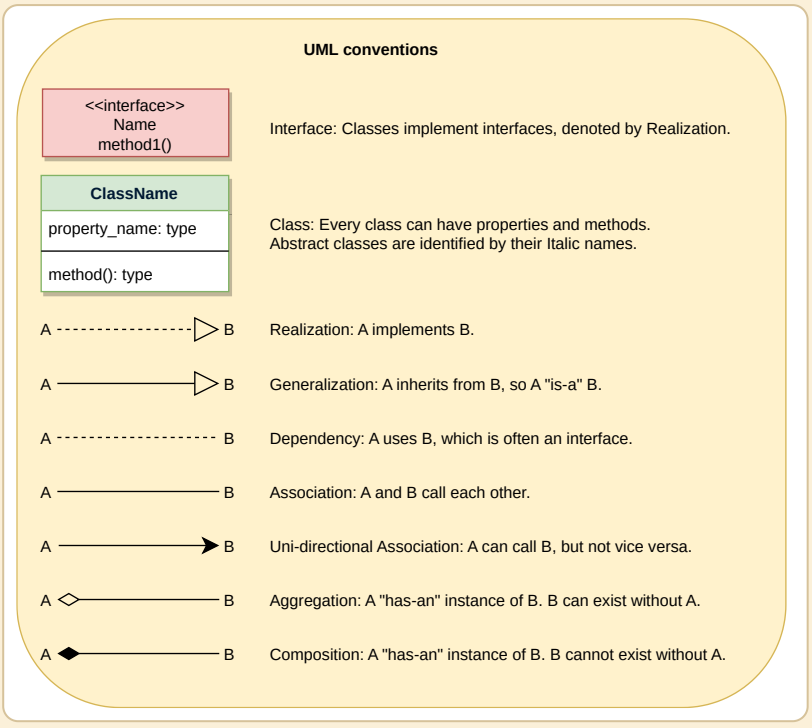


Figure — How to read a class box (name, `property: type`, `method(): type`) and the arrow styles used in these diagrams (course diagram)

Stage 1 answers requirements 4, 7 and 8, which are all about *remembering what happened*. Because each record stores the copy's barcode and the member's id, both halves of "who took this book / what does this member have" are answered by scanning these records.

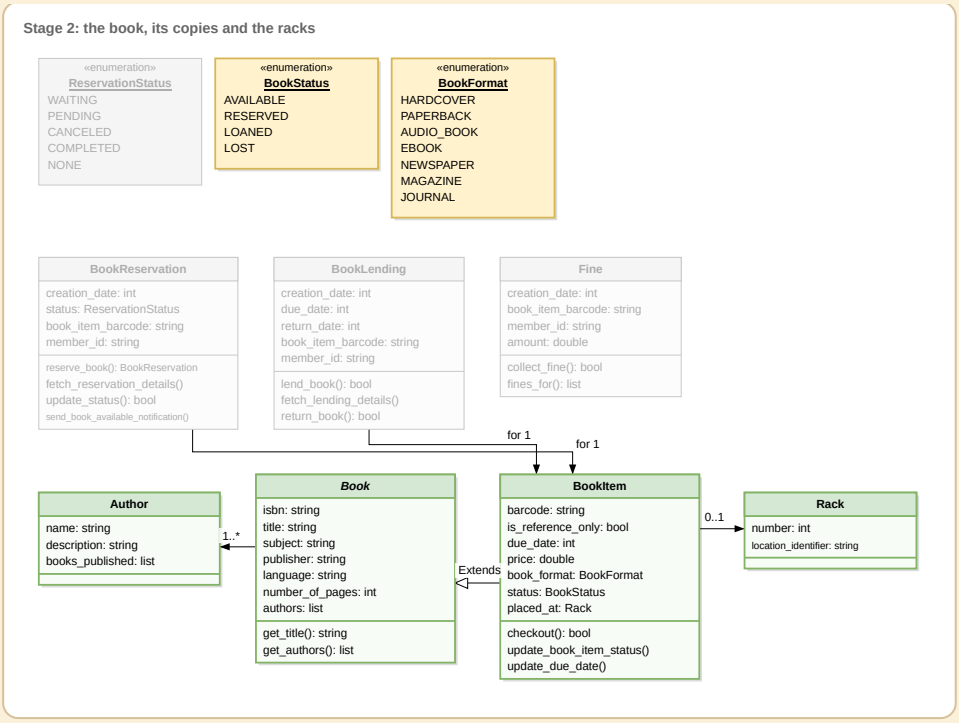


Figure — Stage 2: `Book` (the title) and `BookItem` (one copy with barcode, price and rack), plus `Author` and `Rack`; every reservation and every loan refers to one single copy (course diagram)

Stage 2 is the split that carries the whole design. In the course's diagram `BookItem` *inherits* from `Book`: the reading is that a copy carries all of the title's details and adds its own barcode, price and rack position. In our Java we use **`composition`** instead (`BookItem` *has a* `Book`) — see *Design decisions* for why.

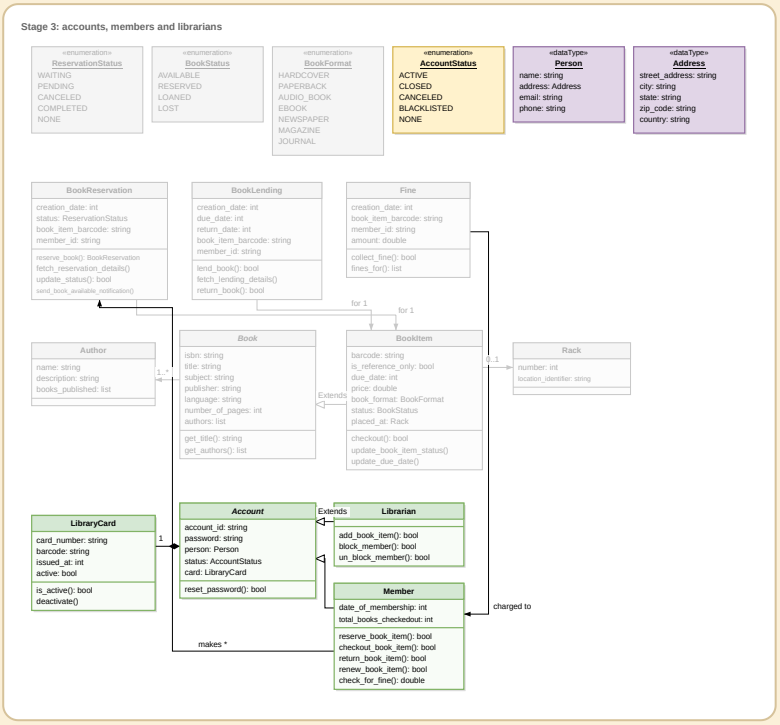


Figure — Stage 3: abstract `Account` with `Member` and `Librarian`; the account owns its `LibraryCard` (filled diamond = composition); a fine is charged to a member; a member makes reservations (course diagram)

Requirements 5 and 6 restrict *what one member may do*, so they belong to the people classes. `Account` is an **`abstract class`**: you can never create a plain `Account`, only a `Member` or a `Librarian`. The **`filled diamond`** is composition — the card has no life of its own, so closing the account destroys the card too.

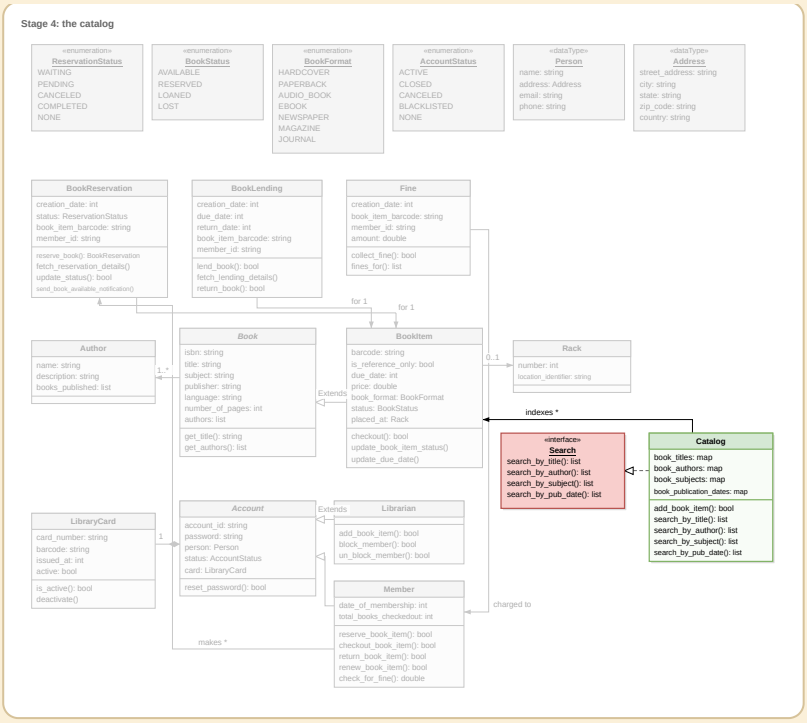


Figure — Stage 4: the `Search` interface (four methods) and `Catalog` implementing it (dashed line, hollow head), indexing book items without owning them (course diagram)

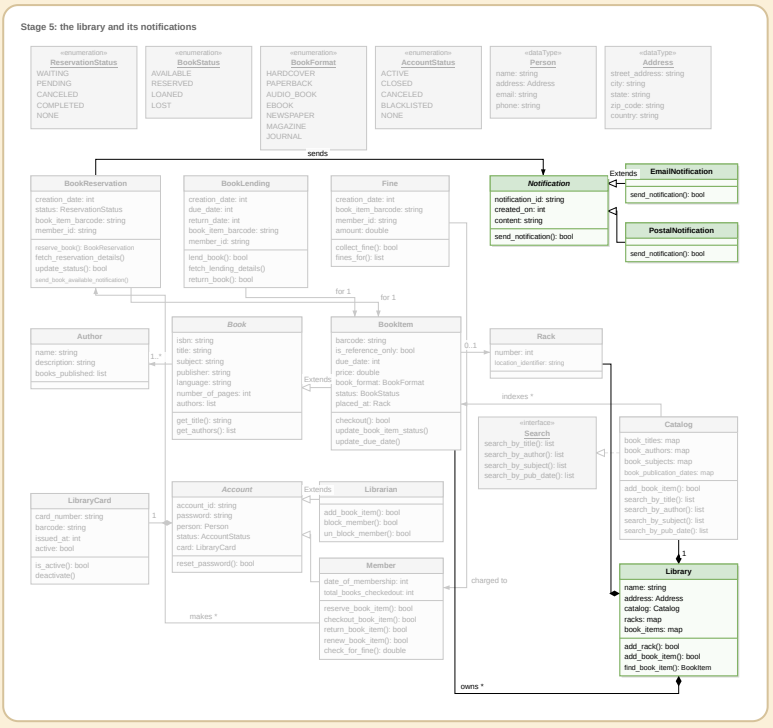


Figure — Stage 5: `Library` is the owner of the catalog, every rack and every book item; `Notification` is abstract with one subclass per delivery channel; a reservation triggers a notification when its copy comes back (course diagram)

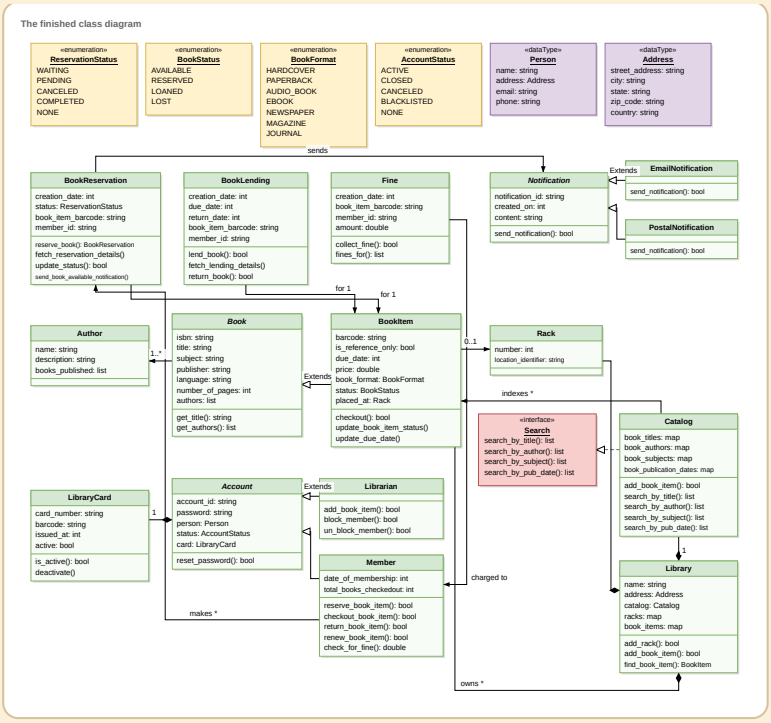
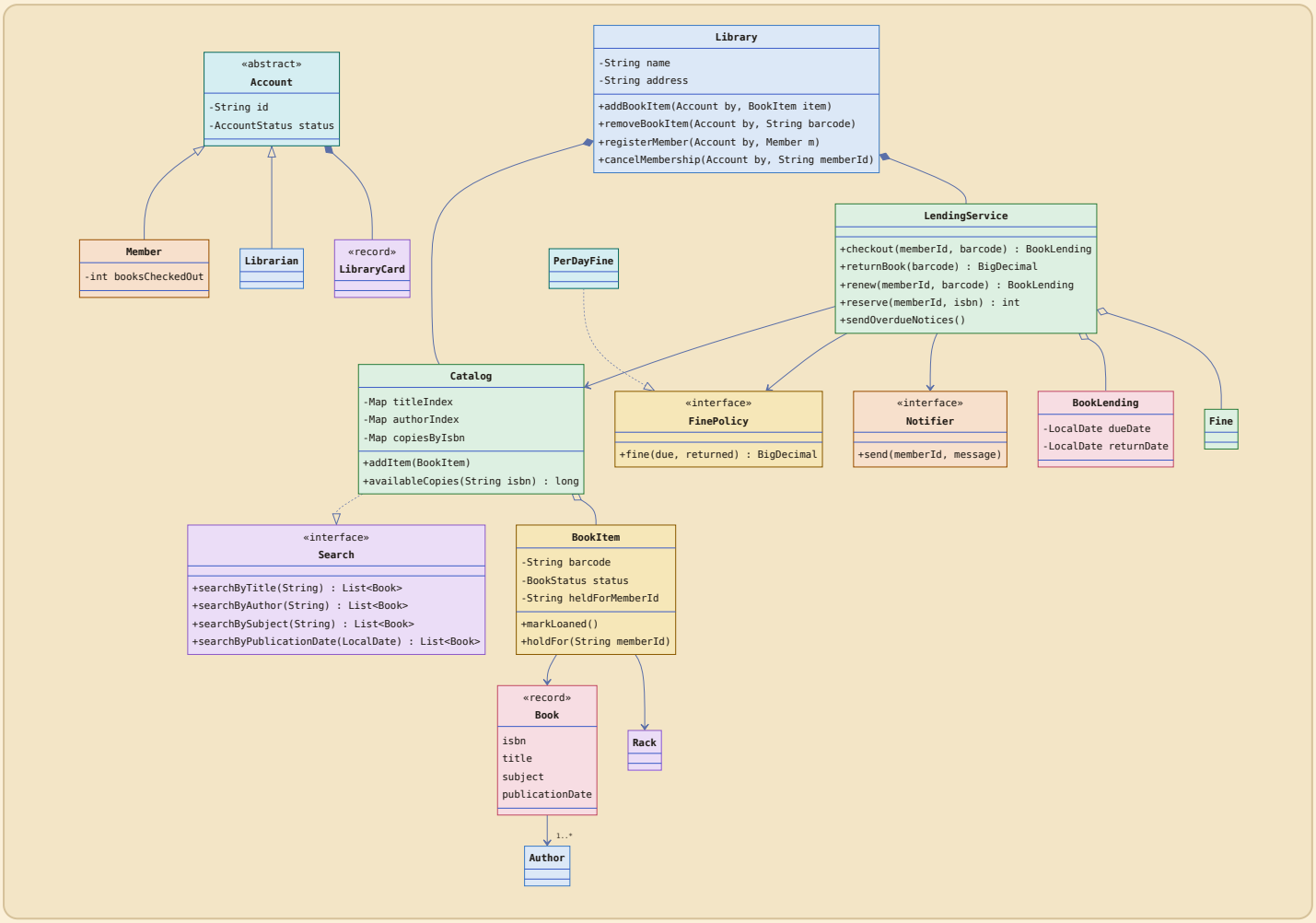


Figure — The finished library class diagram: every class from the five stages in full colour (course diagram)

Here is the class diagram of **our Java code** (the one you would draw on the whiteboard):



Key flows

The course gives three activity diagrams — check out, return and renew.

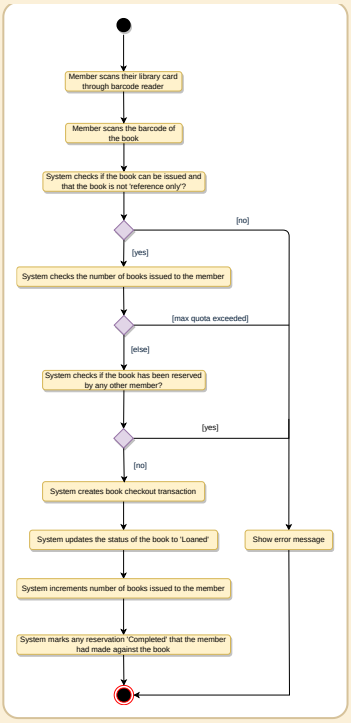


Figure — Check out: scan the book, check it can be issued (not reserved by someone else), check the member's quota, create the checkout transaction, mark the copy "Loaned", increment the member's count, complete any reservation this member had (course diagram)

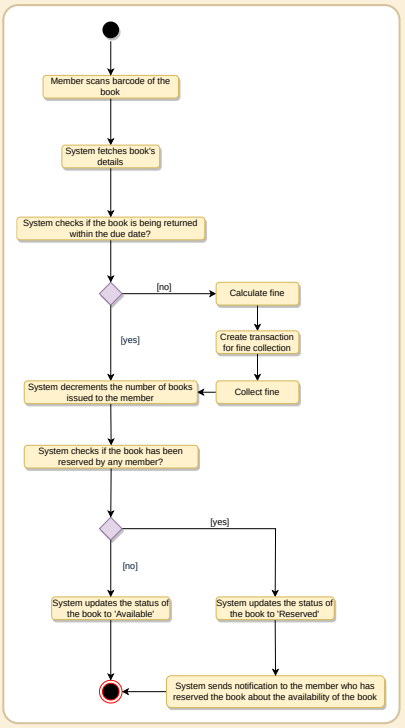


Figure — Return: scan the book, check whether it is late (if so, calculate and collect the fine), decrement the member's count, then check whether the book is reserved — if yes, notify the member who reserved it and mark it reserved, otherwise make it available (course diagram)

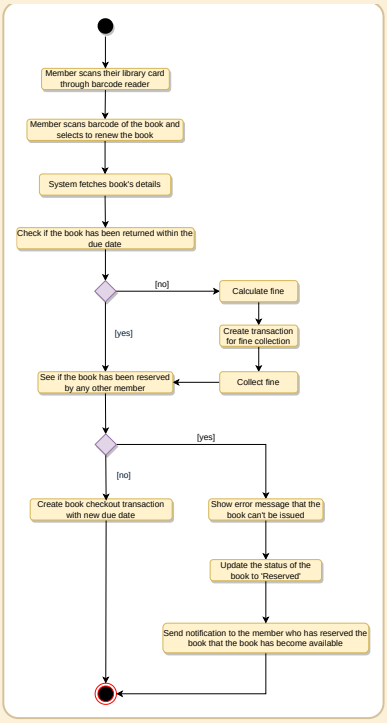
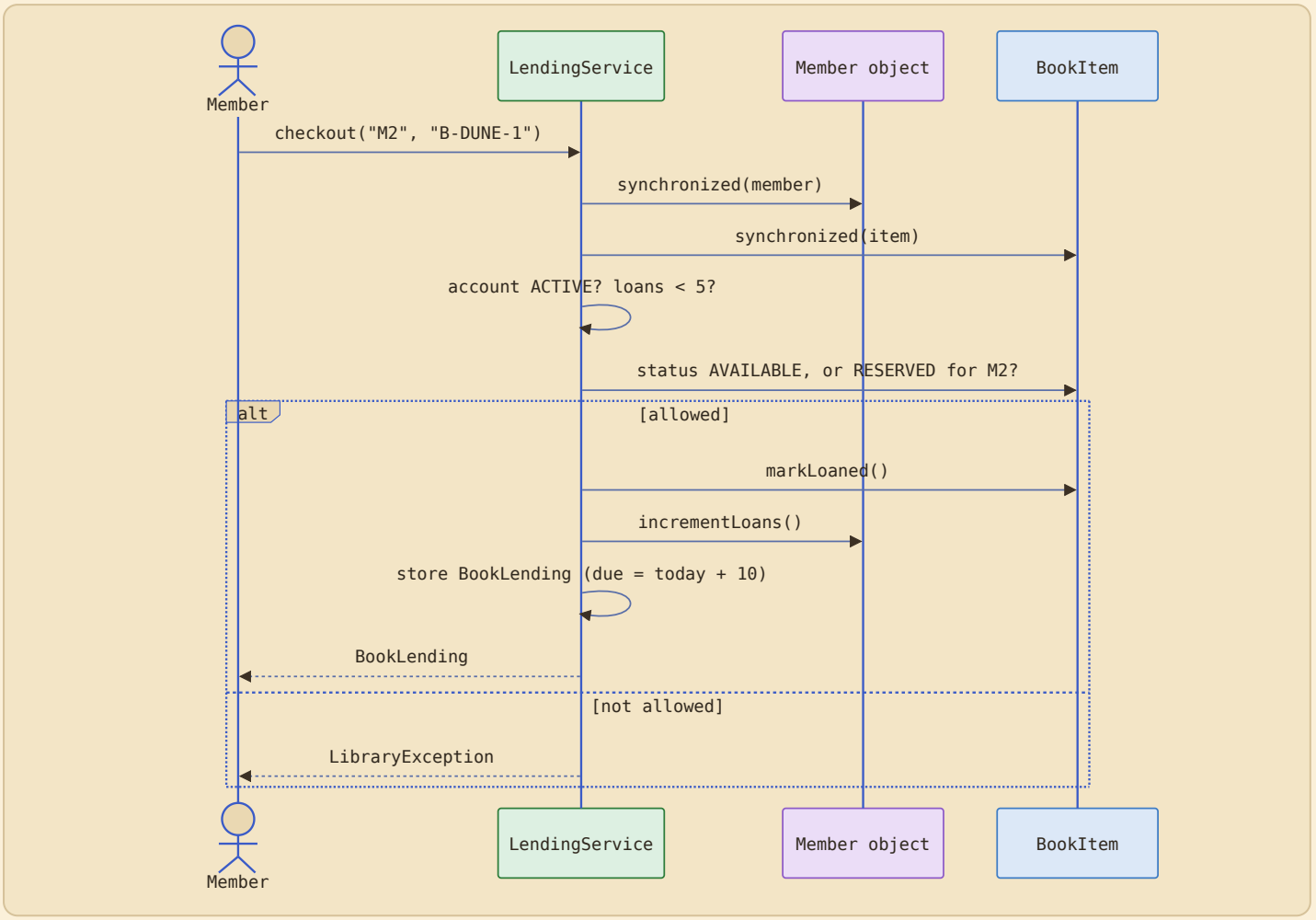
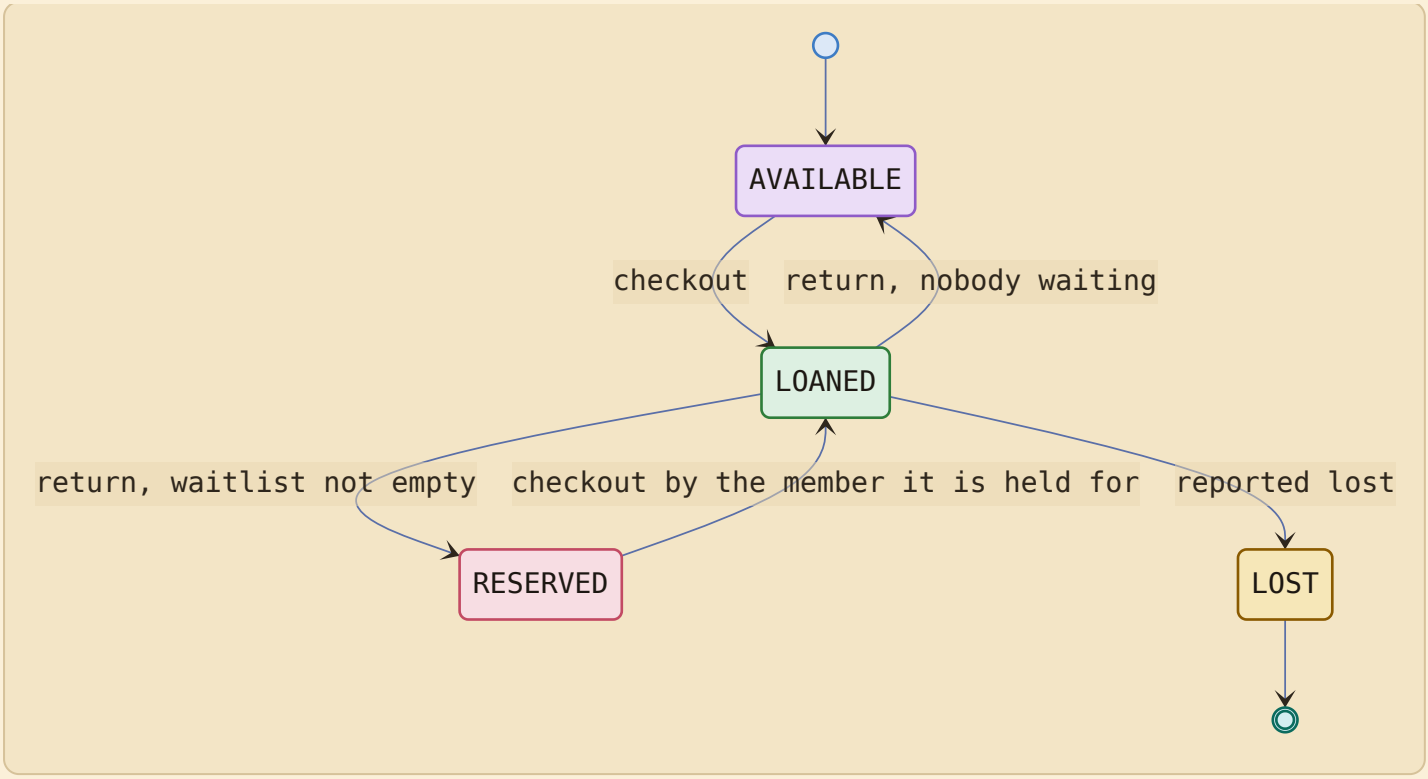


Figure — Renew: scan card and book, collect a fine if it is already late, check whether another member reserved it (if so, refuse and show an error), otherwise create a new checkout transaction with a new due date (course diagram)

The same check-out flow as a sequence diagram (messages between objects in time order, top to bottom) of our code, including the locks:



A book copy's life is a small **state machine** (a fixed set of states plus the moves allowed between them):


AVAILABLE
checkout
return, nobody waiting
LOANED
return, waitlist not empty
checkout by the member it is held for
reported lost
RESERVED
LOST

Design decisions & patterns used

Decision	Why	Alternative rejected
`Book` vs `BookItem` split	A member borrows a *copy*; search works on *titles*. Every later class depends on this split, so say it early.	One `Book` class with a `copies` counter — cannot say *which* copy is late, on which rack, or who has it.
Composition (`BookItem` has a `Book`) instead of the course's inheritance	Many copies share one immutable title; with inheritance each copy would duplicate every title field, and changing a title's subject would mean editing every copy. "A copy *is a* title" also fails a strict substitution test (the Liskov Substitution Principle: a subclass must work wherever its parent is expected): a title has no barcode or status.	`BookItem extends Book` (the course diagram) — works for a small model, and is worth mentioning as the other valid reading.
Lending rules in `LendingService`, not on `Member`	The course itself flags that a `Member` doing checkout/return/renew has *two reasons to change* (a person, and the lending rules) — a Single Responsibility violation. Moving the flows to a service keeps `Member` a plain person.	Methods on `Member` (course version) — fine for a first sketch.
Reservation = FIFO (first in, first out) waitlist per *title*	A member wants "a copy of Dune", not "copy #3". When *any* copy is returned, the first person in line gets it.	Reservation per copy (course's `BookReservation` keyed by barcode) — a member could wait on copy #3 while copy #1 sits on the shelf.
Strategy for fines (`FinePolicy`)	Fine rules change (caps, student discounts, holidays). The service calls `fine(due, returned)` and does not care how.	`if/else` inside `returnBook`.
Observer-style `Notifier`	(Observer: the sender announces events to whoever subscribed.) Lending code announces events; delivery (email, SMS, push) is someone else's job. The course's abstract `Notification` with one subclass per channel is the same idea.	Sending email directly from `LendingService`.
Facade (`Library`)	Desk software calls one object; role checks (only librarians add stock) live there.	Exposing the catalog and service directly to the UI.
Catalog keeps one index per search	Every lookup is a map read. The course names the cost: each new way of searching adds another map, whereas a single index keyed by (field, value) would stay one structure.	Scanning all books per search.

Java implementation

Package `lld.ch15.library`. Files: `Model.java` (enums, records, entities), `Catalog.java`, `Support.java` (fine policy, notifier, `SimClock`, money formatting), `LendingService.java`, `Library.java`, `Main.java`. Essential lines:

```java
// Model.java
enum BookFormat { HARDCOVER, PAPERBACK, AUDIO_BOOK, EBOOK, NEWSPAPER, MAGAZINE, JOURNAL }
enum BookStatus { AVAILABLE, RESERVED, LOANED, LOST }
enum AccountStatus { ACTIVE, CLOSED, CANCELED, BLACKLISTED }
record Author(String name) {}
record Rack(int number, String location) {}                    // where a copy physically sits

/** The TITLE: one per ISBN, however many copies. Immutable. */
record Book(String isbn, String title, String subject, String publisher,
            LocalDate publicationDate, List<Author> authors) {
    Book { authors = List.copyOf(authors); }
}

/** One PHYSICAL COPY: own barcode, rack, status. Guarded by its own monitor. */
final class BookItem {
    private final String barcode;
    private final Book book;                       // composition: the copy HAS a title
    private final BigDecimal price;                // money: BigDecimal, never double
    private BookStatus status = BookStatus.AVAILABLE;
    private String heldForMemberId;                // set only while status == RESERVED
    // state changes: only LendingService calls them, while holding this monitor
    void markLoaned()             { status = BookStatus.LOANED;    heldForMemberId = null; }
    void markAvailable()          { status = BookStatus.AVAILABLE; heldForMemberId = null; }
    void holdFor(String memberId) { status = BookStatus.RESERVED;  heldForMemberId = memberId; }
}

record LibraryCard(String cardNumber, String barcode, LocalDate issuedOn) {}   // owned by its account
abstract sealed class Account permits Member, Librarian { /* id, name, card, volatile status */ }
final class Member extends Account { private int booksCheckedOut; /* guarded by the member's monitor */ }
final class Librarian extends Account { }

final class BookLending {                          // "who has this copy, since when, due when"
    private final String barcode, memberId;
    private final LocalDate creationDate;
    private volatile LocalDate dueDate;            // volatile: read by the overdue job without the item lock
    private volatile LocalDate returnDate;
}
record Fine(String memberId, String barcode, BigDecimal amount, LocalDate date) {}
```

```java
// Catalog.java - one index per search, so every lookup is a map read; also knows every copy.
// Reads vastly outnumber writes: concurrent maps + copy-on-write lists fit well.
interface Search {
    List<Book> searchByTitle(String title);
    List<Book> searchByAuthor(String author);
    List<Book> searchBySubject(String subject);
    List<Book> searchByPublicationDate(LocalDate date);
}
final class Catalog implements Search {
    private final Map<String, List<Book>> byTitle = new ConcurrentHashMap<>();   // byAuthor, bySubject, byDate
alike
    private final Map<String, List<BookItem>> copies = new ConcurrentHashMap<>(); // isbn -> copies
    private final Map<String, BookItem> itemsByBarcode = new ConcurrentHashMap<>();
    long availableCopies(String isbn) {            // derived, never a counter that can drift
        return copiesOf(isbn).stream().filter(c -> c.status() == BookStatus.AVAILABLE).count();
    }
    // addItem: indexes the title once (keys lower-cased), rejects a duplicate barcode;
    // removeItem: under the item's monitor, refuses a LOANED or RESERVED copy
}
```

```java
// Support.java
interface FinePolicy { BigDecimal fine(LocalDate dueDate, LocalDate returnDate); }   // Strategy
final class PerDayFine implements FinePolicy {                    // the classic $0.50 a day
    @Override public BigDecimal fine(LocalDate due, LocalDate returned) {
        long daysLate = Math.max(0, ChronoUnit.DAYS.between(due, returned));
        return perDay.multiply(BigDecimal.valueOf(daysLate));
    }
}
interface Notifier { void send(String memberId, String message); }  // ConsoleNotifier prints "[notify M2] ..."
// SimClock extends Clock: advance(Duration) moves time forward. Money.fmt: "$" + two decimals.
```

```java
// LendingService.java - Locking rule (prevents deadlock): always lock the MEMBER first, then the BOOK ITEM.
public final class LendingService {
    static final int MAX_BOOKS_PER_MEMBER = 5;
    static final int MAX_LENDING_DAYS = 10;
    private final Map<String, BookLending> activeLoans = new ConcurrentHashMap<>();   // barcode -> loan
    private final Map<String, Deque<String>> waitlists = new ConcurrentHashMap<>();   // isbn -> member ids (FIFO)

    BookLending checkout(String memberId, String barcode) {
        Member m = member(memberId);
        BookItem item = catalog.item(barcode);
        synchronized (m) {
            synchronized (item) {
                if (m.status() != AccountStatus.ACTIVE) throw new LibraryException(memberId + " account is " +
m.status());
                if (m.booksCheckedOut() >= MAX_BOOKS_PER_MEMBER)
                    throw new LibraryException(memberId + " already has " + MAX_BOOKS_PER_MEMBER + " books");
                switch (item.status()) {
                    case AVAILABLE -> { /* ok */ }
                    case RESERVED -> {
                        if (!memberId.equals(item.heldForMemberId()))
                            throw new LibraryException(barcode + " is held for another member");
                    }
                    case LOANED, LOST -> throw new LibraryException(barcode + " is " + item.status());
                }
                item.markLoaned();
                m.incrementLoans();
                LocalDate today = today();
                BookLending loan = new BookLending(barcode, memberId, today, today.plusDays(MAX_LENDING_DAYS));
                activeLoans.put(barcode, loan);
                removeFromWaitlist(item.book().isbn(), memberId);   // their reservation is now fulfilled
                return loan;
            }
        }
    }

    BigDecimal returnBook(String barcode) {
        BookLending loan = activeLoans.get(barcode);              // read first to find the member...
        if (loan == null) throw new LibraryException(barcode + " is not checked out");
        Member m = member(loan.memberId());
        BookItem item = catalog.item(barcode);
        synchronized (m) {
            synchronized (item) {                                 // ...then lock in the same order and re-check
                if (activeLoans.get(barcode) != loan) throw new LibraryException(barcode + " is not checked out");
                LocalDate today = today();
                loan.close(today);
                activeLoans.remove(barcode);
                m.decrementLoans();
                BigDecimal fine = chargeFineIfLate(loan, today);  // records a Fine and notifies when > 0
                handToNextInLine(item);
                return fine;
            }
        }
    }

    BookLending renew(String memberId, String barcode) {          // same locks; refused while anyone waits
        // ... loan must belong to memberId
        if (waitingCount(item.book().isbn()) > 0)
            throw new LibraryException("cannot renew " + barcode + ": another member reserved this title");
        chargeFineIfLate(loan, today);                            // a late renewal still pays the fine
        loan.renewUntil(today.plusDays(MAX_LENDING_DAYS));
    }

    int reserve(String memberId, String isbn) {                   // join the title's FIFO waitlist
        if (catalog.availableCopies(isbn) > 0) throw new LibraryException("a copy is on the shelf, just check it
out");
        Deque<String> q = waitlist(isbn);
        synchronized (q) {
            if (!q.contains(memberId)) q.addLast(memberId);       // joining twice keeps your place
            return new ArrayList<>(q).indexOf(memberId) + 1;
        }
```

```
    }

    private void handToNextInLine(BookItem item) {                // caller holds the item's monitor
        Deque<String> q = waitlist(item.book().isbn());
        String next;
        synchronized (q) { next = q.pollFirst(); }
        if (next == null) { item.markAvailable(); return; }
        item.holdFor(next);
        notifier.send(next, "'" + item.book().title() + "' copy " + item.barcode() + " is held for you");
    }
    // cancelMembership (refused while books are out; drops the member from every waitlist),
    // sendOverdueNotices, whoHas(barcode), loansOf(memberId), fines()
}
```

```
// Library.java - Facade: the one object the desk software talks to
final class Library {
    void addBookItem(Account by, BookItem item) {                 // only a librarian may add stock
        if (!(by instanceof Librarian)) throw new LibraryException(by.id() + " is not a librarian");
        catalog.addItem(item);
    }
    // removeBookItem, registerMember, cancelMembership: same role check, then delegate
}
// Main.java: SimClock at 2026-03-01; PerDayFine($0.50); Dune x1, The Hobbit x6, LOTR x1; members M1-M9;
// section 3 releases 8 threads with a CountDownLatch to race for the only Dune.
```

Output:

```
== 1. Search the catalog ==
  author 'j.r.r. tolkien' -> [The Hobbit, The Lord of the Rings]
  subject 'Fantasy'       -> [The Hobbit, The Lord of the Rings]
  published 1965-08-01    -> [Dune]
  free copies of Hobbit   -> 6

== 2. Checkout limit (max 5 books per member) ==
  M1 took B-HOB-1, due 2026-03-11
  M1 took B-HOB-2, due 2026-03-11
  M1 took B-HOB-3, due 2026-03-11
  M1 took B-HOB-4, due 2026-03-11
  M1 took B-HOB-5, due 2026-03-11
  REFUSED: M1 already has 5 books
  M1 has [B-HOB-1, B-HOB-2, B-HOB-3, B-HOB-4, B-HOB-5]

== 3. 8 members race for the ONLY copy of Dune ==
  winners=1 losers=7 (exactly one copy was lent)
  holder = M9  (which member wins varies from run to run)

== 4. Reserve a title with no free copy (FIFO waitlist) ==
  M2 position 1
  M4 position 2
  M6 reserve Hobbit REFUSED: a copy is on the shelf, just check it out

== 5. Renew is blocked while someone waits ==
  REFUSED: cannot renew B-DUNE-1: another member reserved this title

== 6. 13 days later: overdue notices, late return, fine, hand-over ==
  [notify M9] 'Dune' (B-DUNE-1) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-1) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-2) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-3) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-4) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-5) was due 2026-03-11
  [notify M9] fine of $1.50 for late B-DUNE-1
  [notify M2] 'Dune' copy B-DUNE-1 is held for you
  fine charged = $1.50 (3 days late x $0.50)
  M4 REFUSED: B-DUNE-1 is held for another member
  M2 checks out held copy, due 2026-03-24
  who has B-DUNE-1? M2

== 7. Renew on time with nobody waiting ==
  [notify M1] fine of $1.50 for late B-HOB-1
  M6 renewed B-HOB-1, new due 2026-03-28
  fines on record: 2

== 8. Librarian housekeeping: remove a copy, cancel a membership ==
  REFUSED: B-DUNE-1 is LOANED, cannot remove it now
  copies of LOTR left: 0
  REFUSED: M6 must return 1 book(s) first
  REFUSED: M8 account is CANCELED
```

What to notice in the output: section 2 refuses the sixth book (requirement 5); section 3 lets exactly one of eight concurrent members borrow the only copy; section 5 refuses a renewal because someone is waiting; section 6 sends overdue notices, charges 3 days × $0.50, and hands the returned copy to the first person in the waitlist — the second person is refused because the copy is *held* for the first; section 8 covers the librarian's "remove book item" and "cancel membership" use cases — a copy that is out on loan cannot be removed, a member with a book still out cannot be cancelled, and a cancelled member can no longer borrow.

Concurrency & edge cases

- **Two members, one last copy.** `checkout` locks the copy (`synchronized (item)`), so "is it available? → mark loaned" is one atomic step. The demo's 8-thread race always has exactly one winner.
- **Two librarians scanning for the same member.** The quota check and increment happen under `synchronized (member)`, so a member can never end up with 6 books.

- **Deadlock avoidance.** (A **deadlock** is two threads each holding a lock the other needs, so both wait forever.) Every method locks **member first, then item** — a fixed global lock order. Two threads can never each hold one lock while waiting for the other. (`returnBook` reads the loan first to find the member, then locks in the same order and re-checks the loan is still active.)
- **Renew vs reserve.** Renewal is refused if the title's waitlist is non-empty. The check happens *before* charging any late fine, so a refused renewal never fines the member twice.
- **A known small window.** `reserve` checks "no free copy" and then joins the queue as two steps. If a copy is returned in between, it goes to the shelf while the member sits first in the queue; the *next* return reaches them. We accept this rather than make `reserve` take the queue lock before the copies' locks, because returns take them in the opposite order (copy, then queue) — mixing the two orders is exactly how deadlocks are born.
- **Held copy.** A returned copy with a waitlist becomes `RESERVED` for one member only; everybody else is refused at checkout.
- **Edge cases handled:** unknown barcode or member, duplicate barcodes, inactive/blacklisted/cancelled accounts, removing a copy that is on loan or held, cancelling a membership while books are still out, reserving a title that has a free copy ("just check it out"), joining the same waitlist twice (keeps your place), returning a book that is not out.
- **Across several servers** the `synchronized` blocks are not enough — the same guarantee must come from the database: e.g. `UPDATE book_item SET status='LOANED' WHERE barcode=? AND status='AVAILABLE'` and check that exactly one row changed (a *conditional update*).

Extensions & follow-up questions

- **"Two members reserve the last copy at the same moment — what happens?"** The course calls this out as unguarded in its own design. In ours, `reserve` appends to a FIFO deque (double-ended queue) under a lock, so both are queued in a definite order and the first one gets the next copy.
- **"Where do you keep the number of available copies of a title?"** Don't store a counter that can drift; derive it: `Catalog.availableCopies(isbn)` counts copies with status `AVAILABLE`. If that becomes slow, cache it (keep a ready-made copy for fast reads) and update it in the same lock as the status change.
- **"What is a `BookLending` for?"** It is the loan record: it answers "who has this copy, since when, due when", drives overdue notices and fines, and is the history of the copy.
- **"How is the fine paid?"** Add a `FinePayment` / `PaymentService` and block checkouts while unpaid fines exceed a threshold — a one-line rule in `checkout`.
- **"Hold expiry?"** A held copy should not wait forever: add `heldUntil` to `BookItem` and a daily job that passes the copy to the next person (or shelves it) when the hold lapses — exactly the TTL idea of section 15.2.
- **"E-books?"** An e-book "copy" is a licence: `BookFormat.EBOOK` items with no rack, and many libraries buy a fixed number of concurrent licences — the same copy model works.

Real-world example

Open-source integrated library systems such as **Koha** and **Evergreen** implement exactly this model: bibliographic *records* (titles) with attached *items* (copies with barcodes and shelf locations), circulation (lending and return) rules per patron (library member) category (loan period, maximum items, fine per day), a holds queue per title, and overdue/hold-available notices by email or SMS. Digital lending apps such as **OverDrive's Libby** apply the same waitlist idea to e-books: a library owns a fixed number of licences, borrowers join a holds queue, and when a licence frees up the next person is notified and given a limited window to borrow before it passes on.

In the interview

30–45 minute plan. (1) 3 min — clarify: copies, limits (5 books, 10 days), fines, reservations, notifications. (2) 5 min — actors and use cases. (3) 10 min — entities, *starting with the `Book` / `BookItem` split*, then accounts, lending records, catalog. (4) 10 min — code `checkout` and `returnBook` with the locking; mention lock ordering. (5) 5 min — the reservation hand-over and fines (Strategy). (6) Remaining — follow-ups: concurrency across servers, fine payment, hold expiry.

30-second answer: "The key split is `Book` (the title) vs `BookItem` (a physical copy with a barcode and rack). Members borrow copies; search works on titles through a `Catalog` with one index per search type. A `LendingService` enforces 5 books and 10 days, records `BookLending`s, charges fines via a `FinePolicy` strategy, and passes returned copies to a FIFO waitlist per title with a notification. Checkout locks member then copy so the last copy can't be double-lent."

TL;DR

Title vs copy. Limits on the member, state on the copy, history in `BookLending`. Rules in a `LendingService`; fines as a Strategy; notifications behind an interface. Lock member-then-copy so the last copy is never lent twice. Reservations are a FIFO queue per title.

15.2 Design a Movie Ticket Booking System (BookMyShow)



Figure — Cover image for the movie ticket booking case study (course diagram)

Problem & clarifying questions

An online movie ticket booking system (think **BookMyShow**, **Fandango**, **AMC's app**) lets customers see which movies are playing, pick a show, choose seats on a seat map, pay, and receive a ticket — from anywhere, at any time. It is *the* classic LLD booking question because it has a hierarchy of places (city → cinema → hall → seat), a time dimension (shows), and a genuine race (two people tapping the same seat).

Question	Answer we assume
One city or many?	Many cities with affiliate (partner) cinemas; the app lists cities.
Can a cinema have several halls? Several shows per hall?	Yes and yes — but a hall runs **one show at a time**.
What do customers search by?	Title, language, genre, release date, **city**.
Can a customer pick several seats in one booking?	Yes, from a seat map showing free vs booked seats.
How long is a seat kept while paying?	A few minutes (we use **5**). If payment does not finish, the seats go back.
Payment methods? Discounts?	Credit card or cash (cash at the front desk); optional **discount coupon**.
Refunds?	On cancellation, refund if within the allowed time (we use: up to 2 hours before the show).
Notifications?	New movie, booking made, booking cancelled.
Who else uses it?	Admins (add movies/shows, block customers), front-desk officers (book for walk-ins), guests (search only).

Requirements

Functional (the course's list, in order)

1. List the **cities** where affiliate cinemas are located.
2. Each cinema has many **halls**; each hall runs **one show at a time**.
3. Each movie has many **shows**.
4. **Search** movies by title, language, genre, release date and city.
5. After choosing a movie, show the **cinemas and shows** running it.
6. Pick a show and **book** tickets.
7. Show the **seating plan**; let the customer pick several seats.
8. Distinguish **available** seats from booked ones.
9. **Notify** on a new movie, and on booking or cancellation.
10. Pay by **credit card or cash**.
11. **No two customers can reserve the same seat.**
12. Apply a **discount coupon** to the payment.

Non-functional

- **Strong consistency on seats** (every reader sees the latest confirmed state; see System Design Fundamentals): double booking is the one bug that must never happen.
- **High read, bursty write** (writes arrive in sudden spikes): browsing dwarfs booking, but a blockbuster's first-day shows get many simultaneous seat grabs.
- **Hold expiry**: abandoned checkouts must not block seats (the course lists this as a gap in its own design; we implement it).
- **Failure-safe payments**: a customer must never pay for a seat they don't get.

Use cases

Five actors: **Admin** (add movies and shows, cancel a movie or show, block/unblock customers), **FrontDeskOfficer** (book and cancel tickets for walk-ins), **Customer** (view schedules, book, cancel), **Guest** (search only — must register to book), **System** (notifications).

Top use cases: search movies; create/modify/view booking; make payment; add a coupon to the payment; assign seats (seat map); refund payment on cancellation within the allowed time.

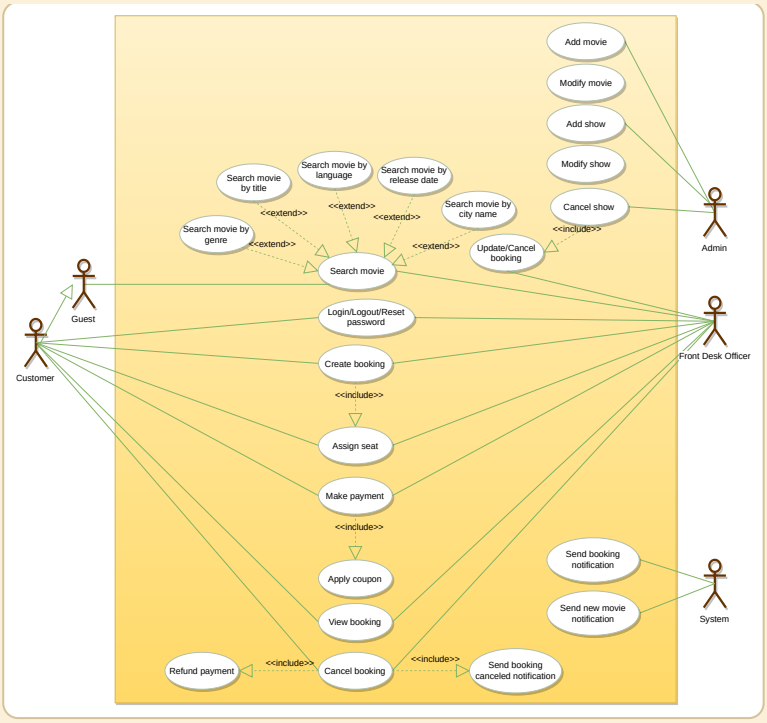


Figure — Movie booking use cases. The hollow arrow from Customer to Guest says a customer can do everything a guest can; the five "search by" use cases extend "Search movie"; "Create booking" includes "Assign seat"; "Make payment" includes "Apply coupon"; "Cancel booking" includes "Refund payment" and the cancellation notification; the admin's "Cancel show" includes updating/cancelling its bookings (course diagram)

Notice in the figure above that a **Customer is a specialised Guest**: everything a guest can do (search), a customer can do too, plus booking. That is the use-case version of the inheritance we enforce in code by giving `BookingService` a `Customer` parameter.

Core entities

Class	Responsibility
`City` (record)	Name and state. Cinemas point to it (association — the city exists without the cinema).
`Cinema`	Name, city; **owns** its halls.
`CinemaHall`	Name, physical seats; refuses an overlapping show.
`HallSeat` (record)	The seat bolted to the floor: id and `SeatType` (`REGULAR`, `PREMIUM`, `ACCESSIBLE`, `EMERGENCY_EXIT`, `OTHER`). **No booking state.**
`Movie` (record)	Title, language, genre, release date, duration.
`Show`	One screening of a movie in a hall at a time; **owns one `ShowSeat` per hall seat**.
`ShowSeat`	A seat *for this show*: price and `booked` flag. **This is what gets booked.**
`SeatLockProvider` / `InMemorySeatLockProvider`	Temporary, expiring **holds** on show seats; all-or-nothing lock, commit, release.
`Booking`	Customer, show, seat ids, status (`PENDING`, `CONFIRMED`, `CHECKED_IN`, `CANCELED`, `ABANDONED`), payment, coupon.
`BookingService`	hold → pay → commit (make final); cancel; seat map.
`Payment`, `PaymentMethod` (`CardPayment`, `CashPayment`), `PaymentGateway`	Money. The gateway (the external service, such as a card processor, that actually moves the money) is the boundary to the real provider.
`Coupon`	Percentage off, applied to the booking amount before payment.
`CancellationPolicy` / `CutoffRefundPolicy`	How much comes back on cancel (Strategy).
`MovieSearch` / `Catalog`	Five searches; "by city" is answered from shows.
`Customer`, `Guest`, `FrontDeskOfficer`, `Admin`	Actors; a `Guest` has no type that `BookingService` accepts.
`Notifier`	Messages to users: new movie (sent by `Catalog.addMovie`), booking confirmed, booking cancelled.

Class diagram

The course grows the model over **six stages**; after each one the system gains a capability it lacked:

Stage	Adds	What works afterwards
1	City, cinema, hall, physical seat	There is a building with seats in it
2	Movie, show, show seat	Every screening carries its own seat list and prices
3	Coupon, payment types, notification types	An amount can be reduced by a coupon and then paid
4	Booking and `BookingService`	One seat can never go to two customers
5	Search interface and catalog	A movie can be looked up in five different ways
6	People and accounts	Booking is restricted to registered customers

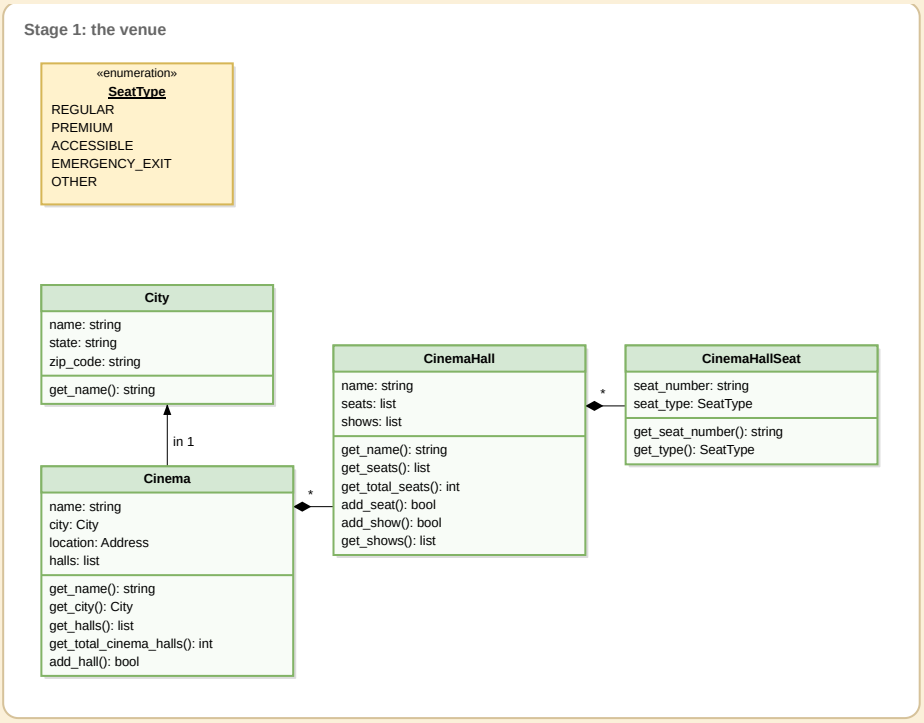


Figure — Stage 1: `City`, `Cinema`, `CinemaHall` and `CinemaHallSeat`; cinema→city is a plain association, cinema→hall and hall→seat are composition (course diagram)

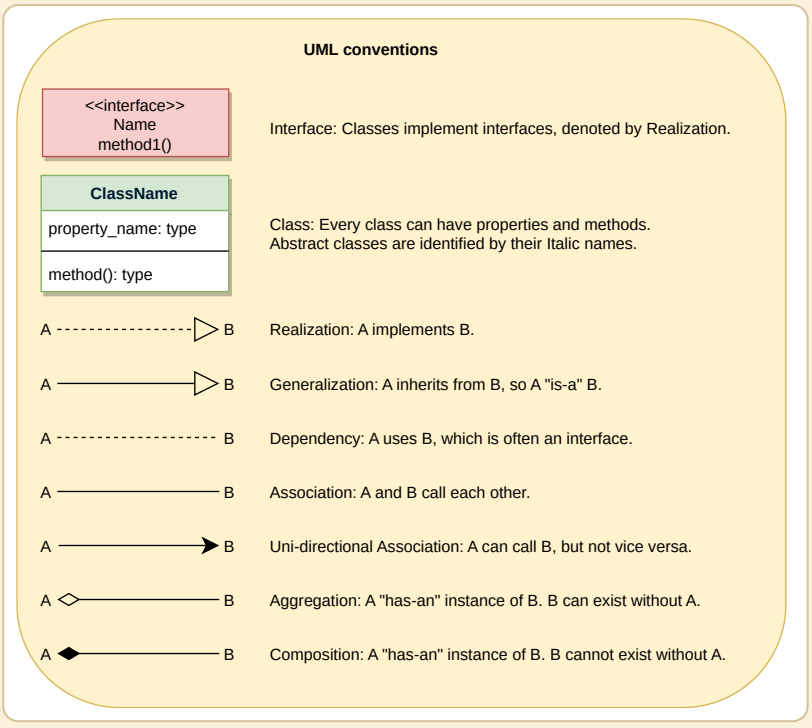


Figure — How to read a class box and the arrow styles used in these diagrams (course diagram)

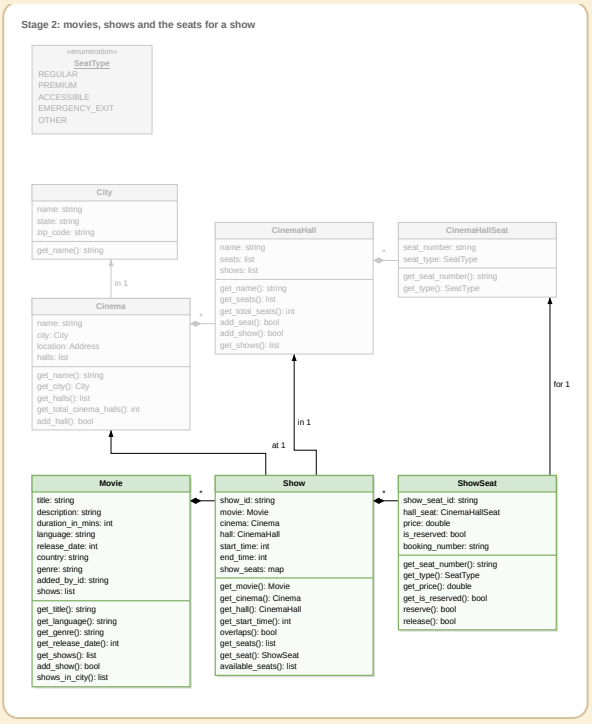


Figure — Stage 2: `Movie` owns its `Show`s; each `Show` owns one `ShowSeat` per physical seat, and each show seat points back at its `CinemaHallSeat` (course diagram)

Stage 2 is **where the design succeeds or fails**. One physical seat can be empty for the 3 pm screening and sold for the 9 pm one, so the thing you reserve cannot be the physical seat. You reserve a `ShowSeat` — one per seat per show — which also lets each show price its seats differently (a premium seat costs more).

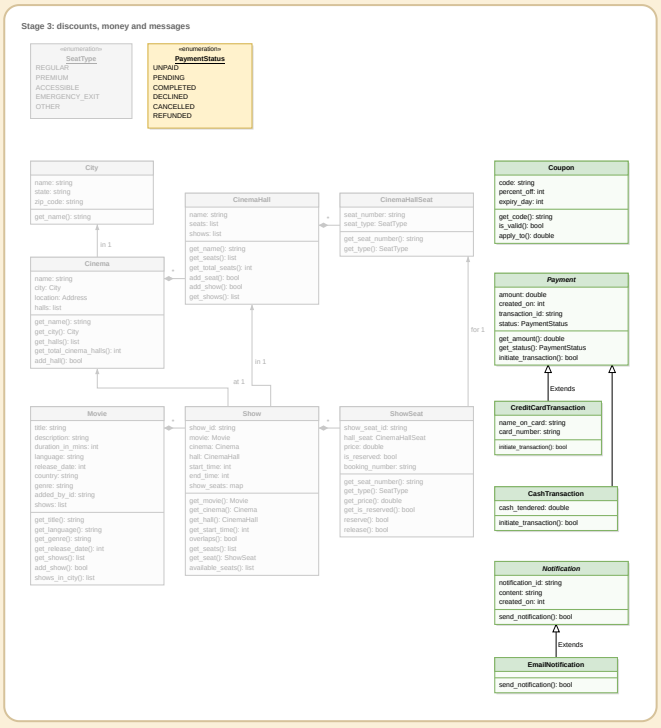


Figure — Stage 3: `Coupon`, abstract `Payment` (credit card and cash subclasses) and abstract `Notification` (one subclass per channel); none of them points at anything yet (course diagram)

The coupon reduces **the booking's amount, not the payment**: by the time any payment object is created the price is final, and no payment type needs its own discount logic.

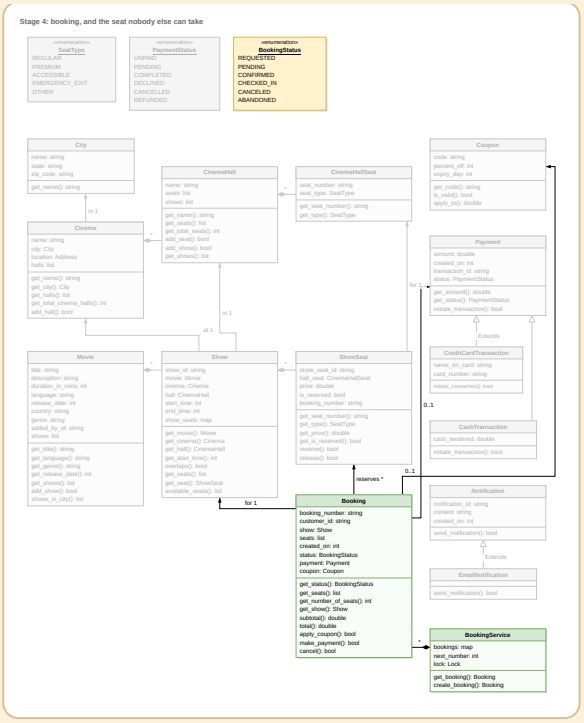


Figure — Stage 4: `Booking` points at the show, seats, payment and coupon without owning them; `BookingService` owns the bookings and is the only class holding a lock (course diagram)

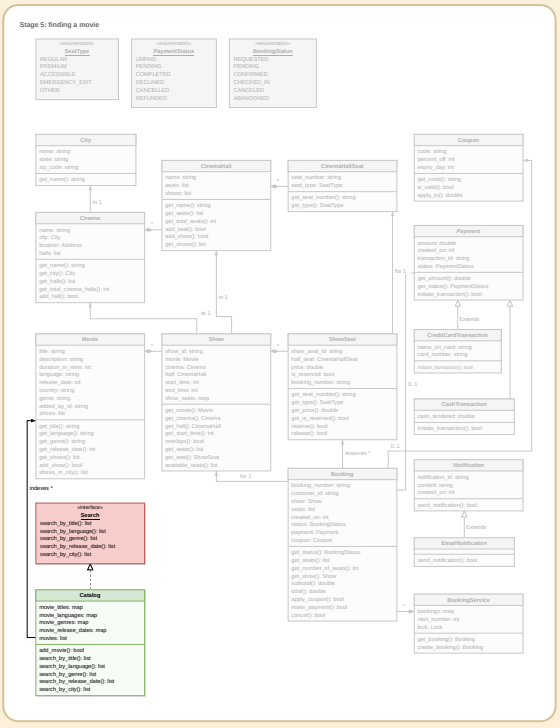


Figure — Stage 5: the `Search` interface with five methods, implemented by `Catalog`; four searches use indexes built when a movie is added, while city search is computed from the scheduled shows (course diagram)

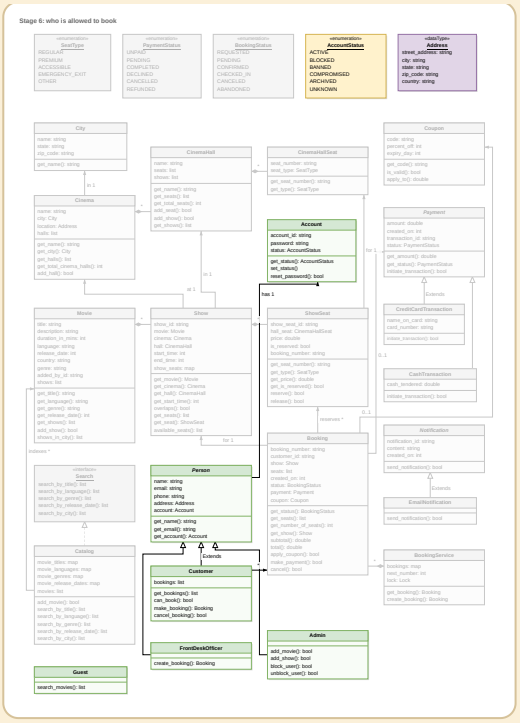


Figure — Stage 6: `Person` with `Customer`, `FrontDeskOfficer` and `Admin`; `Guest` sits outside that tree and has no booking method; a customer keeps the bookings it made (course diagram)

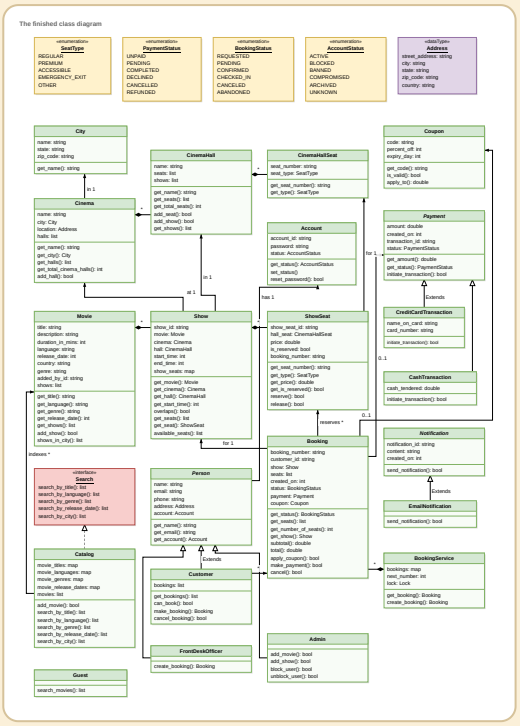
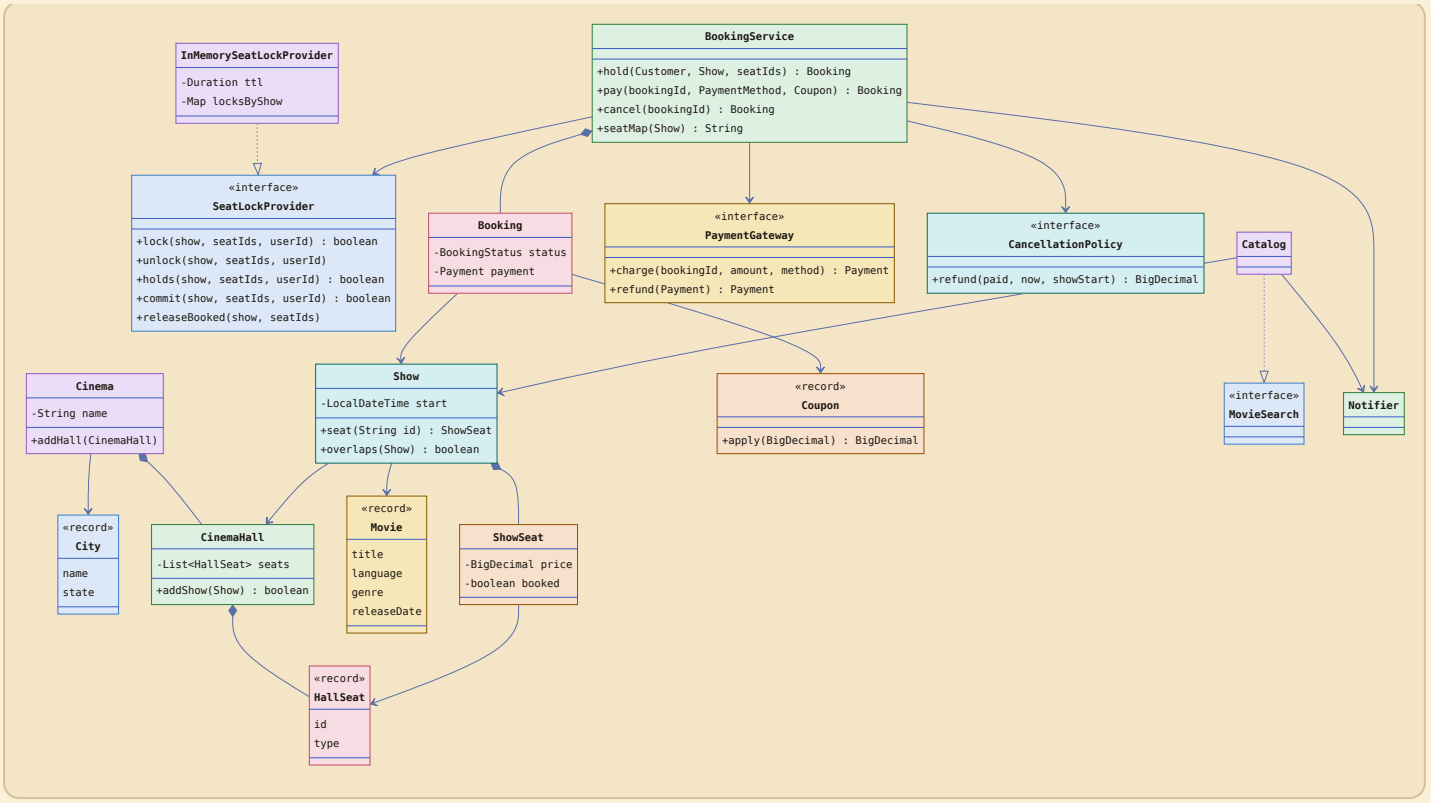


Figure — The finished movie booking class diagram (course diagram)

Our Java design (the main difference from the course: a separate `SeatLockProvider` with **TTL holds**, which the course names as its biggest omission):



Key flows

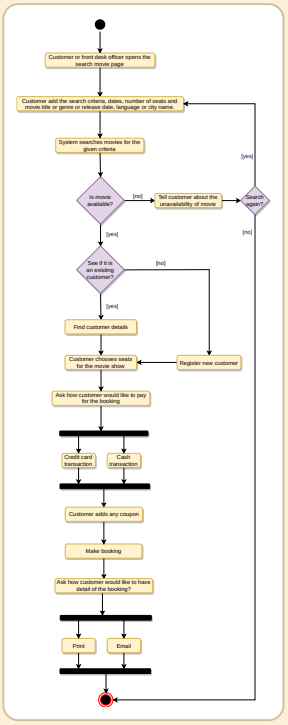


Figure — Make a booking: search movies; if available, find or register the customer, pick seats, ask how they will pay (credit card or cash), and choose how to receive the ticket (course diagram)

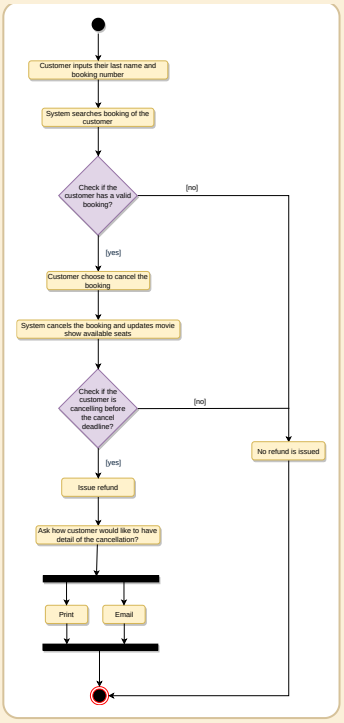
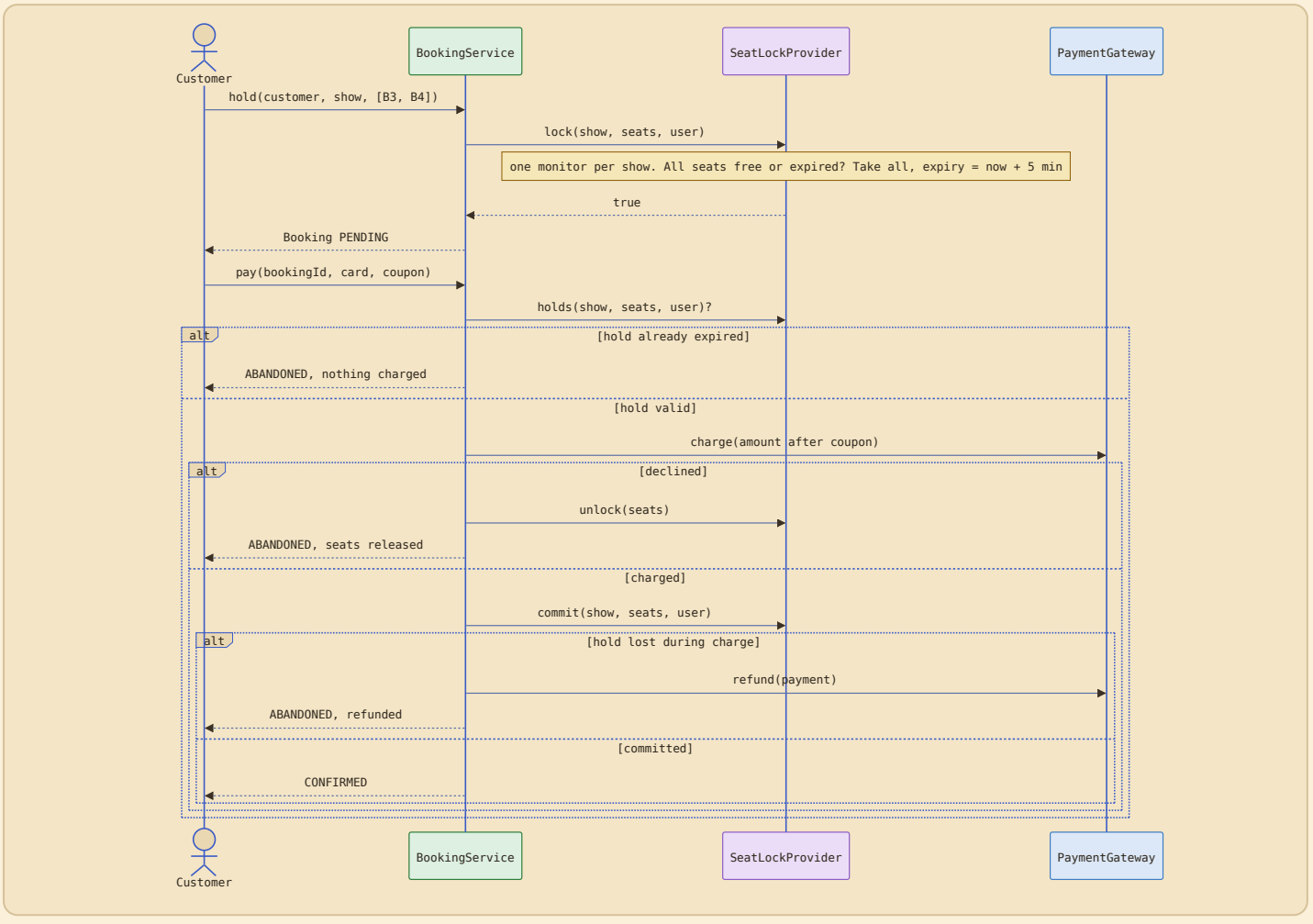


Figure — Cancel a booking: the customer gives last name and booking number; the system finds the booking, issues a refund if the cancellation is within the allowed time (otherwise no refund), cancels, updates the show's available seats, and prints or emails the details (course diagram)

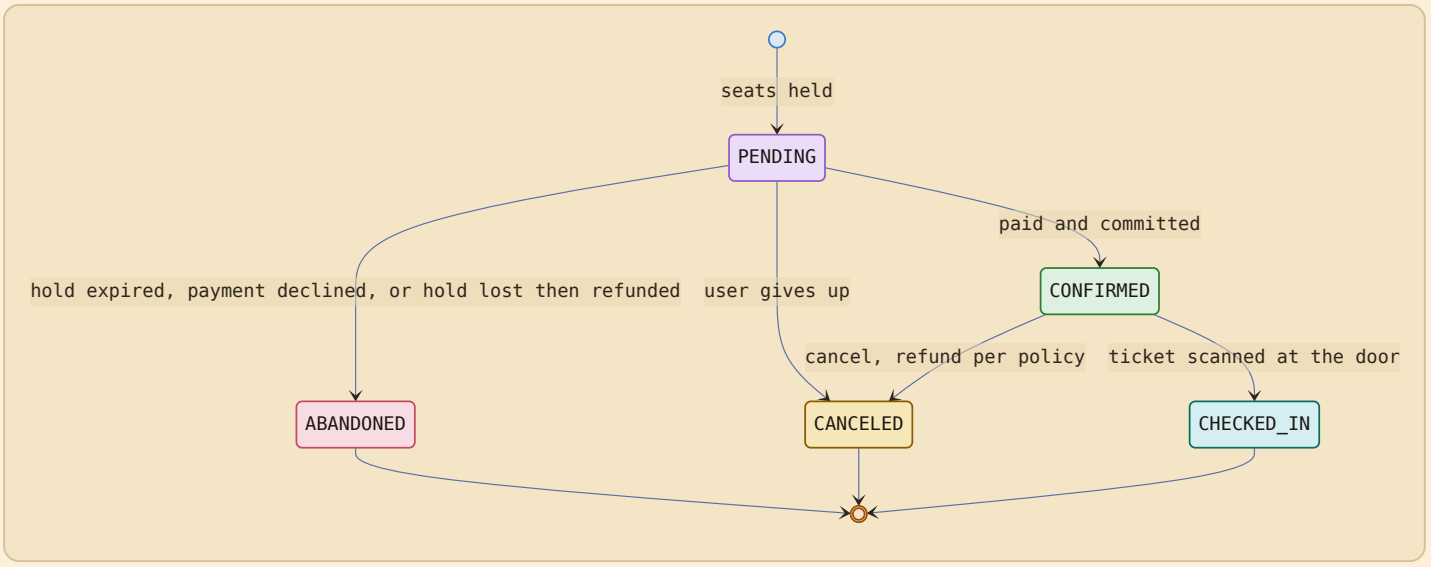
The heart of the design is a **three-step booking path**: *hold → pay → commit*.



Why `commit` must re-check inside the lock: the payment call can take seconds. If the hold expires during it and someone else grabs the seat, a naive "mark booked" would double-book. `commit` verifies "these seats are still held by me and not expired" **and** marks them booked in one atomic step. If that

fails after money was taken, we run a **compensating action** — a refund — instead of a database-style rollback (undoing unfinished changes as if they never happened), because a card charge cannot be "un-happened". This is the essence of the **Saga** idea: each step has an undo.

Booking lifecycle:



A show seat, as the customer sees it on the map: **free** (`.`), **held** by someone paying (`L`), **booked** (`X`). "Held" is not stored on the seat — it lives in the lock provider and simply disappears when it expires.

Design decisions & patterns used

Decision	Why	Alternative rejected
`ShowSeat` per show (course's key point)	The same physical seat is free in one show and taken in another; prices differ per show.	Booking `HallSeat` directly — breaks the moment there are two shows a day.
`SeatLockProvider` interface with TTL holds	Seats are kept only while the customer pays; abandoned checkouts free themselves. The interface lets you swap the in-memory version for **Redis** (a fast in-memory key-value store, often used for locks and caches; `SET key value NX PX 300000` — set only if absent, expire in 300 000 ms) or a database table with an `expires_at` column.	Holding seats until the booking is cancelled (course's version; it names this as the gap).
All-or-nothing lock	A customer asking for 4 seats together must never be left with 2. Check every seat first, then take all.	Locking seats one by one without rollback.
One monitor per show	Shows never block each other; within a show, every decision is serialized (made one at a time). Fine-grained enough for a cinema.	One global lock (the course's single-process lock) — correct but serializes the whole country's bookings.
Commit re-validates + compensation	Closes the "lock expired during payment" gap; the customer never pays for a seat they don't get.	Trusting the earlier check.
Idempotent `pay` (safe to repeat: two calls have the effect of one)	A double-click or network retry must not charge twice: paying a non-PENDING booking returns it unchanged. Real systems add an **idempotency key** per request.	Charging on every call.
Strategy for cancellation refunds and **Strategy-like `PaymentMethod`**	Refund rules and payment types vary independently of booking logic.	`if (method == CARD)` chains.
Adapter boundary (a wrapper that turns a foreign interface into ours) `PaymentGateway`	Wraps the external provider (Stripe, Razorpay…) behind our own interface; tests use `FakeGateway`.	Calling the provider's SDK (software development kit: the provider's own client library) from `BookingService`.
Coupon applied to the booking	Amount is correct before payment; payment types stay simple.	Each payment type applying coupons.
Guest has no booking method	The rule "guests must register to book" is enforced by *types*, not just written down: `hold` takes a `Customer`.	A `Guest.book()` that throws.
"Search by city" from shows	A movie is in a city only once a show is scheduled there; the other four searches are indexed when a movie is added.	Storing a city on `Movie`.

Java implementation

Package `lld.ch15.moviebooking`. Files: `Venue.java` (city, cinema, hall, seats, movie, show), `SeatLockProvider.java` (interface + in-memory TTL implementation), `Payments.java` (payment types, gateway, coupon, cancellation policy, notifier, clock), `Catalog.java` (five searches), `BookingService.java` (actors, booking, the booking path), `Main.java`. Essential lines:

```java
// Venue.java
enum SeatType { REGULAR, PREMIUM, ACCESSIBLE, EMERGENCY_EXIT, OTHER }
record City(String name, String state) {}
record HallSeat(String id, SeatType type) {}      // physical seat: NO booking state

final class CinemaHall {
    synchronized boolean addShow(Show show) {     // one show at a time: refuse an overlap
        for (Show s : shows) if (s.overlaps(show)) return false;
        shows.add(show);
        return true;
    }
}
// Cinema: name, City (association), owns its halls (composition)
record Movie(String id, String title, String language, String genre, LocalDate releaseDate, int durationMinutes)
{}

final class Show {                                // owns one ShowSeat per physical seat
    Show(String id, Movie movie, Cinema cinema, CinemaHall hall, LocalDateTime start, Map<SeatType, BigDecimal>
prices) {
        // ...
        for (HallSeat hs : hall.seats())
            seats.put(hs.id(), new ShowSeat(hs, prices.getOrDefault(hs.type(), BigDecimal.ZERO)));
    }
    boolean overlaps(Show o) { return start.isBefore(o.end()) && o.start().isBefore(end()); }   // end = start +
duration
}
final class ShowSeat {                            // what gets booked; only SeatLockProvider flips `booked`
    private final HallSeat seat;
    private final BigDecimal price;
    private volatile boolean booked;
}
```

```java
// SeatLockProvider.java - an interface so the in-memory version can be swapped for Redis (SET NX PX) or a DB row lock
interface SeatLockProvider {
    boolean lock(Show show, List<String> seatIds, String userId);     // all-or-nothing
    void unlock(Show show, List<String> seatIds, String userId);      // payment failed, user gave up
    boolean holds(Show show, List<String> seatIds, String userId);    // still mine, none expired?
    boolean commit(Show show, List<String> seatIds, String userId);   // atomically: verify, mark BOOKED, drop holds
    void releaseBooked(Show show, List<String> seatIds);              // cancellation
    Set<String> lockedSeats(Show show);
}
record SeatLock(String userId, Instant expiresAt) {
    boolean expired(Instant now) { return !now.isBefore(expiresAt); }
}

/** One monitor per show: shows never block each other; within a show every decision is serialized. */
final class InMemorySeatLockProvider implements SeatLockProvider {
    private final Map<String, Map<String, SeatLock>> locksByShow = new ConcurrentHashMap<>();   // show id -> seat -> lock

    @Override public boolean lock(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            Instant now = clock.instant();
            for (String id : seatIds) {                          // 1. check every seat first
                if (show.seat(id).isBooked()) return false;
                SeatLock l = t.get(id);
                if (l != null && !l.expired(now) && !l.userId().equals(userId)) return false;
            }
            SeatLock mine = new SeatLock(userId, now.plus(ttl));
            for (String id : seatIds) t.put(id, mine);           // 2. then take them all
            return true;
        }
    }
    @Override public boolean commit(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            if (!holdsLocked(t, seatIds, userId, clock.instant())) return false;   // every seat: mine and not expired
            for (String id : seatIds) { show.seat(id).setBooked(true); t.remove(id); }
            return true;
        }
    }
    // unlock removes only this user's locks; releaseBooked clears `booked`; lockedSeats lists unexpired holds -
    // all under the same per-show monitor
}
```

```java
// Payments.java
enum PaymentStatus { UNPAID, PENDING, COMPLETED, DECLINED, CANCELLED, REFUNDED }
sealed interface PaymentMethod permits CardPayment, CashPayment {}   // exactly card or cash
record CardPayment(String cardNumberLast4) implements PaymentMethod {}
record CashPayment() implements PaymentMethod {}
record Payment(String txnId, BigDecimal amount, PaymentMethod method, PaymentStatus status) {}

interface PaymentGateway {                          // adapter boundary to the real provider
    Payment charge(String bookingId, BigDecimal amount, PaymentMethod method);
    Payment refund(Payment p);
}
// FakeGateway: declines listed cards and can run a hook "while the bank is thinking" (setDuringCharge)

record Coupon(String code, int percentOff) {       // applied to the booking amount BEFORE payment
    BigDecimal apply(BigDecimal amount) {          // rounded to whole paise/cents, half-up
        BigDecimal off = amount.multiply(BigDecimal.valueOf(percentOff)).divide(BigDecimal.valueOf(100), 2,
RoundingMode.HALF_UP);
        return amount.subtract(off).setScale(2, RoundingMode.HALF_UP);
    }
}
interface CancellationPolicy { BigDecimal refund(BigDecimal paid, LocalDateTime now, LocalDateTime showStart); }
record CutoffRefundPolicy(Duration cutoff) implements CancellationPolicy {   // full refund up to cutoff, nothing
after
    @Override public BigDecimal refund(BigDecimal paid, LocalDateTime now, LocalDateTime showStart) {
        return now.plus(cutoff).isAfter(showStart) ? BigDecimal.ZERO : paid;
    }
}
```

```java
// Catalog.java - four searches indexed when a movie is added; "by city" answered from the shows
interface MovieSearch {
    List<Movie> searchByTitle(String title);
    List<Movie> searchByLanguage(String language);
    List<Movie> searchByGenre(String genre);
    List<Movie> searchByReleaseDate(LocalDate date);
    List<Movie> searchByCity(String cityName);
}
final class Catalog implements MovieSearch {
    void addMovie(Movie m) {                       // index four ways, then announce it
        // put(byTitle/byLanguage/byGenre/byRelease, ...)
        notifier.send("subscribers", "new movie: " + m.title() + " (" + m.language() + ", " + m.genre() + ")");
    }
    boolean addShow(Show s) { if (!s.hall().addShow(s)) return false; shows.add(s); return true; }
    @Override public List<Movie> searchByCity(String city) {
        return shows.stream().filter(s -> s.cinema().city().name().equalsIgnoreCase(city))
                .map(Show::movie).distinct().sorted(Comparator.comparing(Movie::title)).toList();
    }
    // showsOf(movie, city): that movie's shows in that city, by start time
}
```

```
// BookingService.java - HOLD seats (TTL) -> PAY -> COMMIT; refund if the hold was lost mid-charge
enum BookingStatus { PENDING, CONFIRMED, CHECKED_IN, CANCELED, ABANDONED }
enum AccountStatus { ACTIVE, BLOCKED, BANNED, COMPROMISED, ARCHIVED, UNKNOWN }
final class Customer { /* id, name, volatile status */ }          // only customers can book
record Guest(String sessionId) { Customer register(String id, String name) { return new Customer(id, name); } }
record FrontDeskOfficer(String id, String name) {}
record Admin(String id, String name) { void block(Customer c) { c.setStatus(AccountStatus.BLOCKED); } }
// Booking: customerId, show, seatIds, baseAmount, status, payment, coupon - transitions under its own monitor

final class BookingService {
    Booking hold(Customer c, Show show, List<String> seatIds) {      // step 1, all-or-nothing
        if (c.status() != AccountStatus.ACTIVE) throw new IllegalStateException(c.id() + " is " + c.status());
        if (seatIds.isEmpty()) throw new IllegalArgumentException("no seats");
        List<String> sorted = seatIds.stream().distinct().sorted().toList();
        if (!locks.lock(show, sorted, c.id()))
            throw new SeatUnavailableException(sorted + " not available for " + show.id());
        BigDecimal base = sorted.stream().map(id -> show.seat(id).price()).reduce(BigDecimal.ZERO,
BigDecimal::add);
        Booking b = new Booking("BK-" + seq.incrementAndGet(), c.id(), show, sorted, base);
        bookings.put(b.id(), b);
        return b;
    }
    // holdFor(FrontDeskOfficer, Customer, ...): same flow for a walk-in

    Booking pay(String bookingId, PaymentMethod method, Coupon coupon) {   // steps 2 + 3; idempotent
        Booking b = get(bookingId);
        synchronized (b) {
            if (b.status() != BookingStatus.PENDING) return b;
            if (!locks.holds(b.show(), b.seatIds(), b.customerId())) {       // fast fail, no charge
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " hold expired before payment, nothing charged");
                return b;
            }
            b.setCoupon(coupon);
            BigDecimal amount = coupon == null ? b.baseAmount() : coupon.apply(b.baseAmount());
            Payment p = gateway.charge(b.id(), amount, method);
            b.setPayment(p);
            if (p.status() != PaymentStatus.COMPLETED) {
                locks.unlock(b.show(), b.seatIds(), b.customerId());           // free seats at once
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " payment " + p.status() + ", seats released");
                return b;
            }
            if (!locks.commit(b.show(), b.seatIds(), b.customerId())) {       // hold lost during charge
                b.setPayment(gateway.refund(p));                              // COMPENSATION
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " hold expired during payment, " + Money.fmt(amount) + "
refunded");
                return b;
            }
            b.setStatus(BookingStatus.CONFIRMED);
            notifier.send(b.customerId(), b.id() + " CONFIRMED " + b.seatIds() + " for '" +
b.show().movie().title()
                    + "' at " + b.show().start() + ", paid " + Money.fmt(amount));
            return b;
        }
    }

    Booking cancel(String bookingId) {             // under the booking's monitor
        // PENDING   -> unlock the holds, CANCELED
        // CONFIRMED -> releaseBooked; refund = cancellationPolicy.refund(paid, now, showStart);
        //              refund via gateway if > 0; CANCELED; notify
        // otherwise -> IllegalStateException ("cannot be canceled from ...")
    }
    // seatMap(show): "." free, "L" held by someone paying, "X" booked
}
// Main.java: Bengaluru / PVR Forum / Audi 1 = 2 rows x 5 seats (A regular Rs.200, B premium Rs.350);
// TTL 5 min; FakeGateway declines card "0002"; CutoffRefundPolicy(2 h); 10 threads race for B3+B4.
```

Output:

```
== 1. Admin adds movies (subscribers told) and schedules shows; the hall refuses an overlap ==
  [notify subscribers] new movie: Inception (English, Sci-Fi)
  [notify subscribers] new movie: RRR (Telugu, Action)
  add SH1 Inception 18:00 -> true
  add SH2 RRR 19:00 same hall -> false

== 2. Search ==
  city Bengaluru -> [Inception]
  genre Action   -> [RRR] (no show yet, but indexed)
  Inception shows in Bengaluru -> [SH1@2026-05-01T18:00]

== 3. 10 customers click B3+B4 at the same instant ==
  holds granted = 1 (the other 9 saw 'not available')
  A . . . . .
  B . . L L .

== 4. The winner pays by card with a 10% coupon ==
  [notify U10] BK-1 CONFIRMED [B3, B4] for 'Inception' at 2026-05-01T18:00, paid Rs.630.00
  paying twice is harmless -> CONFIRMED
  A . . . . .
  B . . X X .

== 5. Hold expires (TTL 5 min) before paying ==
  Ravi re-held A1,A2 after Asha's hold expired -> BK-3
  [notify ASHA] BK-2 hold expired before payment, nothing charged
  [notify RAVI] BK-3 CONFIRMED [A1, A2] for 'Inception' at 2026-05-01T18:00, paid Rs.400.00

== 6. Hold expires WHILE the bank is charging -> refund (compensation) ==
  [notify MEENA] BK-4 hold expired during payment, Rs.200.00 refunded
  BK-4 [A5] ABANDONED REFUNDED Rs.200.00

== 7. Declined card releases the seats immediately ==
  [notify NEHA] BK-6 payment DECLINED, seats released
  B1 held by anyone now? false

== 8. Blocked customer, then cancellation with refund ==
  REFUSED: U10 is BLOCKED
  [notify U10] BK-1 canceled, refund Rs.630.00
  BK-1 [B3, B4] CANCELED REFUNDED Rs.630.00
  A X X . . L
  B . . . . .
```

What to notice: section 1 announces each new movie to subscribers (requirement 9) and refuses an overlapping show in the same hall (requirement 2); section 3 lets exactly 1 of 10 simultaneous customers hold B3+B4; section 4 applies the 10% coupon (Rs.700 → Rs.630) and a second `pay` is a no-op; section 5 shows an expired hold being re-sold and the slow customer *not* being charged; section 6 is the compensation path — Meena's card was charged, but her hold expired during the charge and Kiran took A5, so she is refunded; section 7 releases seats immediately on a decline; section 8 blocks a customer and cancels with a full refund.

Concurrency & edge cases

- **Check-then-act is atomic.** `lock`, `holds`, `commit`, `unlock` and `releaseBooked` for one show all run inside `synchronized` on that show's lock table. "Is the seat free? → take it" can't interleave.
- **Expired holds are lazy.** There is no background thread; an expired `SeatLock` is simply treated as absent the next time anyone looks. (In Redis, the key's TTL does the same.)
- **The payment-window race** (hold expires mid-payment) is handled by commit re-validation + refund, as shown in section 6.
- **Lock granularity** (how much data one lock covers)**.** The booking's own monitor makes `pay`/`cancel` on the same booking serial (no double charge, no pay-while-cancel). The show monitor protects seats. The two are always taken in the order booking → show, never the reverse, so no deadlock.
- **Across many servers** (real BookMyShow), in-process locks are not enough. Options, from simplest:
  1. **Conditional update** (course): `UPDATE show_seat SET status='HELD', held_by=?, held_until=? WHERE show_id=? AND seat_id IN (...) AND (status='FREE' OR held_until < now())`, then check the row count equals the

number of seats; if not, roll back.

2. **Pessimistic lock** (lock first, then work, assuming conflicts are likely): `SELECT ... FOR UPDATE` on the seat rows inside a transaction (a group of database statements that succeed or fail together).
3. **Serializable transaction** (course): the result is as if each transaction had run alone, in some order — easiest to reason about, but the slowest option when many buyers collide.
4. **Optimistic locking**: a `version` column; update `WHERE version = ?`; retry on conflict. Good when conflicts are rare.
5. **Unique constraint** (a database rule that rejects a second row with the same values) on `(show_id, seat_id)` in the confirmed-tickets table as the final safety net.
6. **Distributed lock / hold store** (a lock kept in a shared store, so all servers see it) in Redis with TTL for the *hold* phase, with the database as the source of truth (the one copy that wins when copies disagree) for *confirmed* seats. The course's point, in short: the in-process lock only makes the common case quick; once there are two servers, correctness comes from the database.

- **Edge cases:** empty seat list; duplicate seat ids in one request (de-duplicated); unknown seat; blocked customer; overlapping show in a hall; cancelling an abandoned booking (rejected); cancelling too late (no refund).

Extensions & follow-up questions

- **"Show the seat map live."** Push seat changes (held/booked/released) to open seat-map pages over WebSockets or server-sent events (two ways for a server to push updates to an open browser page); the lock provider publishes events (Observer).
- **"Extend the hold when the user reaches the payment page."** Add `extend(show, seats, user, extra)` that refreshes `expiresAt` only if the caller still holds the lock.
- **"Dynamic pricing / different prices per seat type and time."** Replace the price map with a `PricingStrategy` (base by `SeatType`, surcharge for weekends or premieres).
- **"Payment succeeded but our server crashed before commit."** Store the booking as `PENDING` with the gateway's transaction id before charging; a reconciliation job (a periodic task that compares our records with the gateway's and fixes mismatches) queries the gateway for pending bookings and either commits (if the seats are still held) or refunds. Payment providers' **webhooks** (HTTP calls the provider makes to your server when something happens) notify you of the final result.
- **"Cancel a whole show."** Admin cancels → every confirmed booking of the show is cancelled with a full refund and a notification (iterate bookings by show).
- **"Loyalty points, food & beverage add-ons."** Extra line items on the booking; points as another `PaymentMethod`.
- **"Seat gaps rule (no single empty seat left between bookings)."** A validation step in `hold` before locking — a `SeatSelectionRule` chain (Chain of Responsibility).

Real-world example

On **BookMyShow**, after you pick seats the app shows a **countdown timer** on the payment screen; if it runs out the seats are released and you must start again — that timer is the seat hold's TTL. If your bank debits you but the booking does not complete, the app tells you the amount will be **refunded automatically** — that is the compensation path. Big releases put the "one monitor per show" idea under real stress: first-day-first-show bookings for a blockbuster concentrate thousands of users on a handful of shows.

In the interview

30–45 minute plan. (1) 4 min — requirements; ask about hold time, payment, refunds. (2) 4 min — actors/use cases. (3) 8 min — entities: city → cinema → hall → seat; movie → show → **show seat** (say why). (4) 12 min — the booking path: `SeatLockProvider` (all-or-nothing, TTL), `BookingService.hold/pay/cancel`; write `lock` and `commit`. (5) 5 min — payment failure and the expired-during-payment refund. (6) Remaining — multi-server concurrency (conditional update / Redis TTL), follow-ups.

> **30-second answer:** "Venue hierarchy is City → Cinema → Hall → HallSeat. A Show owns a ShowSeat per hall seat, and that's what's booked. Booking is hold → pay → commit: a SeatLockProvider takes all requested seats atomically with a 5-minute TTL; payment goes through a gateway interface; commit re-checks the hold and marks seats booked atomically, and if the hold was lost we refund. In one process a per-show monitor guarantees no double booking; across servers the database's conditional update or a unique constraint is the real guarantee, with Redis TTL keys for holds."

Follow-ups with answers: *Why not lock the whole show table?* Fine for one cinema, but a global lock serializes all shows. *Why TTL?* Without it an abandoned checkout blocks seats forever. *Optimistic or pessimistic?* Pessimistic/conditional-update for hot shows (conflicts common); optimistic when conflicts are rare.

TL;DR

Book `ShowSeat`s, not seats. Hold (all-or-nothing, TTL) → pay → commit (re-validate atomically) → refund if the hold was lost. Coupon on the booking, gateway behind an interface, refund rule as a Strategy. Per-show lock in one JVM (Java Virtual Machine — one running Java process); conditional updates, unique constraints and Redis TTL keys across servers.

15.3 Design a Concert Ticket Booking System

Problem & clarifying questions

A concert ticketing system (think **Ticketmaster**, **Live Nation**, **SeatGeek**, **BookMyShow Live**) sells tickets for a single, huge, one-off event. The *objects* look like section 15.2 — an event, a venue, seats, bookings, payments — so we reuse those ideas and do **not** repeat them. The *focus* is different: **extreme contention**. A movie show has a few hundred seats and demand spread over days; a stadium tour sells 50,000 tickets in minutes to a million people who all arrive at 10:00:00. That changes three things:

1. **Fairness** — who gets to try first? (a *virtual waiting room*)
2. **Throughput under contention** (throughput = requests completed per second; see System Design Fundamentals) — locks that serialize everyone become the bottleneck (the one point that limits overall speed) (so we use *lock-free* compare-and-set)
3. **Abuse** — scalpers (people who buy tickets to resell at a markup) and bots (automated scripts) (so *per-user limits*), and what happens to fans who miss out (a *waitlist*)

Question	Answer we assume
Seat types?	Tiers: **VIP** and **PREMIUM** are numbered seats; **GENERAL_ADMISSION** (GA) is a standing area — just a *count*.
Search?	By artist, venue, date and time (same catalog idea as 15.2 — not repeated).
How do we stop a stampede?	A **virtual waiting room**: everyone gets a queue number; people are admitted in batches, in order.
Limit per person?	**4 tickets** per user per concert.
Hold time?	**10 minutes** to pay.
Sold out?	Fans can join a **waitlist** per tier; released tickets are offered to the waitlist first.
Notifications?	Confirmation and "a ticket freed up for you" (email/SMS).

Requirements

Functional — show concerts with their seating layout and availability; search by artist, venue, date, time; choose seats (or a GA quantity) and buy; **prevent double booking under concurrent requests**; **fair access** for all users; secure payments; confirmation notifications; **waitlist** for sold-out events.

Non-functional — very high write contention for a few minutes; **no overselling** ever (GA count never below zero, a seat never sold twice); fairness (first-come-first-served into the sale); bounded per-user purchases; holds that expire on their own.

Use cases

Actors: **Fan** (join queue, hold, buy, cancel, join waitlist), **Organizer/Admin** (create concert, set tiers and prices, open the sale), **System** (admit batches from the queue, expire holds, offer released tickets to the waitlist, notify), **Payment provider** (external).

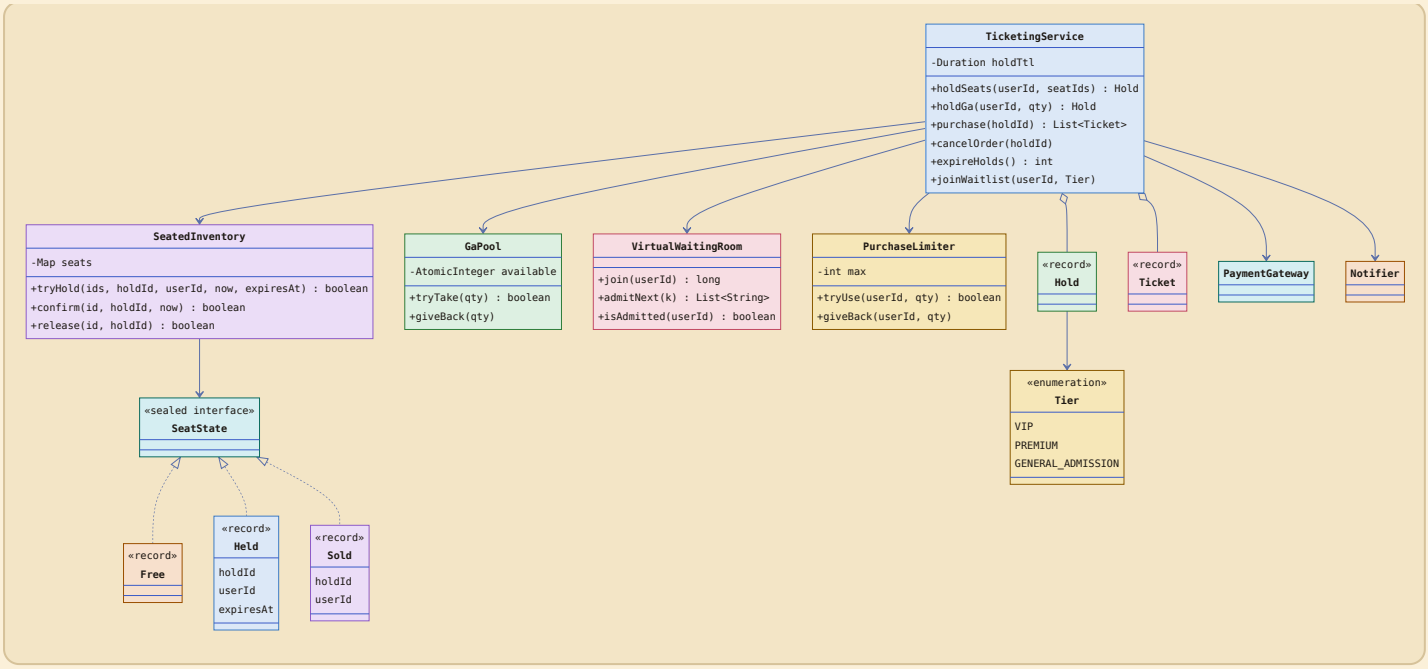
Main use cases: join the waiting room → get admitted → **hold** seats or a GA quantity → **purchase** → receive tickets; **cancel** an order (refund); **join waitlist**; system **reaps** expired holds.

Core entities

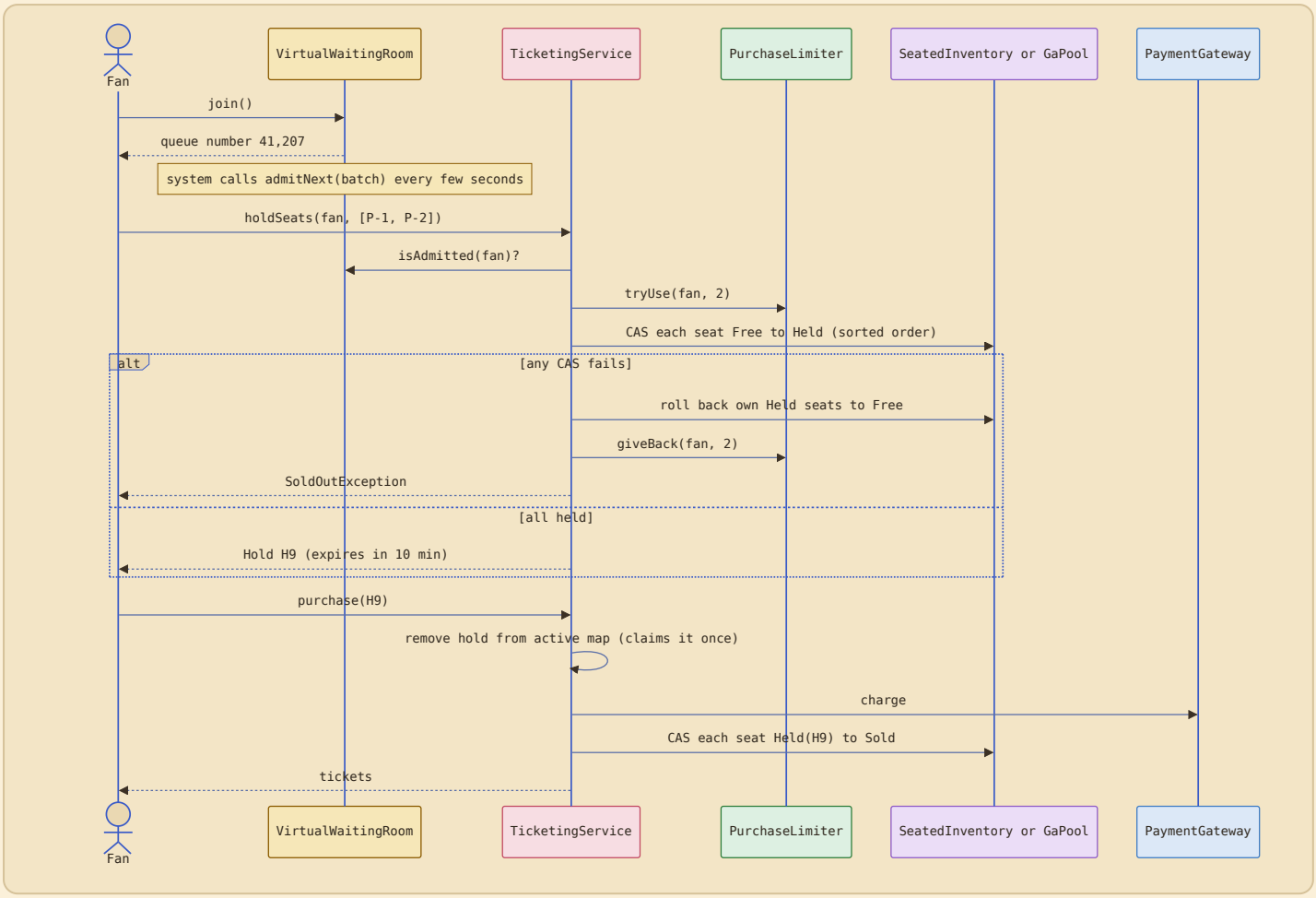
Class	Responsibility
`Concert` (record)	Artist, venue, date/time.
`Tier` (enum)	`VIP`, `PREMIUM` (seated) and `GENERAL_ADMISSION` (unseated), each with a price.
`SeatState` (sealed) → `Free`, `Held`, `Sold`	A seat's whole state as one immutable value.
`SeatedInventory`	One `AtomicReference<SeatState>` (a holder for one object reference that supports CAS) per numbered seat; claims by **CAS**.
`GaPool`	GA capacity as an `AtomicInteger` (an int with atomic updates); take/give back by CAS.
`Hold` (record)	A temporary claim: seats or a GA quantity, amount, expiry.
`Ticket` (record)	One admission: tier and seat id or "GA".
`VirtualWaitingRoom`	Queue numbers in arrival order; admits batches FIFO.
`PurchaseLimiter`	At most N tickets per user, checked and updated atomically.
`TicketingService`	hold → purchase → (cancel); reaper; waitlist offers.
`PaymentGateway`, `Notifier`	External boundaries (as in 15.2).

Class diagram

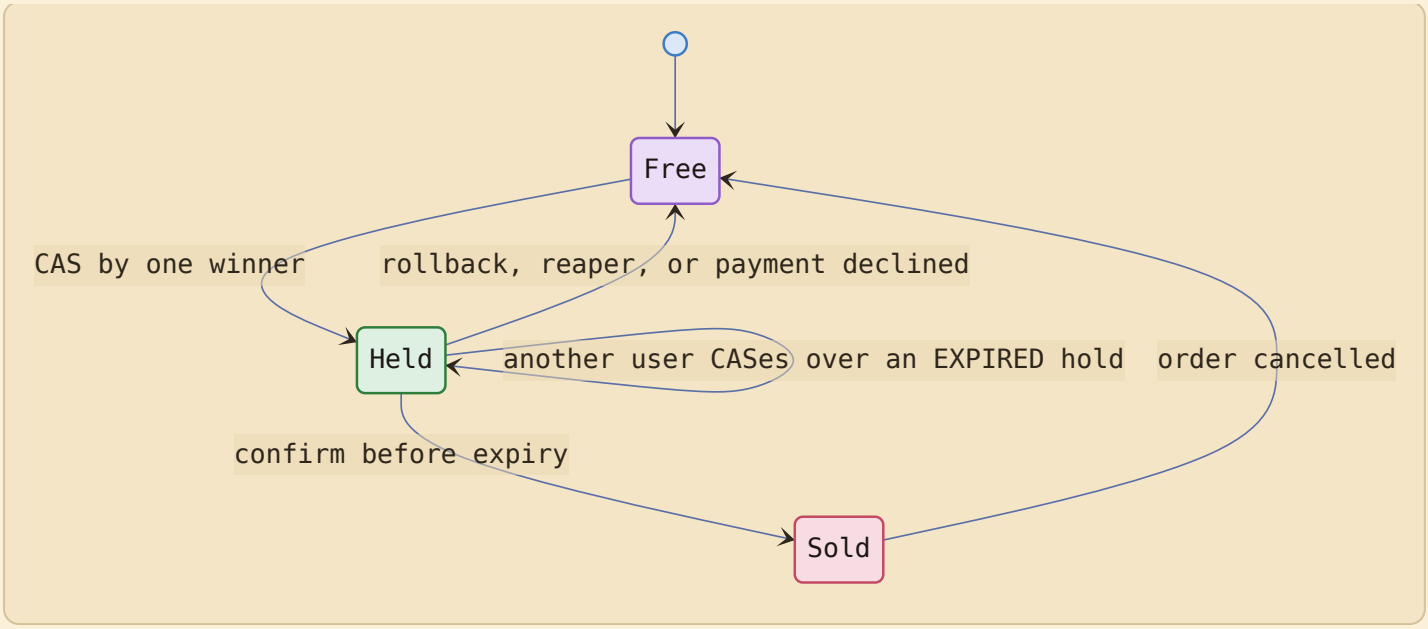
There is no course lesson for this problem; the problem statement comes from the *awesome-low-level-design* problem set. Our design:



Key flows



A numbered seat's state machine — every arrow is a single `compareAndSet`:



When inventory is released (expired hold, cancellation), `TicketingService` **offers it to the tier's waitlist first**: the next waiting fan is admitted directly and given a fresh hold, plus a notification. If they do not pay in time, the reaper releases it again and the next fan gets it.

Design decisions & patterns used

Decision	Why	Alternative rejected
Lock-free CAS per seat (`AtomicReference<SeatState>`)	Buyers of *different* seats never wait for each other; on the *same* seat, exactly one `compareAndSet` wins and the losers learn instantly. No thread is ever blocked holding a lock, so there is no deadlock.	One monitor per concert (15.2's approach) — correct, but at 50,000 simultaneous buyers every request queues behind one lock.
Immutable sealed `SeatState`	CAS compares *references* (which object it is, not what it contains); making each state a fresh immutable record means "the state I read" is exactly "the state I replace". Pattern-matching `switch` / `instanceof` on a sealed type is exhaustive and readable.	Mutable fields (`status`, `heldBy`, `expiry`) updated separately — a reader could see a half-updated seat.
Multi-seat hold in sorted order + rollback	Each seat is claimed independently; on failure we release the ones *we* took (matching our hold id). Sorted order makes two buyers of overlapping sets contend on the same first seat, so one of them fails fast.	Holding several seats with nested locks — deadlock-prone without ordering.
GA as a counter	Standing places are interchangeable — there's nothing to name. A CAS loop on an `AtomicInteger` (`if cur < qty fail; else CAS(cur, cur-qty)`) never oversells.	A `Seat` object per standing place — 40,000 objects for no benefit.
Virtual waiting room	Fairness and load control: arrival order decides who gets to *try*, and admitting in batches caps the load on inventory. Bots gain nothing by hammering "buy". `join` is `synchronized` so number order and queue order can never disagree.	"Whoever's request lands first" — rewards bots and fast connections, and melts the backend (the servers behind the app).
Per-user limit via `ConcurrentHashMap.compute` (a thread-safe map; `compute` updates one key atomically)	`compute` runs the check-and-update for one key atomically, so two parallel requests from the same user can't both slip under the limit.	`get` then `put` — a classic race.
Holds with TTL + a reaper	Seat holds are also reclaimed lazily (a CAS can overwrite an expired `Held`); GA holds have no per-item state, so a periodic `expireHolds()` returns their count. `holds.remove(id, h)` ensures the reaper and `purchase` never both process one hold.	Holds that never expire.
Waitlist offers on release	Released tickets go to the longest-waiting fan (FIFO), not to whoever refreshes fastest.	Returning tickets silently to the pool.
Compensation on lost hold	Same as 15.2: if a seat could not be confirmed after payment, release and refund.	Assuming the hold is still valid.
Singleton?	The *awesome-lld* reference solution makes the booking system a Singleton. We pass collaborators through the constructor instead (dependency injection): it is easier to test and allows one service per concert.	A global Singleton.

Java implementation

Package `lld.ch15.concert`. Files: `Model.java` (tiers, seat states, hold, ticket, interfaces, clock), `Inventory.java` (CAS seat inventory, GA pool, purchase limiter, waiting room), `TicketingService.java`, `Main.java`. The demos use Java 21 **virtual threads** (lightweight threads managed by the JVM rather than

the operating system; `Executors.newVirtualThreadPerTaskExecutor()`), which make 1,000 concurrent buyers cheap. Essential lines:

```
// Model.java
enum Tier {                                          // seated tiers have numbered seats; GA is a count
    VIP(true, "5000.00"), PREMIUM(true, "2500.00"), GENERAL_ADMISSION(false, "800.00");
    final boolean seated;
    final BigDecimal price;                          // money: BigDecimal, never double
    BigDecimal priceFor(int qty) { return price.multiply(BigDecimal.valueOf(qty)); }
}
record Concert(String id, String artist, String venue, LocalDateTime startsAt) {}

sealed interface SeatState permits Free, Held, Sold {}   // a seat's whole life in one immutable value
record Free() implements SeatState {}
record Held(String holdId, String userId, Instant expiresAt) implements SeatState {
    boolean expired(Instant now) { return !now.isBefore(expiresAt); }
}
record Sold(String holdId, String userId) implements SeatState {}

record Hold(String id, String userId, Tier tier, List<String> seatIds, int quantity, BigDecimal amount, Instant
expiresAt) {}
record Ticket(String id, String concertId, String userId, Tier tier, String seatOrGa) {}
interface PaymentGateway { boolean charge(String userId, BigDecimal amount); void refund(String userId, BigDecimal
amount); }
// SoldOutException, Notifier, SimClock as before
```

```java
// Inventory.java
final class SeatedInventory {                          // lock-free: one AtomicReference<SeatState> per seat
    private final Map<String, AtomicReference<SeatState>> seats = new ConcurrentHashMap<>();

    /** All-or-nothing: claim in sorted order, roll back our own claims on failure. */
    boolean tryHold(List<String> ids, String holdId, String userId, Instant now, Instant expiresAt) {
        List<String> taken = new ArrayList<>();
        for (String id : ids.stream().sorted().toList()) {
            if (!tryHoldOne(id, holdId, userId, now, expiresAt)) { taken.forEach(t -> release(t, holdId)); return
false; }
            taken.add(id);
        }
        return true;
    }
    private boolean tryHoldOne(String id, String holdId, String userId, Instant now, Instant expiresAt) {
        AtomicReference<SeatState> ref = seats.get(id);
        while (true) {
            SeatState cur = ref.get();
            boolean claimable = cur instanceof Free || (cur instanceof Held h && h.expired(now));
            if (!claimable) return false;
            if (ref.compareAndSet(cur, new Held(holdId, userId, expiresAt))) return true;
            // CAS lost: someone changed the seat between get() and CAS. Re-read and decide again.
        }
    }
    boolean confirm(String id, String holdId, Instant now) {   // Held-by-this-hold, not expired -> Sold
        AtomicReference<SeatState> ref = seats.get(id);
        SeatState cur = ref.get();
        return cur instanceof Held h && h.holdId().equals(holdId) && !h.expired(now)
                && ref.compareAndSet(cur, new Sold(holdId, h.userId()));
    }
    boolean release(String id, String holdId) {      // back to Free only if still ours (held or sold by this
hold)
        AtomicReference<SeatState> ref = seats.get(id);
        SeatState cur = ref.get();
        boolean ours = (cur instanceof Held h && h.holdId().equals(holdId))
                || (cur instanceof Sold s && s.holdId().equals(holdId));
        return ours && ref.compareAndSet(cur, new Free());
    }
    // freeSeats(tier, now): Free or expired-Held seats of a tier, in seat order
}

final class GaPool {                                   // a counter; the CAS loop can never go below zero
    private final AtomicInteger available;
    boolean tryTake(int qty) {
        while (true) {
            int cur = available.get();
            if (cur < qty) return false;
            if (available.compareAndSet(cur, cur - qty)) return true;
        }
    }
    void giveBack(int qty) { available.addAndGet(qty); }
}

final class PurchaseLimiter {                          // anti-scalping: at most `max` per user, atomically
    private final ConcurrentHashMap<String, Integer> used = new ConcurrentHashMap<>();
    boolean tryUse(String userId, int qty) {
        boolean[] ok = {false};
        used.compute(userId, (u, cur) -> {
            int c = cur == null ? 0 : cur;
            if (c + qty > max) return cur;             // unchanged
            ok[0] = true;
            return c + qty;
        });
        return ok[0];
    }
    void giveBack(String userId, int qty) { used.computeIfPresent(userId, (u, c) -> c - qty <= 0 ? null : c -
qty); }
}

final class VirtualWaitingRoom {                       // strictly increasing numbers; admitted in order, in batches
```

```
    synchronized long join(String userId) {          // synchronized: number order == line order
        Long n = numberOf.get(userId);
        if (n != null) return n;                     // joining twice keeps your place
        long mine = nextNumber.incrementAndGet();
        numberOf.put(userId, mine);
        line.add(userId);
        return mine;
    }
    synchronized List<String> admitNext(int k) { /* poll up to k from the line into `admitted` */ }
    boolean isAdmitted(String userId) { return admitted.contains(userId); }
    void admitDirectly(String userId) { admitted.add(userId); }      // used for waitlist offers
}
```

```java
// TicketingService.java - join room -> admitted -> HOLD (seats or GA qty, TTL) -> PURCHASE;
// released inventory is offered to the tier's waitlist first
final class TicketingService {
    private final Map<String, Hold> holds = new ConcurrentHashMap<>();           // active holds
    private final Map<Tier, Queue<String>> waitlists = new EnumMap<>(Tier.class);

    Hold holdSeats(String userId, List<String> seatIds) {
        requireAdmitted(userId);                    // "... is still in the waiting room"
        // ... one hold = one tier
        if (!limiter.tryUse(userId, seatIds.size()))
            throw new IllegalStateException(userId + " would exceed the per-user ticket limit");
        Instant now = clock.instant();
        Hold h = new Hold("H" + seq.incrementAndGet(), userId, tier, List.copyOf(seatIds), seatIds.size(),
                tier.priceFor(seatIds.size()), now.plus(holdTtl));
        if (!seated.tryHold(seatIds, h.id(), userId, now, h.expiresAt())) {
            limiter.giveBack(userId, seatIds.size());
            throw new SoldOutException(seatIds + " not available");
        }
        holds.put(h.id(), h);
        return h;
    }
    // holdGa(userId, qty): same shape with ga.tryTake(qty) -> "general admission sold out"

    List<Ticket> purchase(String holdId) {
        Hold h = holds.remove(holdId);              // atomic claim: purchased at most once
        if (h == null) throw new IllegalStateException(holdId + " unknown, expired or already used");
        if (h.expired(clock.instant())) { releaseInventory(h); throw new IllegalStateException(holdId + "
expired"); }
        if (!payments.charge(h.userId(), h.amount())) { releaseInventory(h); throw new
IllegalStateException("payment declined"); }
        // seated: confirm each seat; if one fails (hold lost during payment) -> release all, give back the
        //         limit, refund, throw "seat ... lost during payment, refunded". GA: one ticket per unit.
    }

    int expireHolds() {                             // the reaper: every few seconds in production
        Instant now = clock.instant();
        int n = 0;
        for (Hold h : holds.values())
            if (h.expired(now) && holds.remove(h.id(), h)) { releaseInventory(h); n++; }
        return n;
    }
    // cancelOrder(holdId): refund the order, then releaseInventory as if held

    private void releaseInventory(Hold h) {
        limiter.giveBack(h.userId(), h.quantity());
        if (h.tier().seated) {
            h.seatIds().forEach(id -> seated.release(id, h.id()));
            h.seatIds().forEach(id -> offerSeat(h.tier(), id));
        } else {
            ga.giveBack(h.quantity());
            offerGa();
        }
    }
    private void offerGa() {                        // waitlist first: a fresh hold for the longest-waiting fan
        Queue<String> q = waitlists.get(Tier.GENERAL_ADMISSION);
        String next;
        while (ga.available() > 0 && (next = q.poll()) != null) {
            room.admitDirectly(next);
            try {
                Hold h = holdGa(next, 1);
                notifier.send(next, "a GA ticket freed up, held for you as " + h.id() + " until " +
h.expiresAt());
            } catch (RuntimeException e) { /* over their limit or lost the race: try the next person */ }
        }
    }
    // offerSeat(tier, seatId): same idea for one seat; availability(): free seats per tier + GA count
}
// Main.java: VIP-1..4, P-1..6, GA 100; limit 4; TTL 10 min; payment gateway sums collected money;
// 40 virtual threads race for VIP-1; 1,000 virtual threads buy GA.
```

Output:

```
== 1. Seating and availability, then the virtual waiting room ==
  availability: VIP free [VIP-1, VIP-2, VIP-3, VIP-4], PREMIUM free [P-1, P-2, P-3, P-4, P-5, P-6],
GENERAL_ADMISSION free 100
  ana got number 1
  ben got number 2
  cai got number 3
  dev got number 4
  eli got number 5
  ben joins again, keeps number 2
  admitted batch 1: [ana, ben, cai]
  REFUSED: eli is still in the waiting room

== 2. 40 admitted fans CAS for the same seat VIP-1 ==
  holds on VIP-1 = 1

== 3. Flash sale: 1,000 fans, 100 GA places (98 left after test buys) ==
  sold=98 soldOut=902 gaLeft=0 (never negative)

== 4. Per-user limit (4) stops a scalper ==
  ben holds [P-1, P-2, P-3] = 7500.00
  REFUSED: ben would exceed the per-user ticket limit
  ben bought [P-1, P-2, P-3]

== 5. Sold out -> waitlist -> a cancellation is offered to the waitlist ==
  cai: general admission sold out, joins waitlist
  [notify cai] a GA ticket freed up, held for you as H244 until 2026-06-01T09:10:00Z
  GA left after offers = 1

== 6. Unpaid holds expire; the reaper returns them ==
  admitted batch 2: [dev, eli]
  expired holds reaped = 3 (VIP-1, cai's unpaid offer, dev's P-6)
  dev: H245 unknown, expired or already used
  P-6 now Free[], GA left 2
  availability: VIP free [VIP-1, VIP-2, VIP-3, VIP-4], PREMIUM free [P-4, P-5, P-6], GENERAL_ADMISSION free 2
  money collected = 85900.00
```

What to notice: section 1 admits the first three in arrival order and refuses someone still in the queue; section 2 — 40 concurrent CAS attempts on one seat, exactly one hold; section 3 — 1,000 buyers for 98 remaining GA places sell exactly 98 and the counter stops at 0; section 4 — a fifth ticket is refused by the per-user limit; section 5 — a cancellation's released ticket goes straight to the waitlisted fan as a hold; section 6 — the reaper returns three expired holds (VIP-1, the unpaid waitlist offer, and dev's seat) and a late purchase is refused. The availability line at the end shows what is still for sale (three PREMIUM seats, four VIP seats, two GA places). Money collected is 85,900.00 = 98 × 800 + 3 × 2,500 (ana's 2 GA tickets were refunded).

Concurrency & edge cases

- **Why CAS is correct.** `compareAndSet(expected, new)` atomically replaces the value only if it is still the exact object we read. If another thread changed it in between, the CAS fails and we re-read and decide again. The loop in `tryHoldOne` only retries while the seat is still claimable; otherwise it returns `false`.
- **ABA problem?** ABA is when a value goes A → B → A and a CAS wrongly succeeds. Here every transition creates a *new* record object and `AtomicReference` compares references, so an "A" that came back is a different object and the CAS fails safely.
- **Hold claimed exactly once.** `purchase` starts with `holds.remove(holdId)`; the reaper uses `holds.remove(id, h)`. Only one of them can get the hold, so inventory is never released twice.
- **Per-user limit and failures.** Every failure path (sold out, declined, expired, lost seat) gives the limit back, so a user is never locked out by a failed attempt.
- **Lost hold after payment.** `confirm` fails if the hold expired and someone CASed over it; we release, give back the limit, and refund.
- **Fair queue scope.** The in-memory queue is fair *within one process*. At real scale the waiting room is a separate service (often at the CDN edge — a CDN, content delivery network, is a worldwide set of

servers placed close to users; see System Design Fundamentals) that hands out signed, time-limited access tokens (short strings the server can verify it issued); inventory servers only check the token.

- **Across servers.** Seat CAS maps naturally to a database conditional update (`UPDATE seat SET state='HELD', hold_id=?, expires_at=? WHERE id=? AND (state='FREE' OR expires_at < now())`) or Redis `SET NX PX`. GA counts map to Redis `DECRBY` (decrement by an amount) guarded by a Lua script (a small script Redis runs as one atomic step), or to a database row updated with `WHERE available >= ?`.

Extensions & follow-up questions

- **"Best available seats."** Instead of the user choosing, a `SeatAllocator` strategy picks the best adjacent free seats in a tier, then calls the same CAS hold.
- **"Verified fans / presale codes."** Admission to the waiting room requires a token issued to pre-registered fans; the limiter keys on a verified identity, not an email.
- **"Dynamic pricing."** Prices per tier come from a `PricingStrategy` that looks at remaining inventory and time — decided when the hold is created and frozen in the `Hold`.
- **"Resale marketplace."** A sold ticket can be listed; a buyer's purchase transfers ownership and re-issues a new barcode (so the old one stops working).
- **"Waitlist for seated tiers."** Implemented the same way for each released seat (`offerSeat`).

Real-world example

Ticketmaster uses a virtual queue ("Smart Queue") for high-demand on-sales and a **Verified Fan** presale programme that registers fans in advance to filter out bots; purchase limits per account are standard on its events. The November 2022 presale for Taylor Swift's *Eras Tour* is the well-known stress case: demand far beyond what was planned for overwhelmed the system, many fans waited for hours or were dropped, and the general public sale was cancelled — a real lesson that fairness and admission control are as important as seat-locking correctness.

In the interview

30–45 minute plan. (1) Say up front: "the entities are like movie booking; the hard part is contention and fairness." (2) Tiers: seated vs GA, and why GA is a counter. (3) Holds with TTL; code the CAS seat claim and the GA CAS loop. (4) Fairness: waiting room with batches; per-user limit with atomic `compute`. (5) Waitlist on release; reaper. (6) Scale-out: tokens from the waiting room, conditional updates/Redis.

> **30-second answer:** "Seated tiers are an `AtomicReference<SeatState>` per seat, claimed by compare-and-set — one winner, no locks. GA is an `AtomicInteger` decremented by a CAS loop so it can't go negative. Everyone enters through a virtual waiting room that admits in arrival order, and a per-user limit is enforced atomically. Holds expire after 10 minutes; a reaper returns expired GA holds; released tickets go to a FIFO waitlist first. If a hold is lost during payment we refund."

Follow-ups with answers: *Lock-free vs locks?* Lock-free wins when contention is high and critical sections are tiny (one field swap); locks are simpler when an operation must update many things together. *What if the queue service fails?* Fail closed (on failure, deny rather than allow): pause admissions, don't open the floodgates. *How do you stop one person with 50 accounts?* Verified identity, payment-card fingerprinting (recognising the same card across accounts), and limits keyed on those.

TL;DR

Same nouns as movie booking, different enemy: contention. CAS per seat, CAS counter for GA, a FIFO virtual waiting room for fairness, atomic per-user limits, TTL holds with a reaper, a waitlist that gets released tickets first, and refunds when a hold is lost.

15.4 Design a Hotel Management System



Figure — Cover image for the hotel management case study (course diagram)

Problem & clarifying questions

A **Hotel Management System** (often called a *property management system*, PMS) runs a hotel's operations in one place: it knows every room, takes bookings, checks guests in and out, issues key cards, records housekeeping and room service, and produces the bill. Management gets one portal to track available rooms, book them and generate invoices.

The inventory here is different from a cinema: a room is not sold once, it is sold **night by night**. That single fact shapes the whole design.

Question	Answer we assume
Several hotel locations?	Yes — a `Hotel` has many `HotelLocation`s, each with its own rooms.
Room types?	`STANDARD`, `DELUXE`, `FAMILY_SUITE`, `BUSINESS_SUITE`.
What does a guest search by?	Room type and **date range** (availability).
Cancellation policy?	**Full refund if cancelled more than 24 hours before check-in**; otherwise no refund (our assumption for the "otherwise" case).
Extra charges during the stay?	Room service, food (kitchen) and amenities are added to the bill.
How is the bill paid?	**Credit card, check or cash**.
Keys?	Electronic key cards with a barcode and key id; deactivated at check-out.
Housekeeping?	Every task is logged per room (a history, never overwritten).
One booking for several rooms?	No — one booking = one room (a known limitation; see follow-ups).

Requirements

Functional (course list)

1. Support booking different **room types**: standard, deluxe, family suite (and business suite).

2. Guests **search** room inventory and book any **available** room.
3. Retrieve **who booked a room** and **which rooms a customer booked**.
4. Customers can **cancel**; **full refund if more than 24 hours before check-in**.
5. **Notify** the guest shortly before check-in and shortly before check-out.
6. Keep a **housekeeping log** per room.
7. Customers can add **room services and food items**.
8. Customers can ask for **amenities**.
9. Pay bills by **credit card, check or cash**.

Non-functional — never double-book a room-night (even with concurrent bookings from the website and the front desk); the bill must always equal the sum of its lines; each actor may do only their own job; the refund rule must be changeable in one place.

Use cases

Actors (course): **Guest** (search, book), **Receptionist** (add/modify rooms, create bookings, check in/out), **System** (booking/cancellation/reminder notifications), **Manager** (adds new workers), **Housekeeper** (adds/modifies housekeeping records), **Server** (adds/modifies room-service records).

Top use cases: add/remove/edit room; search room (by type and availability); register or cancel an account; book room; **check in**; **check out** (end of booking, return of keys); add room charge; update housekeeping log.

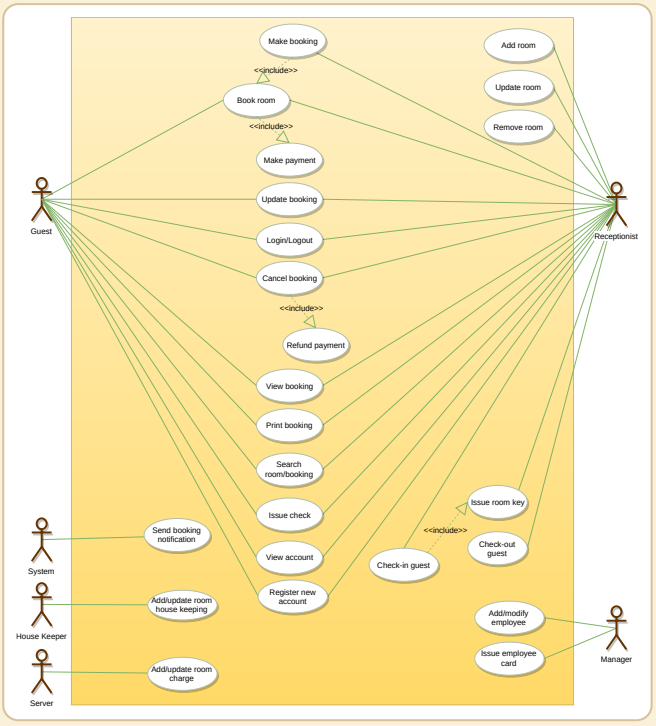


Figure — Hotel use cases: Guest and Receptionist share booking, update, cancel, view, print and search; "Make booking" includes "Book room", which includes "Make payment"; "Cancel booking" includes "Refund payment"; the receptionist's "Check-in guest" includes "Issue room key"; the System sends booking notifications; the House Keeper adds/updates housekeeping; a manager adds/modifies employees (course diagram)

Core entities

Class	Responsibility
`Hotel`	Owns its locations; searches each one in turn.
`HotelLocation` implements `Search`	Owns its rooms; answers "which rooms of this type are free for these dates?".
`Room`	Number, style, price per night; the **set of booked nights**; owns its `RoomKey`s and housekeeping log; current `RoomStatus`.
`RoomStyle`, `RoomStatus` (enums)	Type of room; what it is doing *now* (`AVAILABLE`, `RESERVED`, `OCCUPIED`, `NOT_AVAILABLE`, `BEING_SERVICED`, `OTHER`).
`RoomKey`	Key id, barcode, issue time, master flag, active flag.
`RoomHouseKeeping` (record)	One log line: when, who, what, how long.
`RoomBooking`	Reservation number, guest id, room, check-in/out dates, status, deposit; **owns its** `Invoice`; `refundDue()` holds the cancellation rule.
`BookingStatus` (enum)	`REQUESTED`, `PENDING`, `CONFIRMED`, `CHECKED_IN`, `CHECKED_OUT`, `CANCELLED`, `ABANDONED`.
`Invoice`, `InvoiceItem`	A list of lines; the total is computed from them.
`RoomCharge` (abstract) → `Amenity`, `RoomService`, `KitchenService`	A charge that knows how to write itself onto the invoice (Template Method: the parent fixes the steps, subclasses fill in one).
`Payment` (sealed, abstract) → `CreditCardTransaction`, `CheckTransaction`, `CashTransaction`	Three ways to pay with genuinely different behaviour.
`Person` (sealed) → `Guest`, `Receptionist`, `Server`, `Housekeeper`, `Manager`	Actors; service methods accept only the right actor type.
`FrontDesk`	Every flow of a stay: book, check in, add charge, check out, cancel, housekeeping, reminders, queries.
`Notifier`	Messages to guests.

Class diagram

The course's five stages:

Stage	Adds	What works afterwards
1	Invoice, payment types, notification types	A bill can be produced and settled by card, check or cash
2	Room, key card, housekeeping record	Each room can tell which of its nights are sold
3	Hotel, location, search interface	Free rooms can be listed for any date range
4	Booking and room charges	A stay can be followed from reservation to check-out
5	People and accounts	Every actor is limited to its own tasks

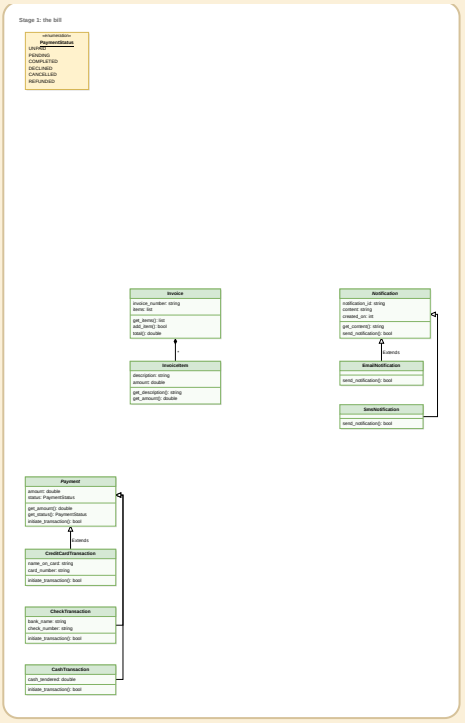


Figure — Stage 1: `Invoice` owns its `InvoiceItem`s (composition) and computes `total()`; abstract `Payment` with credit card, check and cash subclasses; abstract `Notification` with one subclass per channel (course diagram)

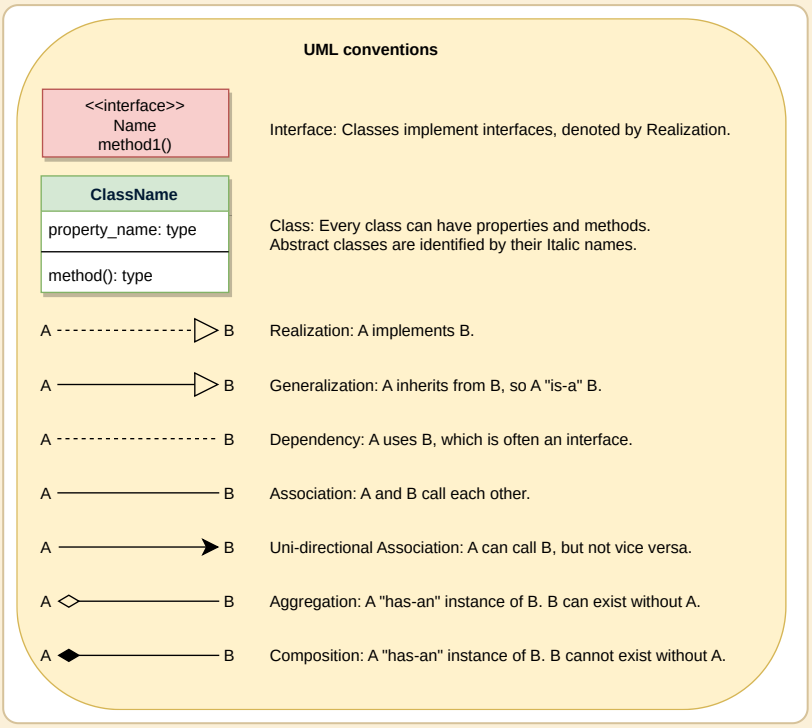


Figure — How to read a class box and the arrow styles used in these diagrams (course diagram)

Requirements 7 and 8 each put extra charges on a stay. That is why the invoice stores **individual lines** and computes its total on demand: a total that is always recomputed can never go out of step with the lines.

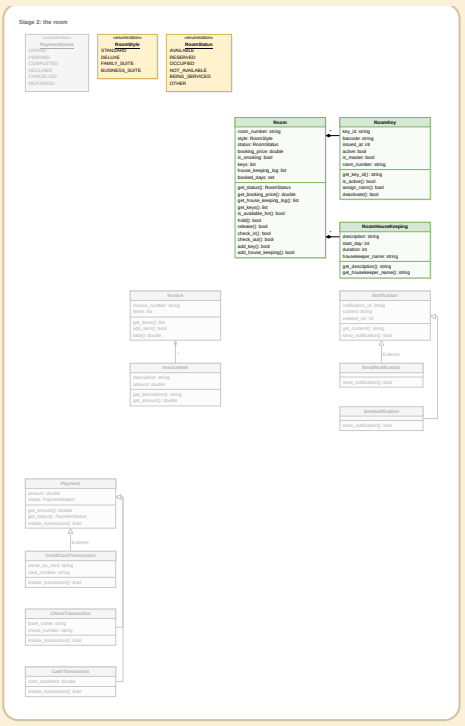


Figure — Stage 2: `Room` (number, style, status, price, and the set of nights it is committed to) owns its `RoomKey`s and its list of `RoomHouseKeeping` records (course diagram)

The key decision of the whole design sits here: **a status field is not availability.** `RoomStatus.OCCUPIED` describes the present moment only. A guest, however, asks about the future ("can I have it from the 10th to the 13th?"), and a single status value cannot answer that. So each room also stores the dates of the nights it has already sold.

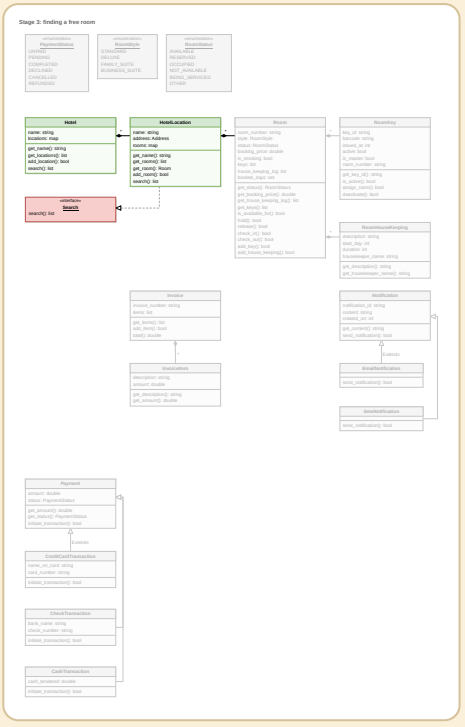


Figure — Stage 3: the `Search` interface implemented by `HotelLocation`, which owns its rooms; `Hotel` owns its locations (course diagram)

Search sits on the **location**, not on `Room`: one room has no knowledge of the other rooms, so it cannot list which of them are free. Putting `search` on `Room` is a frequent beginner mistake.

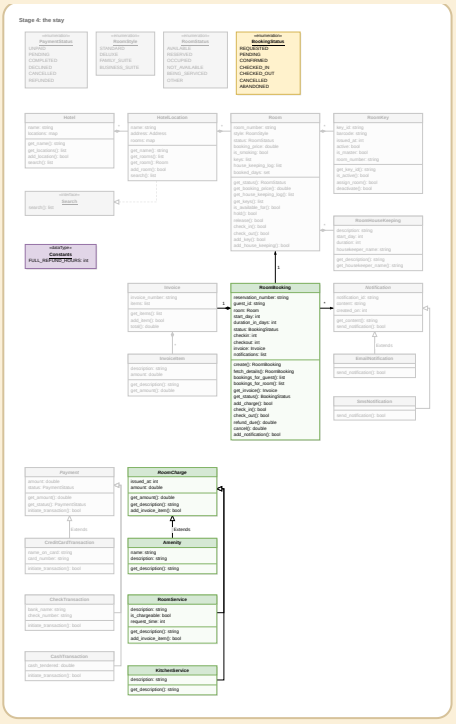


Figure — Stage 4: `RoomBooking` owns its invoice and points at its room (plain association); `RoomCharge` with `Amenity`, `RoomService` and `KitchenService` subclasses each writing an invoice line (course diagram)

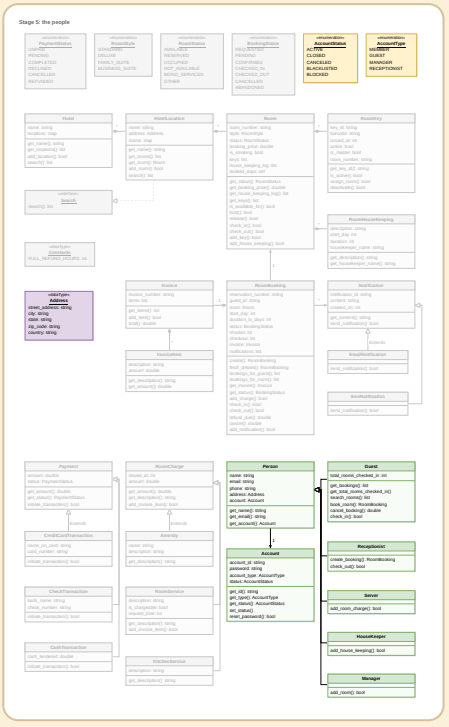


Figure — Stage 5: `Person` with `Guest`, `Receptionist`, `Server`, `HouseKeeper` and `Manager`, each with only its own methods; `Person` refers to its `Account`; there is deliberately no arrow from guest to booking (course diagram)

Notice what the figure does *not* have: an arrow between guest and booking. The booking stores only the guest's **id**, and queries filter bookings by that id — requirement 3 is met without either class holding a reference to the other.

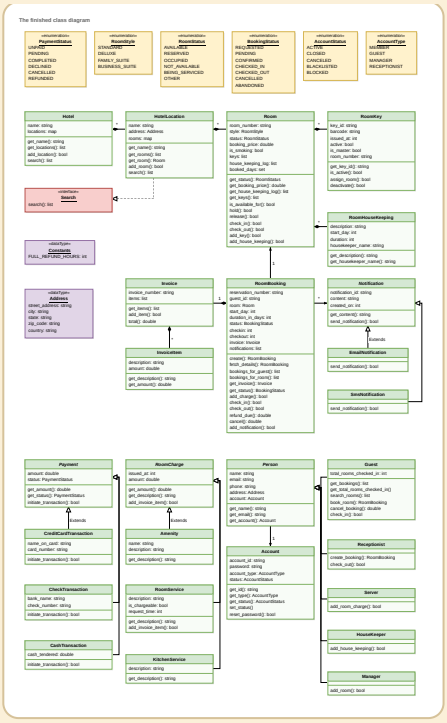
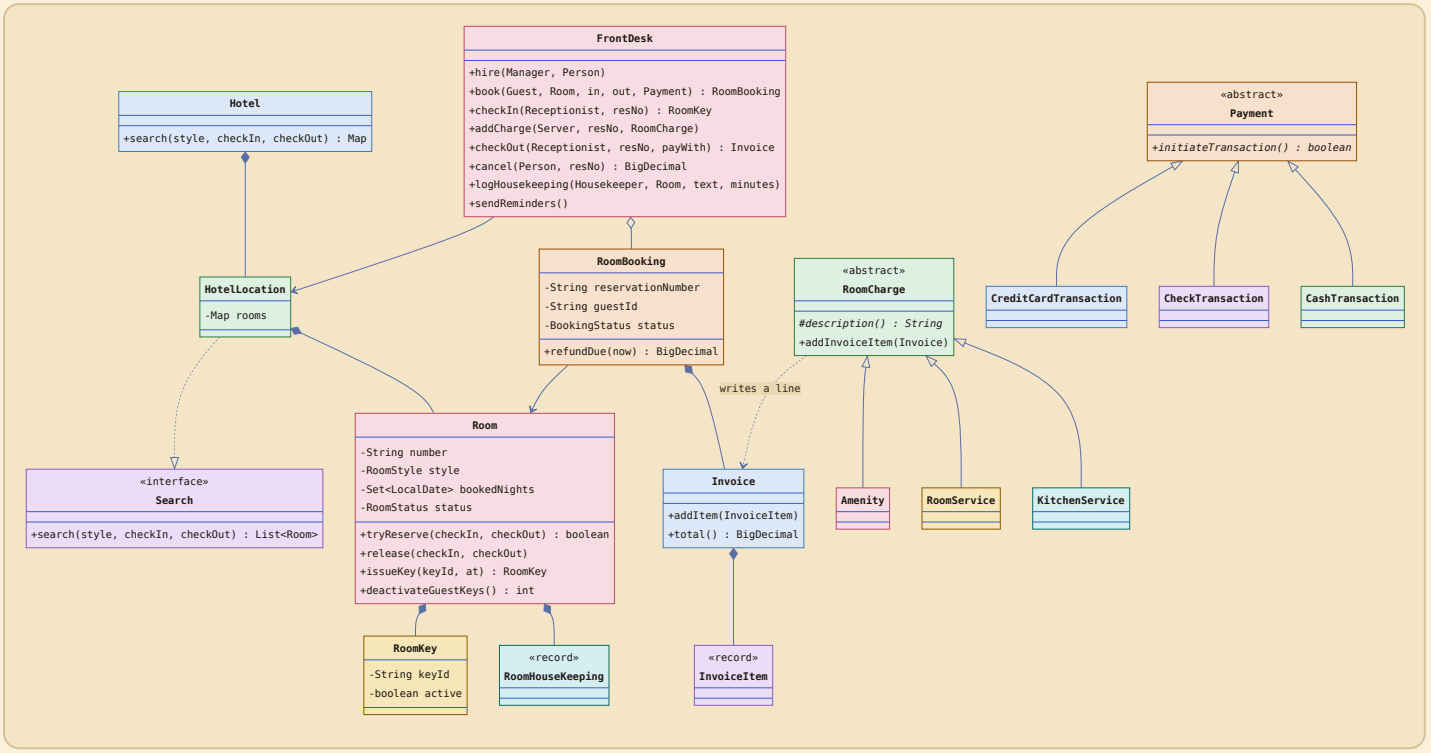


Figure — The finished hotel class diagram (course diagram)

Our Java design:



Key flows

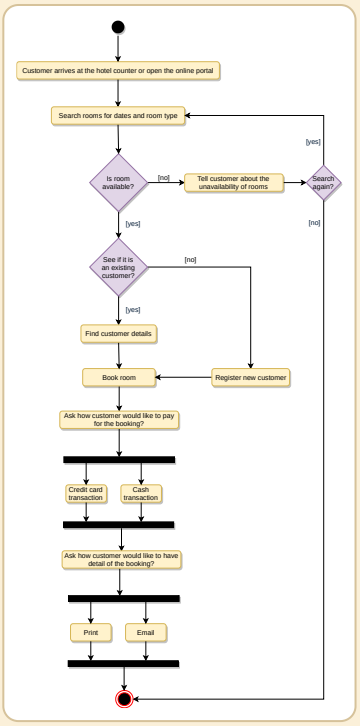


Figure — Make a room booking: search rooms for dates and room type; if none is available tell the customer; otherwise find or register the customer, book the room, ask how they will pay (credit card or cash), and how they want the booking details (course diagram)

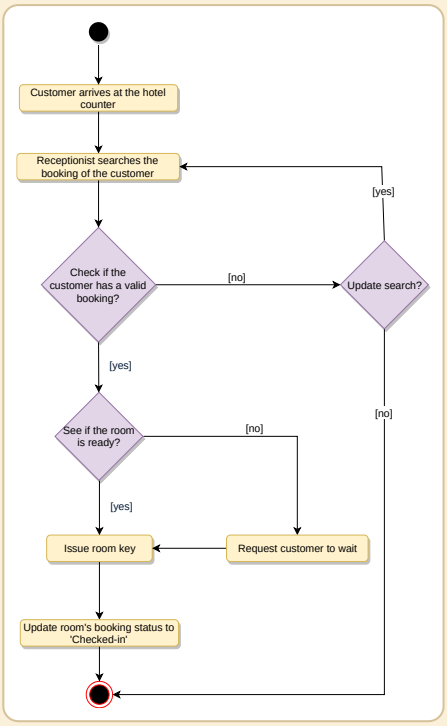


Figure — Check in: the customer arrives at the counter and the receptionist looks up the booking (searching again if it is not found); with a valid booking, check whether the room is ready — if not, the customer is asked to wait; if so, issue the room key and set the booking status to "Checked-in" (course diagram)

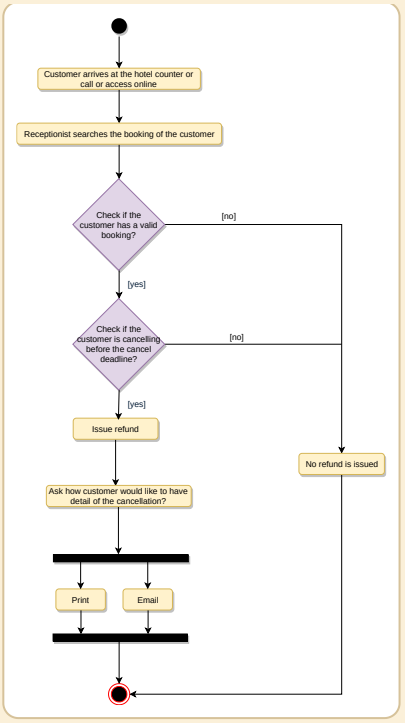
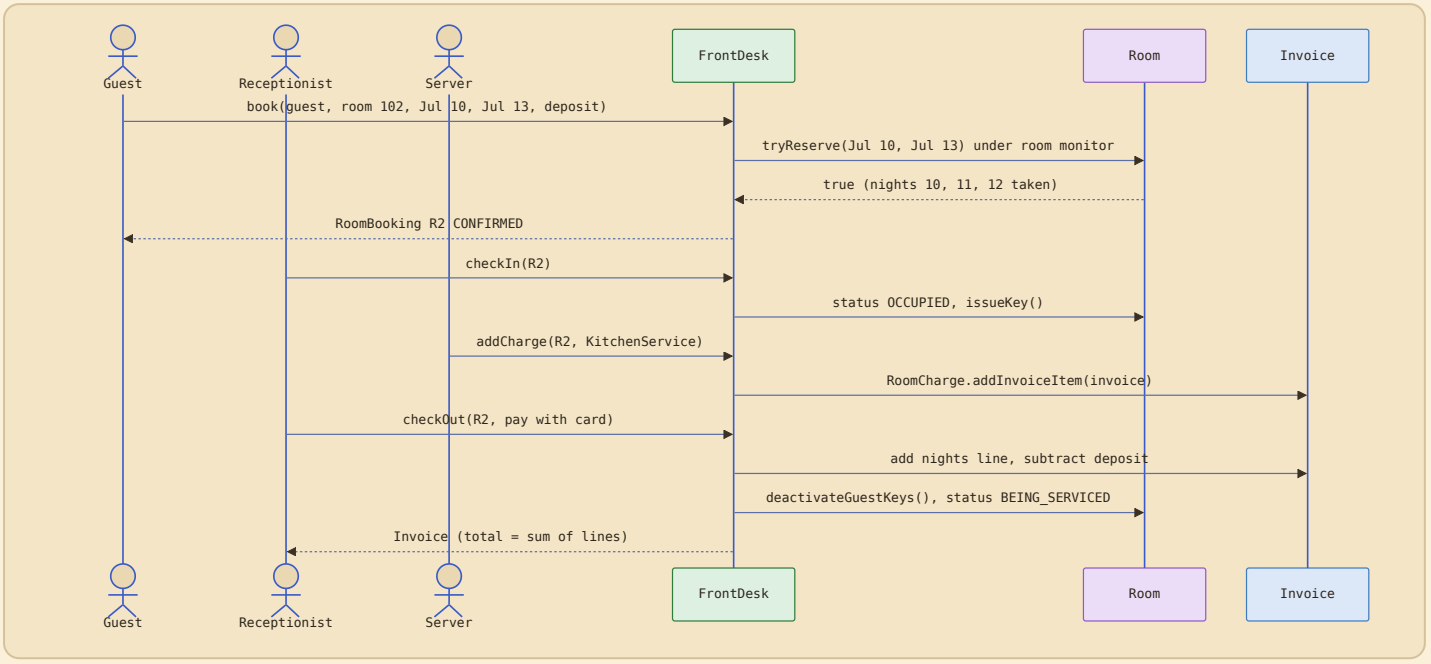
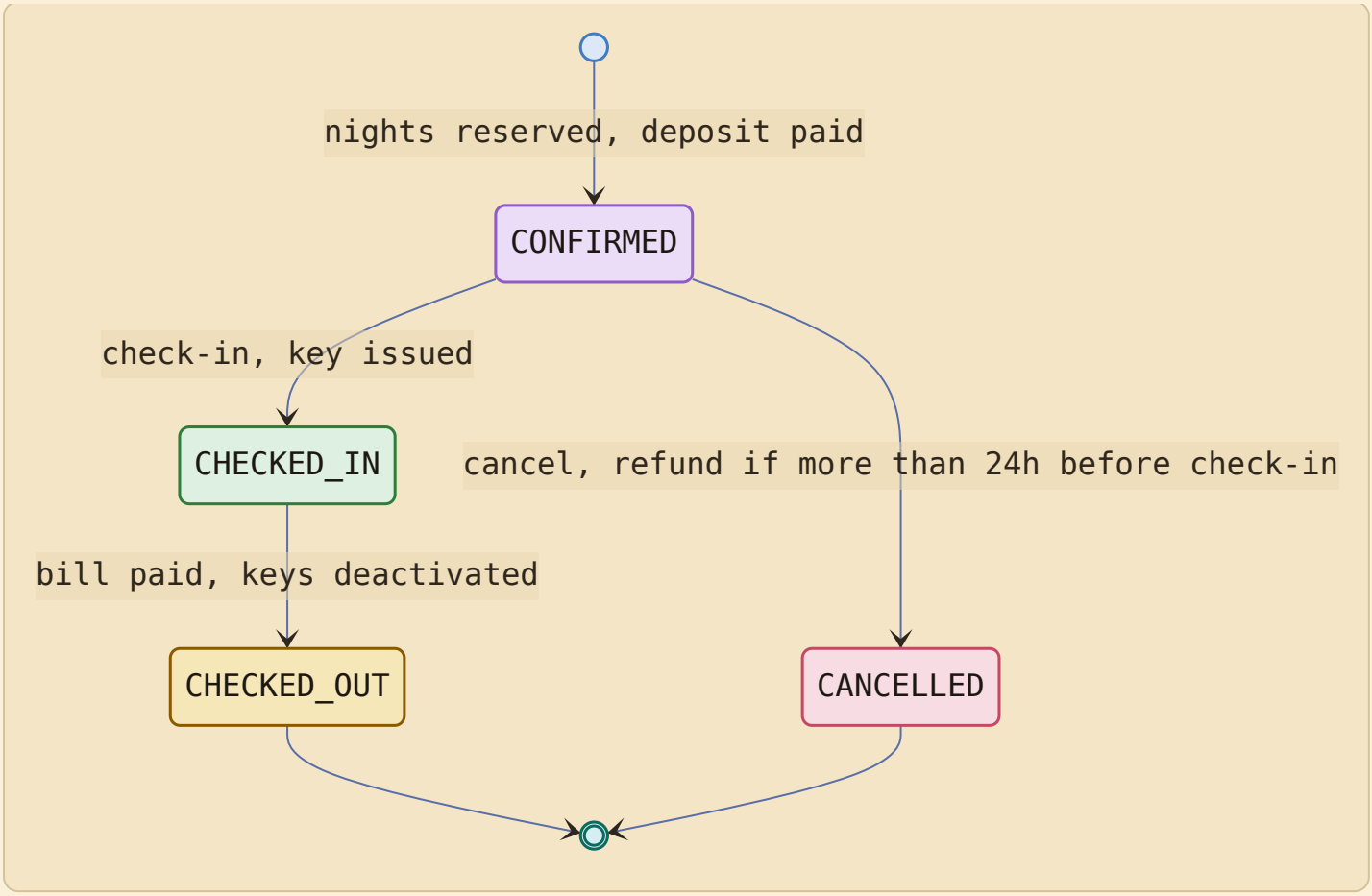


Figure — Cancel a booking: the customer comes to the counter, calls or goes online; the receptionist finds the booking and checks it is valid; if the customer is cancelling before the cancel deadline a refund is issued, otherwise no refund; the cancellation details are printed or emailed (course diagram)

A whole stay as a sequence:



Booking lifecycle:



The room's *current* status cycles independently: `AVAILABLE` → `OCCUPIED` (check-in) → `BEING_SERVICED` (check-out) → `AVAILABLE` (housekeeping logged).

Design decisions & patterns used

Decision	Why	Alternative rejected
Availability = set of booked nights (course's key point)	Answers date-range questions; nights are half-open `[checkIn, checkOut)`, so a guest leaving on the 13th and one arriving on the 13th don't clash.	Using `RoomStatus` for availability — only knows *now*.
`tryReserve` is `synchronized` on the room	"Are all these nights free? → take them" is check-then-act; the room's monitor makes it atomic. Rooms don't block each other.	One hotel-wide lock.
Search on `HotelLocation`	A room can't answer "which rooms are free?".	`Room.search()`.
Invoice = list of lines, total computed	The total can never disagree with the lines (requirements 7 and 8).	A stored running total.
Template Method: `RoomCharge.addInvoiceItem`	The parent fixes the algorithm (describe → price → write a line) and is `final`; subclasses fill in only `description()`. A new kind of charge = a new subclass, no edit to `Invoice` (Open/Closed). The course calls this the clearest Template Method example in its catalogue. Compare with Strategy, where the *whole* method is swapped rather than one step.	A `switch` on charge type inside the invoice.
Three `Payment` subclasses	They really behave differently: a card completes immediately, a **check stays `PENDING`** until it clears, cash can be declined if the amount tendered is short. Each passes the substitution test (each can be asked to take money).	One `Payment` with a `type` field and `if`s.
Refund rule in one method (`refundDue`)	When the policy changes there is exactly one place to edit. To support several policies (non-refundable rates, partial refunds), promote it to a `CancellationPolicy` strategy as in 15.2.	Refund arithmetic scattered through cancel flows.
Actor types in service signatures	`addCharge(Server, …)`, `logHousekeeping(Housekeeper, …)`, `addRoom(Manager, …)` — the compiler enforces who can do what. The receptionist booking for a guest calls the *same* booking method — two actors, one behaviour, no duplication.	Every actor able to call everything.
Booking stores guest id, not a `Guest` reference	No two-way dependency; "which rooms did this guest book?" is a query.	Bidirectional links that must be kept in sync.
Nights released on failed deposit / cancel	Reserve-then-pay needs an undo: if the deposit fails the nights are given back immediately.	Leaving phantom reservations.

Java implementation

Package `lld.ch15.hotel`. Files: `Billing.java` (invoice, payments, room charges), `Rooms.java` (room, key, housekeeping, search, hotel, clock, money), `Stay.java` (booking, actors, front desk), `Main.java`. Essential lines:

```java
// Billing.java
enum PaymentStatus { UNPAID, PENDING, COMPLETED, DECLINED, CANCELLED, REFUNDED }
record InvoiceItem(String description, BigDecimal amount) {}   // money: BigDecimal, never double

final class Invoice {                              // a list of lines; the total is always computed
    synchronized void addItem(InvoiceItem i) { items.add(i); }
    synchronized BigDecimal total() { return items.stream().map(InvoiceItem::amount).reduce(BigDecimal.ZERO,
BigDecimal::add); }
}

abstract sealed class Payment permits CreditCardTransaction, CheckTransaction, CashTransaction {
    /** Take the money. Returns true when the hotel may release the guest. */
    abstract boolean initiateTransaction();
}
final class CreditCardTransaction extends Payment {
    @Override boolean initiateTransaction() { setStatus(PaymentStatus.COMPLETED); return true; }
}
final class CheckTransaction extends Payment {      // bank name, check number
    @Override boolean initiateTransaction() { setStatus(PaymentStatus.PENDING); return true; }   // clears in days
}
final class CashTransaction extends Payment {
    @Override boolean initiateTransaction() {
        if (tendered.compareTo(amount()) < 0) { setStatus(PaymentStatus.DECLINED); return false; }
        setStatus(PaymentStatus.COMPLETED);
        return true;
    }
}

/** TEMPLATE METHOD: addInvoiceItem() is the fixed algorithm; subclasses fill in only description(). */
abstract class RoomCharge {
    private final BigDecimal amount;
    protected abstract String description();                // the one step that varies
    final void addInvoiceItem(Invoice invoice) {            // final, so the order can't change
        invoice.addItem(new InvoiceItem(description(), amount));
    }
}
final class Amenity extends RoomCharge { @Override protected String description() { return "Amenity: " + name; } }
// RoomService -> "Room service: " + request; KitchenService -> "Kitchen: " + dish
```

```java
// Rooms.java
enum RoomStyle { STANDARD, DELUXE, FAMILY_SUITE, BUSINESS_SUITE }
enum RoomStatus { AVAILABLE, RESERVED, OCCUPIED, NOT_AVAILABLE, BEING_SERVICED, OTHER }   // what it does NOW
final class RoomKey { /* keyId, barcode, issuedAt, master flag, volatile active; deactivate() at checkout */ }
record RoomHouseKeeping(LocalDateTime at, String housekeeper, String description, int minutes) {}   // append-only

/** Owns its keys and housekeeping log (composition) and the SET OF NIGHTS it is committed to. */
final class Room {
    private final Set<LocalDate> bookedNights = new HashSet<>();
    private RoomStatus status = RoomStatus.AVAILABLE;

    /** Nights are [checkIn, checkOut): the checkout day itself is free for the next guest. */
    synchronized boolean isFree(LocalDate checkIn, LocalDate checkOut) {
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) if (bookedNights.contains(d)) return
false;
        return true;
    }
    synchronized boolean tryReserve(LocalDate checkIn, LocalDate checkOut) {   // check-then-act, atomic
        if (!isFree(checkIn, checkOut)) return false;              // re-entrant monitor: same lock
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) bookedNights.add(d);
        return true;
    }
    synchronized void release(LocalDate checkIn, LocalDate checkOut) { /* remove those nights */ }
    synchronized int deactivateGuestKeys() { /* deactivate every active non-master key, return how many */ }
    // issueKey(keyId, at), addHousekeeping(h), housekeepingLog()
}

interface Search { List<Room> search(RoomStyle style, LocalDate checkIn, LocalDate checkOut); }   // on the
LOCATION
final class HotelLocation implements Search {
    @Override public List<Room> search(RoomStyle style, LocalDate in, LocalDate out) {
        return rooms.values().stream().filter(r -> r.style() == style && r.isFree(in, out))
                .sorted(Comparator.comparing(Room::number)).toList();
    }
}
// Hotel.search(style, in, out): asks every location in turn (TreeMap location -> rooms)
// Money.fmt prints negatives as "-$120.00"
```

```java
// Stay.java
enum BookingStatus { REQUESTED, PENDING, CONFIRMED, CHECKED_IN, CHECKED_OUT, CANCELLED, ABANDONED }
sealed interface Person permits Guest, Receptionist, Server, Housekeeper, Manager { String name(); }
record Guest(String id, String name, String email) implements Person {}     // others: (id, name)

final class RoomBooking {                          // owns its invoice; points at the room; guest by id
    static final LocalTime CHECK_IN_TIME = LocalTime.of(14, 0);
    static final Duration FULL_REFUND_NOTICE = Duration.ofHours(24);

    /** The refund rule lives in exactly ONE method. */
    BigDecimal refundDue(LocalDateTime now) {
        Duration notice = Duration.between(now, checkIn.atTime(CHECK_IN_TIME));
        // compareTo, not toHours(): toHours() truncates, so 24h30m would wrongly count as "24, not more than 24".
        return notice.compareTo(FULL_REFUND_NOTICE) > 0 ? deposit() : BigDecimal.ZERO;
    }
}

final class FrontDesk {
    void hire(Manager by, Person worker) {         // the manager adds new workers
        if (worker instanceof Guest) throw new IllegalArgumentException("a guest is not a worker");
        staff.add(worker);
    }
    RoomBooking book(Guest guest, Room room, LocalDate checkIn, LocalDate checkOut, Payment deposit) {
        if (!checkIn.isBefore(checkOut)) throw new IllegalArgumentException("check-out must be after check-in");
        if (!room.tryReserve(checkIn, checkOut))
            throw new IllegalStateException("room " + room.number() + " is not free " + checkIn + ".." +
checkOut);
        RoomBooking b = new RoomBooking("R" + seq.incrementAndGet(), guest.id(), room, checkIn, checkOut);
        if (!deposit.initiateTransaction()) {      // payment failed: give the nights back
            room.release(checkIn, checkOut);
            throw new IllegalStateException("deposit " + deposit.status());
        }
        b.setDeposit(deposit.amount());
        bookings.put(b.reservationNumber(), b);
        notifier.send(guest.id(), b.reservationNumber() + " confirmed: room " + room.number() + " " + checkIn +
".." + checkOut);
        return b;
    }
    // bookFor(Receptionist, Guest, ...): same call, second actor
    // checkIn(Receptionist, resNo): under the booking's monitor; CONFIRMED and not too early ->
    //   CHECKED_IN, room OCCUPIED, issue a key

    void addCharge(Server by, String reservationNumber, RoomCharge charge) {
        RoomBooking b = get(reservationNumber);
        synchronized (b) {                         // same monitor as checkOut: no charge after the bill is
totalled
            if (b.status() != BookingStatus.CHECKED_IN) throw new IllegalStateException("guest not in house");
            charge.addInvoiceItem(b.invoice());
        }
    }

    Invoice checkOut(Receptionist by, String reservationNumber, Function<BigDecimal, Payment> payWith) {
        RoomBooking b = get(reservationNumber);
        synchronized (b) {
            // ... must be CHECKED_IN
            BigDecimal price = b.room().pricePerNight();
            List<InvoiceItem> finalLines = new ArrayList<>(List.of(
                    new InvoiceItem(b.nights() + " nights x " + Money.fmt(price),
price.multiply(BigDecimal.valueOf(b.nights()))),
                    new InvoiceItem("Deposit paid", b.deposit().negate())));
            BigDecimal balance = finalLines.stream().map(InvoiceItem::amount).reduce(inv.total(),
BigDecimal::add);
            Payment p = payWith.apply(balance);
            // final lines are written only once payment succeeds, so a declined payment can be retried
            // without the nights and deposit lines being added twice
            if (!p.initiateTransaction()) throw new IllegalStateException("payment " + p.status() + ", guest still
checked in");
            finalLines.forEach(inv::addItem);
            int killed = b.room().deactivateGuestKeys();
            b.setStatus(BookingStatus.CHECKED_OUT);
```

```
            b.room().setStatus(RoomStatus.BEING_SERVICED);
            // notify "checked out, paid ... by ... [status], N key(s) deactivated"
            return inv;
        }
    }

    BigDecimal cancel(Person by, String reservationNumber) {   // only from CONFIRMED, under the booking's monitor
        BigDecimal refund = b.refundDue(now());
        b.room().release(b.checkIn(), b.checkOut());
        b.setStatus(BookingStatus.CANCELLED);
        // notify "... cancelled by <name>, refund ..."
        return refund;
    }
    // logHousekeeping(Housekeeper, room, text, minutes): append a log line, room AVAILABLE
    // sendReminders(): check-in (14:00) or check-out (11:00) within the next 24 hours
    // roomsBookedBy(guestId), whoBooked(roomNumber): queries over bookings, cancelled ones excluded
}
// Main.java: Seaside Hotels (Goa: 101, 102 DELUXE $120; 201 FAMILY_SUITE $250 / Kochi: K-1 DELUXE $90);
// SimClock at 2026-07-01T09:00Z; 6 threads race for room 101 (Jul 10-12 or Jul 10-13).
```

Output:

```
== 0. Manager adds rooms and hires staff: [Rita, Sam, Hari] ==

== 1. Search DELUXE for Jul 10-13 across locations ==
  Goa -> [101, 102]
  Kochi -> [K-1]

== 2. 6 guests try to book room 101 for overlapping dates at once ==
  [notify G6] R1 confirmed: room 101 2026-07-10..2026-07-12
  successful bookings = 1 of 6
  who booked 101? [G6@2026-07-10]

== 3. Back-to-back stays share the changeover day ==
  [notify ASHA] R2 confirmed: room 102 2026-07-10..2026-07-13
  [notify BEN] R3 confirmed: room 102 2026-07-13..2026-07-15
  rooms booked by ASHA: [102]

== 4. Cancellation: >24h before check-in = full refund, later = none ==
  [notify ASHA] R4 confirmed: room 201 2026-07-10..2026-07-13
  [notify CARL] R5 confirmed: room 201 2026-07-02..2026-07-03
  [notify CARL] R5 cancelled by Carl, refund $0.00
  [notify ASHA] R4 cancelled by Asha, refund $250.00

== 5. Reminders, check-in (key), room service, check-out (invoice) ==
  [notify G6] reminder: check-in for R1 at 2026-07-10T14:00
  [notify ASHA] reminder: check-in for R2 at 2026-07-10T14:00
  issued key K1 active=true
  [notify ASHA] reminder: check-out for R2 at 2026-07-13T11:00
  [notify BEN] reminder: check-in for R3 at 2026-07-13T14:00
  cash short: payment DECLINED, guest still checked in
  [notify ASHA] checked out, paid $299.00 by card(Asha) [COMPLETED], 1 key(s) deactivated
    Room service: Extra towels        $5.00
    Kitchen: Fish curry dinner       $24.00
    Amenity: Spa access              $30.00
    3 nights x $120.00              $360.00
    Deposit paid                   -$120.00
    TOTAL                           $299.00
  key active after checkout? false, room 102 is BEING_SERVICED

== 6. Housekeeping log ==
  RoomHouseKeeping[at=2026-07-13T09:00, housekeeper=Hari, description=Full clean, linen change, minutes=40]
  room 102 is AVAILABLE
```

What to notice: section 0 — only a `Manager` can add rooms and hire staff (the course's "Manager adds new workers"); section 2 — six concurrent bookings of room 101 for overlapping dates produce exactly one booking (which guest wins varies per run); section 3 — Ben's stay starting the 13th fits right after Asha's ending the 13th (half-open ranges); section 4 — Carl cancels 19 hours before check-in and gets nothing, Asha cancels 8 days ahead and gets a full refund; section 5 — reminders fire within 24

hours of check-in/check-out, the key is issued at check-in and dead after check-out, a declined cash payment leaves the guest checked in and the invoice untouched (the retry by card does not duplicate any line), and the invoice lines add up (room service $5 + kitchen $24 + spa $30 + 3 nights $360 − $120 deposit = $299); section 6 — housekeeping is logged and the room becomes available.

Concurrency & edge cases

- **Double booking a room-night.** Prevented by `Room.tryReserve` being `synchronized`: the check and the add of every night are one atomic step. The demo's 6-thread race always has one winner.
- **Search is advisory.** `search` may show a room that is gone a millisecond later; the booking itself (`tryReserve`) is the real check — so always handle "no longer available" on book.
- **Booking-level transitions** (`checkIn`, `checkOut`, `cancel`) run under the booking's monitor, so a cancel can't interleave with a check-in. Lock order is always booking → room.
- **The 24-hour rule, exactly.** `refundDue` compares `Duration`s (`notice.compareTo(24h) > 0`). The tempting `Duration.toHours() > 24` is wrong: `toHours()` drops the minutes, so a cancellation 24 h 30 min ahead would count as "24, not more than 24" and lose its refund.
- **Checkout is retry-safe.** The nights and deposit lines are written to the invoice only after the payment succeeds, and `addCharge` takes the same booking monitor as `checkOut`, so no charge can be added between totalling the bill and closing it.
- **Edge cases:** check-out date not after check-in; checking in before the check-in date; adding charges when the guest is not in house; cancelling after check-in; cash tendered below the amount (declined); a check payment that stays `PENDING`.
- **Chain-wide inventory.** The course notes that storing nights on each room is simple but doesn't scale to a chain selling through many channels: real systems keep availability in one place (an **inventory table** keyed by `(room_type, date)` with a count, updated with a conditional `UPDATE ... SET sold = sold + 1 WHERE sold < capacity`), so two sites or two booking channels can't sell the same night.

Extensions & follow-up questions

- **"A family wants two rooms in one booking."** The course flags this: move the dates and the invoice *up one level* to a `Reservation` that holds several `RoomStay`s (each with its room and nights). Reserving several rooms then needs all-or-nothing across rooms — lock rooms in a fixed order (by room number) to avoid deadlock, or do it in one database transaction.
- **"Book by room type, assign the room at check-in."** Many hotels sell a *type* for a date range and pick the physical room later. Keep per-type-per-night counts for selling; assign a specific room at check-in (housekeeping status permitting).
- **"Seasonal rates."** A `RatePlan`/`PricingStrategy` gives the price per night (weekends, seasons, loyalty tiers); the nights line becomes one line per night.
- **"Overbooking."** Allow selling capacity × (1 + overbooking factor) of a type; the ones who can't be housed are "walked" to a partner hotel — a business policy, pluggable as a strategy on the availability check.
- **"Separate the booker from the occupants."** The course names this as missing: add `Occupant`s to the stay (names for fire safety/registration) distinct from the paying guest.
- **"Lost key."** Deactivate all active guest keys for that room and issue a new one — `deactivateGuestKeys()` + `issueKey()`.

Real-world example

Property management systems such as **Oracle Hospitality OPERA** run the front desk this way: reservations are made against room types and dates, a specific room is assigned by check-in, the folio (the guest's bill) collects charges from restaurants, minibar and spa as separate postings, and housekeeping status (dirty, clean, inspected) controls whether a room can be assigned. At check-in, the desk encodes an RFID (radio-frequency identification, a contactless chip) key card with an electronic lock system (for example ASSA ABLOY's VingCard or dormakaba) that is valid only until the check-out time. Online, **Booking.com** shows each rate's cancellation policy (for example "free cancellation until 24 hours before arrival") — exactly the one-method refund rule of this design, but chosen per rate plan.

In the interview

30–45 minute plan. (1) Clarify: room types, date-range search, 24-hour refund, charges, payment types, keys, housekeeping. (2) Actors and use cases. (3) Entities, stating early: **"a status field is not availability"** — rooms keep booked nights. (4) Code `tryReserve` (synchronized), `book`, `checkIn`, `checkOut` with the invoice. (5) Template Method for charges; payment subclasses; refund rule. (6) Follow-ups: two rooms per booking, chain-wide inventory, overbooking.

> **30-second answer:** "A `Hotel` has `HotelLocation`s which own `Room`s and implement search by type and date range. Each room keeps the set of nights it's committed to, and reserving nights is a synchronized check-then-act so two guests can't get the same night. A `RoomBooking` owns an `Invoice` made of lines; room service, kitchen and amenities are `RoomCharge` subclasses using Template Method to add a line. Payment is credit card, check or cash subclasses. Cancellation refunds in full if more than 24 hours before check-in — one method. Check-in issues a key card; check-out totals the invoice, takes payment, kills the keys and hands the room to housekeeping."

TL;DR

Rooms are sold night by night: each room keeps its booked nights and reserves them atomically. Search lives on the location. The bill is a list of lines filled by `RoomCharge` Template Methods; three payment subclasses; one refund method (24 hours). Keys issued at check-in, killed at check-out; housekeeping is an append-only log (entries are only added, never changed or removed).

Chapter summary

Problem	Inventory unit	How a hold/booking is made safe	Payment failure handling	Cancellation
Library	`BookItem` (copy)	`synchronized` member → item (fixed order)	— (fines recorded)	Return → FIFO waitlist per title gets the copy
Movie tickets	`ShowSeat` (seat × show)	Per-show monitor; all-or-nothing lock with TTL; atomic commit	Declined → release; lost hold → refund (compensation)	Refund per `CancellationPolicy`; seats back on sale
Concert tickets	Numbered seat state / GA counter	Lock-free CAS; waiting room; per-user limit	Same compensation as movies	Released tickets offered to waitlist first
Hotel	Room × night	`synchronized` `Room.tryReserve` over a date range	Failed deposit → nights released	Full refund if > 24h before check-in

Five sentences to remember for any booking interview:

1. Name the *real* unit of inventory first (copy, show seat, room-night, GA count).
2. Every "if free then take" must be one atomic step — a monitor, a CAS, or a conditional `UPDATE`.
3. Temporary holds need an expiry, or abandoned checkouts block inventory forever.
4. Money can't be rolled back — failures after payment need a compensating refund.
5. In-process locks protect one server; with many servers, the database (constraints, conditional updates) or a TTL store like Redis is the real guarantee.

Chapter 16 — Case Studies: Booking & Inventory (Part 2)

> **Where this fits.** Chapter 15 covered the first group of booking problems. This chapter finishes the family with five more: a car rental company, a restaurant, an airline, a university's course registration and a meeting-room scheduler. They look different on the surface, but they are the same problem underneath: **a limited resource, a calendar, and many people trying to claim it at once.** Once you see that shared skeleton, each new booking problem takes minutes to set up in an interview.

The shared skeleton of every booking problem

Idea	Car rental	Restaurant	Airline	Course registration	Meeting scheduler
The thing being booked	a vehicle	a table	a seat on one flight instance	a seat in one section	a room for a time range
The "calendar" it is booked on	days	hourly time slots	the flight instance (route + date)	the term	continuous time (half-open intervals)
The check-then-claim step that must be atomic	`Vehicle.tryBook`	`Table.tryHold`	`FlightInstance.holdSeats`	`Section.enroll`	`Room.tryReserve`
What happens when it is full	search another car	try the next table	pick another seat or flight	waitlist	try the next fitting room
Money	bill with lines + late fee	check split by seat + tax	fare per seat class, paid per itinerary	none (fees are an extension)	none

A **resource** is anything with a limited supply (a car, a table, a seat). A **thread** is one independent line of execution; a server runs many side by side (**concurrency**: several things in progress at once). A **race condition** is a bug where the result depends on which thread runs first — here, two people both "win" the same seat because both checked "free?" before either wrote "taken". A step is **atomic** when no other thread can see it half-done or slip in between its parts.

How to run every example in this chapter

```
cd chapters/16-case-booking-2/code
javac --release 21 -d /tmp/lld_16 $(find . -name '*.java')
java -cp /tmp/lld_16 lld.ch16.carrental.Main          # or restaurant, airline, courseregistration,
meetingscheduler
```

`javac` compiles `.java` files to `.class` files, `--release 21` targets Java 21, and `-cp` (the **classpath**) tells `java` where the compiled classes are. All five `Main` classes use only the standard library; the output under each is its real output. Where threads race, the *counts* are fixed but *which* racer wins varies per run. The listings below show only the essential lines; the complete files are in each design's `code/` folder.

> **Code-style note.** One file sometimes holds several small package-private types (visible only within the same package) — e.g. `Types.java` holds the enums and records. An **enum** is a type with a fixed list of named values (`CAR`, `TRUCK` …); a **record** is a compact class holding a few fixed values,

with `equals`, `hashCode` and `toString` generated. In production each top-level type would usually get its own file.

Money. All prices use `java.math.BigDecimal`, never `double`: a binary floating-point `double` cannot represent 0.1 exactly, so sums of prices drift by fractions of a cent; `BigDecimal` stores exact decimal digits.

Design a Car Rental System

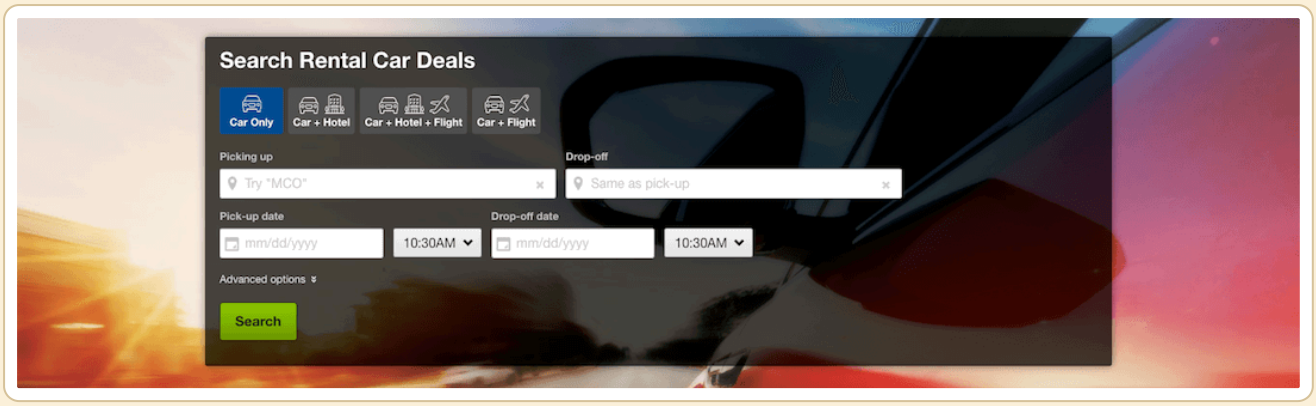


Figure — A car rental company rents vehicles from many branches (course diagram)

Problem & clarifying questions

A **car rental system** is the software a rental company uses to rent vehicles for a short time — from a few hours to a few weeks. Companies usually have many branches, often near airports and city centres, and a customer may return a car to a different branch from the one they picked it up at.

Questions worth asking in the first two minutes, with the answers we assume:

Question	Assumed answer
Which vehicle kinds?	Cars, trucks, SUVs, vans and motorcycles.
One branch or many?	Many branches, each with its own vehicles.
Can the drop-off branch differ from the pick-up branch?	Yes, the reservation records both.
How is price worked out?	A daily rate per vehicle type, plus add-ons, plus a late fee per day late.
What can be added?	Insurance, equipment (navigation, child seat, ski rack), services (roadside assistance, wifi) and additional drivers.
How are vehicles identified?	Each has a unique barcode; a barcode reader scans it at pick-up and return.
Do we need history?	Yes: a vehicle log of every event (reserved, picked up, returned, fuelling, cleaning, damage).
Concurrency?	Many members book at once; two must never get the same car on overlapping days.

Requirements

Functional

1. Rent cars, trucks, SUVs, vans and motorcycles; each vehicle has a unique barcode (read by a scanner) and details including its parking stall number.

2. Members search the inventory, reserve any available vehicle and can cancel.
3. Answer "which member took this vehicle?" and "which vehicles has this member rented?".
4. Charge a late fee after the due date.
5. Notify when a pick-up date is near, a vehicle is near its due date, and when it is overdue.
6. Keep a vehicle log of all events on a vehicle.
7. Add-ons: rental insurance, equipment (navigation, child seat, ski rack), services (roadside assistance, additional driver, wifi).

Non-functional

- **Correctness under concurrency:** no double booking of a vehicle for overlapping days.
- **Extensibility:** adding a new kind of add-on or a new pricing rule should not change the reservation code (the **Open/Closed Principle**: code should be open to extension by adding new classes, but closed to editing what already works).
- **Auditability:** every charge is a visible bill line, every vehicle event is a log entry.

Use cases

Four **actors** (people or systems that use the software):

- **Receptionist** — adds and edits vehicles and workers; can also reserve vehicles for a customer.
- **Member** — searches the catalogue, reserves, picks up and returns a vehicle.
- **Worker** — handles returned vehicles and updates the vehicle log (cleaning, refuelling, damage).
- **System** — sends notifications (pick-up reminders, overdue alerts, cancellations).

A **use case** is one goal an actor can achieve with the system, such as "reserve a vehicle". Top use cases: add/remove/edit vehicle; search catalogue by type and availability; register or cancel membership; reserve vehicle; check out (pick up) vehicle; return vehicle; add equipment or services; update the vehicle log.

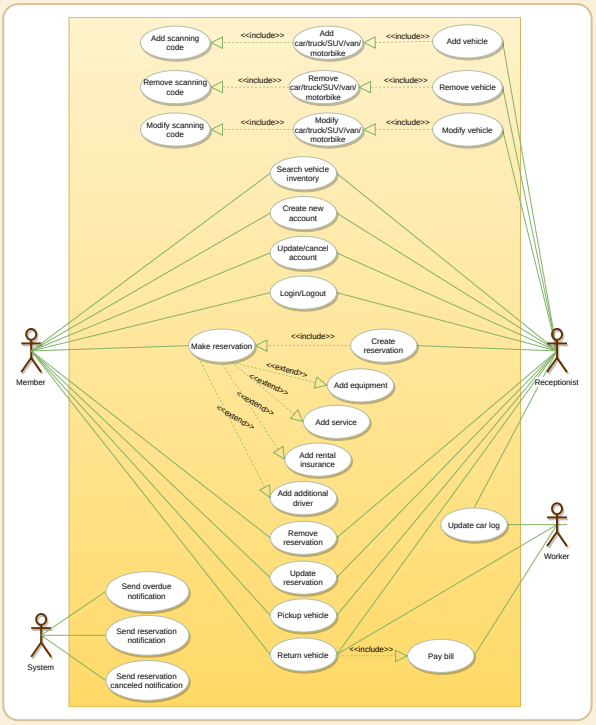


Figure — Use case diagram: receptionist, member, worker and system and the use cases each triggers (course diagram)

A **use case diagram** is a UML (Unified Modeling Language, a standard set of software diagrams) picture: stick figures are actors, ovals are use cases, and lines show who triggers what. In the figure above, notice that reserving is shared by the member and the receptionist, while updating the log belongs to the worker.

Core entities

Class	Responsibility
`CarRentalSystem`	Top-level object; owns the branches and the reservation service.
`CarRentalLocation`	One branch (name, code) with its own `VehicleInventory`.
`VehicleInventory`	Three indexes (lookup maps) over a branch's vehicles: by barcode, by type, by model. Answers "which SUVs are free on these days?".
`Vehicle`	Barcode, plate, make, model, type, seats, parking stall, mileage, **the set of booked days**, and its log.
`VehicleLogEntry`	One event on a vehicle (date, type, note).
`DateRange`	Half-open range of days `[start, end)` (includes `start`, excludes `end`); `end` is the due date.
`Reservation`	The rental: member, vehicle, dates, pick-up and return branch, add-ons, drivers, bill, status. Holds the business rules.
`ReservationAddOn` (+ `RentalInsurance`, `Equipment`, `Service`)	Anything that adds a charge to a rental.
`AdditionalDriver`	A named extra driver on the rental (no charge in our model).
`Bill`, `BillItem`	The one place that knows what a rental costs: a list of lines.
`PricingStrategy`, `LateFeePolicy`	Pluggable (swappable without code changes) rules for the base charge and the late fee.
`ReservationService`	Creates reservations atomically, answers the two "who rented what" questions, runs reminders.
`Notifier`, `PaymentMethod`	Ports (interfaces to outside systems) for sending messages and taking money.

Class diagram

A **class diagram** shows the classes, their fields and methods, and how they relate (owns, refers to, inherits from). The course builds its model in seven stages (money and messages, add-ons, vehicles, search, reservations, actors, the system and its branches); the finished result:

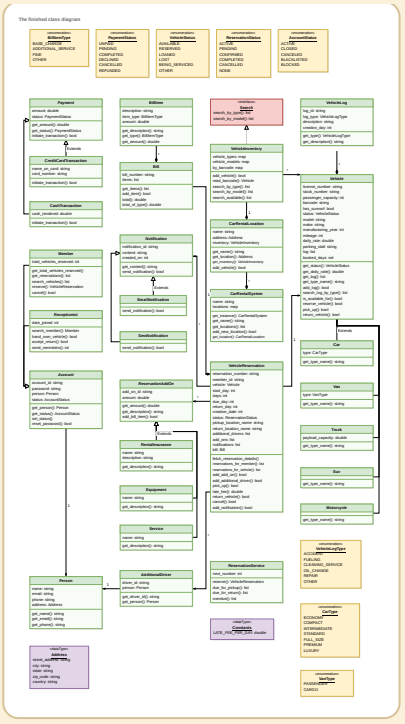
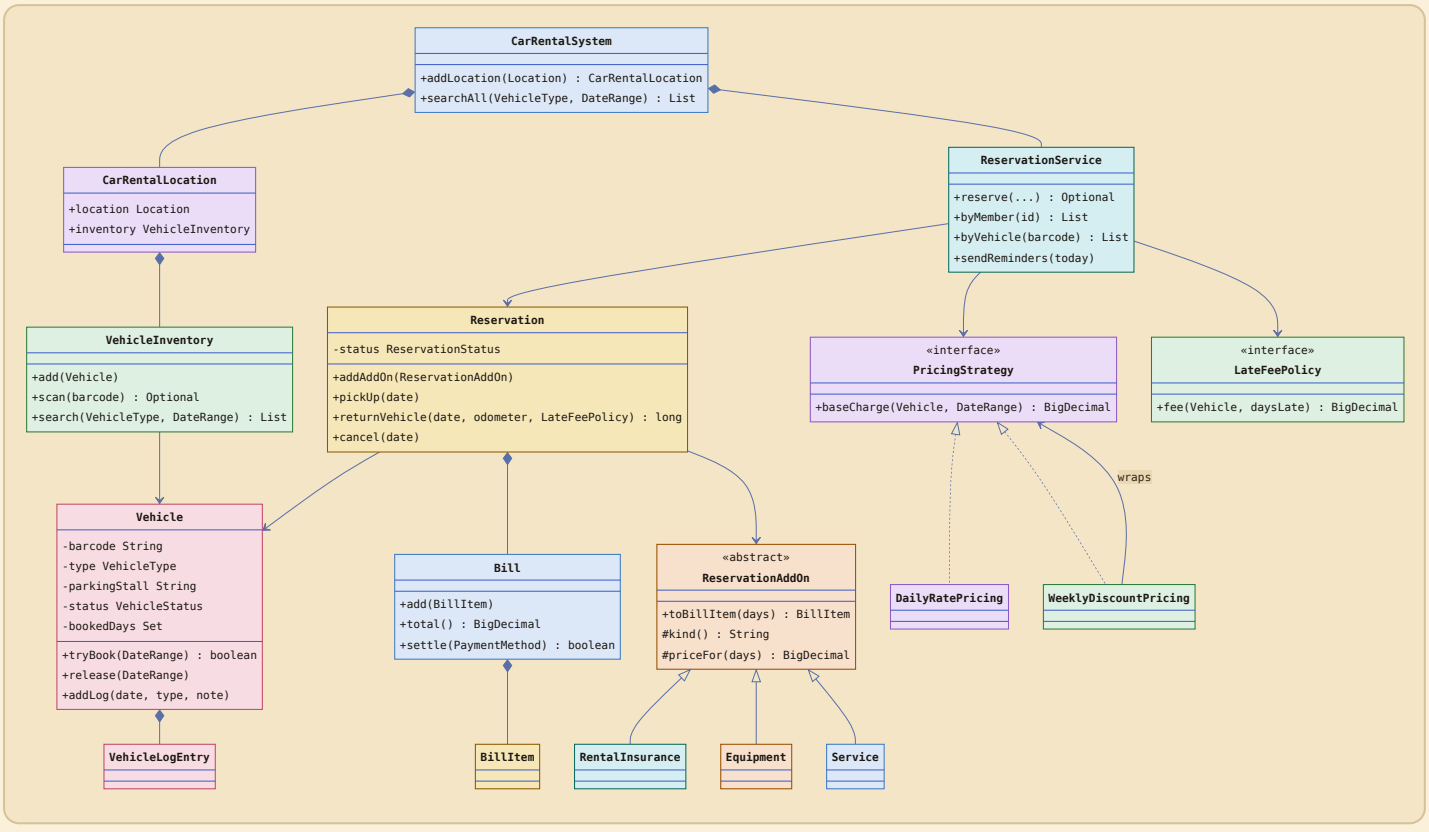


Figure — The finished class diagram of the car rental system (course diagram)

Two ownership chains matter: the system owns its branches and each branch owns its inventory (filled diamonds, **composition** — the part dies with the whole), while the inventory only *refers to* vehicles (plain arrow, **association**). A vehicle owns its log; a reservation owns its bill but only refers to the vehicle, add-ons, drivers and notifications.

Our Java design follows the same shape, with two deliberate differences: vehicle kinds are an enum (`VehicleType`) rather than five empty subclasses (classes that inherit from a parent class), because the subclasses add no behaviour; and pricing is a strategy object (an object you plug in to decide one varying rule) rather than a constant.



Key flows

The figures below are **activity diagrams** — flowcharts of one task, with diamonds for decisions. **Pick up a vehicle.** The receptionist scans the customer's licence (or searches by name), checks there is a valid reservation, scans the vehicle's barcode, and hands over the keys. If there is no reservation, the member can reserve a free vehicle on the spot.

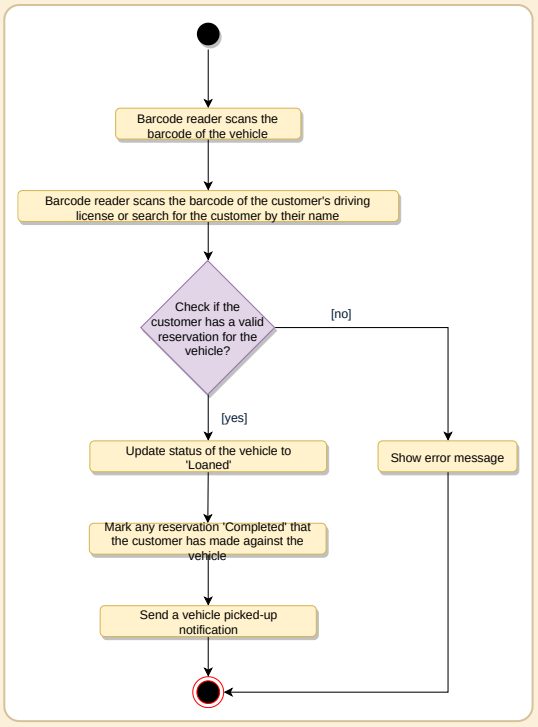


Figure — Activity diagram: picking up a vehicle (course diagram)

Return a vehicle. A worker scans the vehicle barcode; if it is past the due date, the system adds a late fee; the worker checks for damage and fuel; the bill is settled.

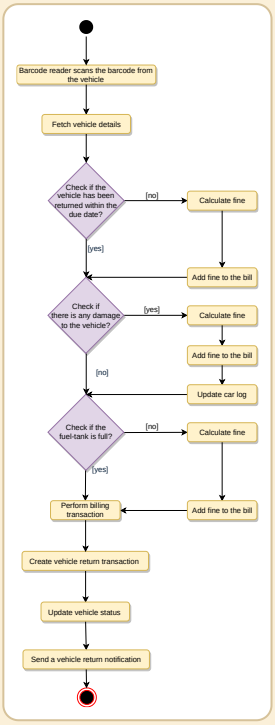
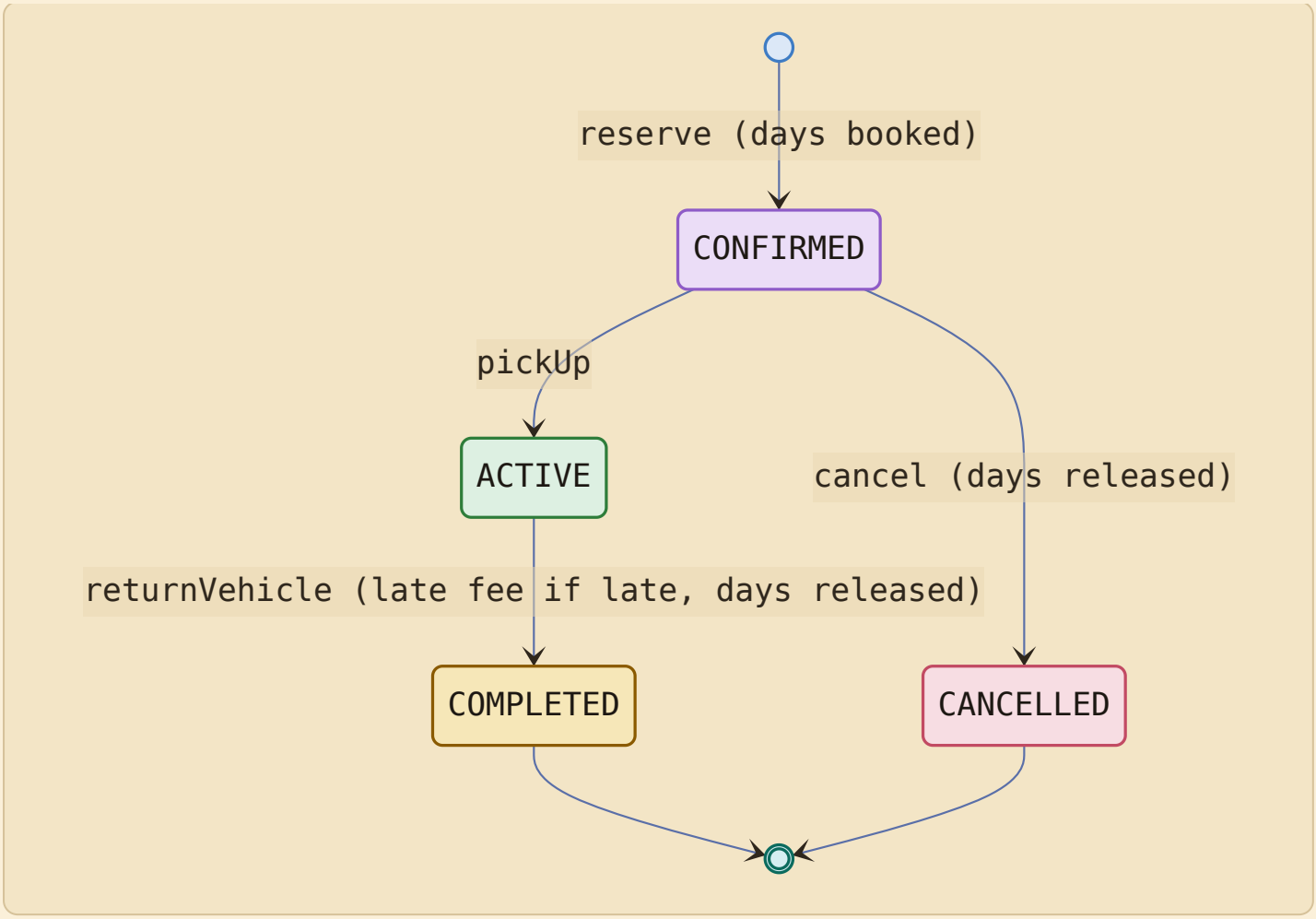


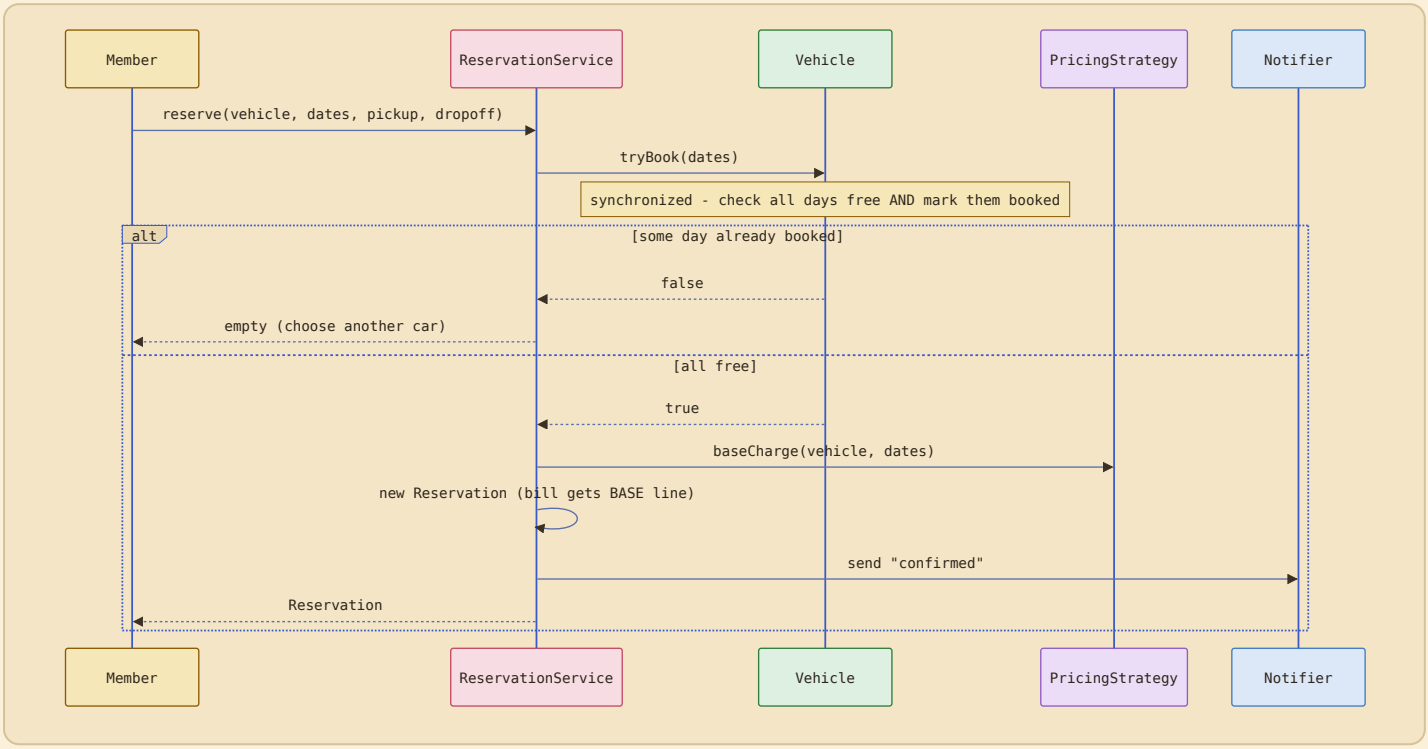
Figure — Activity diagram: returning a vehicle, with the late-fee branch (course diagram)

As the figure above shows, the late-fee decision is a branch on the return path — in our code it is one line inside `Reservation.returnVehicle`.

Reservation lifecycle (a **state machine** — a fixed set of states and the only moves allowed between them):



Reserve flow, including the race we must win (a **sequence diagram**: each column is an object, and arrows are calls in time order from top to bottom):



Design decisions & patterns used

Decision	Why	Alternative rejected
Availability comes from the days each vehicle has already sold, not from a status field	Members ask about future dates ("can I have it from the 10th to the 13th?"). A car that is out on loan right now may be back by then, and a car sitting on the lot today may already be promised to someone else next week.	A `RESERVED` status: cannot answer future-date questions, and a car can only hold one reservation.
Keep `VehicleStatus` too	Staff still need "where is the car right now" (on the lot, loaned, in service, lost).	Dropping status: loses the physical-location answer.
`Vehicle.tryBook` does check **and** book in one `synchronized` method (Java lets only one thread at a time run it on the same object)	Closes the check-then-act race (see below).	Separate `isFree()` then `book()` calls: two members can both see "free".
Template Method (a base class fixes the steps of an algorithm; subclasses fill in some steps) on `ReservationAddOn`	Billing is written once in the `final` (cannot be overridden) method `toBillItem`; subclasses only say what they are (`kind`) and may override the pricing hook (a step meant to be replaced) (`Service` is a flat fee). A fourth add-on is one new subclass.	A type enum with `switch` in the reservation: every new add-on edits the reservation.
Strategy (one interface, interchangeable implementations of a rule) for `PricingStrategy` and `LateFeePolicy`	Seasonal or length-based pricing changes one class, not the reservation.	Hard-coded rate constants.
Decorator (an object that wraps another of the same type and adds behaviour) `WeeklyDiscountPricing` wraps any `PricingStrategy`	Stacks a discount on top of whatever base rule is in place.	A subclass per combination (daily+weekly, seasonal+weekly …).
`Bill` is the only place charges live	Base, add-ons and late fees are lines on one bill; the total can never be computed in two places and disagree.	Each add-on keeping its own running total.
`ReservationService` separate from `Reservation`	Creating and indexing reservations is a different job from holding one reservation's rules (Single Responsibility: one class, one reason to change).	Static factory (a `static` method that builds objects) on `Reservation`.
Actors call the rules, they never compute them	The course's stage 6: a member calls the reservation service (search, reserve, cancel); a receptionist calls the reservation (hand over, take back, take payment, send reminders). An actor that starts working out fees is doing the reservation's job. In our code the actors are plain data (`Member`) and every rule sits in `Reservation`/`ReservationService`.	`Member.reserve()` / `Receptionist.computeLateFee()` methods.
`Bill` methods are `synchronized` and `settle` is idempotent (calling it twice has the same effect as once)	A late-fee line can never be added while the bill is being paid, and a double-clicked "pay" charges once.	Unsynchronized list + a second charge on retry.
The course makes `CarRentalSystem` a **Singleton** (a class allowed	A singleton is easy to reach but hard to replace in tests. Constructing one and passing it (dependency injection: an object receives	`static getInstance()`.

Decision	Why	Alternative rejected
only one global instance); we pass it in instead	what it needs from outside instead of creating or looking it up itself) gives the same "one per app" without a global.	

Java implementation

Package `lld.ch16.carrental`. Full runnable code is in `code/lld/ch16/carrental/`: `Types.java` (the enums `VehicleType`, `VehicleStatus`, `ReservationStatus`, `BillItemType`, `VehicleLogType` and the records `DateRange`, `Member`, `Location`, `VehicleLogEntry`, `AdditionalDriver`), `Vehicle.java`, `Billing.java` (`Bill`, `BillItem`, `PaymentMethod` with `CardPayment`/`CashPayment`, the add-ons), `Pricing.java` (`DailyRatePricing`, `WeeklyDiscountPricing`, `FlatLateFee`, `Notifier`), `Reservation.java`, `RentalSystem.java` (`VehicleInventory`, `CarRentalLocation`, `ReservationService`, `CarRentalSystem`) and `Main.java`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```
/** Half-open range [start, end): 10 Oct to 13 Oct is 3 rental days and the car is due back on 13 Oct. */
record DateRange(LocalDate start, LocalDate end) {
    DateRange { if (!end.isAfter(start)) throw new IllegalArgumentException("end must be after start"); }
    long days() { return ChronoUnit.DAYS.between(start, end); }
    Stream<LocalDate> dates() { return start.datesUntil(end); }
}

final class Vehicle {                       // barcode, plate, make, model, type, seats, parkingStall, mileage, log ...
    private VehicleStatus status = VehicleStatus.AVAILABLE;      // where it is RIGHT NOW
    private final Set<LocalDate> bookedDays = new HashSet<>();   // future availability

    synchronized boolean isFreeFor(DateRange r) { return r.dates().noneMatch(bookedDays::contains); }

    /** Check AND book inside one lock, so two members can never both win the same days. */
    synchronized boolean tryBook(DateRange r) {
        if (!isFreeFor(r)) return false;
        r.dates().forEach(bookedDays::add);
        return true;
    }
    synchronized void release(DateRange r) { r.dates().forEach(bookedDays::remove); }
    synchronized void setMileage(int m) {
        if (m < mileage) throw new IllegalArgumentException("odometer cannot go backwards");
        mileage = m;
    }
}
```

```java
/** The ONLY place that knows what a rental costs. Every method is synchronized on the bill. */
final class Bill {
    synchronized void add(BillItem item) {
        if (paid) throw new IllegalStateException("bill already settled");
        items.add(item);
    }
    /** Idempotent: settling a bill that is already paid does not charge the member a second time. */
    synchronized boolean settle(PaymentMethod method) {
        if (paid) return true;
        paid = method.charge(total());
        return paid;
    }
}

/** Template Method: toBillItem() is fixed; subclasses say what they are and may override the price hook. */
abstract class ReservationAddOn {
    final BillItem toBillItem(long rentalDays) {
        return new BillItem(kind() + ": " + name, BillItemType.ADD_ON, priceFor(rentalDays));
    }
    protected abstract String kind();
    protected BigDecimal priceFor(long rentalDays) { return price.multiply(BigDecimal.valueOf(rentalDays)); } //
hook: per day
}
final class Service extends ReservationAddOn {   // RentalInsurance and Equipment keep the per-day default
    protected String kind() { return "Service"; }
    @Override protected BigDecimal priceFor(long rentalDays) { return price; }   // flat fee per rental
}

/** Decorator over any PricingStrategy: 10% off rentals of 7 days or more. */
final class WeeklyDiscountPricing implements PricingStrategy {
    public BigDecimal baseCharge(Vehicle v, DateRange p) {
        BigDecimal base = inner.baseCharge(v, p);
        return p.days() >= 7 ? base.multiply(new BigDecimal("0.90")).setScale(2, RoundingMode.HALF_UP) : base;
    }
}
```

```
final class Reservation {   // the rules live here, not in the actors
    /** Returns the number of late days charged. The due date is period.end(). */
    synchronized long returnVehicle(LocalDate on, int odometer, LateFeePolicy lateFee) {
        require(ReservationStatus.ACTIVE, "return");
        long daysLate = Math.max(0, ChronoUnit.DAYS.between(period.end(), on));
        if (daysLate > 0)
            bill.add(new BillItem("Late fee: " + daysLate + " day(s)", BillItemType.LATE_FEE, lateFee.fee(vehicle,
daysLate)));
        vehicle.setMileage(odometer);
        vehicle.setStatus(VehicleStatus.AVAILABLE);
        vehicle.release(period);
        vehicle.addLog(on, VehicleLogType.RETURNED, id + " at " + returnLocation.code() + ", odometer " +
odometer);
        status = ReservationStatus.COMPLETED;
        return daysLate;
    }
    synchronized void cancel(LocalDate on) {
        require(ReservationStatus.CONFIRMED, "cancel");
        status = ReservationStatus.CANCELLED;
        vehicle.release(period);   // a cancelled rental must give its days back
    }
}

final class ReservationService {
    Optional<Reservation> reserve(Member m, Vehicle v, DateRange period, Location pickup, Location dropoff,
LocalDate today) {
        if (!v.tryBook(period)) return Optional.empty();
        var r = new Reservation("R" + seq.incrementAndGet(), m, v, period, pickup, dropoff, pricing.baseCharge(v,
period));
        reservations.put(r.id(), r);
        v.addLog(today, VehicleLogType.RESERVED, r.id() + " for " + period);
        notifier.send(m, "Reservation " + r.id() + " confirmed: " + v.model() + " " + period);
        return Optional.of(r);
    }
    // byMember(id), byVehicle(barcode): filters over the one reservations map.
    // sendReminders(today): pick-up tomorrow / due tomorrow / OVERDUE - run once a day by a scheduler.
}
```

`VehicleInventory.add` fills all three indexes (`byBarcode`, `byType`, `byModel`) in one call so they cannot drift apart, and rejects a duplicate barcode. `Main.java` builds two branches (SFO Airport, Downtown), prices with `WeeklyDiscountPricing(DailyRatePricing(...))` and a `FlatLateFee` of 40.00/day, and in step 3 releases 8 threads at once with a `CountDownLatch` to race for VH-002.

Output:

```
1) Search SUVs at SFO for 2026-10-10..2026-10-13
   VH-001 Toyota RAV4 (SUV, 5 seats, plate 7ABC123) stall A-12
   VH-002 Honda CR-V (SUV, 5 seats, plate 7ABC124) stall A-13
2) Alice reserves VH-001 (pick up SFO, return Downtown) and adds extras
    [notify alice@example.com] Reservation R1001 confirmed: RAV4 2026-10-10..2026-10-13
3) 8 members race for VH-002 on the same dates
    [notify racer7@example.com] Reservation R1002 confirmed: CR-V 2026-10-11..2026-10-14
   successful reservations for VH-002: 1
   SUVs free for 2026-10-10..2026-10-13 now: 0
4) Daily reminder job on 2026-10-09
    [notify alice@example.com] Pick-up tomorrow for R1001 at SFO Airport
5) Pick up on 10 Oct, return at Downtown on 15 Oct (due 13 Oct)
   VH-001 status while out: LOANED
    [notify alice@example.com] OVERDUE: R1001 was due 2026-10-13
    [notify alice@example.com] Returned 2 day(s) late, late fee added to R1001
6) Alice's bill
   Base: RAV4 x 3 days                     BASE_CHARGE   240.00
   Insurance: Collision damage waiver      ADD_ON        45.00
   Equipment: Child seat                   ADD_ON        24.00
   Service: Roadside assistance            ADD_ON        25.00
   Late fee: 2 day(s)                      LATE_FEE      80.00
   TOTAL 414.00
    charged 414.00 to card token tok_alice_visa
   paid: true, settle again (double click) -> true, no second charge
7) Who rented VH-001? What has Alice rented?
   R1001 -> Alice 2026-10-10..2026-10-13 COMPLETED
   Alice: R1001 RAV4
8) Vehicle log for VH-001
   2026-10-01 RESERVED R1001 for 2026-10-10..2026-10-13
   2026-10-10 PICKED_UP R1001 by Alice at SFO
   2026-10-15 RETURNED R1001 at DTN, odometer 20640
   2026-10-15 CLEANING interior clean by worker W7
9) The VH-002 winner cancels; the days come back
    [notify racer7@example.com] Reservation R1002 cancelled
   SUVs free for 2026-10-10..2026-10-13: 2
10) A 7-day Civic rental gets the weekly discount
    [notify alice@example.com] Reservation R1003 confirmed: Civic 2026-11-01..2026-11-08
   base charge 315.00 (7 x 50 = 350, minus 10%)
11) Rules are enforced
   rejected: cannot add an add-on to reservation R1001 in state COMPLETED
   rejected: end must be after start
```

What to notice in the output: step 3 has eight threads racing for one car and exactly one wins (which racer wins varies per run); step 5 shows the overdue reminder and the 2-day late fee (2 × 40.00); step 6 shows that settling an already-paid bill does not charge again; step 9 shows cancellation giving the days back.

Concurrency & edge cases

> **Key term — lock:** a **lock** (mutex) is a token only one thread can hold at a time; others wait for it (§12.7). A `synchronized` method locks the object itself. A *global lock* makes everyone queue; a *per-object lock* (one per vehicle here) lets different objects proceed in parallel (§12.11).

- **Double booking.** The course lists this as an open gap in its own design (checking the days and booking them are two separate steps there). Two members call `reserve` for the same car at the same moment. Because `tryBook` is `synchronized` on the vehicle, the check "are all days free?" and the write "mark them booked" happen as one step, so the second caller sees the first caller's days. Locking per vehicle (not one global lock) lets different cars be booked in parallel. In a database-backed system the same idea becomes a unique constraint on `(vehicle_id, day)` (the database refuses a second row with the same pair) or a `SELECT … FOR UPDATE` (a read that also locks the rows it returns) in one transaction (a group of database changes that succeed or fail together).
- **Search is advisory.** `search` may say a car is free and then `reserve` fails because someone else won. That is correct: the client (the app or web page the member is using) simply shows "no longer available". Never reserve based on a stale (out-of-date) search result without re-checking.

- **Late return collides with the next rental.** If Alice returns two days late and Bob booked the car from the 14th, Bob's car is not on the lot. Real systems keep a buffer (for example a cleaning day) between bookings and, when a car is overdue, reassign the next reservation to another car of the same type.
- **Early return.** We release the whole booked range; the unused days become bookable. Whether the member is refunded is a policy question.
- **Cancelling after pick-up** is rejected by the state machine (`CONFIRMED` only).
- **Add-ons after the rental ends** are rejected (see step 11 of the output).
- **Invalid ranges** (end not after start) are rejected in the `DateRange` constructor.
- **Odometer going backwards** is rejected in `Vehicle.setMileage`.
- **Different return branch.** The reservation records the return branch, but the vehicle stays in the pick-up branch's inventory. The course calls this out as a known gap: moving a car between branches is a *transfer* the design does not model yet (see Extensions).

Extensions & follow-up questions

- **"How do you handle one-way rentals?"** — On return at a different branch, remove the vehicle from the old `VehicleInventory` and add it to the new one inside `returnVehicle`, and log a `TRANSFER` event. Add a one-way fee as another `BillItem`.
- **"Seasonal or dynamic pricing?"** — Add a `SeasonalPricing` strategy (rate per date) or wrap with another decorator (`SurgePricing`). The reservation does not change.
- **"Deposits and card holds?"** — A card *hold* reserves money on the card without taking it yet. A `PaymentMethod.authorize(amount)` at pick-up and `capture(amount)` at return; the difference is released.
- **"Mileage limits, fuel level, damage claims?"** — Record odometer and fuel at pick-up and return; add `BillItem`s of type `FINE` for extra miles, missing fuel or damage (the course's enum has a `FINE` bill item type for exactly this kind of charge).
- **"How do reminders run?"** — A scheduler (for example a `ScheduledExecutorService`, Java's built-in timer for repeating tasks, or a cron job, an operating-system entry that runs a command on a fixed timetable) calls `sendReminders(today)` once a day.
- **"How would you scale this?"** — Partition vehicles by branch (split the data so each branch's vehicles live on their own database or server); the booked-days table becomes `(vehicle_id, day)` with a unique key; searches go to a read replica (a read-only copy of the database kept in sync with the main one) or a search index (a separate store built for fast lookups, such as Elasticsearch); reservations are written with a transaction. These scaling ideas are explained in System Design Fundamentals.
- **"How do workers update the log?"** — `Vehicle.addLog(date, CLEANING, note)` as in step 5; restrict who can call it by role in the service layer (the classes, like `ReservationService`, that sit between callers and the domain objects and enforce who may do what).

In the interview

30-second pitch: "Branches own inventories, inventories index vehicles by barcode, type and model. The key decision: availability is a question about *dates*, so each vehicle keeps its booked days, and `tryBook` checks and books them atomically. A reservation owns a bill; every charge — base, insurance, equipment, services, late fee — is a line on it. Add-ons share a template-method base class; pricing and late fees are strategies."

Suggested 40-minute plan:

Minutes	Do
0–5	Clarify vehicle types, branches, one-way, add-ons, late fees.
5–10	Actors and use cases; list the entities.
10–20	Class diagram. Say "availability is dates, not status" early.
20–35	Code `Vehicle.tryBook`, `Reservation` (state machine + bill + late fee), `ReservationService.reserve`.
35–40	Concurrency, late-return collision, one-way transfers.

Likely follow-ups: "Can I book it for next month while it is loaned now?" (yes — dates, not status). "Two people click reserve together?" (atomic `tryBook`). "Add a new add-on?" (one subclass).

Self-check

1. **Q:** Why is a `RESERVED` status not enough to answer "is this car free next week?" **A:** Status describes *now*. A car can be loaned today and free next week, or on the lot today but promised next week. Only per-day bookings answer date questions.
2. **Q:** What pattern does `ReservationAddOn.toBillItem` use and what is the "hook"? **A:** Template Method; `toBillItem` is the fixed algorithm and `priceFor` is the overridable hook (`Service` overrides it to a flat fee).
3. **Q:** Why is `tryBook` one method rather than `isFree` + `book`? **A:** To make check-then-act atomic under the vehicle's lock, so two threads cannot both see "free".
4. **Q:** How is the late fee computed? **A:** Days between the due date (`period.end`) and the actual return date, times the per-day fee from `LateFeePolicy`, added as a `LATE_FEE` bill line.
5. **Q:** Which class answers "who rented VH-001"? **A:** `ReservationService.byVehicle`, from the one list of reservations.

TL;DR

- Availability = booked **days** per vehicle; status = where it is now.
- `tryBook` makes check-and-book atomic per vehicle.
- A reservation owns one bill; every charge is a line. Late fee = days late × rate.
- Template Method for add-ons, Strategy (+ Decorator) for pricing.
- Known gaps to mention: branch transfers, buffer between rentals.

Real-world example. Rental companies such as Hertz and Enterprise let you book a *car class* ("intermediate SUV or similar") rather than a specific plate; the exact vehicle is assigned at the counter. That is the same design one level up: availability is counted per type per day, and a specific `Vehicle` is bound at pick-up. Their counters and return lanes scan the car and compute extra-day and fuel charges on the spot — the "return" activity above.

Design a Restaurant Management System



Figure — A restaurant with branches, tables, waiters and a kitchen (course diagram)

Problem & clarifying questions

A **restaurant management system** runs the day-to-day of a restaurant from one place: which tables are free, table reservations, taking orders, sending them to the kitchen, and producing and settling the bill.

Question	Assumed answer
One restaurant or a chain?	A restaurant with several branches; each branch has its own menu, tables and staff.
Menu structure?	Menu → sections (Starters, Mains …) → items with prices.
How are orders taken?	A waiter creates an order for a table and adds meals **per seat**; each meal has meal items, each pointing at a menu item.
Reservations?	Receptionist searches tables by date/time and party size and reserves one; customers can cancel; the system reminds them before the time.
Walk-ins?	Yes: show tables free right now.
Seating layouts?	A branch can have several seating arrangements (for example "Weekday" and "Banquet").
Payment?	Credit card, cheque or cash.
Kitchen?	Orders go to a chef queue; chefs cook them in order and the waiter is told when food is ready.

Requirements

Functional

1. Several branches, each with a menu of sections → menu items.
2. A waiter creates an order for a table with meals per seat; each meal item corresponds to a menu item.
3. Show tables free now for walk-ins.
4. Reservations: the receptionist searches free tables by date and time and reserves one; customers can cancel; a notification goes out as the time approaches.

5. Pay by credit card, cheque or cash.
6. A branch can have several seating arrangements of tables.
7. (Added) Orders go to a kitchen queue; chefs take them one at a time and the waiter is notified when an order is ready.

Non-functional

- No two parties may be given the same table for overlapping time.
- Prices live in one place (the menu item) so an order and the menu can never disagree.
- Easy to add a new payment type or a new chef-assignment rule.

Use cases

Actors:

- **Receptionist** — adds and edits tables and layouts; creates and cancels reservations; checks guests in.
- **Waiter** — takes and changes orders.
- **Manager** — hires staff and edits the menu.
- **Chef** — views and cooks orders.
- **Cashier** — produces the check (bill) and takes payment.
- **System** — sends reservation reminders and cancellations.

Top use cases: add/modify tables; search tables; place order; update order; create reservation; cancel reservation; check in; make payment.

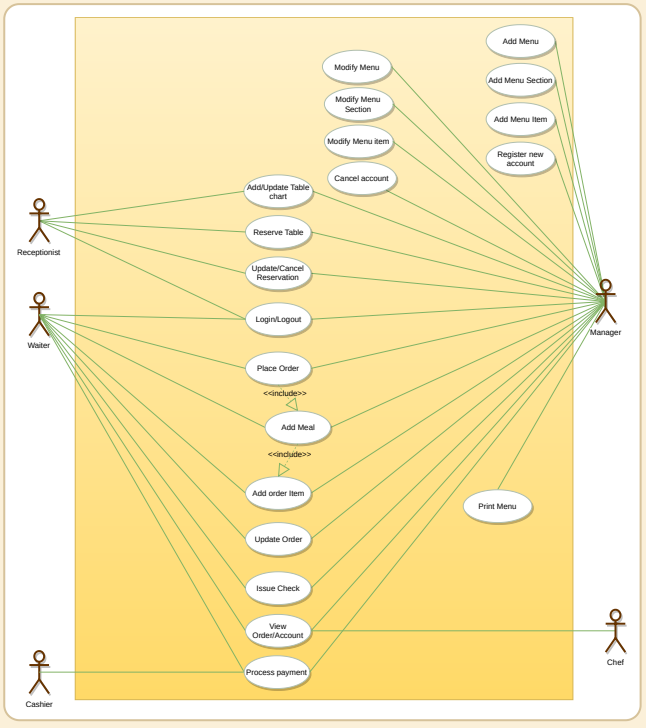


Figure — Use case diagram: receptionist, waiter, manager, chef, cashier and system (course diagram)

In the figure above, each role has a small, separate set of use cases. That is a hint for the code: each role class gets only its own methods.

Core entities

Class	Responsibility
`Restaurant`	Owns its branches. In the course the restaurant also registers its employees: closing (deactivating) the restaurant deactivates every employee account with it.
`Branch`	Owns its menu, kitchen, seating charts and reservation service; employs staff.
`Menu`, `MenuSection`, `MenuItem`	The menu tree. The **price lives only on `MenuItem`**.
`Table`, `TableSeat`	A table with a capacity and numbered seats. Keeps both its **current status** and the **time slots it is promised to**.
`TableChart`	A named seating arrangement; refers to tables (a table can appear in several charts).
`Reservation`	Customer, party size, time, table, status. `updatePeopleCount` refuses a party bigger than the reserved table seats (otherwise the booking would say six while the table holds four).
`ReservationService`	Searches tables (best fit), reserves atomically, cancels, checks in, sends reminders, lists walk-in tables.
`Order`	One per table visit: meals keyed by seat, status with allowed transitions, listeners.
`Meal`, `MealItem`	One meal per seat; a meal item = menu item × quantity.
`Kitchen`, `Chef`	Kitchen hands each order to a chef by a pluggable rule; each chef cooks from a FIFO (first in, first out) queue.
`Employee` and roles (`Receptionist`, `Waiter`, `Manager`, `Chef`, `Cashier`)	Each role's own operations.
`Check`	The bill: computed from the order (subtotal, tax, total), never stored separately.
`Payment` (`CardPayment`, `CashPayment`, `CheckPayment`)	Three ways to pay with three different behaviours.

Class diagram

The course grows the model in seven stages (payments and notifications, menu, tables, reservations, orders, staff, the branch); the finished result:

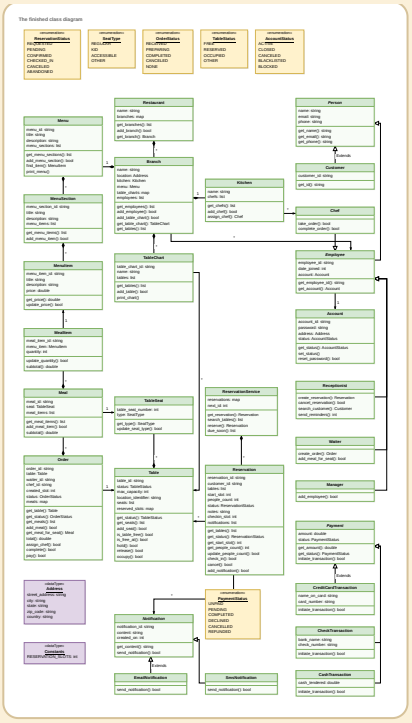
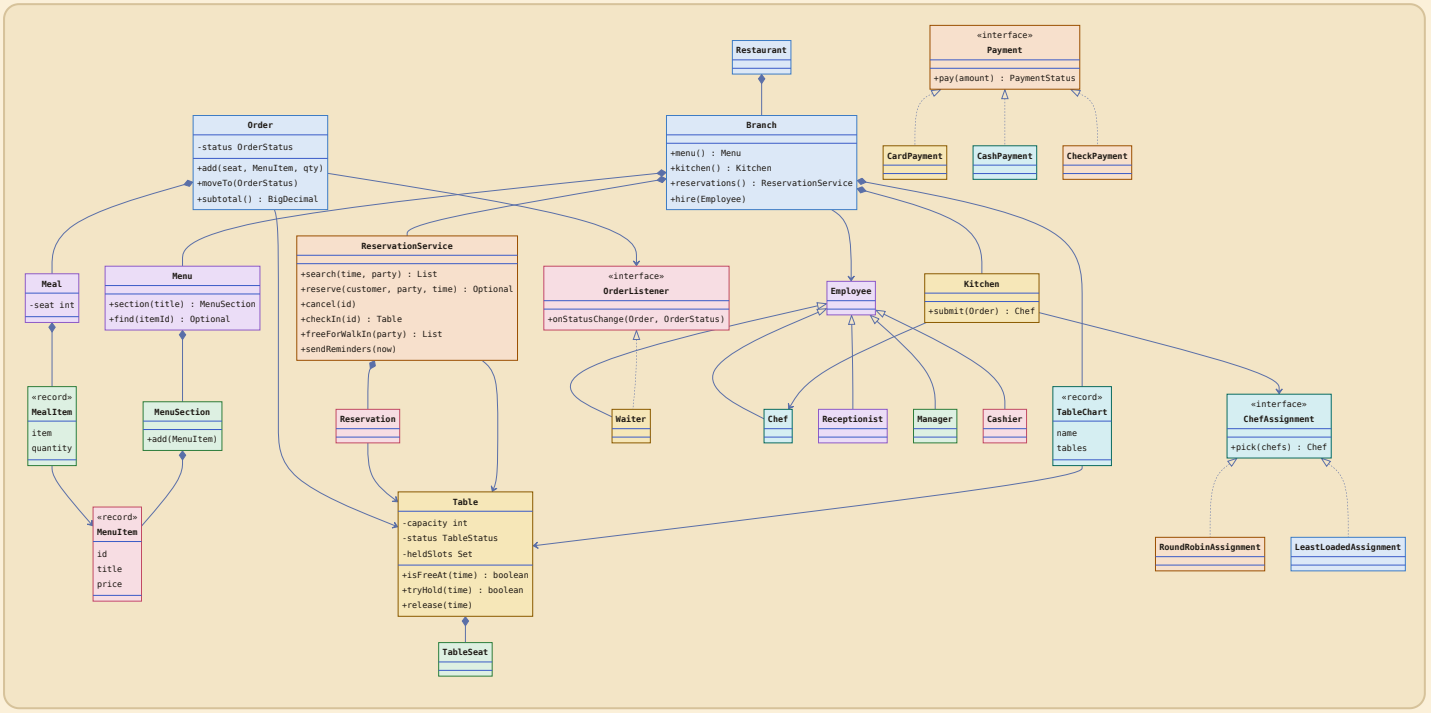


Figure — The finished class diagram of the restaurant management system (course diagram)

A table owns its seats while a chart only refers to tables; an order (one per table) owns one meal per seat, and each meal owns its meal items. The branch is the hub: its menu, kitchen and seating charts are composition (they do not exist without the branch), while staff are only associated, because staff can move between branches. Note also what is *not* there: no separate `Bill` class that stores a total. The course's argument is that the order already holds every priced line, so storing the total again would create a second copy that could disagree. Our `Check` follows that rule — it is computed from the order on demand.



Key flows

Place an order. The waiter opens an order for the table, then for each seat adds a meal with its items, and sends the order to the kitchen.

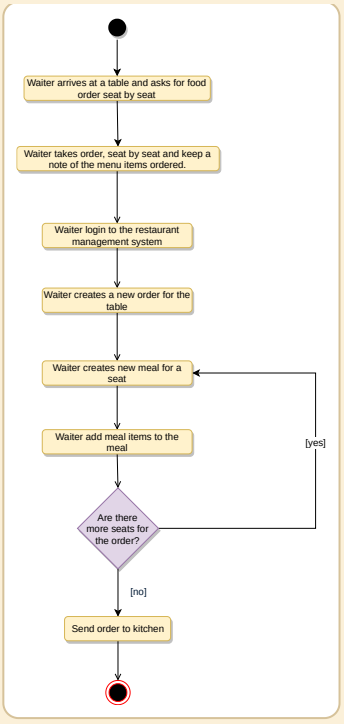


Figure — Activity diagram: placing an order seat by seat and sending it to the kitchen (course diagram)

Make a reservation. The customer calls or arrives; the receptionist searches tables for the time and party size; if one is free it is reserved and a confirmation is sent, otherwise the customer is told none is available.

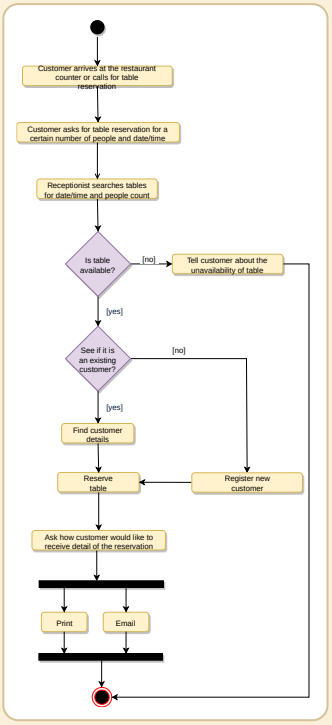


Figure — Activity diagram: making a table reservation (course diagram)

Cancel a reservation. The receptionist looks up the booking; if it is valid, it is cancelled, the table is released and the customer is notified.

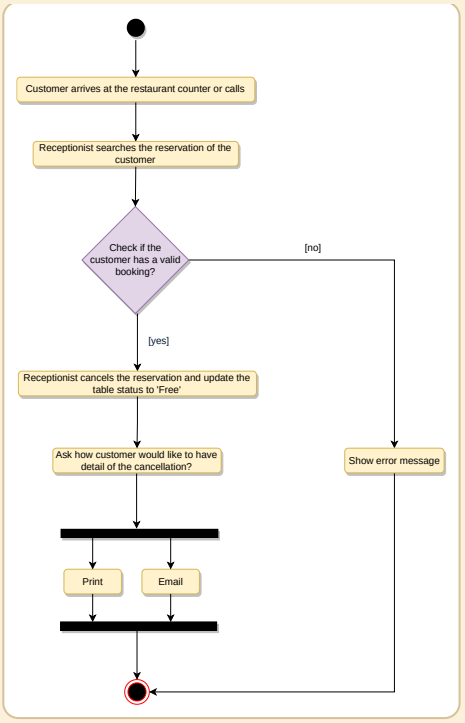
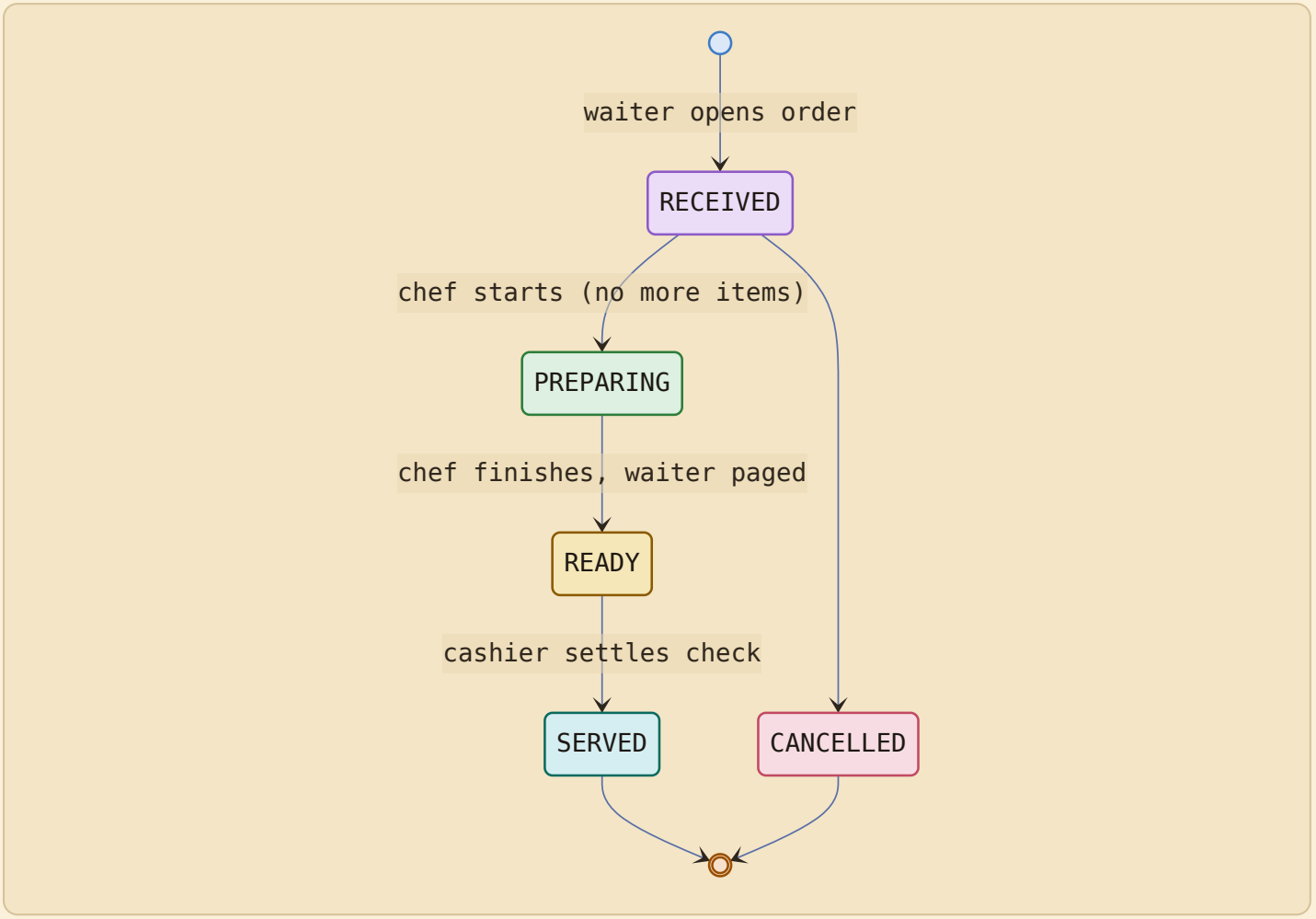
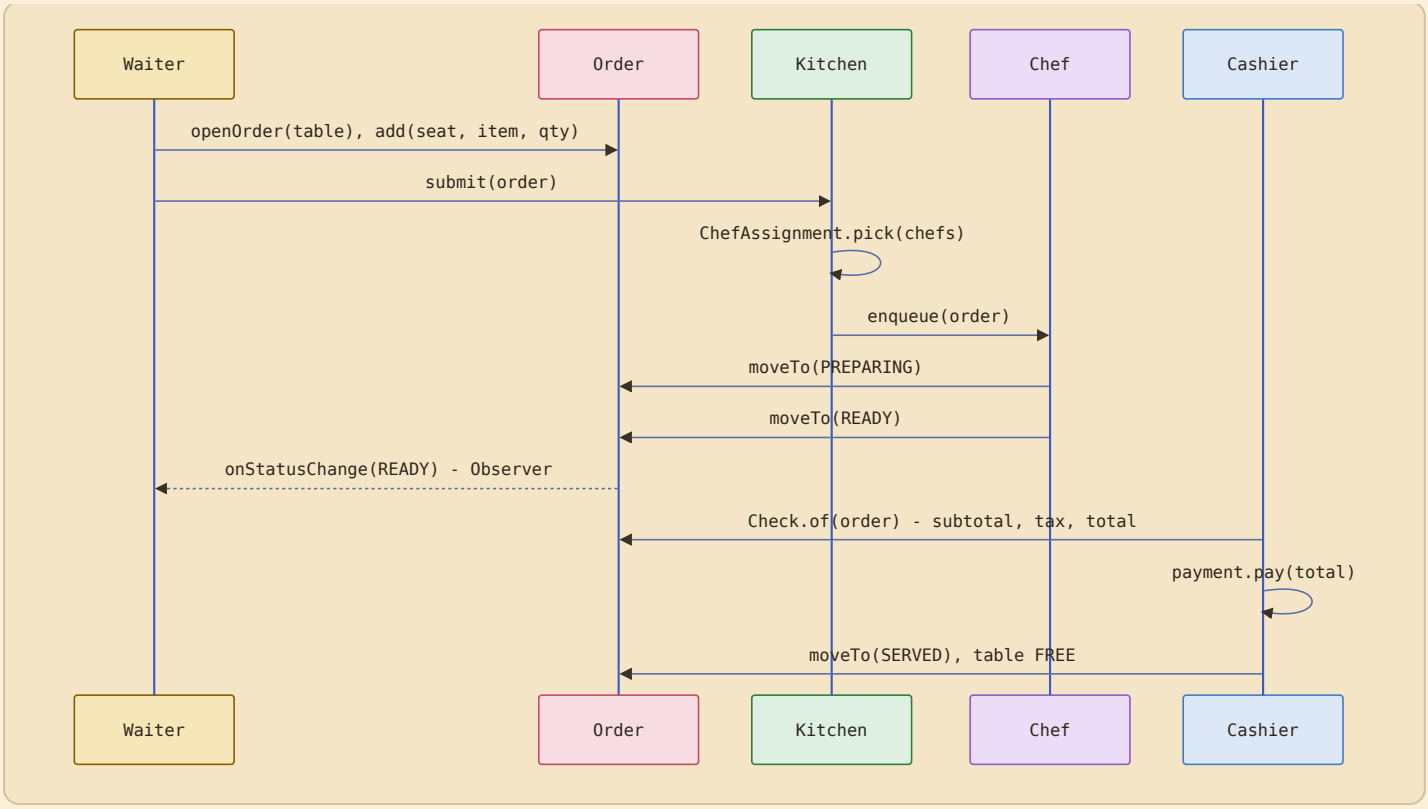


Figure — Activity diagram: cancelling a reservation (course diagram)

Order lifecycle — the transitions `Order.moveTo` allows:



From order to payment:


Waiter
Order
Kitchen
Chef
Cashier
openOrder(table), add(seat, item, qty)
submit(order)
ChefAssignment.pick(chefs)
enqueue(order)
moveTo(PREPARING)
moveTo(READY)
onStatusChange(READY) - Observer
Check.of(order) - subtotal, tax, total
payment.pay(total)
moveTo(SERVED), table FREE
Waiter
Order
Kitchen
Chef
Cashier

Design decisions & patterns used

Decision	Why	Alternative rejected
One meal per seat (`Map<seat, Meal>`)	Requirement 4 asks for it; it tells the kitchen which plate goes to which guest and makes split bills trivial.	A flat list of dishes per order.
Table keeps held time slots as well as status	Status answers "now" (walk-ins); slots answer "is it free at 19:00 on Friday" (reservations). A booking holds two hourly slots.	Status only: cannot take future bookings.
Search lives on `ReservationService`, not on `Table`	One table only knows about itself; "which tables are free at seven" needs all of them. The course notes an older version had a static search on `Table` that could not work.	Static `Table.search`.
Best fit when choosing a table	Smallest table that seats the party keeps large tables free for large parties.	First free table.
Observer (`OrderListener`; listeners register with a subject and are told when it changes)	The kitchen does not need to know who cares about "ready" — the waiter, a display screen, a delivery app can all subscribe.	Chef calling the waiter directly (tight coupling: classes that know too much about each other to change separately).
Strategy (`ChefAssignment`)	The course's kitchen rotates chefs; that is fair but not load-aware (it ignores how busy each chef already is). Swapping to `LeastLoadedAssignment` is one class, not a rewrite — the course's own point about keeping the rule in one place.	Round-robin (taking turns in a fixed rotation) hard-coded in `Kitchen`.
State machine in `Order.moveTo`	Illegal moves (adding food after cooking starts, serving before ready) fail loudly.	Free `setStatus`.
Three `Payment` classes behind a `sealed` interface (only the listed classes may implement it)	Card completes at once, cash may be short (declined) and returns change, a cheque must clear (pending). Different behaviour → different classes (Strategy).	One `Payment` class with a `type` field and `if`s.
Check computed, not stored	Prices live on menu items; the check is derived, so it cannot disagree with the order.	A stored `Bill` total.
Role classes extend `Employee`; `Customer` is not an employee	Only staff have employee ids and accounts; this keeps an account off the customer.	One `Person` class with a role flag.

Java implementation

Package `lld.ch16.restaurant`. Full runnable code is in `code/lld/ch16/restaurant/`: `Types.java` (enums `TableStatus`, `SeatType`, `ReservationStatus`, `OrderStatus`, `PaymentStatus`; records `MenuItem`, `Customer`, `TableSeat`), `Menu.java` (`Menu` → `MenuSection` → `MenuItem`; prices live only on `MenuItem`), `Tables.java` (`Table`, `TableChart`), `Reservations.java`, `Orders.java` (`MealItem`, `Meal`, `Order`, `OrderListener`), `Kitchen.java`, `Staff.java` (`Employee` roles, `Check`, payments, `Branch`, `Restaurant`) and `Main.java`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```java
/** status answers "what is this table doing NOW" (walk-ins); heldSlots answers "is it free at 19:00 on Friday".
*/
final class Table {
    static final int SLOTS_PER_BOOKING = 2;   // a booking holds the table for two hourly slots
    private final Set<LocalDateTime> heldSlots = new HashSet<>();
    private TableStatus status = TableStatus.FREE;

    /** Atomic check-then-hold. */
    synchronized boolean tryHold(LocalDateTime start) {
        if (!isFreeAt(start)) return false;          // none of slots(start) already held
        heldSlots.addAll(slots(start));
        return true;
    }
}
/** A named seating arrangement. It REFERS to tables (association); a table can appear in several charts. */
record TableChart(String name, List<Table> tables) {}

/** Searching belongs here, not on Table: one table only knows about itself. */
final class ReservationService {
    /** Best fit first: the smallest table that seats the party, so big tables stay free for big parties. */
    List<Table> search(LocalDateTime time, int partySize) {
        return tables.stream().filter(t -> t.capacity() >= partySize && t.isFreeAt(time))
                .sorted(Comparator.comparingInt(Table::capacity)).toList();
    }
    Optional<Reservation> reserve(Customer c, int partySize, LocalDateTime time) {
        for (Table t : search(time, partySize))
            if (t.tryHold(time)) {     // another receptionist may have taken it since search(): try the next one
                var r = new Reservation("RES-" + seq.incrementAndGet(), c, partySize, time, t);
                reservations.put(r.id(), r);
                notifier.send(c, "Table " + t.id() + " booked for " + partySize + " at " + time);
                return Optional.of(r);
            }
        return Optional.empty();
    }
    // sendReminders(now): run every few minutes; reminds CONFIRMED bookings starting within the next hour,
    // once each (Reservation.markReminded returns true only the first time).
}
// Reservation.updatePeopleCount(n) refuses n larger than the reserved table; cancel() releases the slots;
// checkIn() marks the table OCCUPIED. freeForWalkIn(size) picks FREE tables by status, best fit.
```

```
final class Order {
    private final Map<Integer, Meal> meals = new TreeMap<>();      // one meal per seat
    private final List<OrderListener> listeners = new CopyOnWriteArrayList<>();

    /** Items can only be added before the kitchen starts cooking. */
    synchronized void add(int seat, MenuItem item, int qty) {
        if (status != OrderStatus.RECEIVED) throw new IllegalStateException("order " + id + " is already " +
status);
        if (seat < 1 || seat > table.capacity()) throw new IllegalArgumentException("table " + table.id() + " has
no seat " + seat);
        meals.computeIfAbsent(seat, Meal::new).add(new MealItem(item, qty));
    }
    void moveTo(OrderStatus next) {
        synchronized (this) {
            boolean ok = switch (status) {
                case RECEIVED -> next == OrderStatus.PREPARING || next == OrderStatus.CANCELLED;
                case PREPARING -> next == OrderStatus.READY;
                case READY -> next == OrderStatus.SERVED;
                case SERVED, CANCELLED -> false;
            };
            if (!ok) throw new IllegalStateException("order " + id + ": " + status + " -> " + next + " not
allowed");
            status = next;
        }
        listeners.forEach(l -> l.onStatusChange(this, next));   // notify outside the lock
    }
}

interface ChefAssignment { Chef pick(List<Chef> chefs); }          // Strategy
final class RoundRobinAssignment implements ChefAssignment {
    public synchronized Chef pick(List<Chef> chefs) { return chefs.get(next++ % chefs.size()); }
}
final class LeastLoadedAssignment implements ChefAssignment {
    public Chef pick(List<Chef> chefs) { return
chefs.stream().min(Comparator.comparingInt(Chef::load)).orElseThrow(); }
}
// Chef keeps a FIFO queue: startNext() -> PREPARING, finishCurrent() -> READY. Waiter implements OrderListener.
```

```
/** The check is computed from the order's lines, never stored separately. */
record Check(Order order, BigDecimal subtotal, BigDecimal tax, BigDecimal total) {
    static final BigDecimal TAX_RATE = new BigDecimal("0.08");
    static Check of(Order o) {
        var sub = o.subtotal();
        var tax = sub.multiply(TAX_RATE).setScale(2, RoundingMode.HALF_UP);
        return new Check(o, sub, tax, sub.add(tax));
    }
}
/** Strategy: three ways to pay, three behaviours. */
sealed interface Payment permits CardPayment, CashPayment, CheckPayment { PaymentStatus pay(BigDecimal amount); }
// CardPayment -> COMPLETED; CashPayment -> DECLINED if short, else prints change and COMPLETED;
// CheckPayment -> PENDING (a paper check has to clear at the bank, so it is not money yet).

final class Cashier extends Employee {
    PaymentStatus settle(Check check, Payment payment) {
        // Check the order state BEFORE taking money: a refused state change after payment would leave the guest
charged.
        if (check.order().status() != OrderStatus.READY)
            throw new IllegalStateException("order " + check.order().id() + " is " + check.order().status() + ",
not READY");
        var status = payment.pay(check.total());
        if (status != PaymentStatus.DECLINED) {
            check.order().moveTo(OrderStatus.SERVED);
            check.order().table().setStatus(TableStatus.FREE);
        }
        return status;
    }
}
```

A `Branch` owns its menu, kitchen and charts; staff are associated (they can move branches), and `Branch.hire` adds a `Chef` to the kitchen. `Main.java` opens a "Spice Route" Downtown branch with tables T1 (2 seats), T2 and T3 (4), T4 (6), round-robin chefs Chen and Maria, and an SMS notifier.

Output:

```
1) Manager builds the menu
   Starters: [Tomato soup, Garlic bread]
   Mains: [Margherita pizza, Pasta arrabbiata]
   Drinks: [Lemonade]
   charts: [Default, Banquet]
2) Ravi searches tables for 4 at 19:00: [T2, T3, T4]
    [sms 555-0101] Table T2 booked for 4 at 2026-10-09T19:00
    [sms 555-0102] Table T3 booked for 4 at 2026-10-09T20:00
   Asha -> T2, Ben -> T3 (T2 is held 19:00-21:00)
   party of 8 at 19:00: no single table fits
   Asha's party grows to 6: rejected, table T2 seats 4, not 6
    [sms 555-0102] Reservation RES-2 cancelled
   after Ben cancels, free for 4 at 20:00: [T3, T4]
3) Reminder job at 18:15
    [sms 555-0101] Reminder: your table T2 at 19:00
4) Asha checks in; a couple walks in
   Asha seated at T2, walk-in couple seated at T1
5) Wendy takes two orders, one meal per seat
   O1 -> chef Chen, O2 -> chef Maria
6) Kitchen works the queues
   rejected: order O1 is already PREPARING
    [Wendy's pager] order O1 for table T2 is READY
    [Wendy's pager] order O2 for table T1 is READY
7) Check for O1 (split by seat)
   seat 1: [1x Tomato soup, 1x Margherita pizza] = 18.50
   seat 2: [1x Pasta arrabbiata, 2x Lemonade] = 20.50
   seat 3: [1x Garlic bread] = 4.00
   subtotal 43.00, tax 3.44, total 46.44
    change due: 13.56
   paid cash: COMPLETED
   O2 total 25.92 paid by check: PENDING
   table statuses: [T1=FREE, T2=FREE, T3=FREE, T4=FREE]
```

Step 2 shows best fit and slot holding (Ben cannot get T2 at 20:00 because Asha's 19:00 booking holds 19:00 and 20:00). Step 6 shows the state machine rejecting a late addition and the Observer paging the waiter. Step 7 shows the per-seat split and the three payment behaviours.

Concurrency & edge cases

- **Two receptionists book the same table.** `search` is only advisory; `reserve` calls `Table.tryHold`, which is `synchronized` and checks-and-holds both slots in one step. If it fails (someone else won) the loop tries the next table in best-fit order.
- **Notifying outside the lock.** `Order.moveTo` changes state inside `synchronized` but calls listeners *after* leaving it. Calling unknown code while holding a lock is a classic way to cause deadlocks or long waits. A **deadlock** is when two threads each hold a lock the other needs, so both wait forever.
- **Reminder spam.** The reminder job may run every few minutes; `markReminded` makes it send once per reservation.
- **Party larger than any table.** Returns "no single table fits". Combining tables is out of scope (see Extensions). The course notes the design already allows it, since a reservation can hold a list of tables.
- **No-shows.** `NO_SHOW` exists as a status; a job can release tables for bookings not checked in 15 minutes after start.
- **Party grows after booking.** `updatePeopleCount(6)` on a four-seat table is rejected (step 2 of the output); the receptionist must re-book.
- **Paying for food that is not ready.** `Cashier.settle` checks the order is `READY` *before* calling the payment, so a refused state change can never happen after the guest's money has been taken.
- **Unknown reservation id** on cancel or check-in fails with a clear `IllegalArgumentException` (Java's standard "you passed a bad value" error), not a `NullPointerException` (the confusing crash you get

when code uses a missing object).

- **Seat number out of range** on an order is rejected.
- **Cash short** → `DECLINED`, the order stays `READY` and the table stays occupied.
- **Cheque** → `PENDING`: the meal is served, but the money is not yet confirmed.

Extensions & follow-up questions

- **"Large party needs two tables?"** — Let a reservation hold `List<Table>`; search for combinations of adjacent tables (tag tables with a zone); hold all of them atomically by locking tables in a fixed order (for example by id) to avoid deadlock.
- **"Split the bill?"** — Already easy: `Meal.subtotal()` per seat; create one `Check` per seat group.
- **"Tips and service charge?"** — Add lines to `Check` (percentage or amount chosen at payment).
- **"Waiting list for walk-ins?"** — A FIFO queue per party size; when a table frees, notify the head.
- **"Kitchen stock?"** — An `Inventory` of ingredients; each `MenuItem` has a recipe; placing an order reserves ingredients and marks items "86'd" (unavailable) when stock runs out.
- **"Real concurrency in the kitchen?"** — Each chef becomes a worker thread (a thread that loops, picking up jobs) taking from a `BlockingQueue<Order>` (a thread-safe queue whose `take()` waits when it is empty); `take()` blocks until an order arrives.
- **"Shift schedules?"** — A `Shift` entity (employee, branch, start, end); `Kitchen` only picks chefs on shift.

In the interview

30-second pitch: "Branch owns a menu tree, a kitchen, seating charts and a reservation service. Tables keep both current status for walk-ins and held time slots for bookings; the service searches best-fit and holds slots atomically. Orders have one meal per seat, move through a strict state machine, and notify the waiter through an observer. The kitchen assigns chefs by a strategy. The check is computed from the order, and payment is a strategy with card, cash and cheque."

Minutes	Do
0–5	Clarify branches, seating layouts, per-seat meals, payment types.
5–10	Actors (six) and use cases.
10–20	Class diagram — say "meal per seat" and "status vs time slots" out loud.
20–35	Code `Table.tryHold`, `ReservationService.reserve`, `Order` state machine, `Kitchen.submit`.
35–40	Races, large parties, kitchen load balancing.

Self-check

1. **Q:** Why does a table need both a status and held slots? **A:** Status answers "what is it doing now" (walk-ins); slots answer "is it free at a future time" (reservations).
2. **Q:** Why is search on the service, not the table? **A:** A single table only knows about itself; finding all free tables needs the collection.
3. **Q:** Which pattern tells the waiter the food is ready, and why notify outside the lock? **A:** Observer; calling foreign code while locked risks deadlock and slows other threads.
4. **Q:** Why is there no stored bill total? **A:** The order's lines already hold every price; a second stored total could disagree.
5. **Q:** Why is a cheque payment `PENDING`? **A:** A cheque has to clear at the bank, so it is not money yet.

TL;DR

- Menu → section → item; price only on the item.
- Table = status (now) + held slots (later); service searches best-fit and holds atomically.
- Order → meal per seat → meal item; strict state machine; Observer for "ready".
- Kitchen assigns chefs by Strategy (round-robin or least-loaded).

- Check is computed; card/cash/cheque are three payment strategies.

Real-world example. Restaurant point-of-sale (POS — the till software where orders are entered and paid) systems such as Toast send each order to a kitchen display screen as a "ticket", show it by station and in order, and let the front of house see when it is done — the Observer and FIFO queue above. Reservation platforms such as OpenTable model table inventory by time slot and party size, exactly like `Table.heldSlots`.

Design an Airline Management System



Figure — An airline flies passengers between airports with owned or leased aircraft (course diagram)

Problem & clarifying questions

An **airline management system** runs an airline's operations: scheduling flights, reserving tickets, cancelling flights, supporting customers and managing staff, plus daily flight updates.

Question	Assumed answer
What does a customer search by?	Date, source airport and destination airport.
One flight or a trip of several?	A customer can build an **itinerary** (the full planned trip) of several flights.
Several passengers per booking?	Yes — a reservation seats several passengers (who need not have accounts).
Seat classes?	Economy, economy plus, preferred economy, business, first; seat types regular, accessible, emergency exit, extra legroom.
Who manages flights?	Only the admin adds aircraft, flights and schedules, and cancels flights; everyone affected is notified.
Crew?	Pilots and crew are assigned to flights and can view their schedule.
Payment?	Paid per itinerary.
Notifications?	When a reservation is made or changed and when a flight's status changes.

Requirements

Functional

1. Search flights by date, source and destination; view schedules (departure, arrival, free seats).

2. Reserve seats for several passengers on any scheduled flight; build an itinerary of several flights.
3. Only the admin adds aircraft, flights and schedules, and can cancel a flight (everyone affected is notified).
4. Customers cancel a reservation or a whole itinerary; payment is taken for reservations.
5. Assign pilots and crew to flights.
6. Message the customer whenever a booking is created or modified, and whenever one of their flights changes status.

Non-functional

- A seat on a flight instance is sold at most once, even under heavy concurrent booking.
- Several passengers in one booking get all their seats or none (**atomic**).
- Fare rules (class multiplier, last-minute surcharge) change without touching booking code.

Use cases

Five actors:

- **Admin** — adds flights and schedules, cancels flights, manages staff and crew.
- **Front desk officer** — reserves and cancels tickets for walk-in customers.
- **Customer** — views schedules, reserves and cancels.
- **Pilot / Crew** — views assigned flights.
- **System** — sends itinerary and flight-status notifications.

Top use cases: search flights; create/modify/view reservation; assign seats to passengers; pay for a reservation; update the flight schedule; assign pilots and crew.

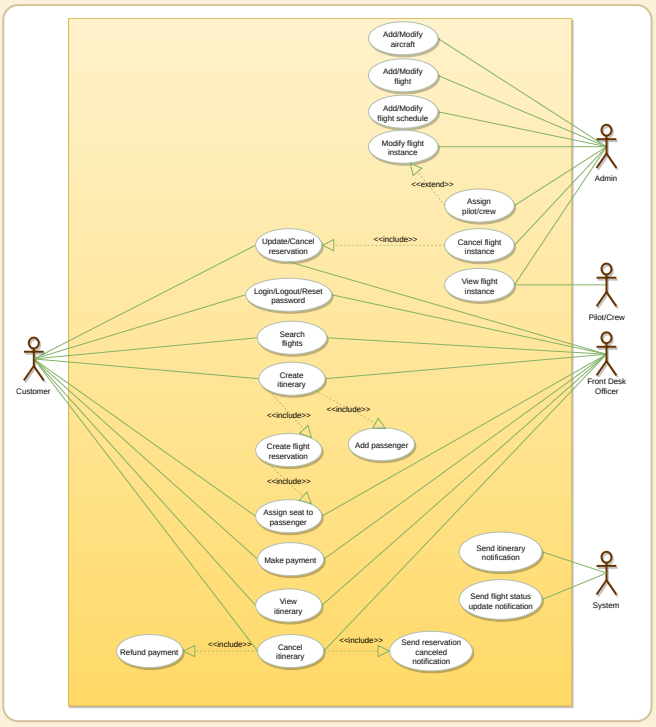


Figure — Use case diagram: admin, front desk officer, customer, crew and system (course diagram)

In the figure above, the front desk officer and the customer share the booking use cases: when an officer books for someone at the counter, it is the very same operation, just triggered by another actor.

Core entities

The one distinction that makes this design work is **four different "seat-ish" and "flight-ish" ideas**:

Idea	Class	Example
The route	`Flight`	GA212 London → Tokyo, 14 h, base fare 500
The route on one date	`FlightInstance`	GA212 on 8 Oct 2026 at 10:00
The physical seat on the aircraft	`Seat`	12A, extra legroom, economy plus
That seat on that one instance	`FlightSeat`	12A on GA212 8 Oct — booked by PNR147 (PNR = booking reference)

Class	Responsibility
`Airport`	Code and city.
`Aircraft`	Registration, model and its physical seats.
`Seat`	Number, type, class — nothing about bookings.
`Flight`	Route, duration, aircraft, base fare, weekly schedules; generates instances.
`WeeklySchedule`	"Every Tuesday at 10:00".
`FlightInstance`	Departure time, status, its `FlightSeat`s and assigned crew; holds seats atomically.
`FlightSeat`	One seat on one instance and which reservation holds it.
`Passenger`	A name on a ticket — deliberately **not** an account holder.
`Customer`	The account holder who books and pays.
`Admin`, `FrontDeskOfficer`	Staff actors. Admin-only methods on `Airline` require an `Admin` argument; the front desk books through `bookFor`.
`CrewMember`	Pilot, co-pilot or cabin crew.
`FlightReservation`	One leg: one instance, map passenger → seat, amount, status.
`Itinerary`	A trip of legs; owns the connection rule; the unit that is paid.
`FareStrategy`	Pluggable pricing: class multiplier, decorated with a last-minute surcharge.
`Airline`	Facade (one simple front object that hides many classes behind it) for admin and customer operations; keeps the index instance → reservations.

Class diagram

The course builds six stages (money and messages, people, what the airline owns, flights and their seats, reservations and itineraries, the actors); the finished result:

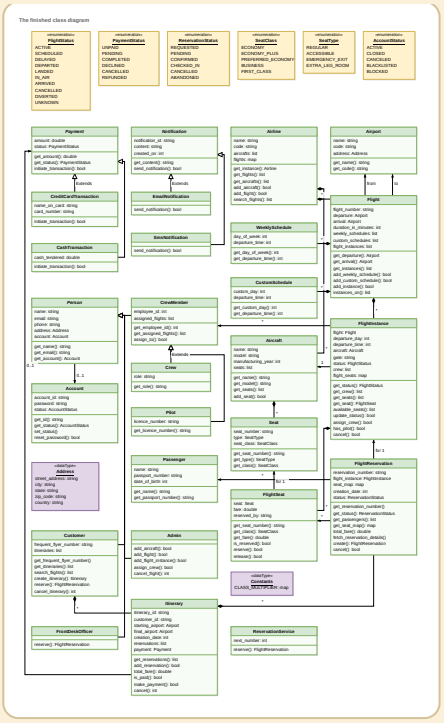
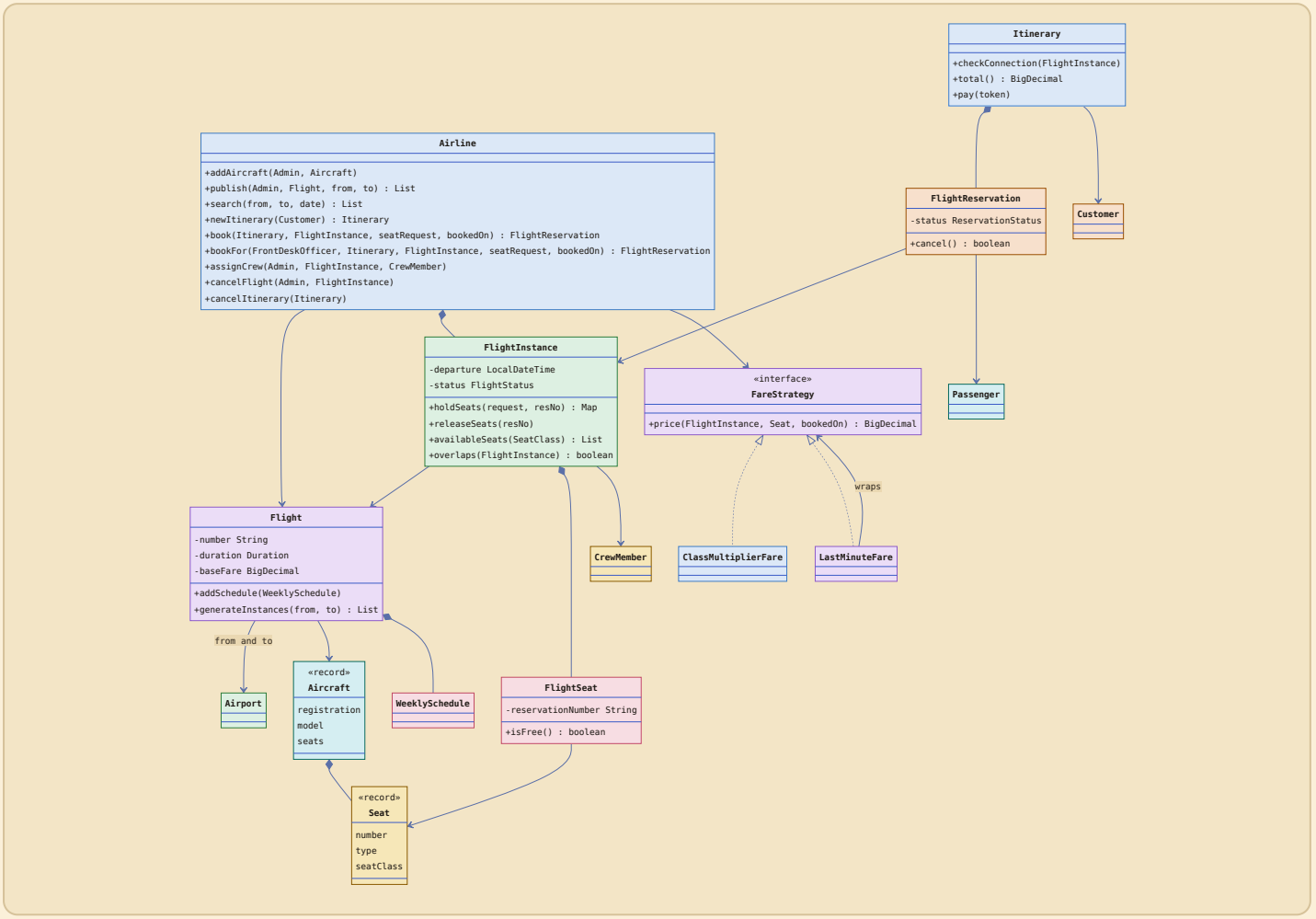


Figure — The finished class diagram of the airline management system (course diagram)

`Flight` is the route, `FlightInstance` the route on a day, `FlightSeat` a seat on that day; a `FlightReservation` maps passengers to flight seats and an `Itinerary` owns its reservations. All the ownership (filled-diamond) arrows run one way: schedules and instances belong to their flight, and flight seats belong to their instance. Airports, the aircraft and the crew are only referred to — cancelling a flight does not destroy an airport. `Passenger` sits outside the `Person`/account tree: the course's reason is that a passenger may never log in, and a customer can book for other people.



Key flows

Reserve a ticket. The customer (or front desk officer) searches; if a flight is available they choose seats and passengers, pay, and receive a confirmation; otherwise they search again.

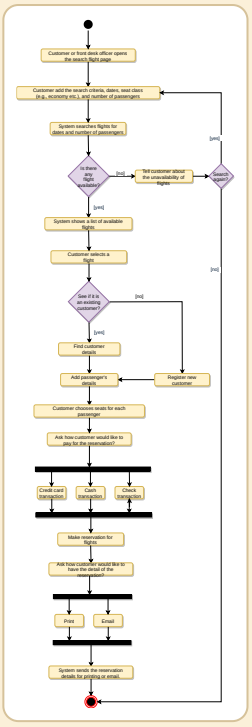


Figure — Activity diagram: reserving a ticket (course diagram)

Cancel a reservation. The system checks that the customer has a valid reservation; if so it cancels it, releases the seats and notifies the customer; if not, it reports the error.

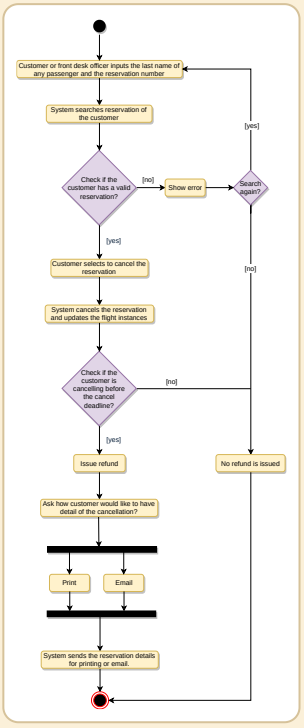
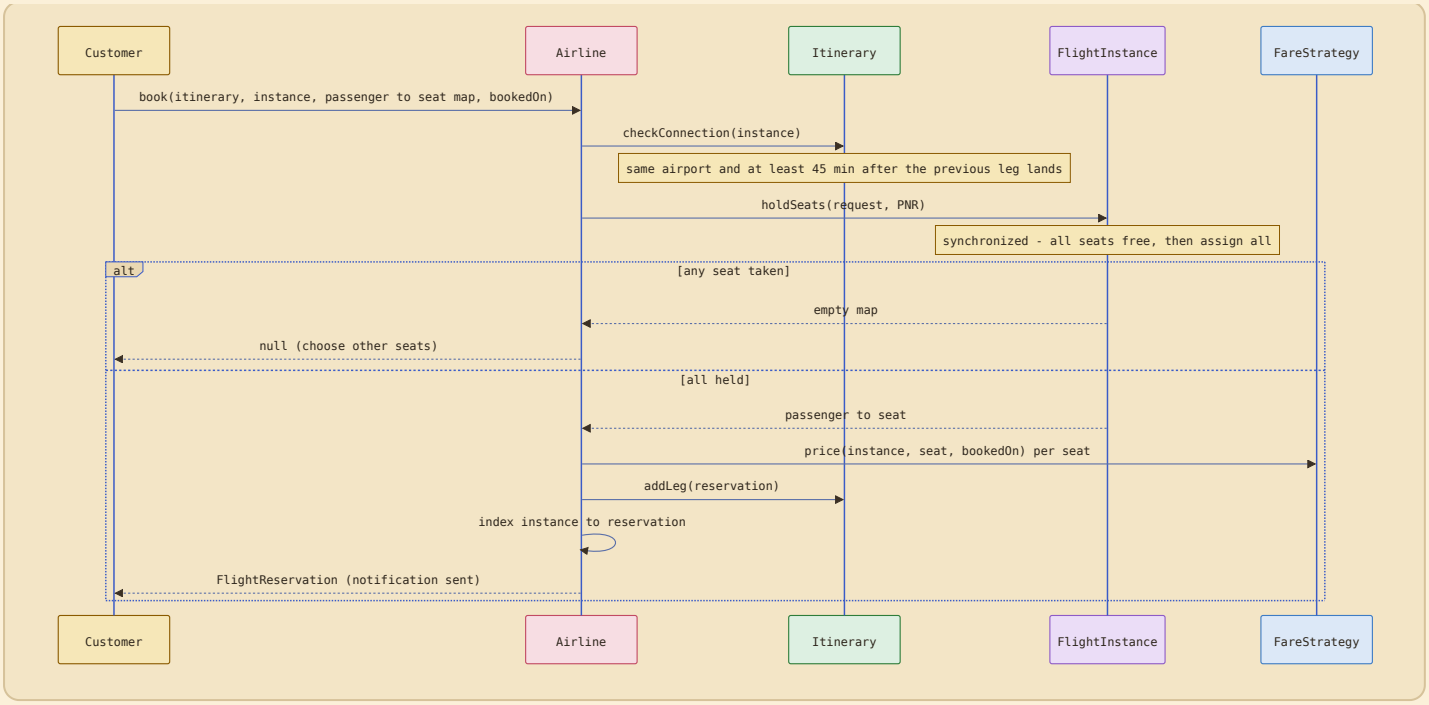
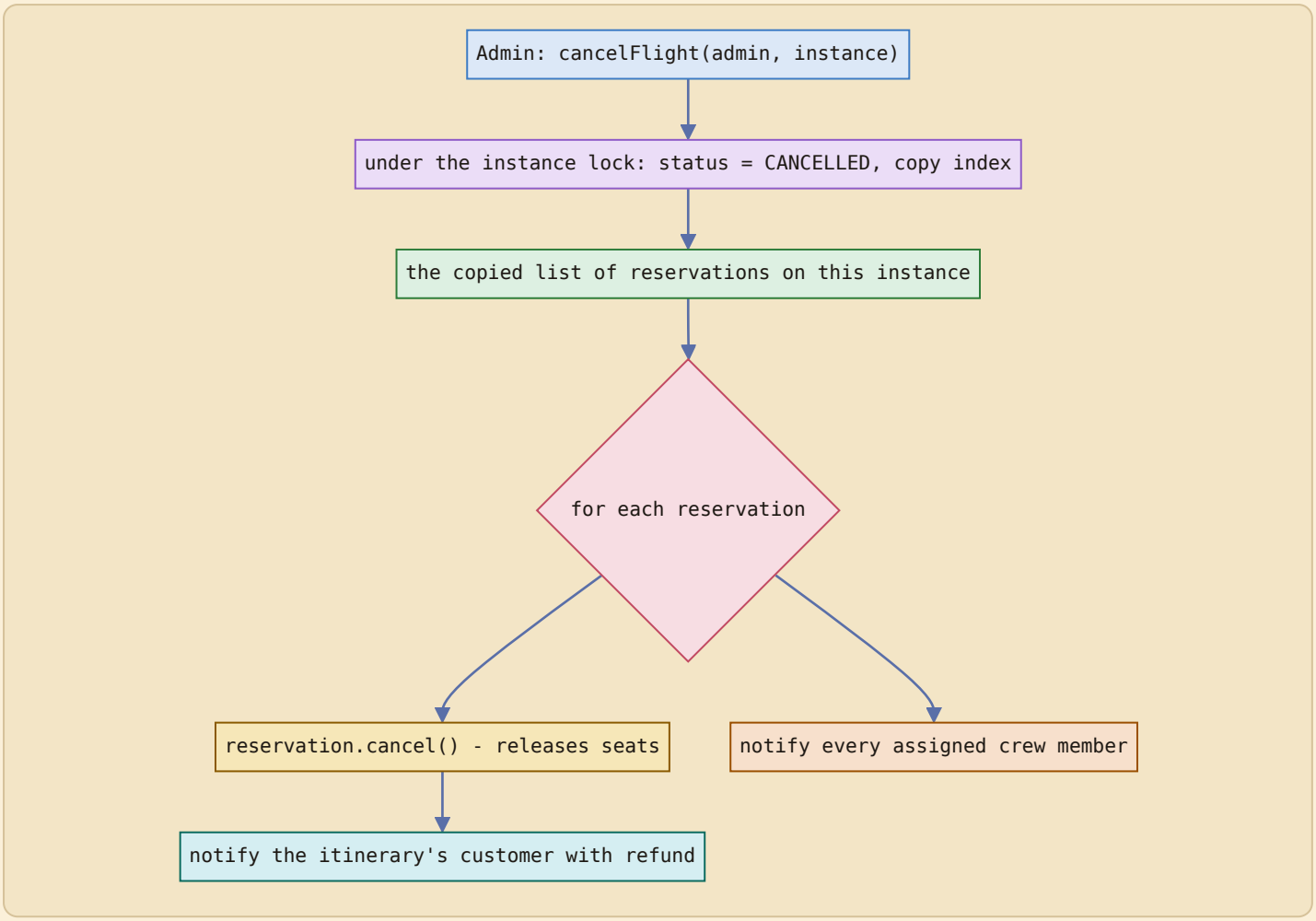


Figure — Activity diagram: cancelling a reservation (course diagram)

Booking one leg of an itinerary:



Admin cancels a flight instance:



Design decisions & patterns used

Decision	Why	Alternative rejected
Flight vs FlightInstance vs Seat vs FlightSeat	12A is free on Tuesday and taken on Thursday, so "booked" and the fare belong to the seat *on an instance*. The course says an older version of its code had drifted from its diagram here — the code is what runs.	A `booked` flag on `Seat`: one booking would block the seat forever.
`holdSeats` is **all-or-nothing** and `synchronized` per instance	A family must get all seats or none; per-instance locks let different flights sell in parallel.	Holding seats one by one (half-booked families), or one global lock (no parallelism).
`Passenger` is not an account	A passenger is a name on a ticket; a customer books for others.	`Passenger extends Person` would force an account on a child.
Itinerary is paid, not the leg	A customer pays for the trip.	Paying per reservation.
Connection rule on `Itinerary`	The course names "no connecting-time check" as a gap; the rule belongs to the thing that holds the legs. We check same airport and a 45-minute minimum.	No check (impossible trips).
Index instance → reservations	The course's other named gap: cancelling a flight told the crew but not passengers, because there was no path from instance to its reservations. The index fixes it.	Scanning every itinerary.
Strategy + Decorator for fares	The course suggests Strategy for fares (base, seasonal, last minute). `ClassMultiplierFare` uses the course's multipliers (1.0, 1.3, 1.5, 2.5, 4.0); `LastMinuteFare` wraps any fare and adds 25% within 3 days.	Price field on the seat.
Crew overlap check	A pilot cannot be on two instances whose times overlap.	Trusting the admin.
Facade (`Airline`)	One entry point for admin and customer operations. Admin-only operations (`addAircraft`, `publish`, `assignCrew`, `cancelFlight`) take an `Admin` parameter, so "only the admin" is enforced by the compiler; `bookFor(FrontDeskOfficer, …)` is the same call as `book` made by a different actor, exactly as in the course's stage 6.	Actors holding the rules.
The course makes `Airline` a Singleton	We construct it once and pass it in, to stay testable.	Global `getInstance()`.

Java implementation

Package `lld.ch16.airline`. Full runnable code is in `code/lld/ch16/airline/`: `Types.java` (enums `FlightStatus`, `ReservationStatus`, `SeatClass`, `SeatType`, `CrewRole`; records `Airport`, `Seat` — the physical seat, which knows nothing about bookings — `Passenger` — a name on a ticket, not an account — `Customer`, `CrewMember`, `Admin`, `FrontDeskOfficer`), `Fleet.java` (`Aircraft`, `WeeklySchedule`, `Flight` — the route, which generates one `FlightInstance` per scheduled day; all times in UTC), `FlightInstance.java` (`FlightSeat`, `FlightInstance`), `Booking.java` (fares, `FlightReservation`, `Itinerary`), `Airline.java` (the facade) and `Main.java`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```
/** One seat on one flight instance: 12A can be free on Tuesday and taken on Thursday. */
final class FlightSeat { private String reservationNumber; /* null = free */ }

final class FlightInstance {   // the route flown on one date: owns its seats, crew list and status
    /** All-or-nothing: a family of three either gets all three seats or none. */
    synchronized Map<Passenger, Seat> holdSeats(Map<Passenger, String> request, String reservationNumber) {
        if (status != FlightStatus.SCHEDULED && status != FlightStatus.DELAYED)
            throw new IllegalStateException(key() + " is " + status);
        for (var seatNo : request.values()) {
            var fs = seats.get(seatNo);
            if (fs == null) throw new IllegalArgumentException("no seat " + seatNo + " on " + key());
            if (!fs.isFree()) return Map.of();
        }
        if (request.values().stream().distinct().count() != request.size())
            throw new IllegalArgumentException("two passengers asked for the same seat");
        Map<Passenger, Seat> out = new LinkedHashMap<>();
        request.forEach((p, no) -> { var fs = seats.get(no); fs.assign(reservationNumber); out.put(p, fs.seat());
});
        return out;
    }
    boolean overlaps(FlightInstance other) {
        return departure.isBefore(other.arrival()) && other.departure().isBefore(arrival());
    }
}
```

```
/** Decorator on any fare: +25% when booked less than 3 days before departure. */
final class LastMinuteFare implements FareStrategy {      // ClassMultiplierFare: baseFare x 1.0/1.3/1.5/2.5/4.0
    public BigDecimal price(FlightInstance fi, Seat seat, LocalDate bookedOn) {
        var p = inner.price(fi, seat, bookedOn);
        long daysBefore = ChronoUnit.DAYS.between(bookedOn, fi.departure().toLocalDate());
        return daysBefore < 3 ? p.multiply(new BigDecimal("1.25")).setScale(2, RoundingMode.HALF_UP) : p;
    }
}

/** A trip of one or more legs (FlightReservations). The itinerary, not each leg, is what gets paid for. */
final class Itinerary {
    static final Duration MIN_CONNECTION = Duration.ofMinutes(45);
    /** Rule the itinerary owns: the next leg must leave from where the last one landed, with time to change. */
    synchronized void checkConnection(FlightInstance next) {
        if (legs.isEmpty()) return;
        var last = legs.getLast().instance();
        if (!last.flight().to().equals(next.flight().from()))
            throw new IllegalArgumentException("leg must depart from " + last.flight().to().code());
        if (next.departure().isBefore(last.arrival().plus(MIN_CONNECTION)))
            throw new IllegalArgumentException("connection too short: " + next.key() + " leaves before " +
last.key() + " lands + 45 min");
    }
    /** Idempotent: a second pay() (double click, retry) does not charge again. */
    synchronized void pay(String cardToken) { if (paid) return; /* charge total() */ paid = true; }
}
// FlightReservation.cancel() is idempotent: returns true only for the call that did the cancelling, then releases
seats.
```

```
/** Facade. Keeps an index flight-instance -> reservations so a cancelled flight can reach every passenger. */
final class Airline {
    FlightReservation book(Itinerary it, FlightInstance fi, Map<Passenger, String> seatRequest, LocalDate
bookedOn) {
        it.checkConnection(fi);
        String resNo = "PNR" + seq.incrementAndGet();
        FlightReservation r;
        synchronized (fi) {        // hold the seats AND index the reservation as one step (see cancelFlight)
            var seatMap = fi.holdSeats(seatRequest, resNo);
            if (seatMap.isEmpty()) return null;
            var amount = seatMap.values().stream().map(s -> fares.price(fi, s, bookedOn)).reduce(BigDecimal.ZERO,
BigDecimal::add);
            r = new FlightReservation(resNo, fi, seatMap, amount);
            owner.put(r, it);
            byInstance.computeIfAbsent(fi, k -> new CopyOnWriteArrayList<>()).add(r);
        }
        it.addLeg(r);
        // notifier.send(... "Booked <key> seats [...] (<PNR>, <amount>)")
        return r;
    }
    void cancelFlight(Admin by, FlightInstance fi) {
        List<FlightReservation> affected;
        synchronized (fi) {        // same lock as book(): no booking can slip past the cancel
            fi.setStatus(FlightStatus.CANCELLED);
            affected = List.copyOf(byInstance.getOrDefault(fi, List.of()));
        }
        for (var r : affected)
            if (r.cancel()) { /* email the itinerary's customer: refunded amount, new itinerary total */ }
        fi.crew().forEach(c -> notifier.send(c.id() + "@crew", "Stand down: " + fi.key() + " cancelled"));
    }
    /** A crew member cannot be on two instances whose times overlap. */
    synchronized void assignCrew(Admin by, FlightInstance fi, CrewMember m) { /* scan instances with overlaps(),
then add */ }
    /** The front desk books for a walk-in customer: the same call, made by a different actor. */
    FlightReservation bookFor(FrontDeskOfficer by, Itinerary it, FlightInstance fi, Map<Passenger, String> req,
LocalDate on) {
        return book(it, fi, req, on);
    }
    // also: addAircraft(Admin, ...), publish(Admin, Flight, from, to), search(from, to, date), scheduleFor(crew),
cancelItinerary(it)
}
```

`Main.java` sets up Global Air with `LastMinuteFare(ClassMultiplierFare)`, one six-seat layout (1A/1B business, 10A–10C economy, 12A economy-plus) on two aircraft, and flights GA212 LHR→NRT (Tue/Thu 10:00, 14 h, base 500), GA100 JFK→LHR (Wed 20:00, 7 h, base 400) and GA300 LHR→JFK (Thu 12:00, 8 h); step 5 races 20 threads for seat 10C.

Output:

```
1) Admin adds flights (routes) and weekly schedules, then publishes instances
   GA212 instances: [GA212@2026-10-06, GA212@2026-10-08, GA212@2026-10-13, GA212@2026-10-15]
2) Search JFK->LHR on 7 Oct and LHR->NRT on 8 Oct
   GA100@2026-10-07 dep 2026-10-07T20:00 arr 2026-10-08T03:00, free seats 6
   GA212@2026-10-08 dep 2026-10-08T10:00 arr 2026-10-09T00:00, free seats 6
3) Priya books a two-leg itinerary for herself and her son
    [email priya@example.com] Booked GA100@2026-10-07 seats [10A, 10B] (PNR102, 800.00)
    [email priya@example.com] Booked GA212@2026-10-08 seats [1A, 1B] (PNR103, 2500.00)
   itinerary total 3300.00 (2 x 400 economy + 2 x 1250 business)
    charged 3300.00 to tok_priya_card for itinerary IT-101
4) Bad connection is rejected
    [email priya@example.com] Booked GA100@2026-10-07 seats [12A] (PNR105, 520.00)
   rejected: connection too short: GA212@2026-10-06 leaves before GA100@2026-10-07 lands + 45 min
    [email priya@example.com] Itinerary IT-104 cancelled
5) 20 customers race for seat 10C on GA212@2026-10-08
    [email racer7@example.com] Booked GA212@2026-10-08 seats [10C] (PNR115, 500.00)
   successful bookings of 10C: 1, free seats left: 3
6) Walk-in at the front desk, last-minute (booked 7 Oct for 8 Oct): pays +25%
    [email leo@example.com] Booked GA212@2026-10-08 seats [12A] (PNR147, 812.50)
   economy-plus fare 500 x 1.3 = 650.00, charged 812.50
7) Admin assigns crew
   rejected: Capt. Kim already flies GA212@2026-10-08 at that time
   Kim's schedule: [GA212@2026-10-08]
8) Admin cancels GA212@2026-10-08: every passenger and crew member is told
    [email priya@example.com] GA212@2026-10-08 is cancelled. Reservation PNR103 refunded 2500.00. Itinerary IT-101
now totals 800.00
    [email racer7@example.com] GA212@2026-10-08 is cancelled. Reservation PNR115 refunded 500.00. Itinerary IT-107
now totals 0.00
    [email leo@example.com] GA212@2026-10-08 is cancelled. Reservation PNR147 refunded 812.50. Itinerary IT-146
now totals 0.00
    [email K1@crew] Stand down: GA212@2026-10-08 cancelled
    [email L1@crew] Stand down: GA212@2026-10-08 cancelled
    [email A1@crew] Stand down: GA212@2026-10-08 cancelled
   search LHR->NRT on 8 Oct now returns 0 flights
9) Priya cancels the rest of her trip; seats on GA100@2026-10-07 come back
    [email priya@example.com] Itinerary IT-101 cancelled
   free economy seats on GA100@2026-10-07: [10A, 10B, 10C]
```

Step 3 prices 2 × 400 economy + 2 × (500 × 2.5) business = 3300, and the second `pay` call charges nothing. Step 5 has 20 threads racing for seat 10C and exactly one wins (which racer wins, and therefore the PNR and itinerary numbers printed for them in steps 5 and 8, changes from run to run). Step 6 is 500 × 1.3 = 650, plus 25% last-minute = 812.50. Step 8 reaches every passenger through the index — the gap the course points out.

Concurrency & edge cases

- **Selling one seat twice.** `holdSeats` is `synchronized` on the instance: it checks every requested seat and only then assigns them all. No other thread can slip in between.
- **Booking while the admin cancels the flight.** `book` holds the seats *and* adds the reservation to the instance → reservations index inside one `synchronized (fi)` block, and `cancelFlight` sets `CANCELLED` and copies that index under the same lock. So every reservation is either in the copy (and gets cancelled and notified) or is refused by `holdSeats` because the flight is already `CANCELLED` — none can slip through un-notified.
- **Double payment.** `Itinerary.pay` returns at once if the itinerary is already paid (step 3 calls it twice and charges once).
- **Duplicate seat in one request** (two passengers asking for 12A) is rejected.
- **Booking a cancelled or departed flight** throws.
- **Idempotent cancel.** `FlightReservation.cancel()` returns `false` if already cancelled, so a customer cancelling while the admin cancels the flight does not double-refund.
- **Crew double-booking** across overlapping instances is rejected (`assignCrew` is `synchronized` over the scan and the add).
- **Time zones.** Our example keeps all times in one reference zone. Real systems store departures as `ZonedDateTime` (a date-time plus its time zone) or `Instant` (an exact moment on the global UTC timeline) with each airport's zone, because London and Tokyo are 8–9 hours apart.

- **Seat holds during payment.** Real booking flows put a seat on a short *hold* (for example 10–15 minutes) while the customer pays, and release it automatically if payment fails. Add a `heldUntil` timestamp on `FlightSeat` and a sweeper (a background job that regularly frees expired holds).

Extensions & follow-up questions

- **"Overbooking on purpose?"** — Airlines sell a few more seats than exist because some passengers do not show up. Model a per-class `overbookLimit` and assign physical seats at check-in.
- **"Check-in and boarding passes?"** — Add `CHECKED_IN` per passenger, a `BoardingPass` (seat, gate, group), and a check-in window.
- **"Baggage?"** — `Baggage(passenger, weight, tag)` on the reservation; fees as extra lines.
- **"Change fees and fare rules?"** — Fare class with rules (refundable, change fee); cancellation refund = fare − fee.
- **"Codeshares?"** — A codeshare is one physical flight sold under several airlines' flight numbers. A `MarketingFlight` (partner's number) pointing at the operating `FlightInstance`.
- **"Delays?"** — `FlightInstance.setStatus(DELAYED)` plus a new departure; notify via the same index; re-check itinerary connections and rebook broken ones.
- **"Scale?"** — Seat inventory per instance is a hot row (one database row that very many requests try to change at once); use one row per seat with a conditional update (`UPDATE … WHERE reservation IS NULL` — it only changes the row if the condition still holds, so exactly one buyer wins) or optimistic locking (no lock is taken; each row carries a version number and a write fails if the version changed since you read it, so you retry), and a cache (a fast in-memory copy of data that is read often) for search.

In the interview

30-second pitch: "Four separate ideas carry the design. `Flight` is the published route; `FlightInstance` is one departure of it on a particular day; `Seat` is the chair fixed in the aircraft; `FlightSeat` is that chair on that one departure, and it is where the fare and the booking live. Reservations map passengers to flight seats and are held atomically per instance. An itinerary groups legs, checks connections and is what gets paid. Fares are a strategy with a last-minute decorator. An index from instance to reservations lets a cancelled flight notify every passenger."

Minutes	Do
0–5	Clarify search, itineraries, multiple passengers, crew, cancellation.
5–10	Actors and use cases.
10–20	The four-way distinction, then the class diagram.
20–35	Code `FlightInstance.holdSeats`, `Airline.book`, `Itinerary.checkConnection`, `cancelFlight`.
35–40	Seat holds during payment, overbooking, time zones.

Expected follow-up: "A flight is cancelled — how do its passengers find out?" — "Through an index from instance to reservations; without it you can only reach the crew."

Self-check

1. **Q:** Where does the "booked" flag live and why? **A:** On `FlightSeat` — a seat's booking is per instance (per date), not per aircraft.
2. **Q:** Why is `Passenger` not a subclass of `Person`/account? **A:** A passenger may never log in; a customer books for other people.
3. **Q:** How do you guarantee a family gets all seats or none? **A:** `holdSeats` checks all requested seats and assigns them in one synchronized block.
4. **Q:** What does `LastMinuteFare` add and which pattern is it? **A:** +25% within 3 days of departure; a Decorator over any `FareStrategy`.
5. **Q:** What two gaps did the course admit and how did we close them? **A:** No passenger notification on flight cancel (fixed by the instance → reservations index) and no connection-time check (fixed in `Itinerary.checkConnection`).

TL;DR

- Flight (route) ≠ FlightInstance (route on a date); Seat (physical) ≠ FlightSeat (seat on that date).
- Hold seats all-or-nothing under the instance's lock.
- Itinerary = legs + connection rule + payment.
- Fares: Strategy + Decorator. Keep an instance → reservations index for cancellations.

Real-world example. Airline systems separate a *flight number* (a published schedule) from a *flight leg on a date*; inventory is controlled per leg and per booking class, and the booking record (a PNR, passenger name record) groups passengers and segments — the same shape as `FlightReservation` and `Itinerary` here. When an airline cancels a flight, its systems find every PNR on that leg and rebook or notify passengers, which is exactly why the index matters.

Design a Course Registration System

Problem & clarifying questions

A **course registration system** lets university students sign up for classes each term. It looks simple, but it is one of the harshest concurrency problems in this chapter: at 09:00 on registration day, thousands of students click "register" for the same few popular sections within seconds.

Terms first:

- A **course** is a catalogue entry (CS201 Data Structures, 4 credits).
- A **section** is one offering of a course with its own instructor, meeting time and capacity (CS201-A, Mon/Wed 10:00–11:30, 30 seats). Students register for **sections**, not courses — this is the same split as Flight vs FlightInstance.
- A **prerequisite** is a course you must have completed first.
- A **waitlist** is a first-come-first-served queue of students who want a seat in a full section.
- A **registration window** is the period when a student may register; universities often open it earlier for seniors (priority registration).

Question	Assumed answer
Register for courses or sections?	Sections.
Prerequisites?	Must be completed (not just in progress).
Capacity?	Per section; when full, join a bounded waitlist.
Waitlist promotion?	Automatic when someone drops, in FIFO order, after re-checking the student's timetable.
Limits?	A per-student credit limit; no two sections that meet at the same time; no two sections of the same course.
Windows?	Opens by class year (seniors first); closes for everyone at one time.
Search?	By course code or title.
Concurrency?	Many students at once; a section must never exceed its capacity; one student must not double-register from two tabs.

Requirements

Functional

1. Students register for sections and view their registered sections.

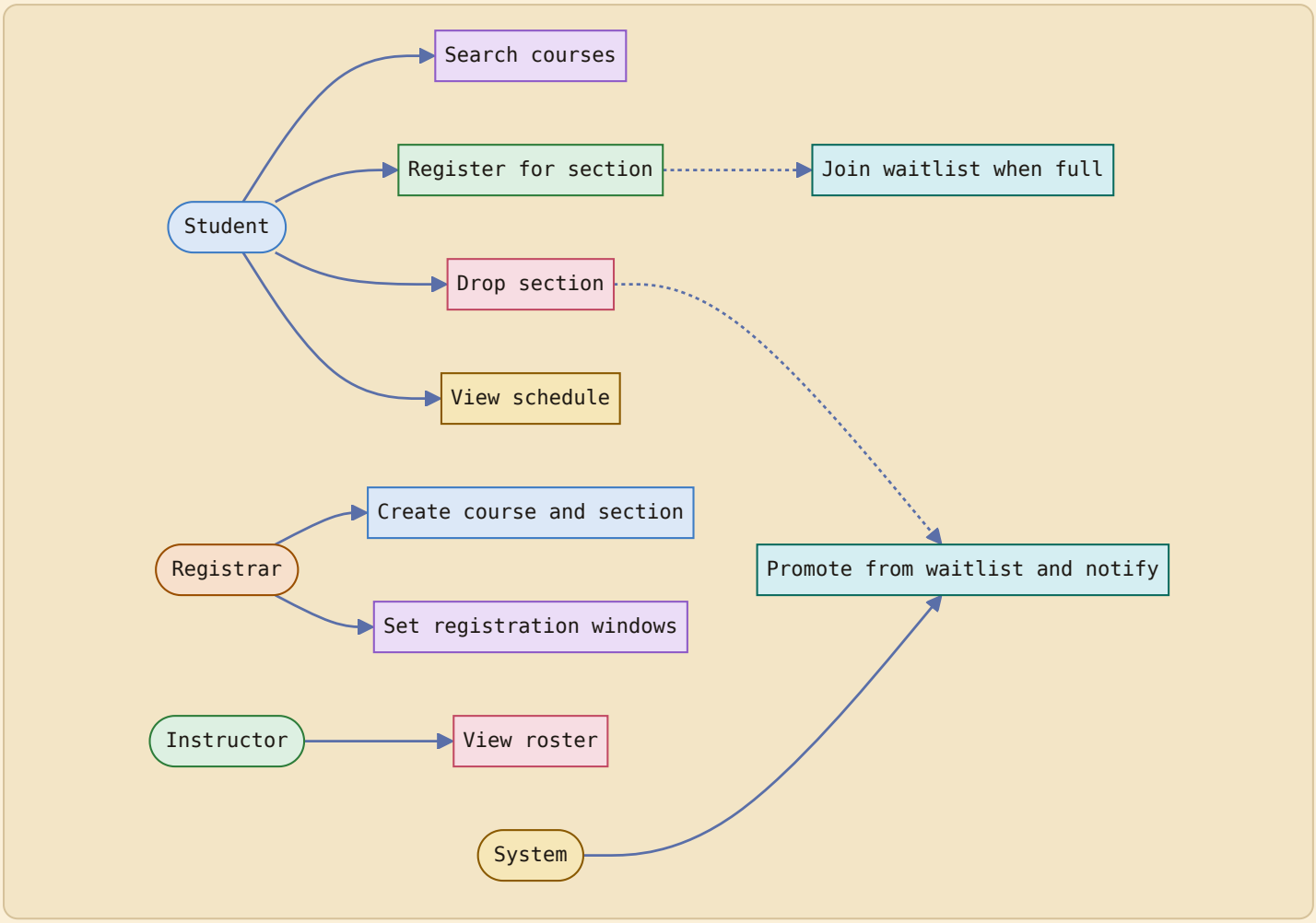
2. Each course has a code, name and credits; each section has an instructor, time slot and enrolment limit.
3. Search courses by code or name.
4. Never enrol beyond a section's capacity; when full, put the student on the waitlist (if it has room).
5. Enforce prerequisites, credit limit, time conflicts, one section per course, and the registration window.
6. On drop, promote the first student on the waitlist automatically and notify them.

Non-functional

- Handle concurrent registrations safely: no race conditions, consistent data.
- Rules must be easy to add or remove (for example "department majors only").
- Extensible for future features (fees, holds, cross-listed courses).

Use cases

- **Student** — searches courses, registers, drops, views schedule, joins a waitlist.
- **Registrar / Admin** — creates courses and sections, sets capacity and windows.
- **Instructor** — views the roster.
- **System** — enforces rules, promotes from the waitlist, sends notifications.

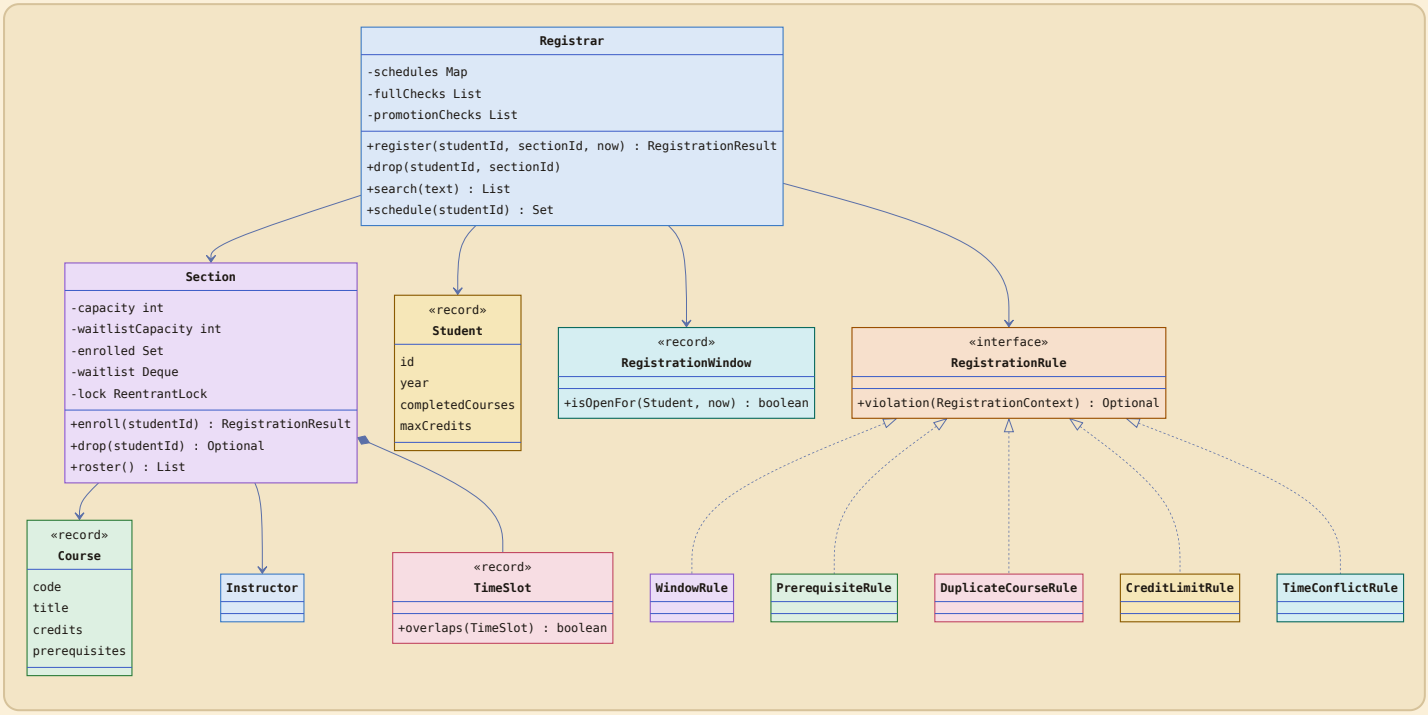


Core entities

Class	Responsibility
`Course`	Code, title, credits, prerequisite codes.
`Section`	One offering: instructor, time slot, capacity, enrolled set, waitlist. **The only place a seat is taken**, under its own lock.
`TimeSlot`	Days + start/end; knows whether it overlaps another slot.
`Instructor`	Who teaches.
`Student`	Year, completed courses, credit limit.
`RegistrationWindow`	Opening time per class year and a common closing time.
`RegistrationRule` (+ `WindowRule`, `PrerequisiteRule`, `DuplicateCourseRule`, `CreditLimitRule`, `TimeConflictRule`)	One rule each; returns the reason for a violation.
`RegistrationContext`	Everything a rule may look at.
`RegistrationResult`, `Outcome`	`ENROLLED`, `WAITLISTED` or `REJECTED` + message.
`Registrar`	The service: runs the rule chain, takes the seat, drops and promotes, keeps each student's schedule, and defines the lock order.

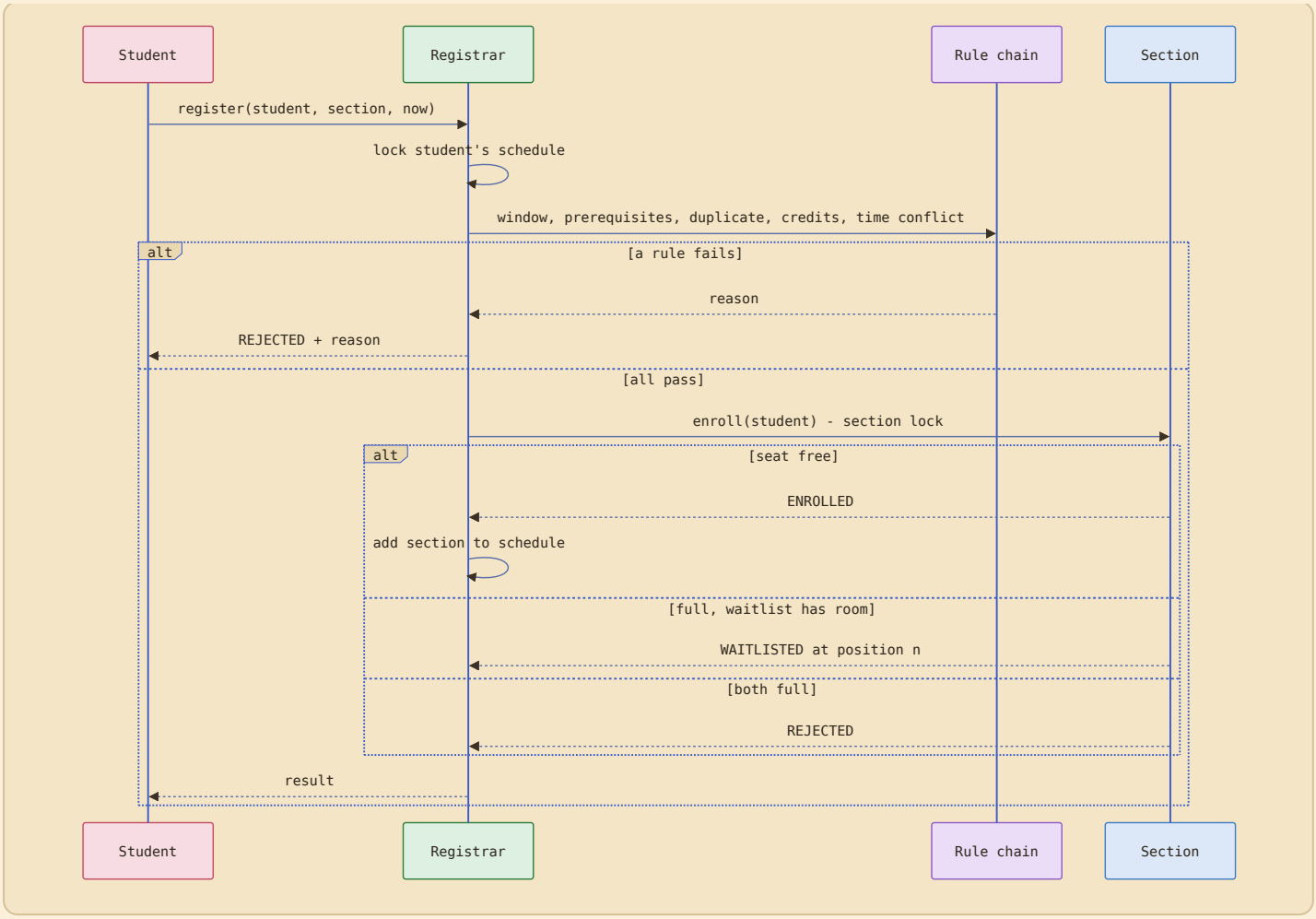
Class diagram

There is no course lesson for this problem; the statement comes from the *awesome-low-level-design* problem set, and every requirement in it is covered above (instructor and capacity sit on the `Section`). The diagram of our Java design:

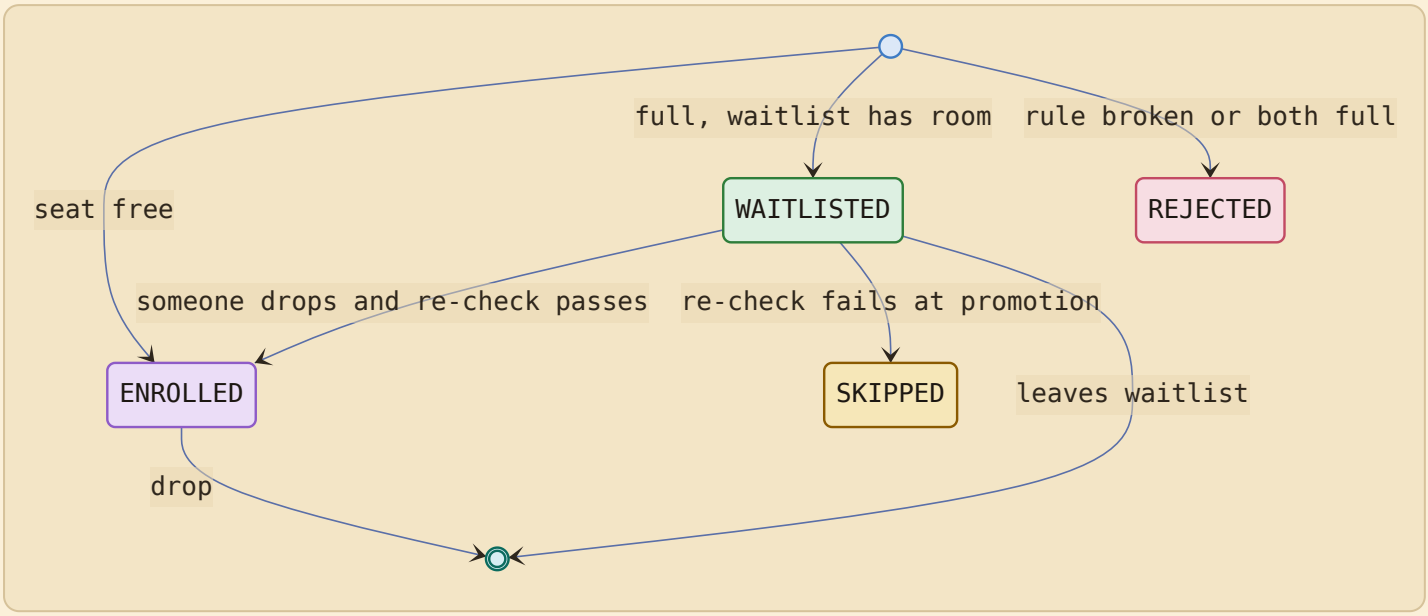


Key flows

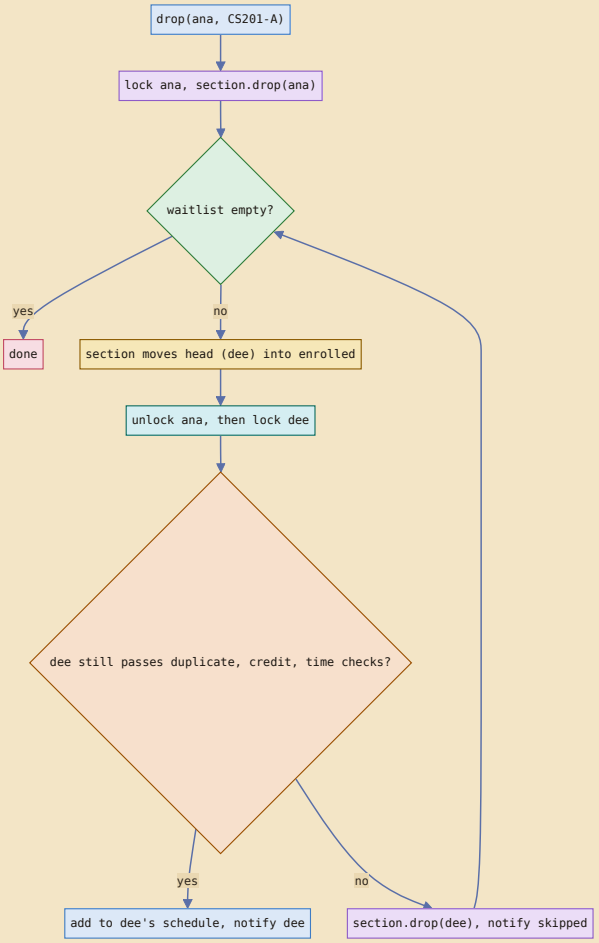
Register:



A student's status in one section:



Drop and promote:


drop(ana, CS201-A)
lock ana, section.drop(ana)
waitlist empty?
yes
no
done
section moves head (dee) into enrolled
unlock ana, then lock dee
dee still passes duplicate, credit, time checks?
yes
no
add to dee's schedule, notify dee
section.drop(dee), notify skipped

Design decisions & patterns used

Decision	Why	Alternative rejected
Course vs Section	Capacity, time and instructor belong to an offering, not the catalogue entry.	Capacity on `Course` (cannot have two sections).
Seat taken only inside `Section.enroll`, under a `ReentrantLock` (Java's explicit lock object, which the same thread may re-acquire) per section	Check-capacity-then-add is atomic; different sections do not block each other.	`synchronized` on the whole registrar (every student in the university waits in one line).
Per-student lock around rules + enroll	Credit limit and time-conflict checks read the schedule; two tabs registering the same student would both pass without it (step 6 of the output).	Lock only the section (a student could exceed credits via two sections in parallel).
Fixed lock order: student, then section	Deadlock needs two threads taking the same two locks in opposite orders. Always taking student before section rules it out. Promotion takes the next student's lock only *after* releasing the dropper's, so two student locks are never held together.	Promoting while still holding the dropper's lock (student → section → student: deadlock risk).
Rules as objects (Specification / Chain of Responsibility style: each check is its own object, and a request passes along the chain until one rejects it)	Each rule is one small class; the registrar runs them in order and stops at the first failure. A new rule ("majors only") is a new class plus one list entry — Open/Closed. The promotion path reuses a *subset* of the same rules.	One long `register` method with nested `if`s.
Waitlist is a FIFO `Deque`, bounded (a double-ended queue with a maximum size)	Fair (first come, first served) and prevents unbounded queues.	Unbounded list; priority by year (a valid extension).
Re-check on promotion	A student may have registered for a clashing section while waiting.	Promote blindly (broken timetables).
Time passed in (`now`)	Makes window logic testable without waiting for real dates.	Calling `LocalDateTime.now()` inside rules.
Singleton? The GitHub problem uses a singleton `CourseRegistrationSystem`.	We construct one `Registrar` and pass it in: one instance, still testable.	Global `getInstance()`.

Java implementation

Package `lld.ch16.courseregistration`. Full runnable code is in `code/lld/ch16/courseregistration/`: `Types.java` (records `Course` — a catalogue entry whose prerequisites are course codes — `Instructor`, `TimeSlot`, `Student`, `RegistrationWindow`, `RegistrationResult`; enum `Outcome` = `ENROLLED`/`WAITLISTED`/`REJECTED`), `Section.java`, `Rules.java`, `Registrar.java` and `Main.java`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```java
/** When a section meets, e.g. Mon+Wed 10:00-11:30. */
record TimeSlot(Set<DayOfWeek> days, LocalTime start, LocalTime end) {
    boolean overlaps(TimeSlot o) {
        boolean sharedDay = days.stream().anyMatch(o.days::contains);
        return sharedDay && start.isBefore(o.end) && o.start.isBefore(end);
    }
}
/** Priority registration: each class year opens at its own time; everyone closes together. */
record RegistrationWindow(Map<Integer, LocalDateTime> opensByYear, LocalDateTime closes) {
    boolean isOpenFor(Student s, LocalDateTime now) {
        var opens = opensByYear.get(s.year());
        return opens != null && !now.isBefore(opens) && now.isBefore(closes);
    }
}

/** One offering of a course. Capacity and waitlist live HERE, guarded by the section's own lock. */
final class Section {
    private final Set<String> enrolled = new LinkedHashSet<>();
    private final Deque<String> waitlist = new ArrayDeque<>();     // FIFO, bounded by waitlistCapacity
    private final ReentrantLock lock = new ReentrantLock();

    /** The only place a seat is taken: check-and-add in one critical section. */
    RegistrationResult enroll(String studentId) {
        lock.lock();
        try {
            if (enrolled.contains(studentId)) return RegistrationResult.rejected("already enrolled in " + id);
            if (waitlist.contains(studentId)) return RegistrationResult.rejected("already on the waitlist of " +
id);
            if (enrolled.size() < capacity) {
                enrolled.add(studentId);
                return new RegistrationResult(Outcome.ENROLLED, "enrolled in " + id + " (" + enrolled.size() + "/"
+ capacity + ")");
            }
            if (waitlist.size() < waitlistCapacity) {
                waitlist.addLast(studentId);
                return new RegistrationResult(Outcome.WAITLISTED, "waitlisted for " + id + " at position " +
waitlist.size());
            }
            return RegistrationResult.rejected(id + " and its waitlist are full");
        } finally {
            lock.unlock();
        }
    }
    /** Removes the student; if a seat opened, moves the head of the waitlist in and returns who it was. */
    Optional<String> drop(String studentId) { /* under lock: remove; enrolled.add(waitlist.pollFirst()) */ }
}
```

```
/** Specification-style rule: returns a reason when the request breaks it. Run as a chain; first violation wins.
*/
interface RegistrationRule { Optional<String> violation(RegistrationContext c); }
// RegistrationContext(student, section, currentSchedule, now, window)
// WindowRule, PrerequisiteRule (lists missing codes), DuplicateCourseRule, CreditLimitRule, TimeConflictRule
final class CreditLimitRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        int current = c.currentSchedule().stream().mapToInt(s -> s.course().credits()).sum();
        int after = current + c.section().course().credits();
        return after <= c.student().maxCredits() ? Optional.empty()
                : Optional.of("credit limit " + c.student().maxCredits() + " exceeded (" + after + ")");
    }
}

/** Lock order is ALWAYS student-then-section, which rules out deadlock. */
final class Registrar {
    // fullChecks = Window, Prerequisite, DuplicateCourse, CreditLimit, TimeConflict
    // promotionChecks = DuplicateCourse, CreditLimit, TimeConflict   (a subset of the same rules)
    RegistrationResult register(String studentId, String sectionId, LocalDateTime now) {
        var schedule = schedules.get(studentId);
        synchronized (schedule) {                                   // 1st lock: the student
            var ctx = new RegistrationContext(student, section, Set.copyOf(schedule), now, window);
            var v = firstViolation(fullChecks, ctx);
            if (v.isPresent()) return RegistrationResult.rejected(v.get());
            var result = section.enroll(studentId);                 // 2nd lock: the section
            if (result.outcome() == Outcome.ENROLLED) schedule.add(section);
            return result;
        }
    }
    void drop(String studentId, String sectionId) {
        Optional<String> promoted;
        synchronized (schedule) { promoted = section.drop(studentId); schedule.remove(section); }
        notifier.accept(studentId + " dropped " + sectionId);
        // Finish promotions OUTSIDE the dropper's lock, one student lock at a time (no nested student locks).
        while (promoted.isPresent()) promoted = finishPromotion(promoted.get(), section);
    }
    // finishPromotion: under the promoted student's lock, re-run promotionChecks (their timetable may have
    // changed while waiting); on success add to schedule, otherwise notify "skipped" and promote the next one.
}
```

`Main.java` opens seniors (year 4) on Nov 2, juniors Nov 3, sophomores Nov 4, first-years Nov 5 at 09:00, closing Nov 20 17:00; CS201-A has capacity 2 and waitlist 2, CS101-A has 30 + 10.

Output:

```
1) Search 'cs2': [CS201-A [MONDAY, WEDNESDAY] 10:00-11:30, CS201-B [TUESDAY, THURSDAY] 14:00-15:30]
2) Rules
   ben  CS201-A on Nov 2  -> REJECTED: registration window is closed for year 3 at 2026-11-02T10:00
   ana  CS201-A on Nov 2  -> ENROLLED: enrolled in CS201-A (1/2)
   ana  MATH201-A         -> REJECTED: time conflict with CS201-A [MONDAY, WEDNESDAY] 10:00-11:30
   ana  CS201-B           -> REJECTED: already registered for CS201 in section CS201-A
   ana  CS301-A           -> REJECTED: missing prerequisites [CS201, MATH201]
   cy   CS201-B on Nov 4  -> REJECTED: missing prerequisites [CS101]
   ana  after close       -> REJECTED: registration window is closed for year 4 at 2026-11-21T09:00
3) Capacity 2 + waitlist 2 on CS201-A (juniors open Nov 3)
   ben  CS201-A           -> ENROLLED: enrolled in CS201-A (2/2)
   dee  CS201-A           -> WAITLISTED: waitlisted for CS201-A at position 1
   eve  CS201-A           -> WAITLISTED: waitlisted for CS201-A at position 2
   fay  CS201-A           -> REJECTED: CS201-A and its waitlist are full
   ben  MATH201-A         -> REJECTED: credit limit 4 exceeded (7)
4) Ana drops CS201-A: the head of the waitlist is promoted
    [notify] ana dropped CS201-A
    [notify] dee promoted from waitlist into CS201-A
   roster [ben, dee], waitlist [eve]
5) 50 first-years grab 30 seats (+10 waitlist) in CS101-A at 09:00 sharp
   ENROLLED=30 WAITLISTED=10 REJECTED=10, roster size 30
6) One student double-clicks: two threads, same request
   {ENROLLED=1, REJECTED=1}, gus schedule [CS201-B]
```

Step 2 shows each rule firing with a clear reason. Step 3 fills two seats and two waitlist places. Step 4 promotes Dee automatically. Step 5 is the "09:00 rush": 50 threads, 30 seats, 10 waitlist places — the counts are exact every run. Step 6 is the double-click: two threads for the same student and section, exactly one succeeds.

Concurrency & edge cases

- **Overselling a section** — prevented by the section lock around "size < capacity → add".
- **Double-click / two tabs** — prevented by the per-student lock; the second request sees the first enrolment and is rejected by `DuplicateCourseRule` (or by the section's "already enrolled" check).
- **Deadlock** — avoided by the fixed lock order (student → section) and by never holding two student locks at once.
- **Waitlisted in one section, enrolled in another section of the same course** — possible while waiting, because waitlist entries are not in the schedule; the promotion re-check (`DuplicateCourseRule`) catches it and skips to the next student.
- **Registration closes while on the waitlist** — promotion does not re-check the window (it is the drop that triggers it). Many universities stop automatic promotion after a deadline; add `WindowRule` to `promotionChecks` if that is the policy.
- **Dropping a section you are not in** → `IllegalArgumentException`.
- **Rules with time** — the window uses half-open time (`opens ≤ now < closes`).
- **Scaling beyond one JVM** (Java Virtual Machine, one running copy of the program) — in-memory locks cover one process. With several app servers behind a load balancer, the seat count moves to the database: `UPDATE section SET enrolled = enrolled + 1 WHERE id = ? AND enrolled < capacity` succeeds for exactly `capacity` requests (an atomic conditional update), or use optimistic locking with a version column. A queue in front of hot sections smooths the 09:00 spike. (Load balancers, queues and databases at scale are covered in System Design Fundamentals.)

Extensions & follow-up questions

- **"Co-requisites or 'prerequisite OR instructor permission'?"** — Model prerequisites as a small expression tree (`And`, `Or` operator nodes over `CourseCompleted`, `Permission` leaves) — the Composite pattern (§9.3), where a group is treated like a single object — evaluated in `PrerequisiteRule`.
- **"Holds (unpaid fees, advising)?"** — A `HoldRule` that rejects if the student has an active hold.
- **"Priority waitlists (seniors first)?"** — Replace the `Deque` with a `PriorityQueue` (a queue that always hands out the smallest item by some ordering, not the oldest) ordered by (year, time joined).
- **"Seat reservations for majors?"** — Split capacity into buckets (majors 20, others 10) and check the bucket in `Section.enroll`.
- **"Linked sections (lecture + lab)?"** — Register both atomically: lock both sections in a fixed order (by id), check both, enrol both.
- **"Notify by email/SMS?"** — The registrar takes a `Consumer<String>` notifier (a function that takes a value and returns nothing); swap in an email service or publish to a message queue (Kafka, RabbitMQ: stores messages until another program reads them).
- **"Course fees and billing?"** — Add a `Bill` per student per term, a line per enrolled section, recomputed on drop.

In the interview

30-second pitch: "Students register for sections, not courses. A section owns capacity and a bounded FIFO waitlist, and takes seats only inside its own lock. The registrar runs a chain of rule objects — window, prerequisites, duplicate course, credit limit, time conflict — while holding the student's lock, then asks the section for a seat. Lock order is always student then section, so no deadlocks. On drop the head of the waitlist is promoted after a re-check."

Minutes	Do
0–5	Clarify course vs section, rules, waitlist, windows, concurrency.
5–12	Entities and class diagram.
12–30	Code `Section.enroll/drop`, the rule interface and two rules, `Registrar.register`.
30–40	Concurrency: run the 50-threads demo in your head; lock order; DB conditional update for scale.

Self-check

1. **Q:** Why do students register for sections, not courses? **A:** Capacity, time and instructor belong to each offering; a course can have many sections.
2. **Q:** Which two locks does `register` take and in what order? **A:** The student's schedule lock, then the section's lock — always in that order.
3. **Q:** Why not simply make `register` `synchronized` on the registrar? **A:** It would be correct but serialise (force to run strictly one at a time) every student in the university; per-section and per-student locks let unrelated requests run in parallel.
4. **Q:** Why re-check rules on promotion? **A:** The student's timetable may have changed while waiting (a clash or the same course in another section).
5. **Q:** How do you enforce capacity with many servers? **A:** An atomic conditional update in the database (`… WHERE enrolled < capacity`) or optimistic locking.

TL;DR

- Course (catalogue) vs Section (offering with capacity, slot, waitlist).
- Rules are small objects run as a chain; add a rule without touching the registrar's logic.
- Seats are taken only inside `Section.enroll` under the section's lock; student lock first, section lock second.
- Drop → promote waitlist head → re-check → notify.
- At scale: conditional update in the database.

Real-world example. University student-information systems (for example Ellucian Banner, used by many US universities) run priority registration windows by class standing, check prerequisites and time conflicts, and keep section waitlists that notify a student when a seat opens (some schools enrol the student automatically; others give a time-limited window to claim the seat). Registration-day slowdowns at such schools are the 09:00 "thundering herd" (a huge crowd of requests all arriving at the same instant) this design addresses with per-section locking and, at scale, database conditional updates.

Design a Meeting Scheduler

Problem & clarifying questions

A **meeting scheduler** is the part of a company calendar that books meeting rooms: an organizer picks a time and the people to invite, the system finds a room that is large enough and free, sends the invitations, and collects the answers. It is the fifth booking problem in this chapter, with two twists the others do not have:

- **The calendar is continuous time, not slots.** A car is booked by the day and a restaurant table by the hour, but a meeting may run 10:15–10:40. Conflict detection must work on arbitrary time ranges.
- **People are booked as well as rooms, but differently.** Two meetings in one room at the same time is impossible. One *person* in two meetings at the same time is merely unwise, and real calendars allow it.

Terms: an **organizer** creates a meeting and **invitees** are asked to attend. An **RSVP** ("please reply") is an invitee's answer — accepted, tentative or declined — and is **pending** until given. A **half-open interval**

`[start, end)` includes its start and excludes its end, so 10:00–11:00 and 11:00–12:00 do **not** overlap; two such intervals overlap exactly when `a.start < b.end && b.start < a.end`. A **hard conflict** blocks the booking (room taken); a **soft conflict** only warns (invitee busy). A **recurring meeting** is a **series** of **occurrences** generated by a rule such as "weekly, 4 times".

Question	Assumed answer
What is booked?	A room for a time interval. A meeting always has exactly one room in the core design (virtual meetings are an extension).
Who picks the room?	Either the organizer names a room, or the system picks one that fits.
What does "fits" mean?	Capacity ≥ organizer + invitees, and the room has the required equipment (video, projector, whiteboard).
Is an invitee's double-booking an error?	No: it is reported as a warning to the organizer; the invitee decides by RSVP.
Can invitees change their answer?	Yes, until the meeting is cancelled.
Who can reschedule or cancel?	Only the organizer. Rescheduling resets every RSVP to pending.
Notifications?	On invite, RSVP (to the organizer), reschedule and cancel. Channel (e-mail, push) is pluggable.
Recurring meetings?	An extension: daily or weekly, N times, all occurrences in the same room or not at all.
Concurrency?	Many organizers book at the same time; a room must never hold two overlapping meetings.

Requirements

Functional

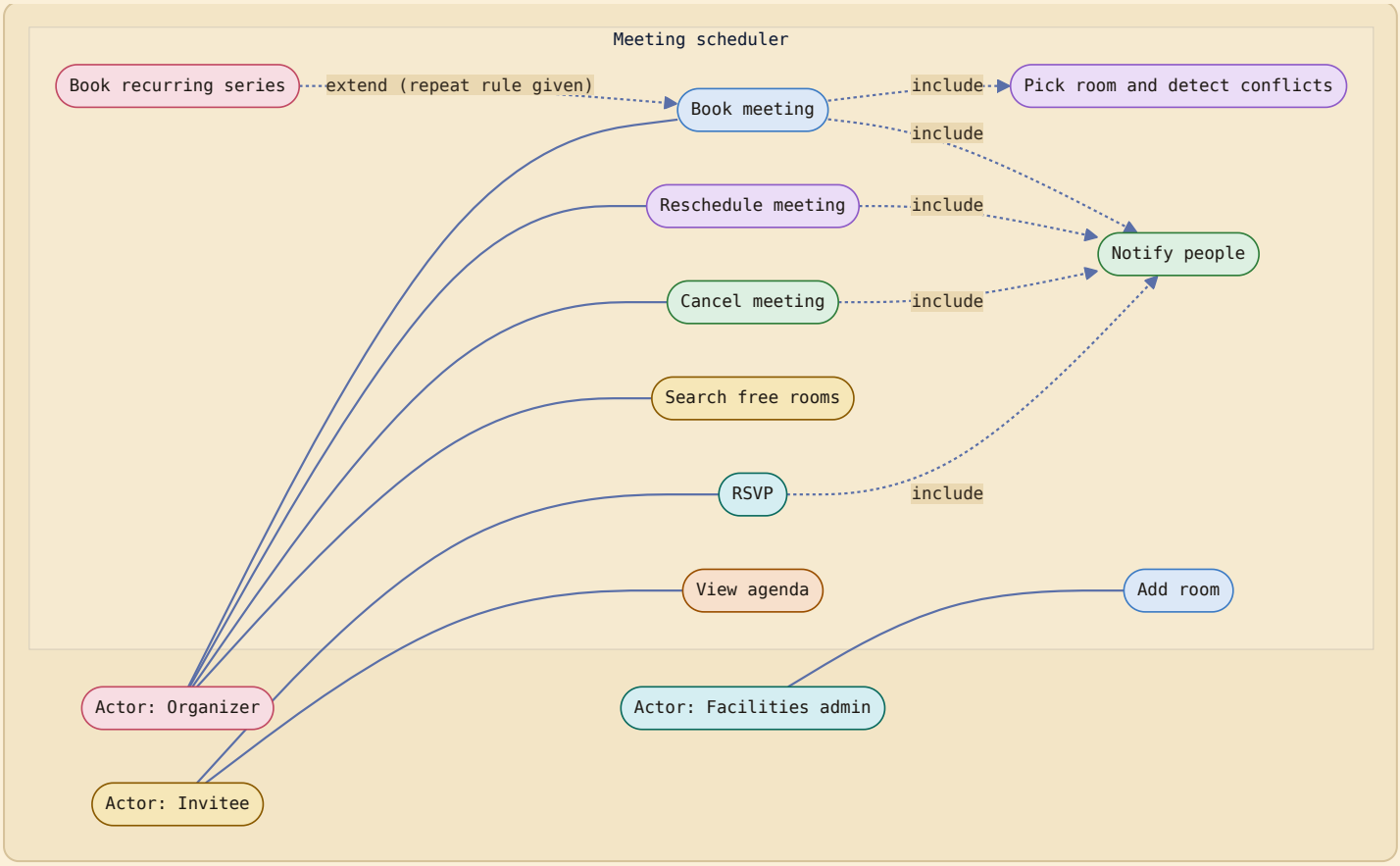
1. Register rooms with an id, name, capacity and equipment.
2. Book a meeting: organizer, title, interval, invitees, required equipment, and optionally a named room.
3. If no room is named, choose one automatically among the rooms that fit and are free.
4. Reject a booking whose room is taken for any part of the interval (hard conflict); allow back-to-back meetings.
5. Warn (but do not block) when the organizer or an invitee is already in another meeting at that time.
6. Send invitations; let invitees accept, decline or reply tentative; tell the organizer.
7. Let the organizer reschedule (same room) or cancel; notify invitees; free the room on cancel.
8. Show a person's agenda for a day and search for free rooms.
9. Extension: book a recurring series and cancel a whole series.

Non-functional

- **No double-booking** of a room, even when many requests arrive at the same instant.
- **Unrelated bookings do not wait for each other** (two different rooms never share a lock).
- **Extensible:** a new room-selection policy or a new notification channel is a new class, not an edit to the scheduler.

Use cases

Room choice, conflict detection and notification are steps the system performs inside other use cases, so they appear as *included* use cases, not as a "System" actor (notation: §3.10).

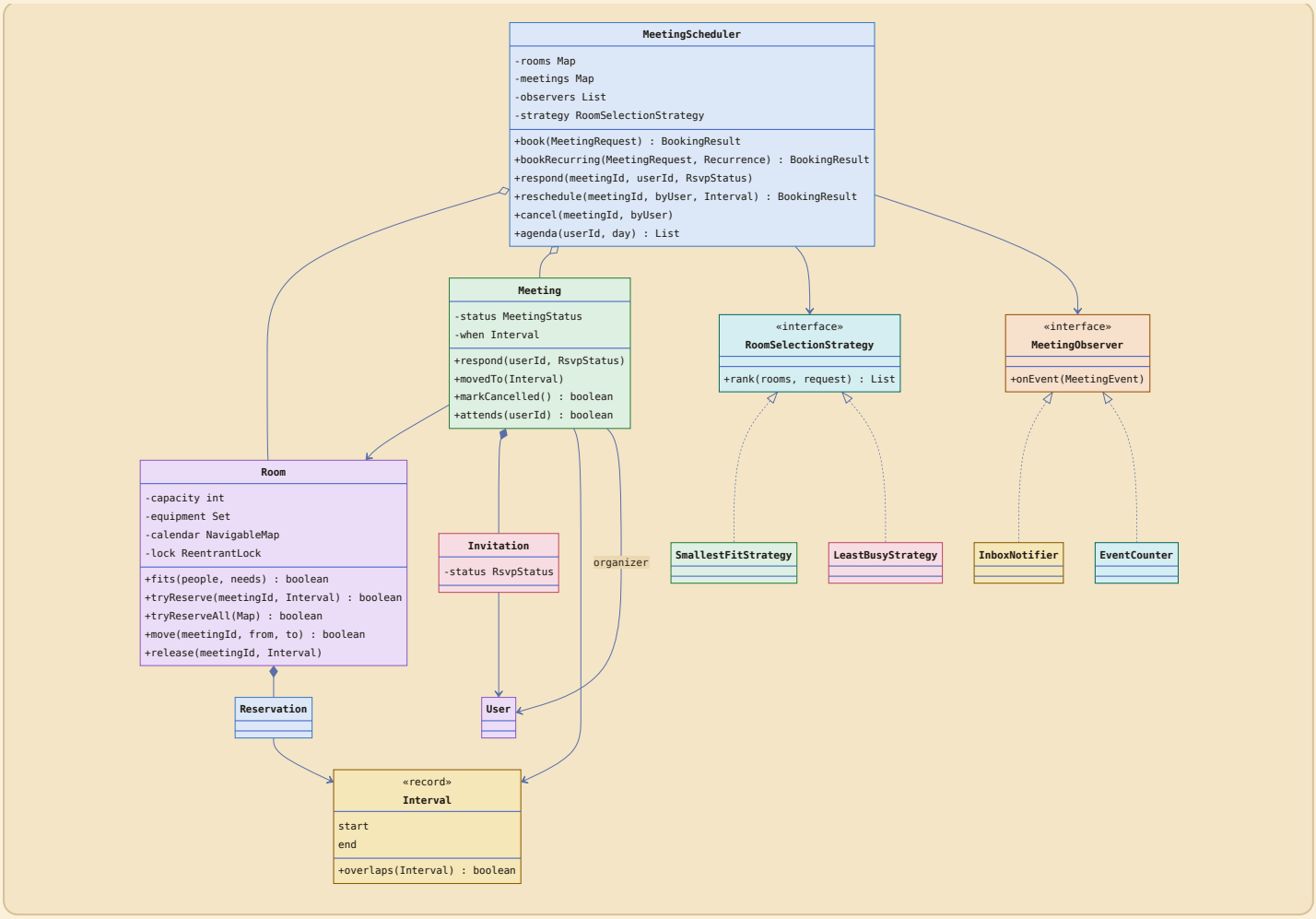


Core entities

Class	Responsibility
`Interval`	Half-open `[start, end)`; knows whether it overlaps another.
`Room`	Capacity, equipment and **its own calendar** of reservations. The **only place a time range is claimed**, under the room's lock.
`Meeting`	Id, title, organizer, room, interval, status (`SCHEDULED` / `CANCELLED`) and the invitations. Optional series id.
`Invitation`	One invitee and their `RsvpStatus` (`PENDING`, `ACCEPTED`, `TENTATIVE`, `DECLINED`).
`MeetingRequest`	What the organizer asks for (people, time, needs, optional room).
`RoomSelectionStrategy` (+ `SmallestFitStrategy`, `LeastBusyStrategy`)	Orders the rooms that fit; the scheduler tries them in that order.
`MeetingObserver` (+ `InboxNotifier`, `EventCounter`)	Listens to `MeetingEvent`s (invited, RSVP, rescheduled, cancelled).
`Recurrence`	Extension: frequency + count; expands one interval into occurrences.
`MeetingScheduler`	The service: book, book a series, RSVP, reschedule, cancel, agenda, soft-conflict warnings. Defines the lock order.
`BookingResult`	Success with the meeting(s) and any warnings, or failure with a reason.

Class diagram

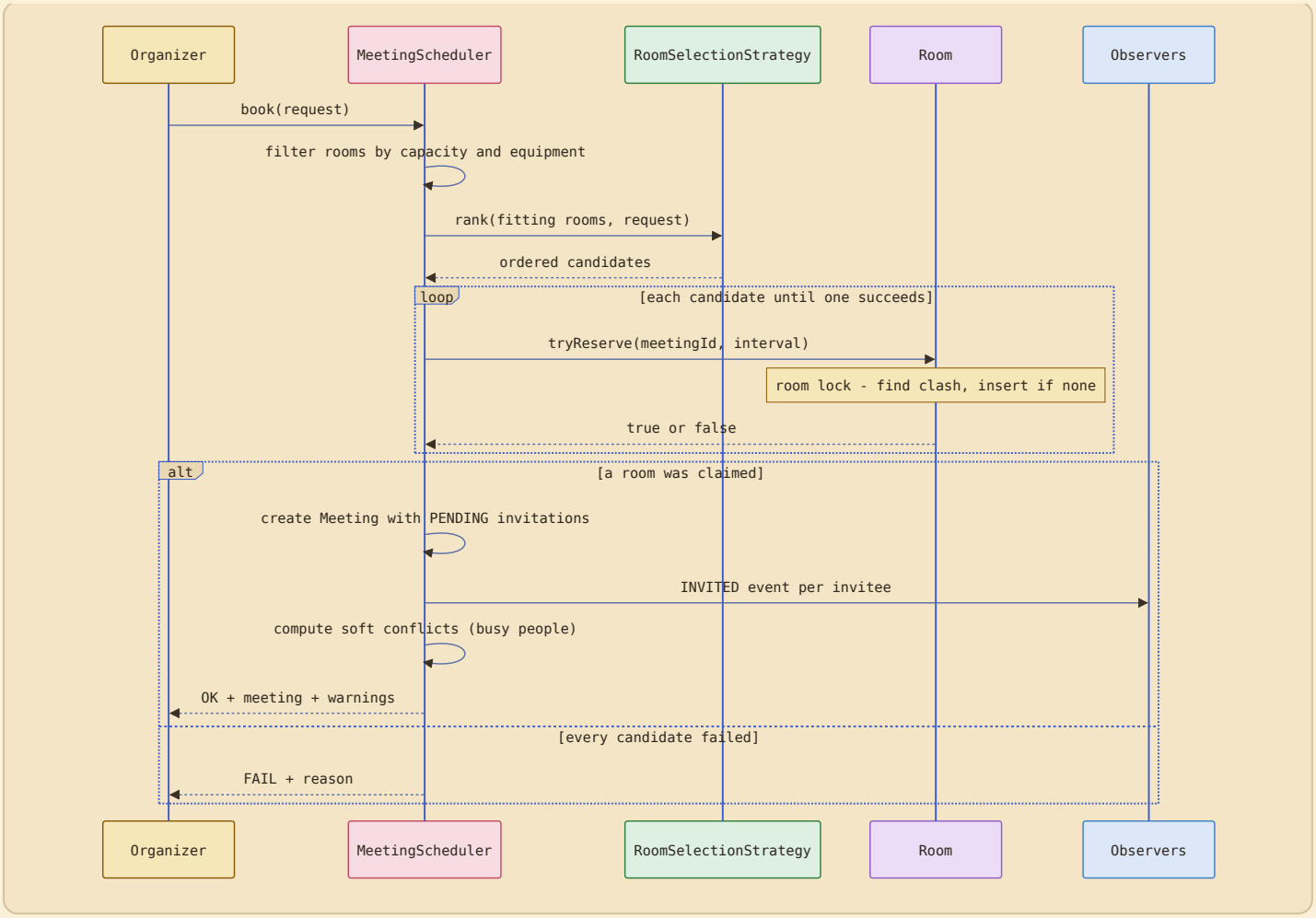
Whiteboard order: `Room` and its calendar, then `Meeting` with its `Invitation`s, then `MeetingScheduler` with its strategy and observers.



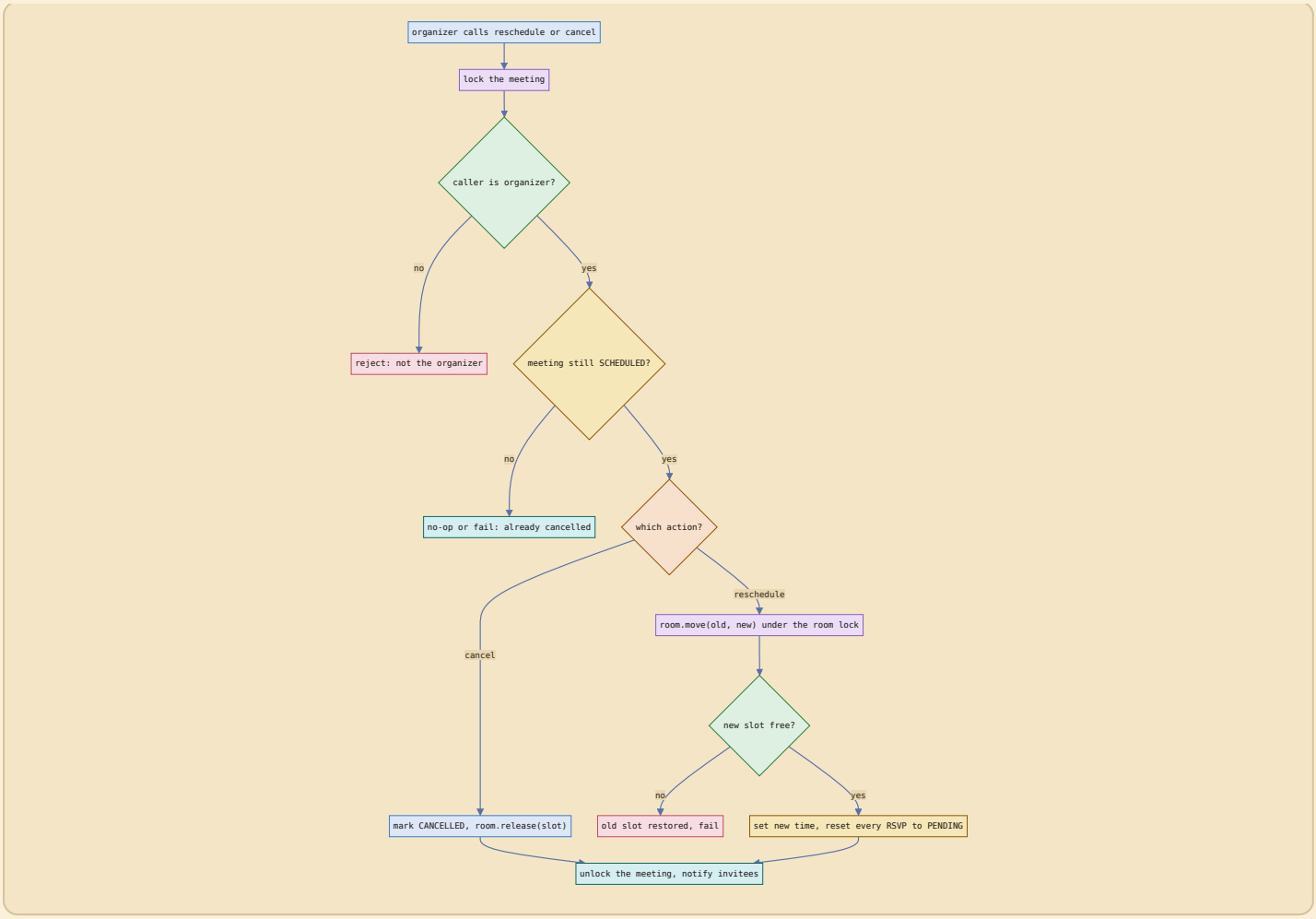
`Meeting --> Room` is plain association (cancelling a meeting does not destroy the room). `Room *-- Reservation` is composition: a reservation lives only in its room's calendar, and the room, not the meeting, owns the truth about which times are taken.

Key flows

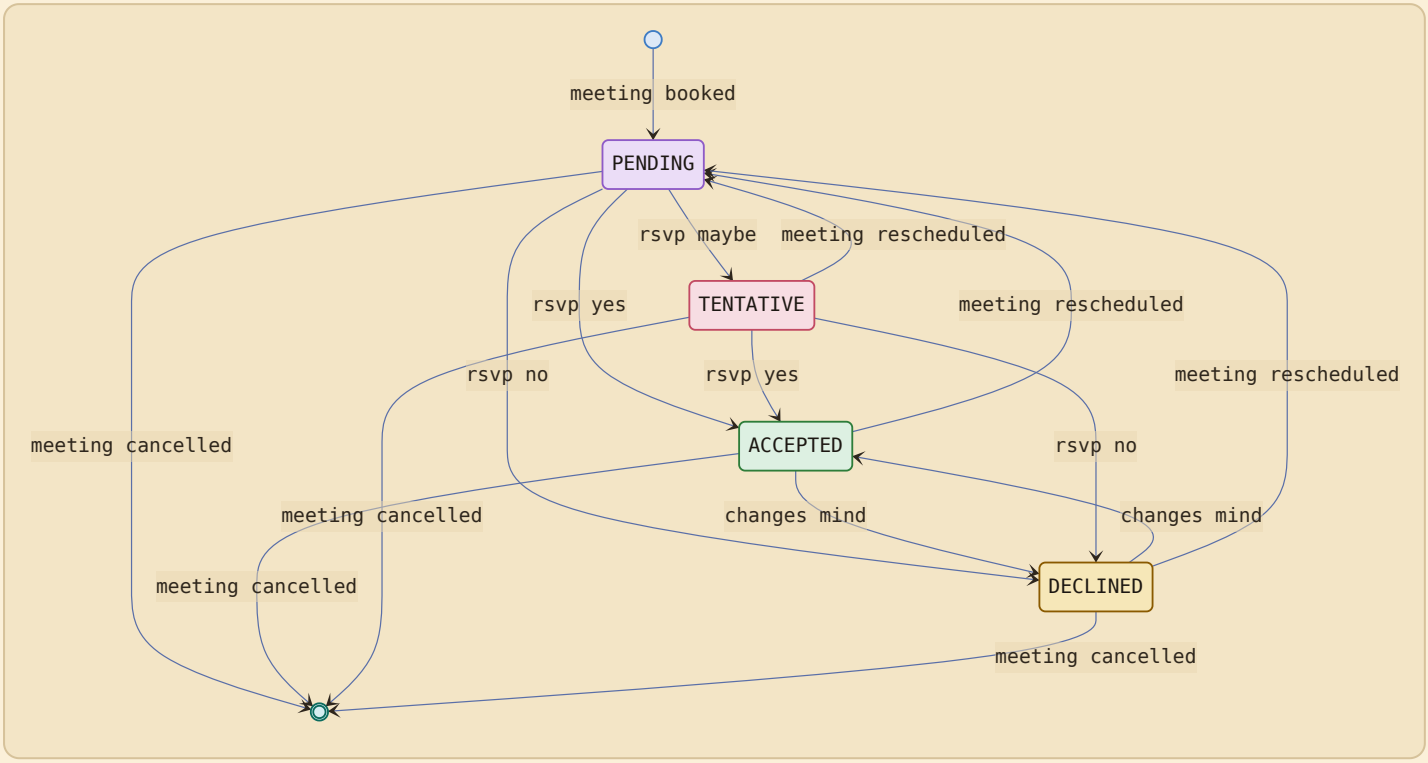
Book a meeting (sequence diagram; the "try the next room" loop is where the race is decided):



Reschedule or cancel (activity diagram; both start by taking the meeting's lock, so they cannot interleave):



Lifecycle of one invitation, inside its meeting (state diagram). The meeting itself has only two states, `SCHEDULED` and `CANCELLED`; cancelling ends every invitation at once.



Design decisions & patterns used

The generic patterns here are already taught elsewhere in the book: notifications use **Observer** (§11.1) and room ranking uses **Strategy** (§11.3), exactly as in the earlier booking cases. The table spends its

words on the choices specific to this problem.

Decision	Why	Alternative rejected
The room owns its calendar (`Room.calendar`), and the room's lock guards it	The invariant "no two overlapping reservations" is about one room, so the room is the object that can enforce it. Different rooms use different locks and never wait on each other.	A global list of meetings scanned for clashes under one global lock (every booking in the company waits in one line).
Calendar = `TreeMap` keyed by start time (a `NavigableMap`: a map kept sorted by key that can answer "the greatest key below x" in O(log n) time)	Because stored reservations never overlap, **only one entry can clash**: the one with the latest start before the new end (`lowerEntry(end)`). If that one ends at or before the new start, every earlier one does too. One lookup instead of a scan.	A `List` scanned from start to end (O(n) per booking); a per-minute bitmap (wastes memory, fixes the granularity).
Half-open intervals everywhere	Back-to-back meetings (10:00–11:00, 11:00–11:30) are legal without special cases, and the overlap test is one line.	Closed intervals `[start, end]` (every back-to-back booking becomes a false conflict, or needs a "minus one minute" hack).
Rank, then *try*, rooms in order	The strategy ranks rooms from a snapshot that can be stale a microsecond later. Instead of trusting it, the scheduler asks each room to claim the slot atomically and moves on if the claim fails. Ranking can be wrong without ever causing a double-booking.	"Find a free room, then book it" (check-then-act across two calls: two organizers find the same free room).
Default strategy is smallest fit	A 3-person call should not take the 20-seat room. `LeastBusyStrategy` spreads load instead; swapping is one constructor argument.	Always the first free room by id.
Room conflicts are hard, people conflicts are soft	A room physically holds one meeting. People routinely double-book themselves and choose by RSVP. The organizer gets a warning list; an invitee who declined the other meeting is not counted as busy (`Meeting.attends`).	Rejecting a meeting because one invitee is busy.
Reschedule resets RSVPs to `PENDING`	"Yes" to Monday 10:00 is not "yes" to Monday 14:00.	Keeping old answers (people silently "accept" a time they never saw).
Reschedule is an atomic move inside the room (`Room.move`)	Remove old slot, test new slot, insert, or restore the old slot — all under one room lock. The meeting is never left with no room, and the old slot is never lost on failure. A meeting may overlap its *own* old slot (moving 10:00–11:00 to 10:30–11:30 works).	Cancel then re-book (another organizer can grab the slot in between, and invitees get a cancel plus a new invite).
Only the organizer may reschedule or cancel; cancel is idempotent (doing it twice has the same effect as once)	A double-clicked "cancel" sends one round of notifications.	Throwing on the second cancel, or notifying twice.
Notify after unlocking	Observers may be slow (e-mail); holding the meeting or room lock while sending would block other bookings.	Publishing inside the critical section.

Java implementation

Package `lld.ch16.meetingscheduler`. Files: `Types.java` (`Interval`, `User`, enums, `Recurrence`, `MeetingRequest`, `BookingResult`), `Room.java`, `Meeting.java` (`Meeting`, `Invitation`), `RoomSelection.java` (the two strategies), `Notifications.java` (events and the two observers), `MeetingScheduler.java`, `Main.java`.

Only the parts that carry the design are shown; the rest is ordinary getters and the observer and strategy classes described above.

The interval and its overlap test:

```java
// File: code/lld/ch16/meetingscheduler/Types.java (excerpt)
/** A half-open time range [start, end): includes start, excludes end, so 10:00-11:00 and 11:00-12:00 do not
clash. */
record Interval(LocalDateTime start, LocalDateTime end) {
    Interval {
        Objects.requireNonNull(start);
        Objects.requireNonNull(end);
        if (!end.isAfter(start)) throw new IllegalArgumentException("end must be after start");
    }

    boolean overlaps(Interval o) { return start.isBefore(o.end) && o.start.isBefore(end); }

    Interval shiftedBy(Duration d) { return new Interval(start.plus(d), end.plus(d)); }
}
```

The room's calendar — the heart of the design. Every method that changes the calendar does its check and its write under the same lock:

```java
// File: code/lld/ch16/meetingscheduler/Room.java (excerpt)
final class Room {
    record Reservation(String meetingId, Interval interval) {}

    /** Bookings keyed by start time. They never overlap, which makes the conflict check one lookup. */
    private final NavigableMap<LocalDateTime, Reservation> calendar = new TreeMap<>();
    private final ReentrantLock lock = new ReentrantLock();

    /**
     * The booking with the latest start before iv.end is the only one that can overlap iv: bookings are
     * disjoint and sorted, so if that one ends at or before iv.start, every earlier one does too. O(log n).
     */
    private Optional<Reservation> clash(Interval iv) {
        var e = calendar.lowerEntry(iv.end());
        return e != null && e.getValue().interval().end().isAfter(iv.start()) ? Optional.of(e.getValue()) :
Optional.empty();
    }

    /** Atomic check-then-claim. */
    boolean tryReserve(String meetingId, Interval iv) {
        lock.lock();
        try {
            if (clash(iv).isPresent()) return false;
            calendar.put(iv.start(), new Reservation(meetingId, iv));
            return true;
        } finally {
            lock.unlock();
        }
    }

    /** All-or-nothing claim of several slots (a recurring series): check every one, then insert every one. */
    boolean tryReserveAll(Map<String, Interval> byMeetingId) {
        lock.lock();
        try {
            for (var iv : byMeetingId.values()) if (clash(iv).isPresent()) return false;
            byMeetingId.forEach((mid, iv) -> calendar.put(iv.start(), new Reservation(mid, iv)));
            return true;
        } finally {
            lock.unlock();
        }
    }

    /** Moves a booking inside this room atomically: the old slot is restored if the new one is taken. */
    boolean move(String meetingId, Interval from, Interval to) {
        lock.lock();
        try {
            var old = calendar.get(from.start());
            if (old == null || !old.meetingId().equals(meetingId)) throw new IllegalStateException(meetingId + "
not booked at " + from);
            calendar.remove(from.start());
            if (clash(to).isPresent()) {
                calendar.put(from.start(), old);
                return false;
            }
            calendar.put(to.start(), new Reservation(meetingId, to));
            return true;
        } finally {
            lock.unlock();
        }
    }
    // release(meetingId, iv) removes the entry only if it still belongs to that meeting.
}
```

A worked example of `clash`: the room holds A = 09:00–10:00 and B = 10:30–11:00. A request for 09:30–10:45 looks up `lowerEntry(10:45)` = B; B ends at 11:00, after 09:30, so it clashes. A request for 10:00–10:30 looks up `lowerEntry(10:30)` = A (B starts *at* 10:30, not before); A ends at 10:00, not after 10:00, so the slot is free — the half-open rule at work.

The scheduler: rank, then try each room; and the meeting-then-room lock order for reschedule and cancel:

```
// File: code/lld/ch16/meetingscheduler/MeetingScheduler.java (excerpt)
BookingResult book(MeetingRequest r) {
    var rooms = candidates(r);   // the named room if it fits, else strategy.rank(every room that fits)
    if (rooms.isEmpty()) return BookingResult.failure("no room fits " + r.headcount() + " people with " +
r.needs());
    String id = "M" + ids.incrementAndGet();
    for (var room : rooms) {
        if (!room.tryReserve(id, r.when())) continue;          // lost this room: try the next one
        var m = new Meeting(id, null, r.title(), r.organizer(), room, r.when(), r.invitees());
        meetings.put(id, m);
        for (var u : r.invitees()) publish(EventType.INVITED, m, u, "");
        return BookingResult.success(List.of(m), "booked " + m, busyPeople(m));
    }
    // ... failure message: who holds the named room, or "all N fitting rooms are taken"
}

BookingResult reschedule(String meetingId, String byUserId, Interval to) {
    var m = meetings.get(meetingId);
    Interval from;
    synchronized (m) {                                          // 1st lock: the meeting
        requireOrganizer(m, byUserId);
        if (!m.isScheduled()) return BookingResult.failure(meetingId + " is cancelled");
        from = m.when();
        if (!m.room().move(meetingId, from, to))                // 2nd lock: the room
            return BookingResult.failure(m.room().name() + " is not free at " + to);
        m.movedTo(to);                                          // new time, every RSVP back to PENDING
    }
    for (var u : m.invitees()) publish(EventType.RESCHEDULED, m, u, from + " -> " + to);
    return BookingResult.success(List.of(m), "moved " + m, busyPeople(m));
}

void cancel(String meetingId, String byUserId) {
    var m = meetings.get(meetingId);
    synchronized (m) {
        requireOrganizer(m, byUserId);
        if (!m.markCancelled()) return;                         // second cancel is a no-op
        m.room().release(meetingId, m.when());
    }
    for (var u : m.invitees()) publish(EventType.CANCELLED, m, u, "");
}

/** Soft conflict: people who are expected somewhere else at the same time. Reported, not blocked. */
List<String> busyPeople(Meeting m) {
    var people = new ArrayList<User>(m.invitees());
    people.add(m.organizer());
    var out = new ArrayList<String>();
    for (var u : people)
        for (var other : meetings.values())
            if (other != m && other.isScheduled() && other.attends(u.id()) && other.when().overlaps(m.when()))
                out.add(u.id() + " is busy in " + other.id());
    return out;
}
```

`Meeting`'s own methods (`respond`, `movedTo`, `markCancelled`, `attends`) are `synchronized` on the meeting, so an RSVP cannot interleave with a reschedule that is resetting the answers. `bookRecurring` follows the same shape as `book` with `tryReserveAll`: generate the occurrences, allocate one id each, and try the ranked rooms until one room accepts the whole series.

`Main` exercises every requirement, then races threads at one room and at the whole building:

```
1) Ana books a 5-person video call; the scheduler picks the smallest room that fits
    [to ben] ana invited you to M1 'Design review' Mon 11-02 10:00-11:00 in Maple
    [to cy] ana invited you to M1 'Design review' Mon 11-02 10:00-11:00 in Maple
    [to dee] ana invited you to M1 'Design review' Mon 11-02 10:00-11:00 in Maple
    [to eve] ana invited you to M1 'Design review' Mon 11-02 10:00-11:00 in Maple
   OK   booked M1 'Design review' Mon 11-02 10:00-11:00 in Maple
2) Conflicts in a named room (half-open intervals)
   FAIL R2 is taken by M1 Mon 11-02 10:00-11:00
    [to cy] ben invited you to M3 'Back-to-back' Mon 11-02 11:00-11:30 in Maple
   OK   booked M3 'Back-to-back' Mon 11-02 11:00-11:30 in Maple
3) Capacity
   FAIL no room fits 12 people with []
   OK   booked M4 'All hands' Mon 11-02 15:00-16:00 in Atrium
4) RSVPs go back to the organizer
    [to ana] ben ACCEPTED for M1
    [to ana] cy DECLINED for M1
    [to ana] dee TENTATIVE for M1
   M1 [ben=ACCEPTED, cy=DECLINED, dee=TENTATIVE, eve=PENDING]
5) Soft conflict: Eve is double-booked, which is allowed but reported
    [to eve] dee invited you to M5 'Hiring sync' Mon 11-02 10:30-11:00 in Huddle
   OK   booked M5 'Hiring sync' Mon 11-02 10:30-11:00 in Huddle warnings=[eve is busy in M1, dee is busy in M1]
    [to cy] dee invited you to M6 'Coffee' Mon 11-02 10:30-11:00 in Oak
   OK   booked M6 'Coffee' Mon 11-02 10:30-11:00 in Oak warnings=[dee is busy in M1, dee is busy in M5]
6) Ana moves the design review to 14:00: same room, RSVPs reset
    [to ben] M1 moved: Mon 11-02 10:00-11:00 -> Mon 11-02 14:00-15:00 - please RSVP again
    [to cy] M1 moved: Mon 11-02 10:00-11:00 -> Mon 11-02 14:00-15:00 - please RSVP again
    [to dee] M1 moved: Mon 11-02 10:00-11:00 -> Mon 11-02 14:00-15:00 - please RSVP again
    [to eve] M1 moved: Mon 11-02 10:00-11:00 -> Mon 11-02 14:00-15:00 - please RSVP again
   OK   moved M1 'Design review' Mon 11-02 14:00-15:00 in Maple
   M1 [ben=PENDING, cy=PENDING, dee=PENDING, eve=PENDING]
   ben cancels -> ben is not the organizer of M1
7) Ana cancels; the slot is free again
    [to ben] M1 'Design review' was cancelled
    [to cy] M1 'Design review' was cancelled
    [to dee] M1 'Design review' was cancelled
    [to eve] M1 'Design review' was cancelled
    [to cy] ben invited you to M7 'Retro' Mon 11-02 14:00-15:00 in Maple
    [to dee] ben invited you to M7 'Retro' Mon 11-02 14:00-15:00 in Maple
   OK   booked M7 'Retro' Mon 11-02 14:00-15:00 in Maple
8) Recurring weekly stand-up x4 in Huddle; a later series must be all-or-nothing
   OK   booked series S8 in Huddle
     M9 'Stand-up' Mon 11-02 09:30-09:45 in Huddle
     M10 'Stand-up' Mon 11-09 09:30-09:45 in Huddle
     M11 'Stand-up' Mon 11-16 09:30-09:45 in Huddle
     M12 'Stand-up' Mon 11-23 09:30-09:45 in Huddle
   FAIL no single fitting room is free for all 3 occurrences
   OK   booked series S17 in Maple
   after cancelSeries, Huddle bookings: 1
9) Race: 20 organizers want Oak at Tue 10:00 at the same instant
   winners=1, Oak bookings at that time=1
10) Race: 20 organizers, any room, Wed 10:00 - 4 rooms exist, so exactly 4 win
   per room {Atrium=1, Huddle=1, Maple=1, Oak=1}, rejected=16
11) Inboxes and event counts
   eve's inbox: 10 messages, first: ana invited you to M1 'Design review' Mon 11-02 10:00-11:00 in Maple
   ana's inbox: [ben ACCEPTED for M1, cy DECLINED for M1, dee TENTATIVE for M1]
   events {INVITED=23, RSVP=3, RESCHEDULED=4, CANCELLED=12}
   ben's Monday: [M3 'Back-to-back' Mon 11-02 11:00-11:30 in Maple, M7 'Retro' Mon 11-02 14:00-15:00 in Maple, M4
'All hands' Mon 11-02 15:00-16:00 in Atrium]
```

Reading the output:

- **Step 1:** only Maple (8) and Atrium (20) fit five people with video; smallest fit picks Maple. (M2 is the id used by step 2's failed attempt: ids are allocated before claiming.)
- **Step 2:** 10:30–11:30 overlaps M1 and fails with the blocker named; 11:00–11:30 is back-to-back and succeeds (half-open).
- **Step 3:** twelve people cannot use Huddle (4 seats), so the named-room request fails before any lock is taken; the automatic request lands in Atrium.

- **Step 5:** Eve and Dee are already in M1, so M5 books with warnings. In M6, Cy is *not* flagged, because Cy declined M1.
- **Step 6–7:** the move keeps Maple, resets RSVPs and notifies; Ben cannot cancel Ana's meeting; the second cancel sends nothing; the freed 14:00 slot is immediately bookable.
- **Step 8:** the daily "Planning" series in week 3 clashes with the 09:30 stand-up on its first day, so the named-room request fails as a whole (no half-booked series). With automatic choice, Huddle fails its all-or-nothing claim and the series lands in Maple. After `cancelSeries`, Huddle keeps only M5.
- **Steps 9–10:** the counts are exact every run: one winner for one room, and four winners — one per room — when 20 threads may use any room. *Which* thread wins varies.

Concurrency & edge cases

The basic technique — one lock per bookable resource, check and claim inside it — is the one used for show seats in BookMyShow (§15.2) and for hotel rooms (§15.4). What is specific to meetings:

- **Overlap, not equality.** Seats and hotel nights are discrete keys ("seat 12F", "night of the 3rd"), so a concurrent set rejects a duplicate key. Meetings are ranges: 10:00–11:00 and 10:30–11:15 have different keys but still collide. A `ConcurrentHashMap.putIfAbsent` on the start time would *not* catch this. That is why the check (`lowerEntry`) and the insert must sit inside one lock on the room's sorted map.
- **Automatic room choice under a race.** The ranked list is a snapshot. In step 10, all 20 threads rank Huddle first; one wins it and the other 19 fail their claim and move on to Maple, and so on. Correctness comes from the per-room claim, not from the ranking.
- **Lock order: meeting, then room; never two rooms.** `reschedule` and `cancel` take the meeting's monitor and then the room's lock. Nothing takes them the other way round (a `Room` never calls back into a `Meeting`), and no code holds two room locks at once, so deadlock (two threads each waiting for a lock the other holds; see §12.17) cannot happen. `book` takes only a room lock, because the new meeting is not visible to anyone yet.
- **Cancel racing reschedule.** Both run under the meeting's monitor; whichever runs second sees the first's result (a cancelled meeting cannot be moved; a moved meeting is cancelled at its new time, and `release` frees the correct slot).
- **RSVP racing reschedule.** Both are `synchronized` on the meeting. If the RSVP runs first, the reschedule resets it to pending; if it runs second, it is an answer to the new time. Either outcome is consistent.
- **Recurring series.** All occurrences are checked and inserted under one room lock, so a competing one-off booking cannot slip into week 3 between the checks and the inserts.
- **Soft-conflict warnings are advisory.** `busyPeople` reads other meetings without a global lock; a meeting booked a millisecond later may be missed. That is acceptable because it is a warning, not an invariant. The room calendar is the only invariant, and it is always exact.
- **Validation:** `end` after `start`; an occurrence must be shorter than its repeat period (a 25-hour "daily" meeting would overlap its own next occurrence); no RSVP to a cancelled meeting.
- **Time zones.** The code uses `LocalDateTime` for readability. A real system stores `Instant` (an absolute point on the UTC timeline) and converts to each person's zone for display, and expands recurring series in the organizer's zone so "09:30 every Monday" survives a daylight-saving change.
- **Beyond one process.** With several app servers the in-memory lock no longer covers every request. Two standard ways: PostgreSQL's **exclusion constraint** (`EXCLUDE USING gist (room_id WITH =, slot WITH &&)` on a `tstzrange` column, with the `btree_gist` extension installed, which makes the database itself reject any overlapping row for the same room), or a row lock on the room (`SELECT ... FROM room WHERE id = ? FOR UPDATE`) held while checking and inserting the booking.

Extensions & follow-up questions

- **"Find a time when everyone is free."** Merge the busy intervals of all attendees (sort by start, merge overlapping ones), then walk the gaps inside working hours and return the first gap at least as long as the meeting. The same sweep over the room calendars answers "earliest time with a free room".

- **"Virtual meetings with no room?"** Make the room optional: a `Location` interface with `Room` and `VideoLink` implementations; only `Room` has a calendar to claim.
- **"Edit one occurrence of a series?"** Store the rule plus **exceptions** (moved or cancelled occurrences by date) instead of one row per occurrence; that is how calendar standards such as iCalendar (RFC 5545) model it. Our extension expands occurrences eagerly, which is simpler but cannot express "forever".
- **"Reminders 10 minutes before?"** A scheduled job (or a `ScheduledExecutorService`, Java's timer-backed thread pool) that publishes a `REMINDER` event to the same observers; nothing in booking changes.

In the interview

30-second pitch: "Each room owns its calendar, a sorted map of half-open intervals keyed by start time. Booking is an atomic check-then-insert under that room's lock, and because stored bookings never overlap, one `lowerEntry(end)` lookup detects any clash. If no room is named, a strategy ranks the rooms that fit, and the scheduler tries to claim them in order, so ranking can be stale without double-booking. Room clashes are hard errors; people clashes are warnings. Invitations carry RSVP state, reschedule resets it, and observers deliver notifications after the locks are released. Recurring meetings are claimed all-or-nothing in one room."

Time plan: clarify (5 min) → entities and diagram (7) → code `overlaps`, `tryReserve`, `book` (16) → RSVP, reschedule, cancel, lock order (7) → concurrency, series, scale, "common free time" (10).

Self-check

1. **Q:** Why does `lowerEntry(end)` find any clash with one lookup? **A:** Stored bookings are disjoint and sorted, so the one with the latest start before the new end has the latest end of all candidates; if it ends at or before the new start, every earlier booking does too.
2. **Q:** Why can't `putIfAbsent` on a concurrent map prevent double-booking here? **A:** Overlapping meetings usually have different start times, so their keys differ; the conflict is about ranges, which needs a check and an insert under one lock.
3. **Q:** The strategy said Huddle was free, yet booking failed over to Maple. Is that a bug? **A:** No. The ranking is a snapshot; another thread claimed Huddle first. The scheduler tries the next room, and the per-room claim keeps the result correct.
4. **Q:** How do you prevent double-booking with several servers? **A:** Let the database enforce it: an exclusion constraint on (room, time range) or a row lock on the room while checking and inserting.

TL;DR

- Room owns its calendar: a `TreeMap` of half-open intervals; claim = check + insert under the room's lock.
- One `lowerEntry(end)` lookup detects any overlap.
- Auto room choice: strategy ranks, scheduler tries each room's atomic claim in order.
- Room conflicts block; people conflicts warn; declined invitees are free.
- Reschedule = atomic move inside the room + RSVPs reset; cancel is idempotent; notify after unlocking.
- Lock order: meeting, then room; never two rooms. Recurring series: all-or-nothing in one room.

Real-world example. Outlook/Exchange and Google Calendar model rooms as bookable resources with their own calendars that can decline conflicting invitations, while people may be double-booked; recurring meetings are a rule plus exceptions (iCalendar).

For a lighter, timetable-only version of the same interval logic (user and team availability without rooms), see the machine-coding Event Calendar in §21.3.

Chapter summary

Problem	Key insight	Atomic claim	Patterns
Car rental	Availability is a question about **dates**, not a status field	`Vehicle.tryBook(range)`	Template Method (add-ons), Strategy + Decorator (pricing), state machine
Restaurant	**Meal per seat**; table = status (now) + slots (later)	`Table.tryHold(time)`	Observer (order ready), Strategy (chef assignment, payment), state machine
Airline	**Flight ≠ FlightInstance, Seat ≠ FlightSeat**	`FlightInstance.holdSeats(map)` (all or nothing)	Strategy + Decorator (fares), Facade, index for cancellation
Course registration	**Course ≠ Section**; rules as objects	`Section.enroll` under a section lock, inside a student lock	Specification / Chain of rules, lock ordering
Meeting scheduler	**Room owns a calendar of half-open intervals**; room clash blocks, people clash warns	`Room.tryReserve(interval)` (one `lowerEntry` lookup under the room lock)	Strategy (room ranking), Observer (notifications), meeting-then-room lock order

The five problems share one lesson: **find the calendar, find the atomic check-then-claim, and put the rules on the object that owns the data.**